



Matryoshka 解説

tozangezan

問題概要

- ・ N 個のマトリョーシカ人形がある (底面の直径 R_i 、高さ H_i)
- ・ Q 個のクエリが与えられる
- ・ クエリ: 底面の直径 A_i 以上、高さ B_i 以下のすべてのマトリョーシカ人形を取り出したとき、最小何個の**入れ子**にまとめることができるか？



例

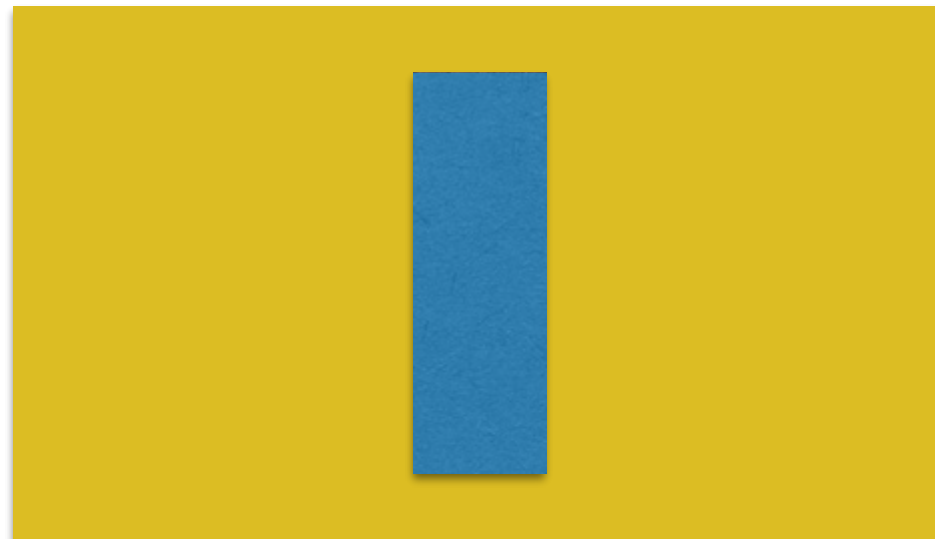
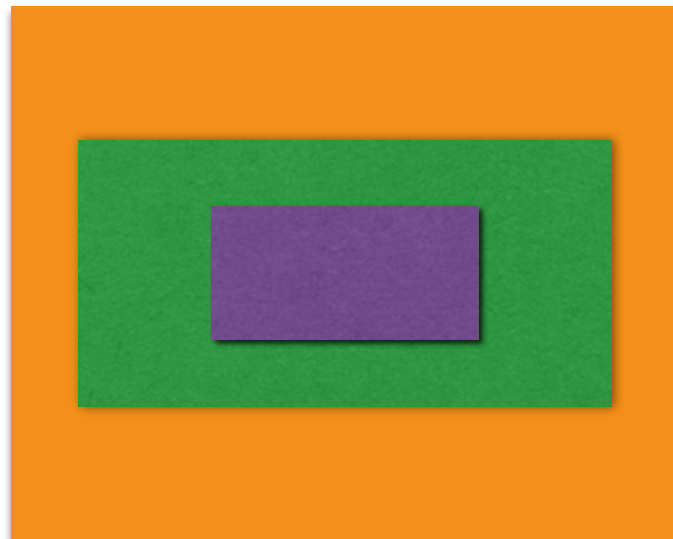
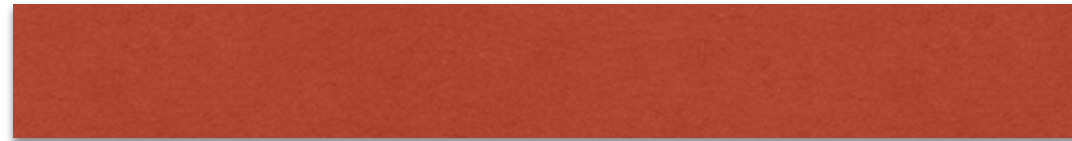
- $R_i = \{1, 4, 2, 8, 5, 7\}$

- $H_i = \{3, 2, 1, 1, 4, 4\}$



例

- $R_i = \{1, 4, 2, 8, 5, 7\}$
- $H_i = \{3, 2, 1, 1, 4, 4\} \rightarrow \text{答え: } 3$

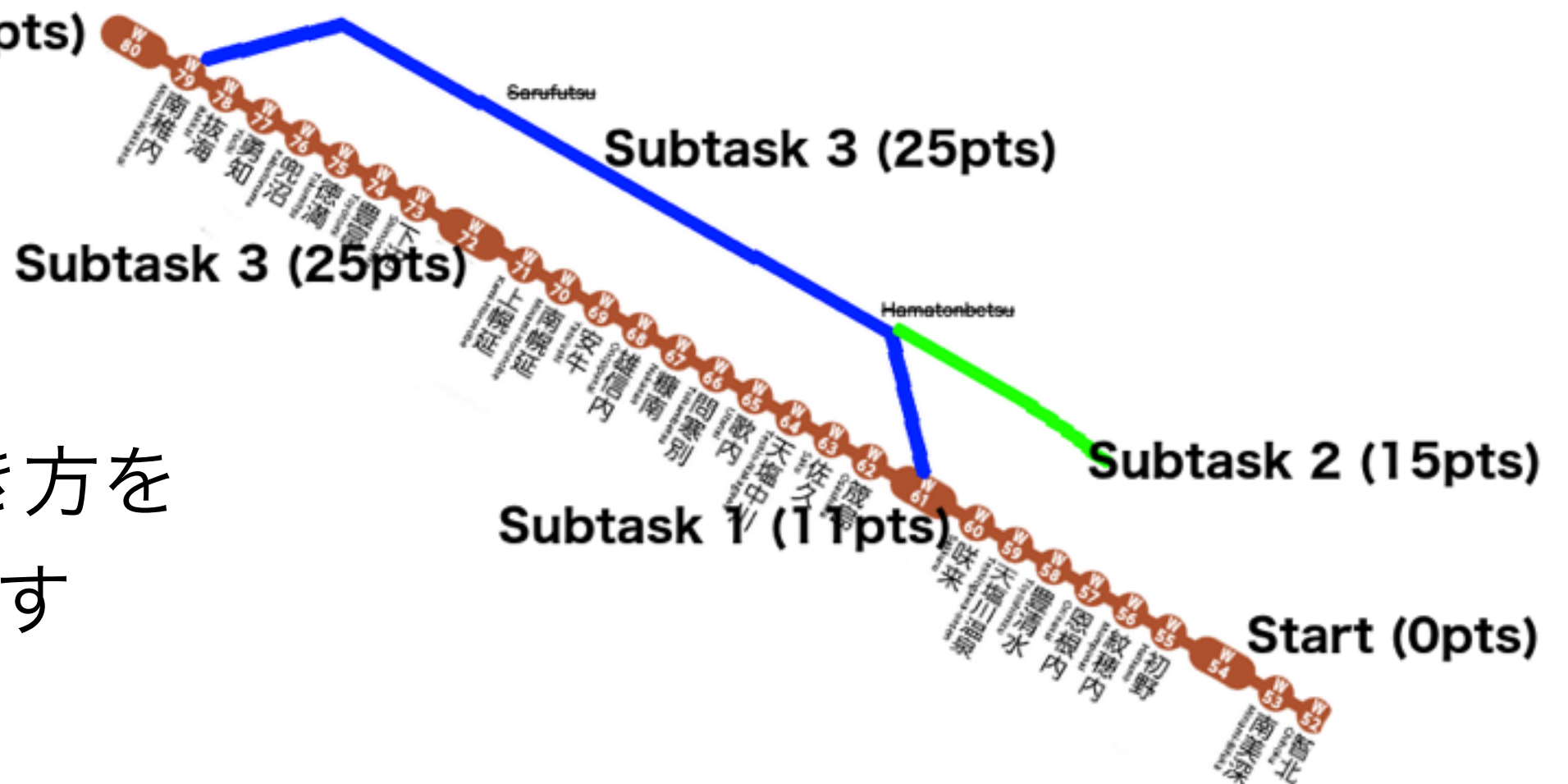


満点解法への道のり



.....

Subtask 4 (49pts)



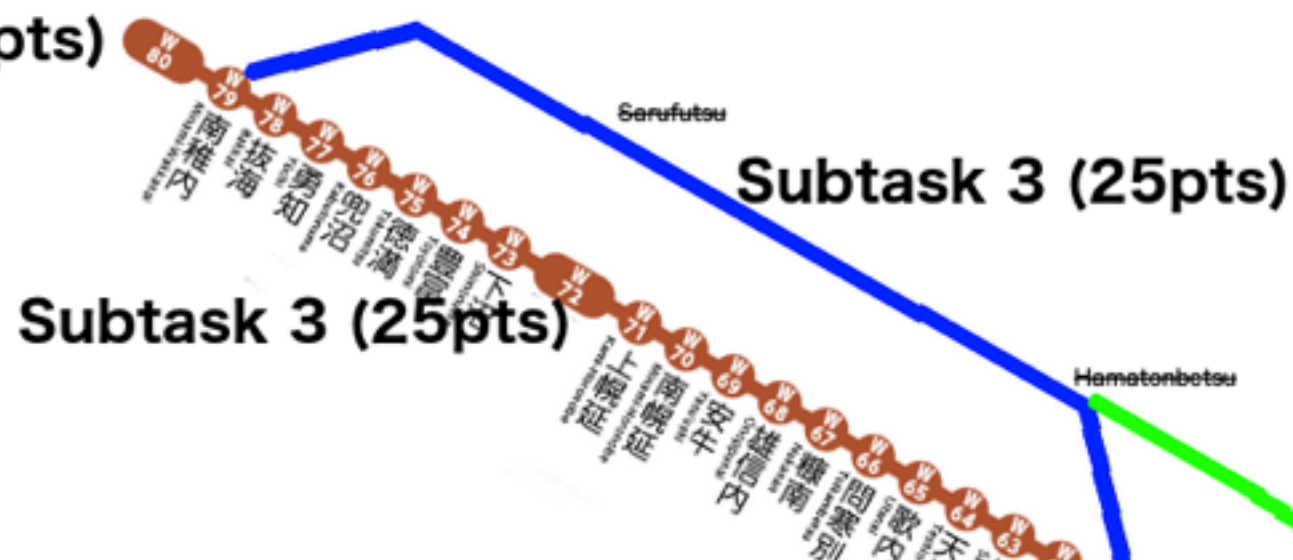
2通りの行き方を
解説します

満点解法への道のり



.....

Subtask 4 (49pts)



Subtask 2 (15pts)

Subtask 1 (11pts)

Start (0pts)

Next: 小課題1

小課題 1 (11点)

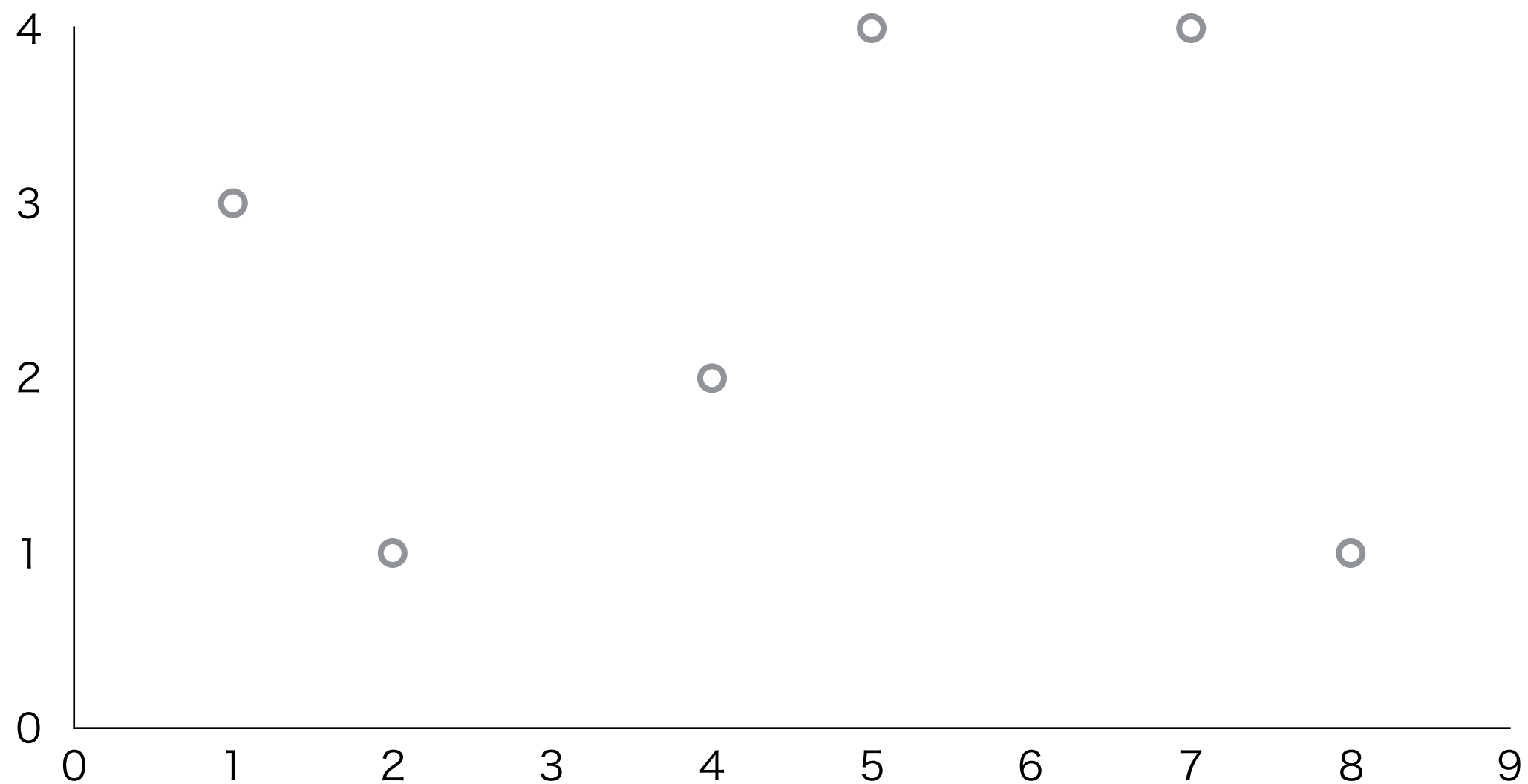
- ・ 全探索でどれの中にどれを入れていくかを試していきます
- ・ DFS で全探索 (枝刈りしなくても間に合います)
- ・ DFS で全探索するのに慣れてない人は PCK の過去問が練習になると思います

とりあえず

- ・ 長方形をたくさん並べたり数字のペアで考えるとややこしいので、現状をより簡単に把握しましょう
- ・ それぞれのマトリョーシカ人形は 2 つのパラメータからなるものです

とりあえず

- ・ x-y 座標上の点にしてみました

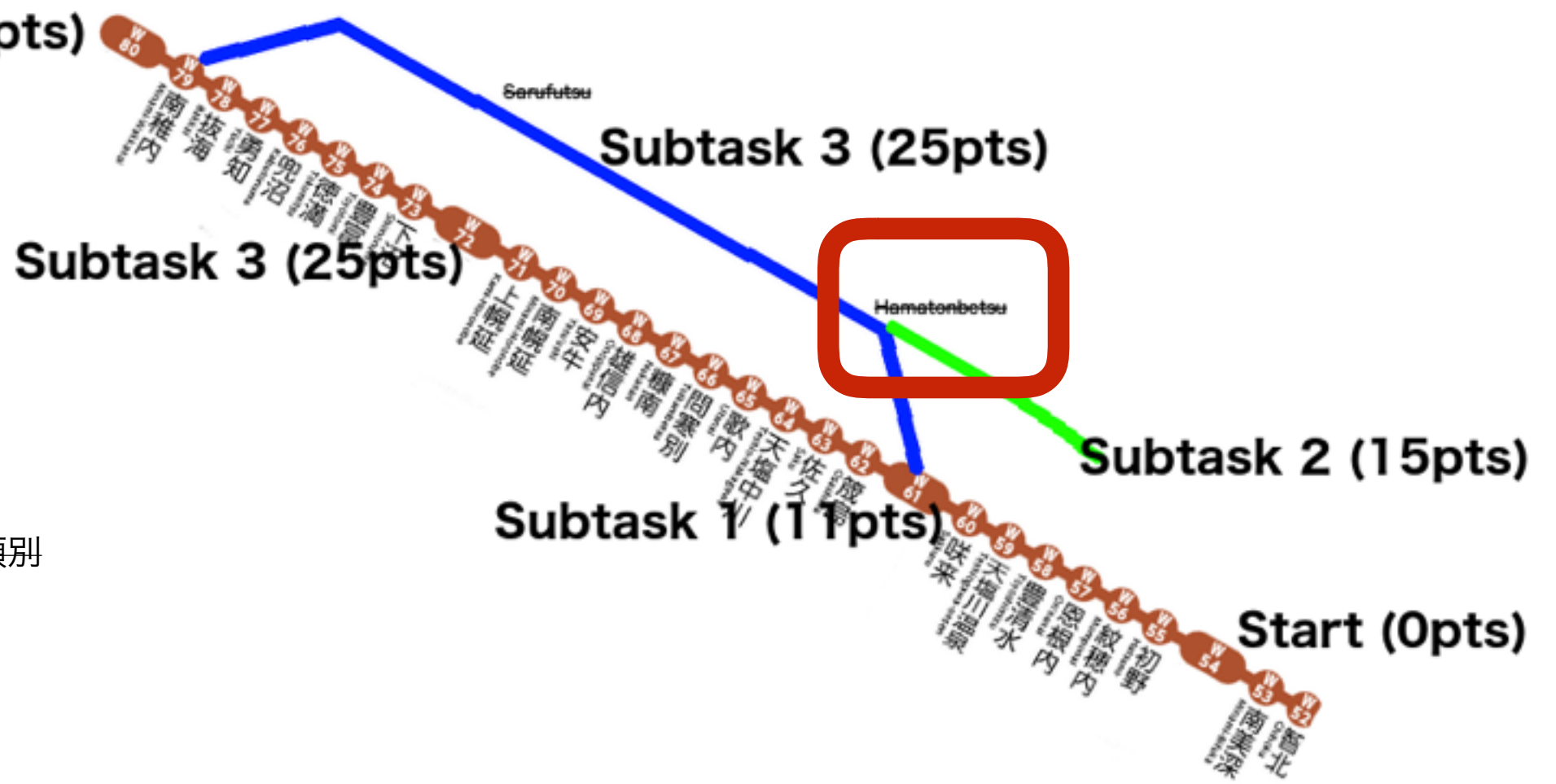


満点解法への道のり



.....

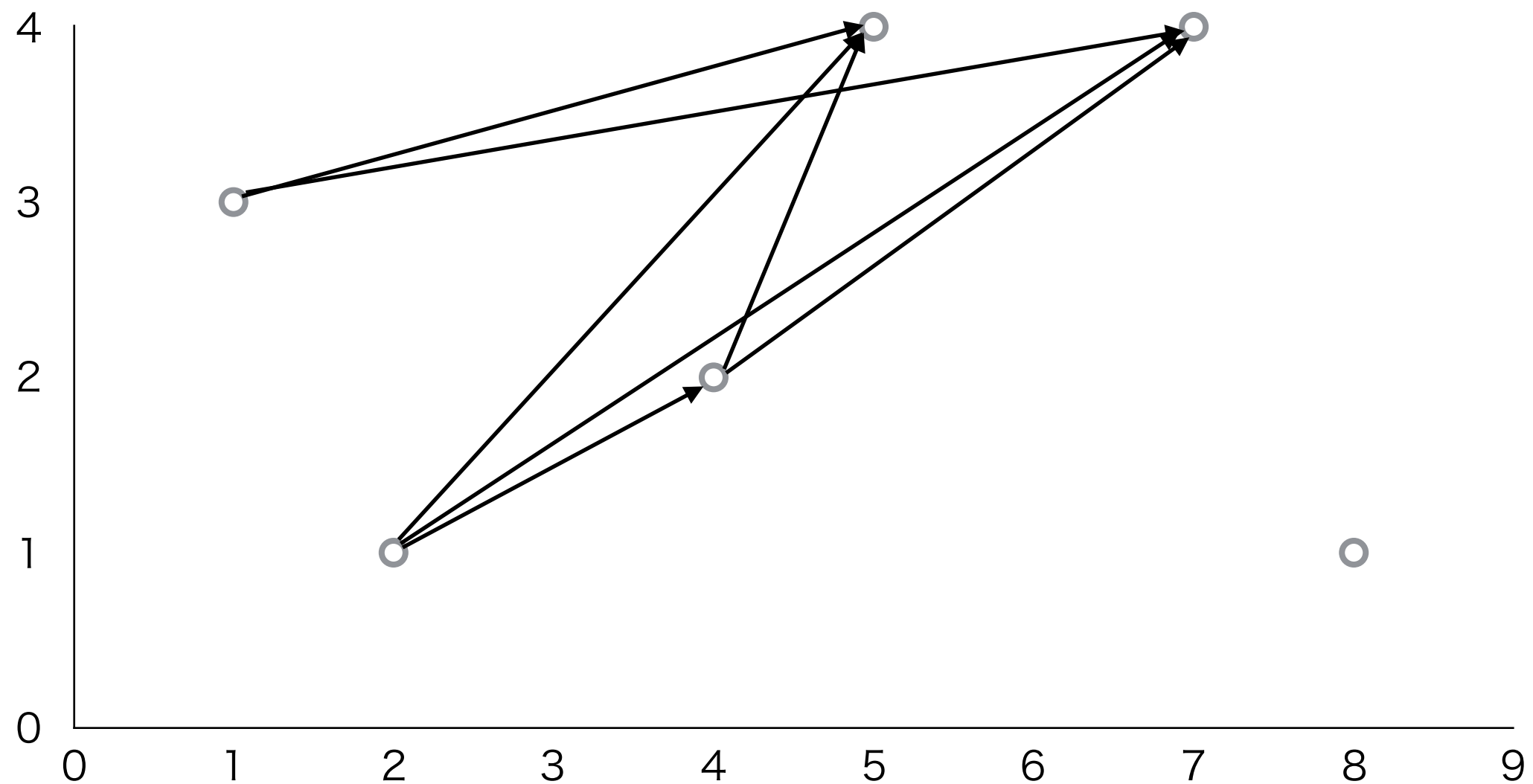
Subtask 4 (49pts)



Next: 浜頓別

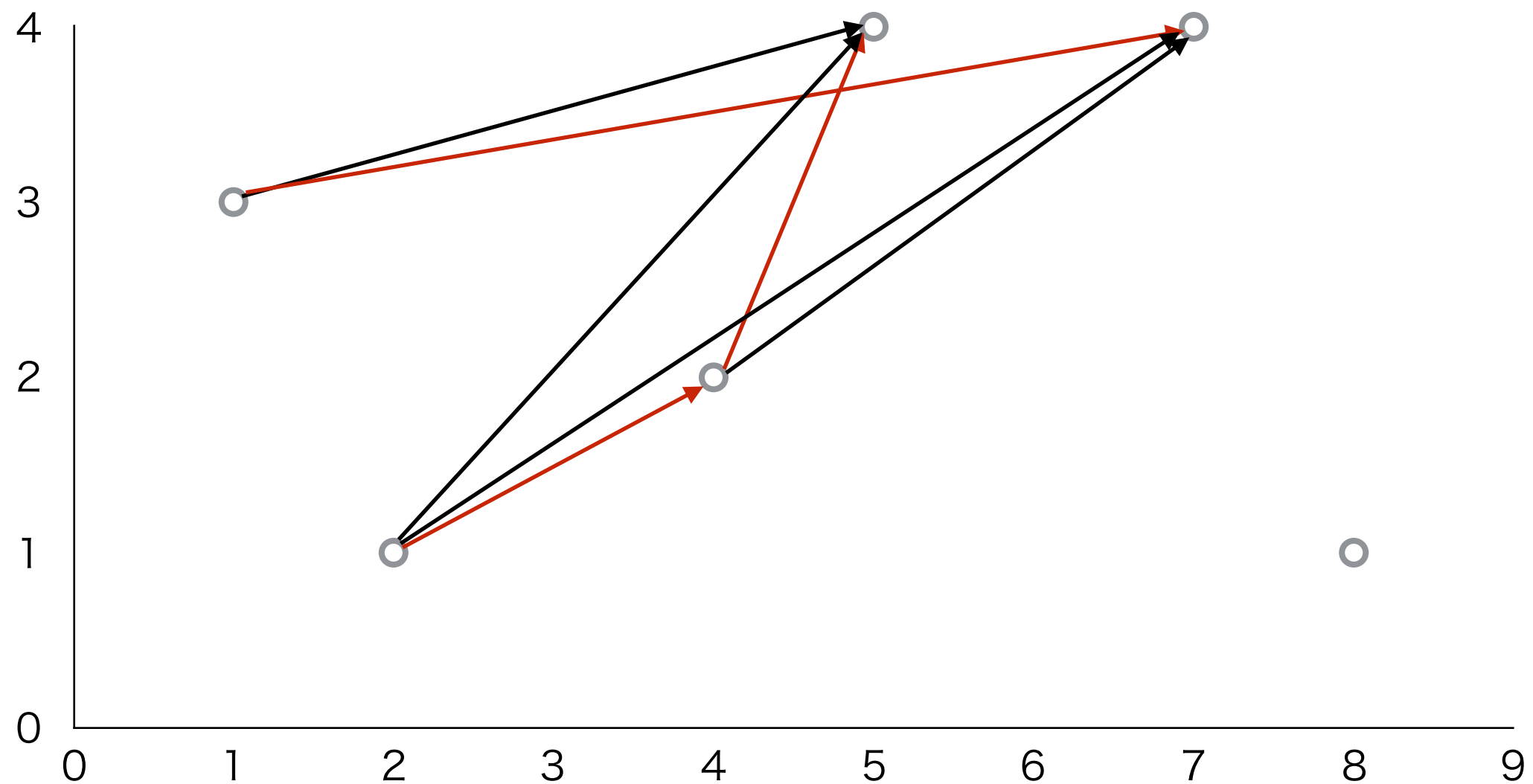
考察 Lv. 1

- ・ 入れ子にできるペアの関係を付け足しました



考察 Lv. 1

- 「例」で入れ子にした関係を赤く塗りました



考察 Lv.1

- ・ 矢印を描いてみてわかったこと
 - ・ 矢印の関係には閉路がない (= DAG)
 - ・ (矢印はどれも右上を向いているので)
- ・ 求める答えは最小いくつのパスですべての点に訪れられるかに等しい (= 最小パス被覆)

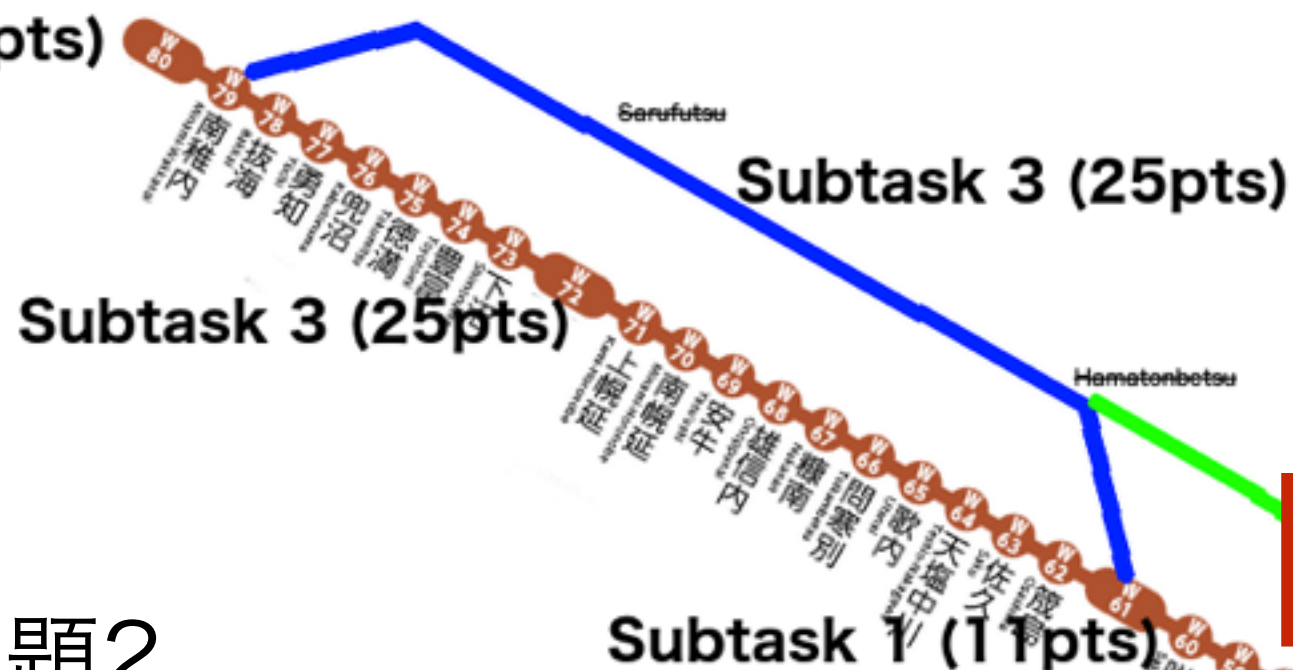
満点解法への道のり

ちょっと寄り道……



.....

Subtask 4 (49pts)

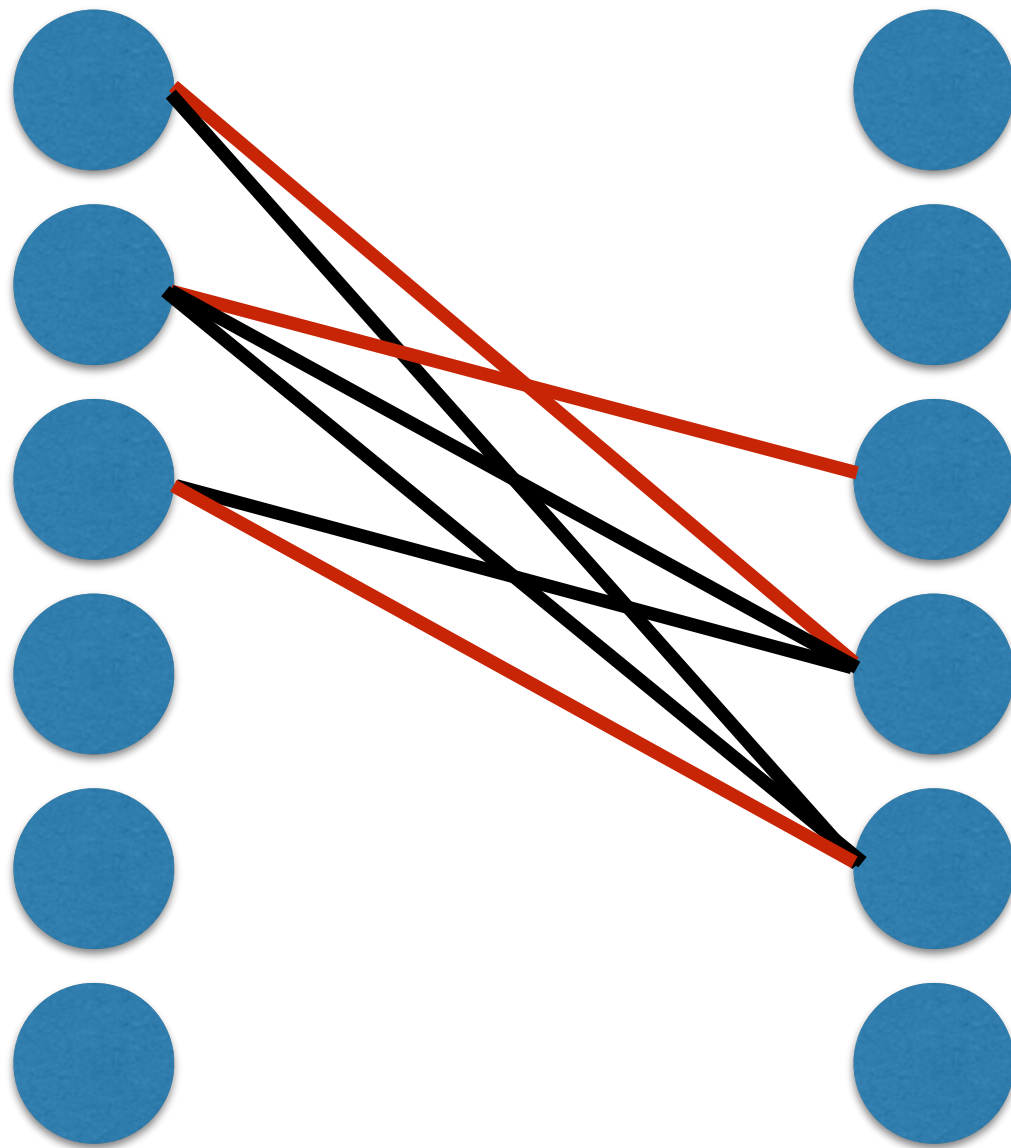


Next: 小課題2

DAG の最小パス被覆

- ・ 実は DAG の最小パス被覆は有名問題です
- ・ N 頂点 - N 頂点の二部グラフを用意します
- ・ DAG で $i \rightarrow j$ という辺があるなら、左の i と 右の j の間に辺を張ります
- ・ 求める答えは $(N - \text{最大マッチング})$ となります

例



先ほどの入力例で作った二部グラフです。

最大マッチングの大きさは 3 です。

求める答えは $6 - 3 = 3$ となります。

フロー

- ・ 二部グラフの最大マッチングを求める必要が出てきたので、ここでフローが必要になります
- ・ 昨日の諸注意にもあったように、JOI 合宿でフローが出題される可能性もあるんですね。

小課題 2

- ・ DAG の 最小パス被覆をフローで求めると、頂点数 $O(N)$ 、辺数 $O(N^2)$ となるので、計算量は $O(N^3)$
- ・ 小課題 2 が解けます (15点)

満点解法への道のり



.....

Subtask 4 (49pts)

Subtask 3 (25pts)

Subtask 3 (25pts)

Subtask 2 (15pts)

Subtask 1 (11pts)

Start (0pts)

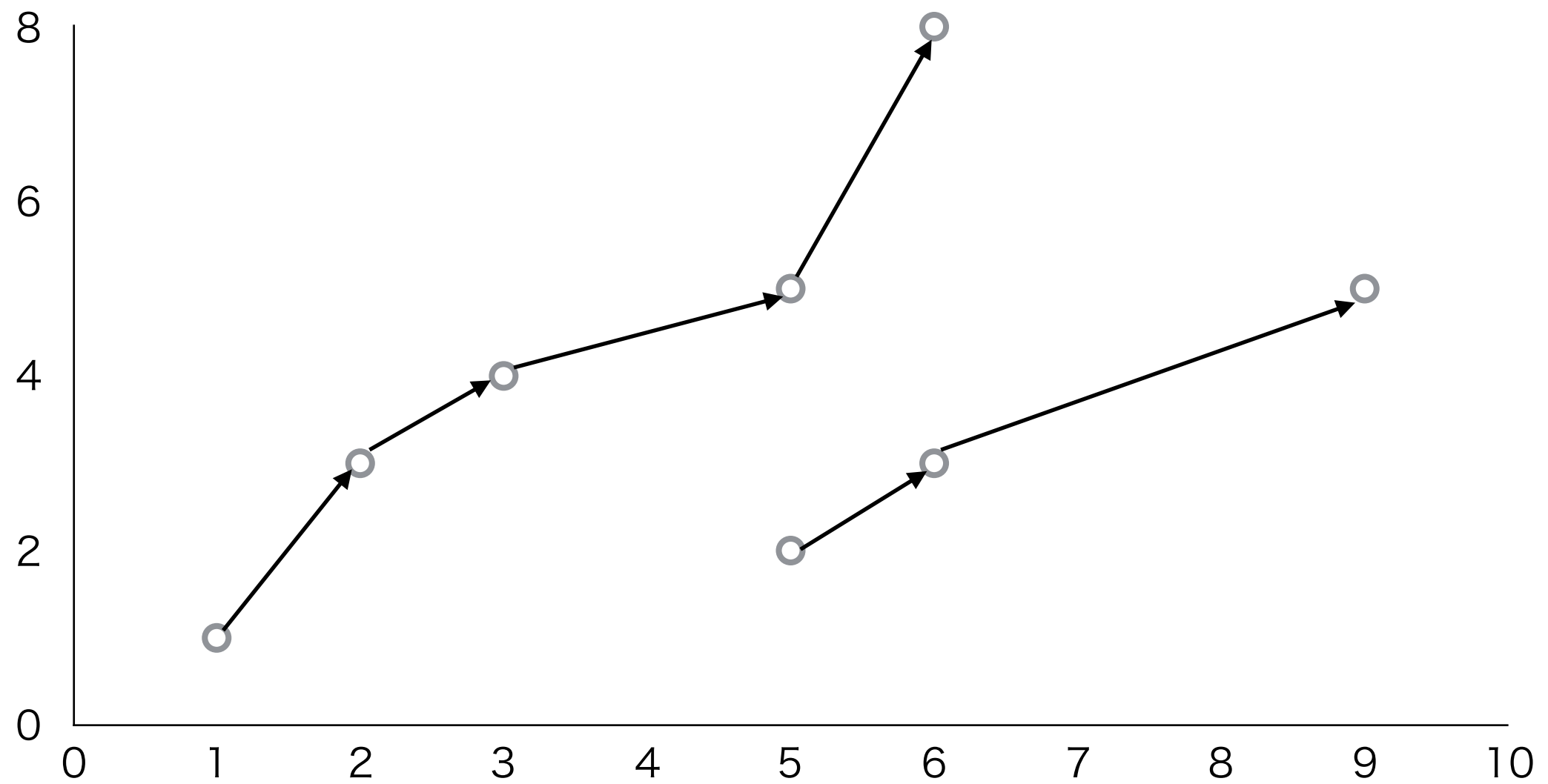
Next: Subtask 3

考察 Lv.2

- ・ パス被覆の個数がどうなるかをいろいろ点を描いて試してみましょう

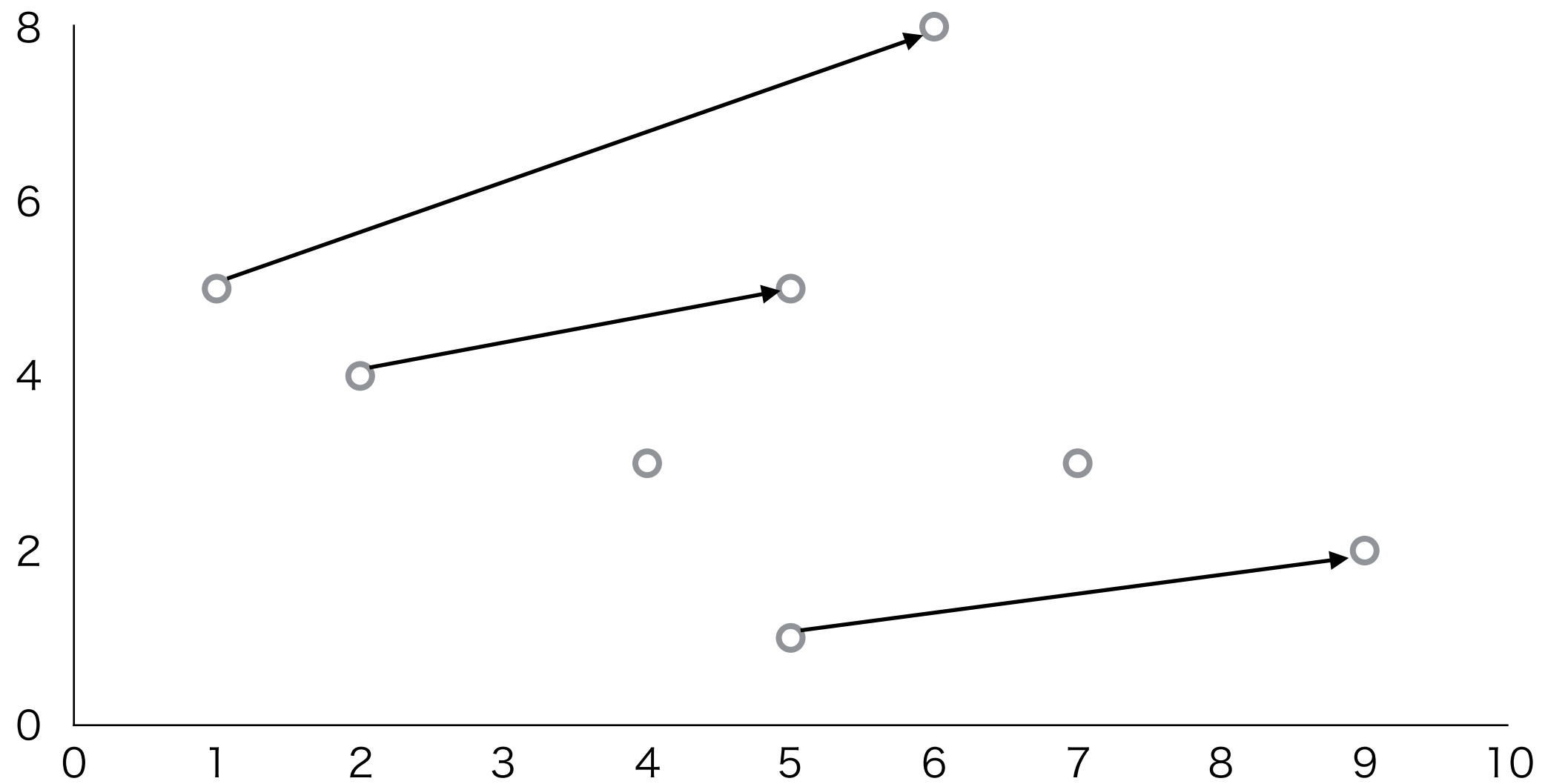
考察 Lv. 2

・ なるほど



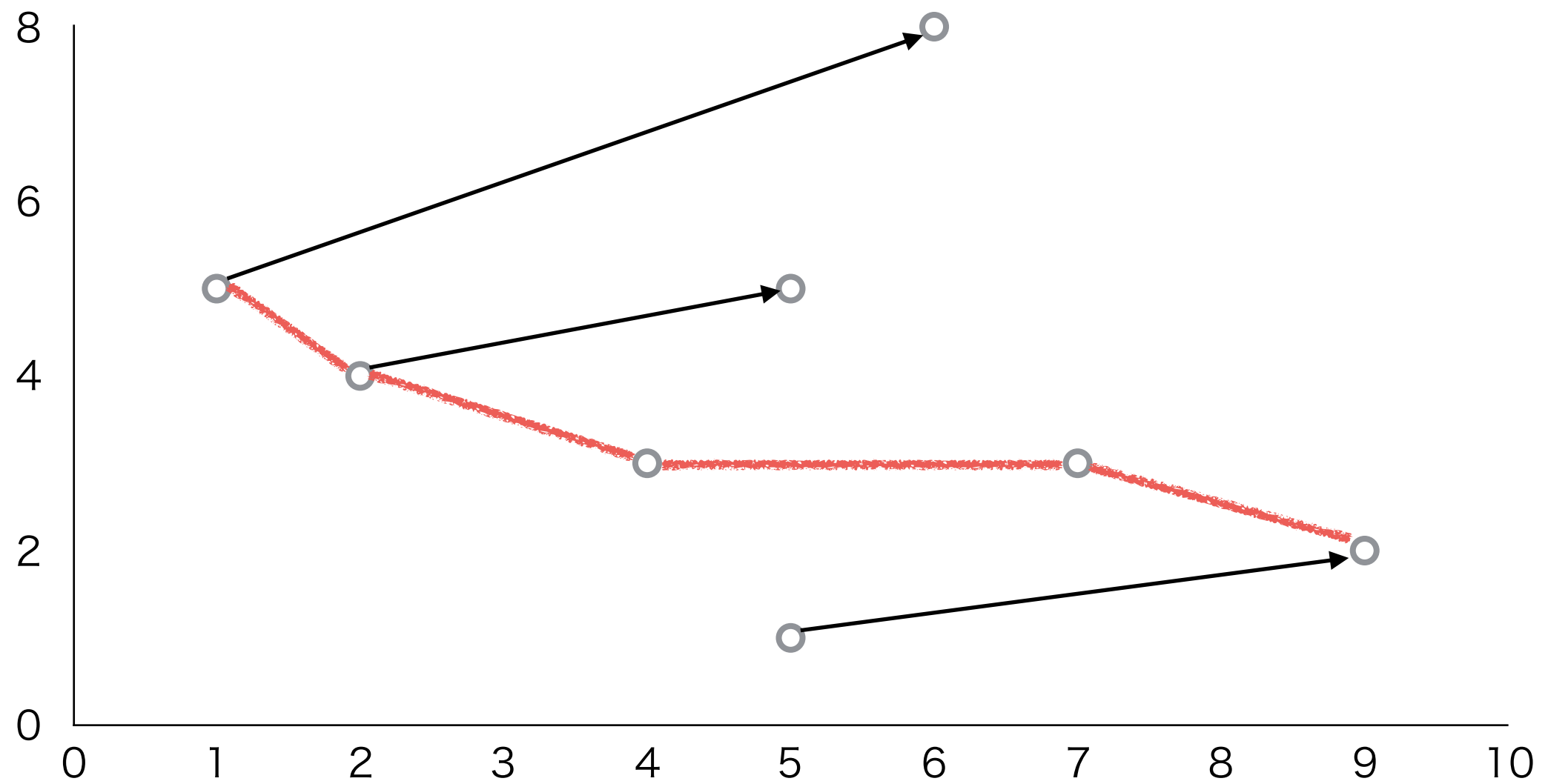
考察 Lv. 2

- ・ なんか落書きしてみようかな



考察 Lv. 2

- 意味ありげな線が引けたぞ！？



思ったことまとめ

- ・ 右下から左上に向かって線を引くと、その線上の点はそれぞれ別のパスで被覆されるので、その折れ線上にある点の個数は最低でも必要っぽい
- ・ この最大個数より多くのパスが必要な場合はあるかな……なさそうだな……それ本当かな……？

思ったことまとめ

それ、正しいです。

・
・
こ
かな...

点
ある

?

Dilworth の定理

- ・ $i \rightarrow j, j \rightarrow k$ の辺がある時、必ず $i \rightarrow k$ の辺もある
ような DAG に対し、最小パス被覆の本数は、任意
の 2 点の組に対し辺が存在しないような頂点集合
の最大サイズに等しい



Robert P. Dilworth
(1950)

問題を変換

- ・ この定理を使うと、次のような問題となります
- ・ 座標上の点で、右下方向 (右、下も含む)に進んでなるべくたくさんの点を通りたい。最大で何個の点を通ることができるか？
- ・ 要は のマイナーチェンジ

問題を変換

- ・ この定理を使うと、次のような問題となります
- ・ 座標上の点で、右下方向 (右、下も含む)に進んでなるべくたくさんの点を通りたい。最大で何個の点を通ることができるか？
- ・ 要は **LIS** のマイナーチェンジ



小課題3 (1) まとめ

- ・ それぞれのクエリで以下のことを行う
- ・ R_i が小さい順 (R_i が等しいときは H_i が大きい順)にソート
- ・ ソートされた列においての H_i のみに着目し、最長非増加部分列問題を解く ($O(N \log N)$ で解ける)
- ・ 全体で $O(QN \log N)$ なので、小課題 3 が解けます

定理知らないんだけど？

- ・ 完全フィードバックなのでとりあえず正しいと思って一度書いてもそんなに損はしない
- ・ けど
- ・ 知らなくてもちゃんと正しいと確信できる解法はないの……？

満点解法への道のり



.....

Subtask 4 (49pts)

Subtask 3 (25pts)

Subtask 3 (25pts)

Subtask 2 (15pts)

Subtask 1 (11pts)

Start (0pts)

Next: Subtask 3

小課題3 別ルート

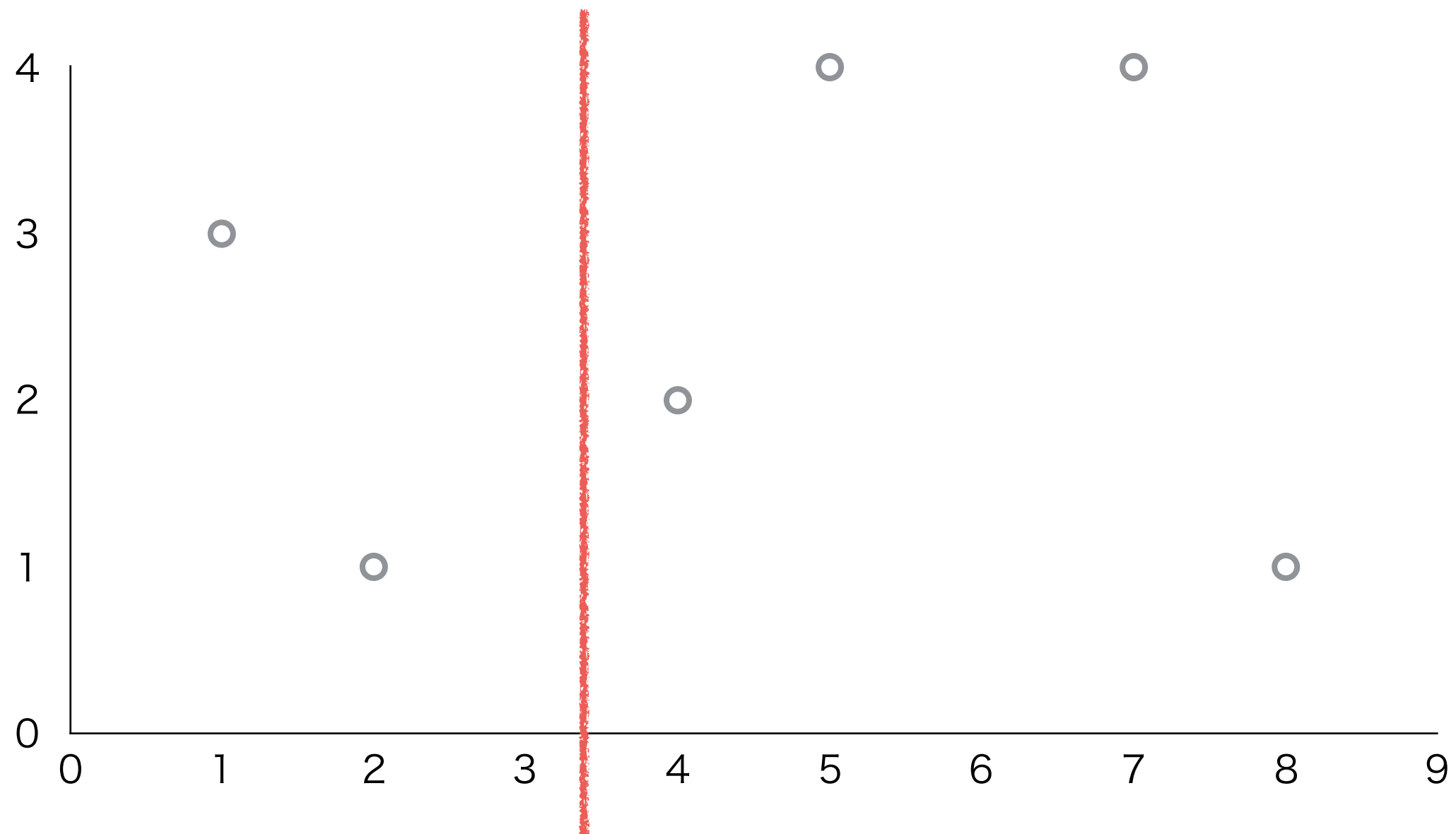
- ・ さっきと同じ順(R_i の昇順、同率は H_i の降順)で並べた状況を考えます
- ・ 並べた順に左から 1 つずつ追加していくことにします

小課題3 別ルート

- ・ あるものを追加するとき、それに入る小さいものが今入れ子にされず残っていたらどうすればいいでしょう？
- ・ 入れない $\rightarrow$ 1 つ余計に増えるだけ、損
- ・ H_i の小さいものを入れる $\rightarrow$ あとで損しそう
- ・ H_i の大きいものを入れる $\rightarrow$ 損要素なし

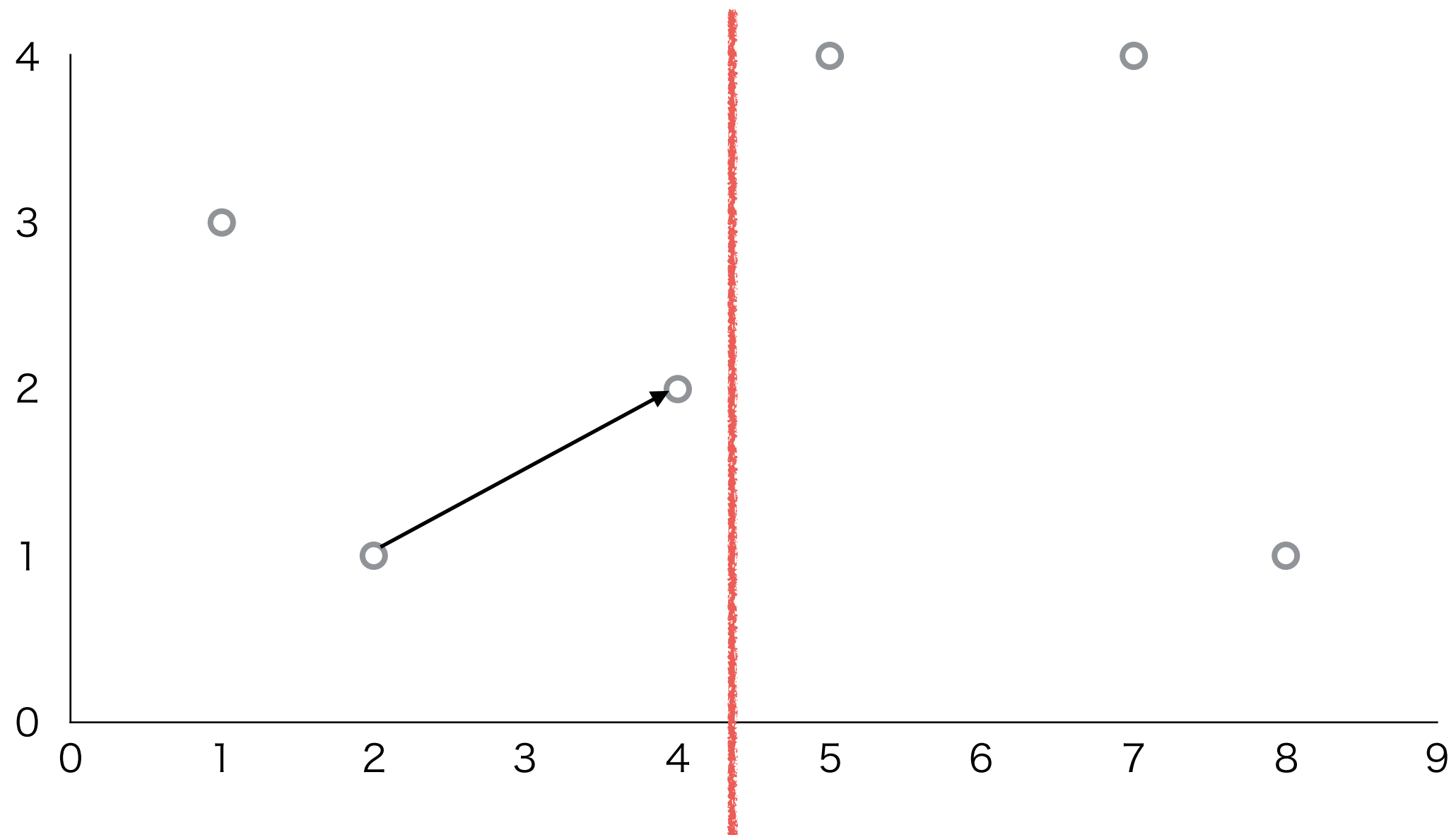
小課題3 例

- ・ ここまでは入れ子にできません



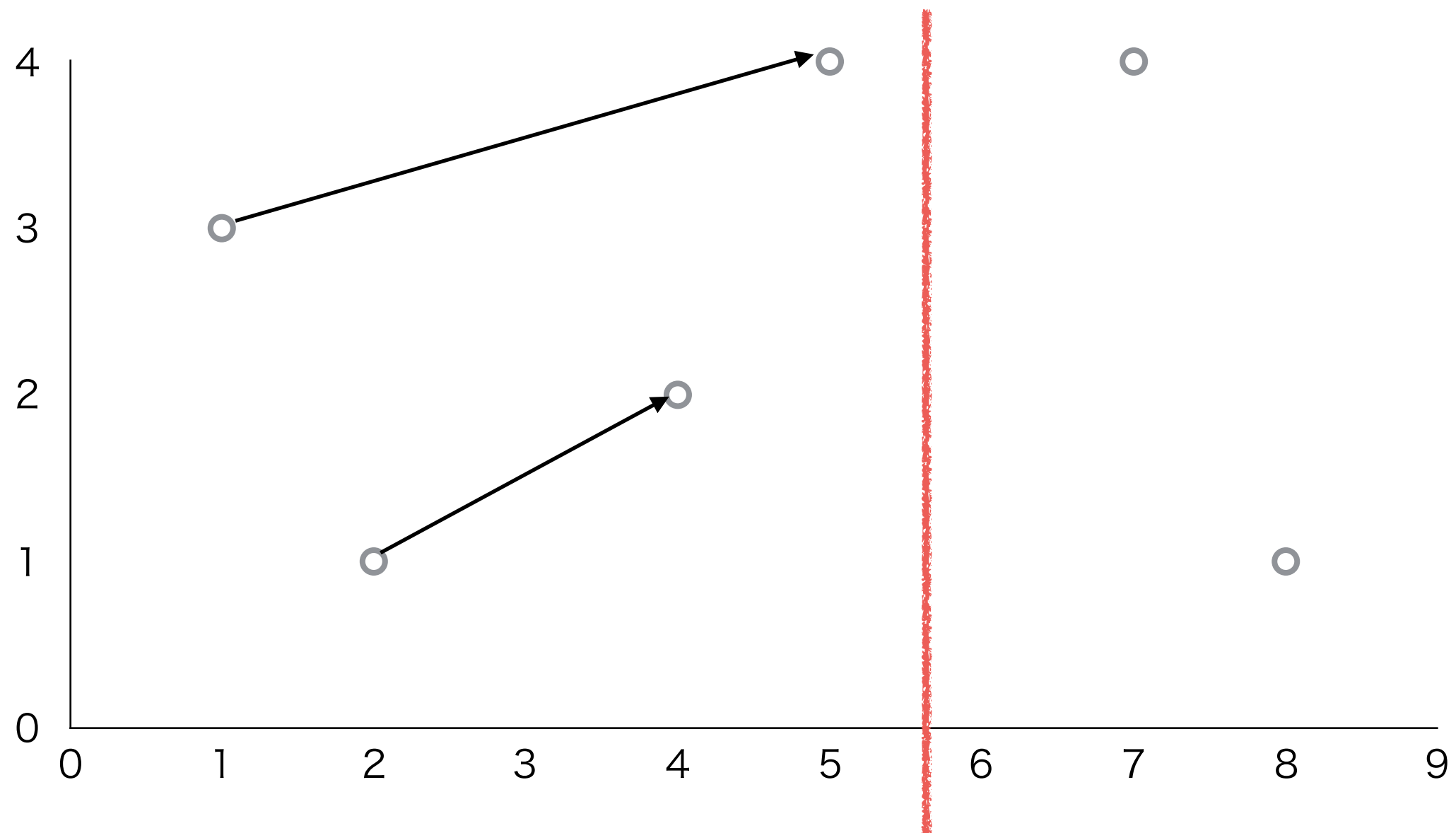
小課題3 例

- ・ 1 つ入れられるものがあるので入れます



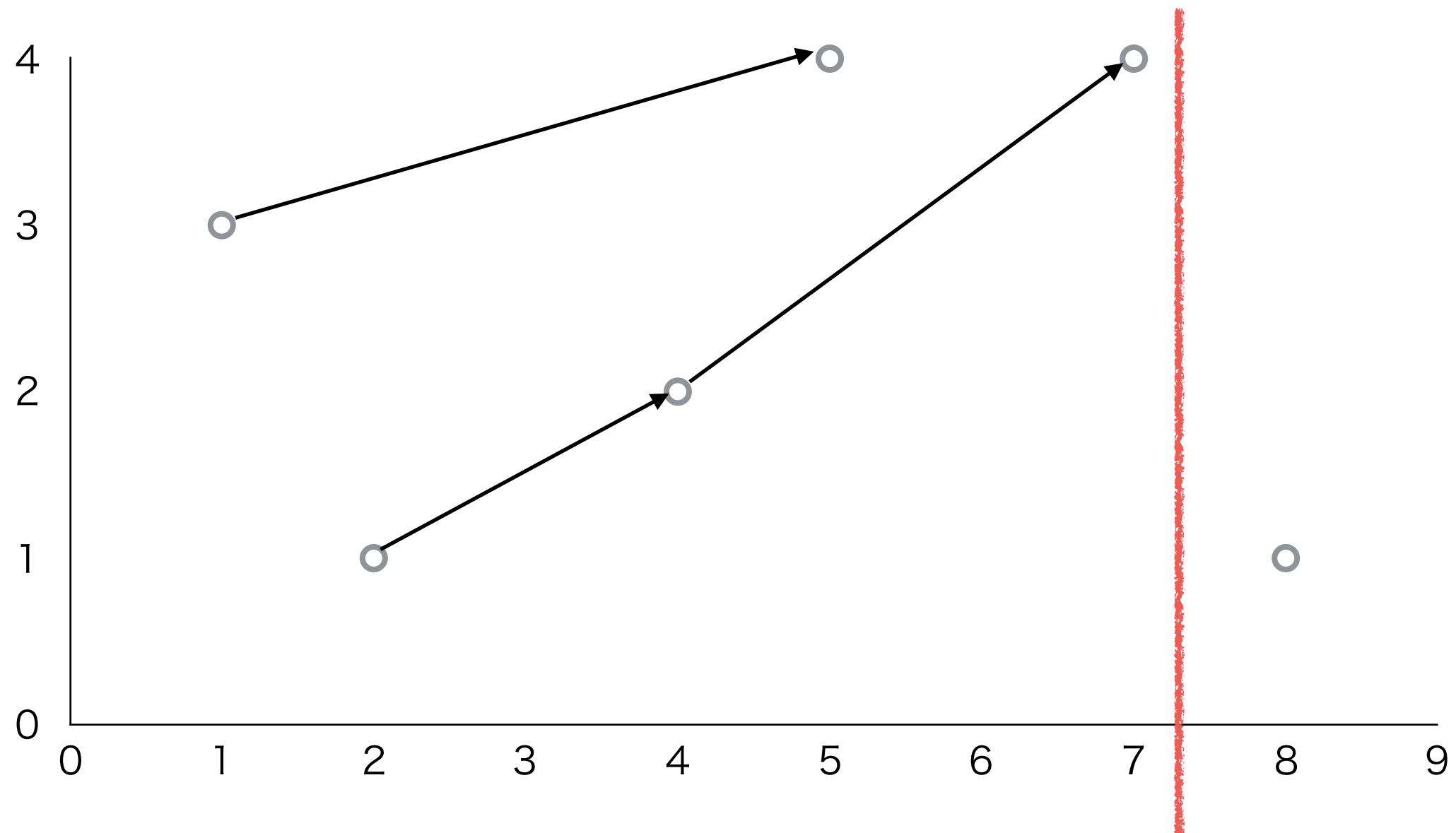
小課題3 例

- 2 つ以上あるので、 H_i の大きい方を入れます



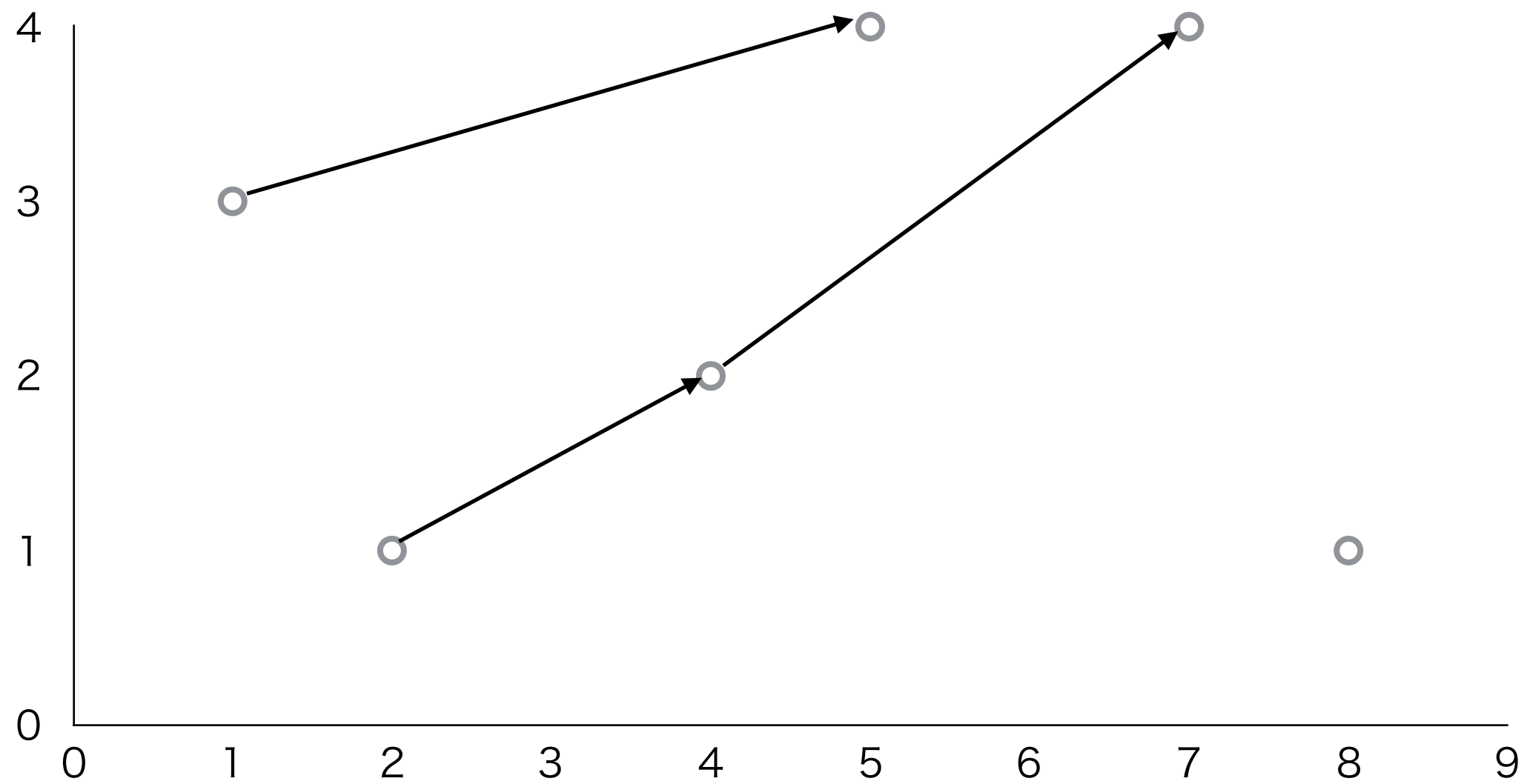
小課題3 例

- ・ 1 つ入れられるものがあるので入れます



小課題3 例

- ・ 答えは 3 です



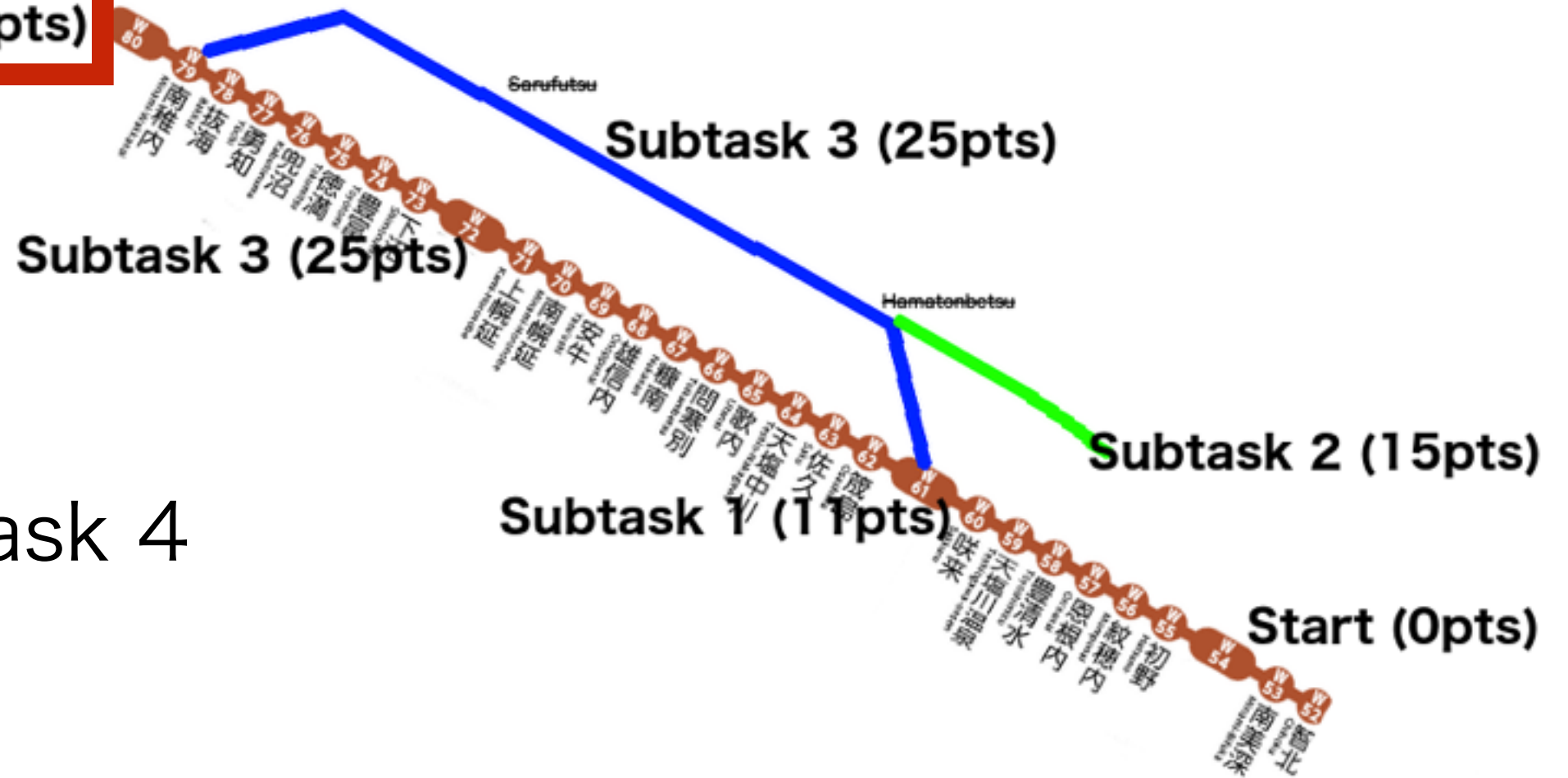
小課題3 (2) まとめ

- ・ 各クエリに対して以下のように求めていく
- ・ 先ほどと同じようにソートしてからGreedy
- ・ ソート列の左から追加していく
- ・ あるものを追加する時、何か入れられるものが残っていれば H_i が入れられるうち最大のものを入れる
- ・ これは `std::set` 等で実装できて、 $O(N \log N)$
- ・ 合計で $O(QN \log N)$ で小課題 3 が解ける

満点解法への道のり



Subtask 4 (49pts)



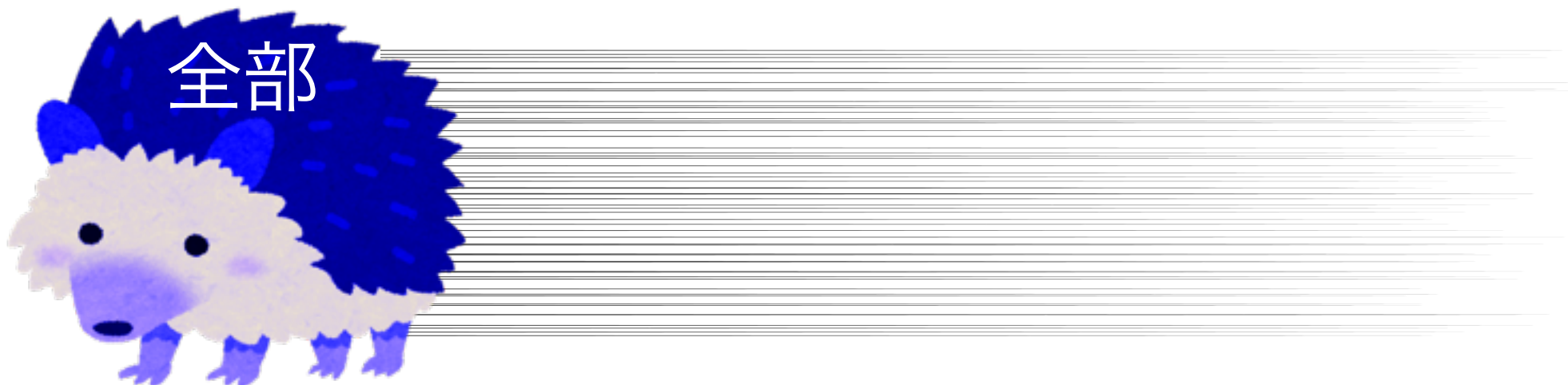
Next: Subtask 4

高速化

- 小課題 3



- 小課題 4

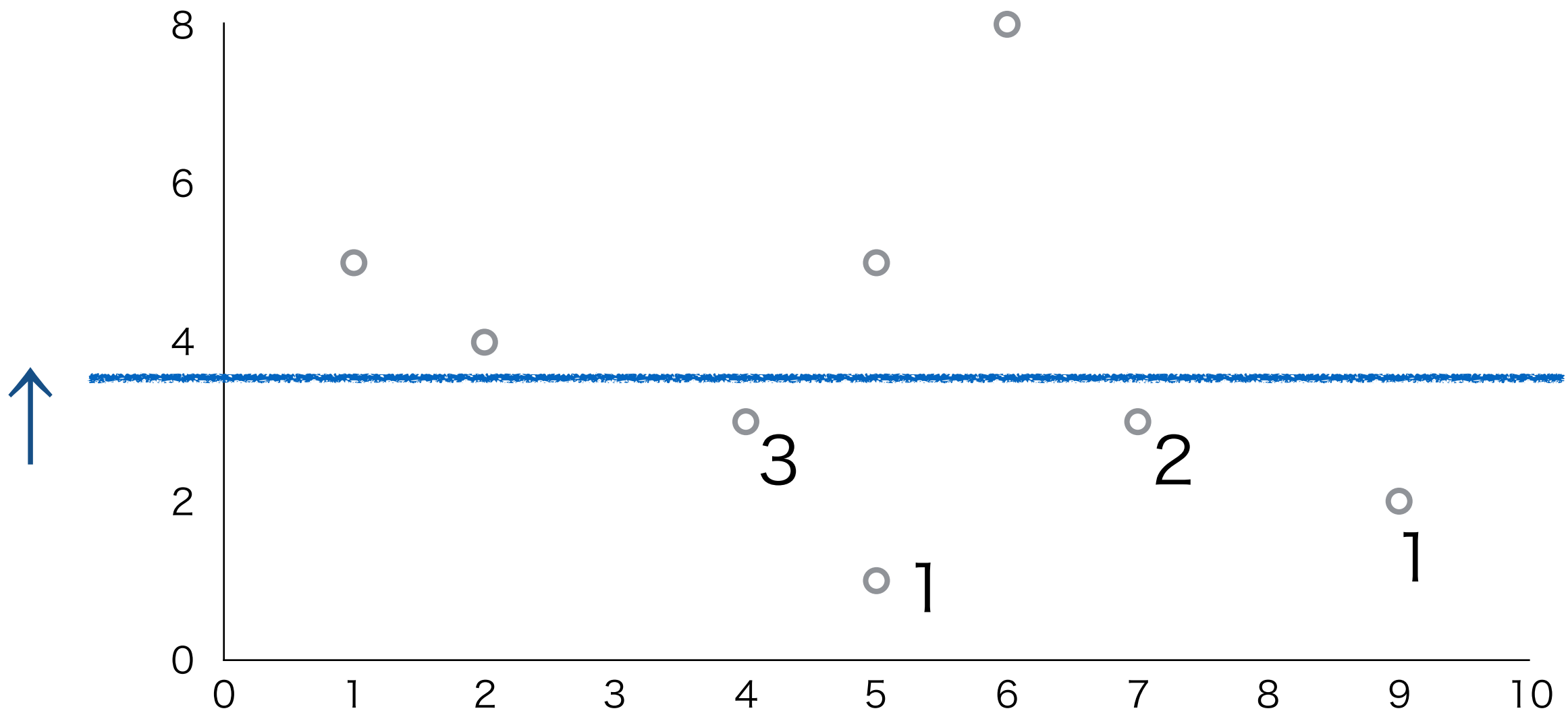


クエリをまとめる

- ・ まずは LIS 版をまとめて求めてみましょう
- ・ クエリは y 座標が小さい順にソートしておきます

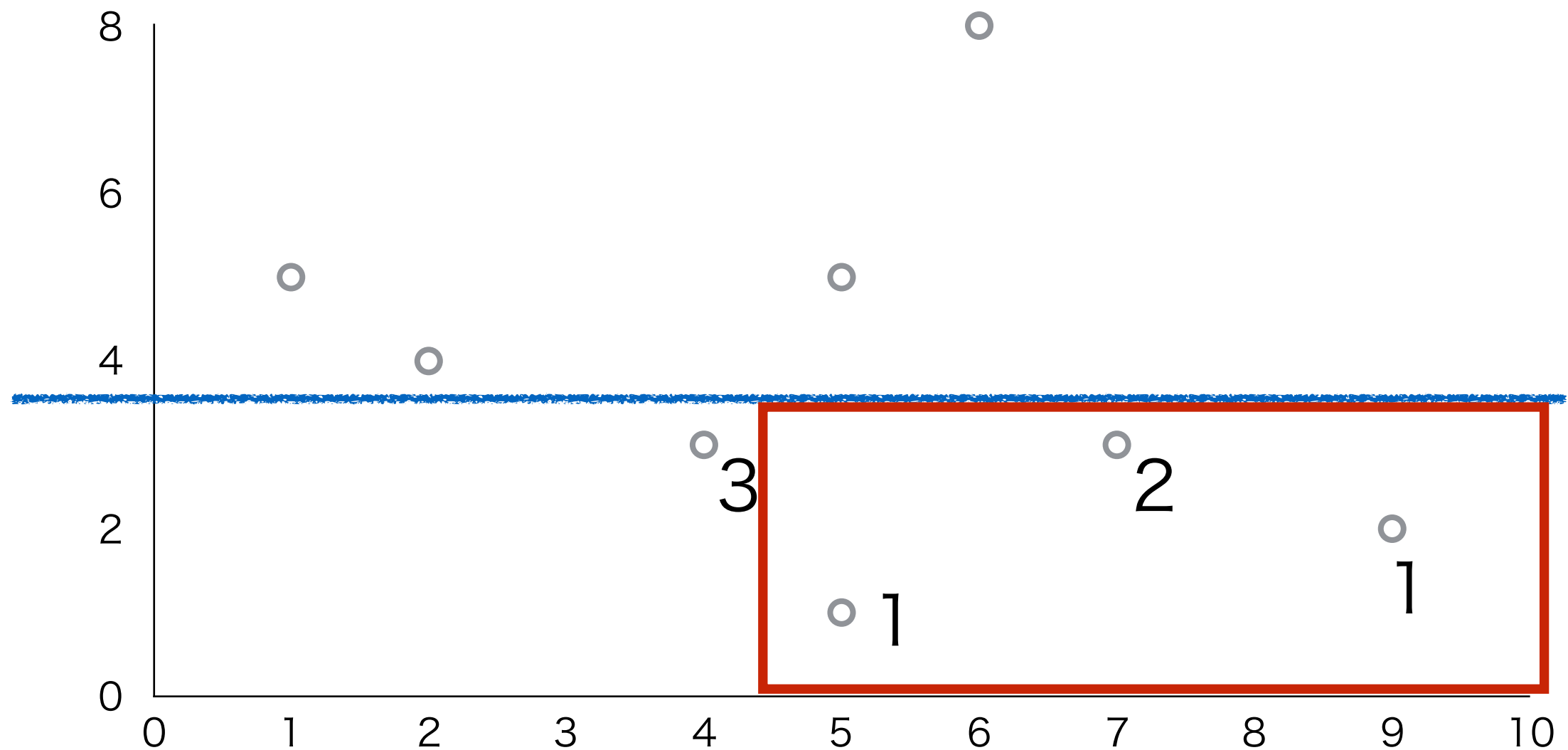
クエリ処理の例

- 各頂点の数字はそれを左端とする LIS 最大長です



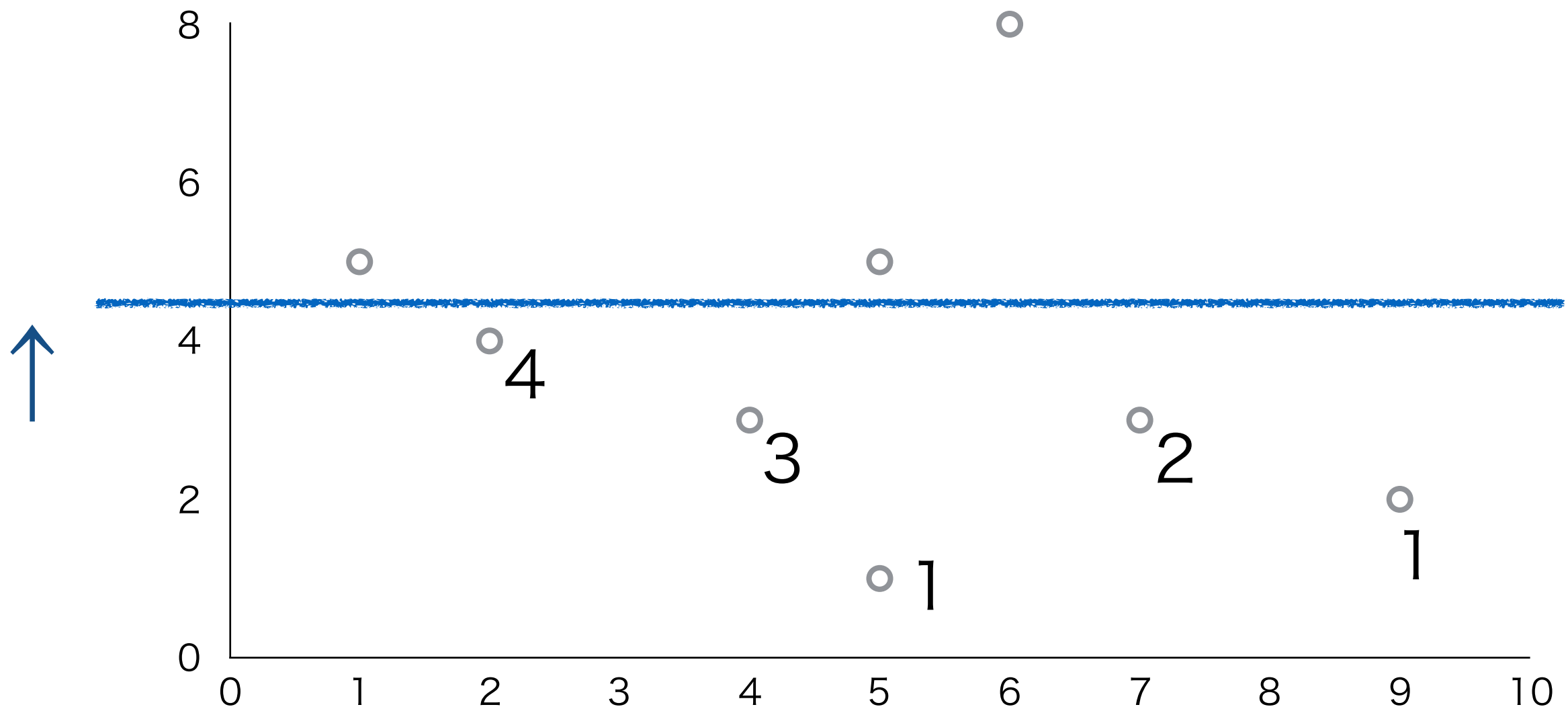
クエリ処理の例

- ・ 赤い範囲にクエリが来たら 2 が答えです



クエリ処理の例

- ・ 走査線を上に動かして、新しい点を追加します



LIS 高速化 まとめ

- ・ クエリと点の y 座標が小さい順 (同じ時は点の追加が先、点は x 座標が大きい順) にソートしておく
- ・ ある範囲の値の最大値を求める
- ・ ある場所を更新する
- ・ という 2 種類ができれば良い

LIS 高速化 まとめ

- ・ クエリと点の y 座標が小さい順 (同じ時は点の追加が先、点は x 座標が大きい順) にソートしておく
- ・ ある範囲の値の最大値を求める
- ・ ある場所を更新する
- ・ という 2 種類ができれば良い → **座圧 Segtree**

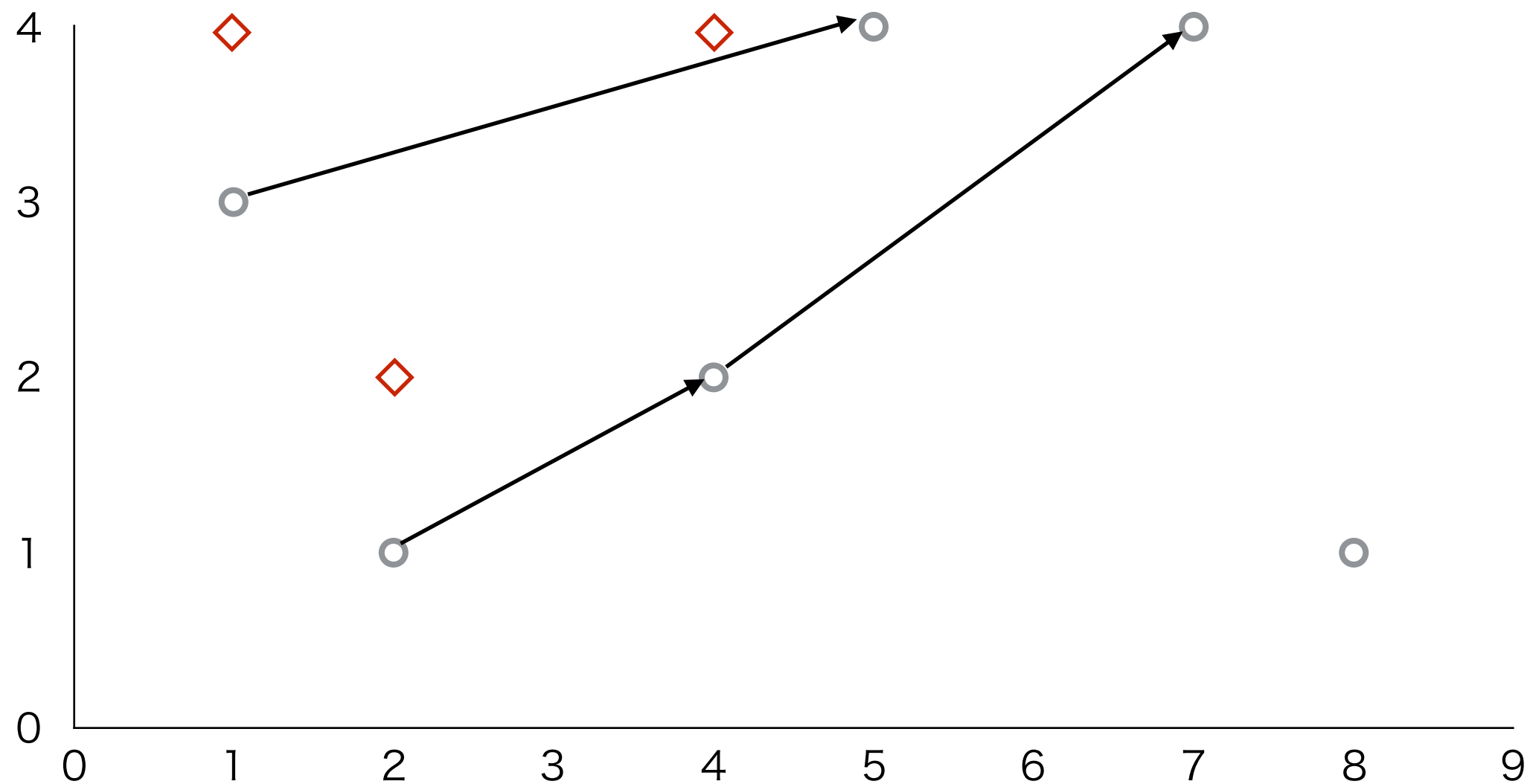


クエリをまとめる

- ・ 次に Greedy 版をまとめて求めてみましょう
- ・ 先ほどの Greedy で入れ子になる組を事前に求めて、(R_i の小さい方, H_i の大きい方) に重み -1 の点を追加します
- ・ 今度は x 座標が**大きい**順に走査していきます

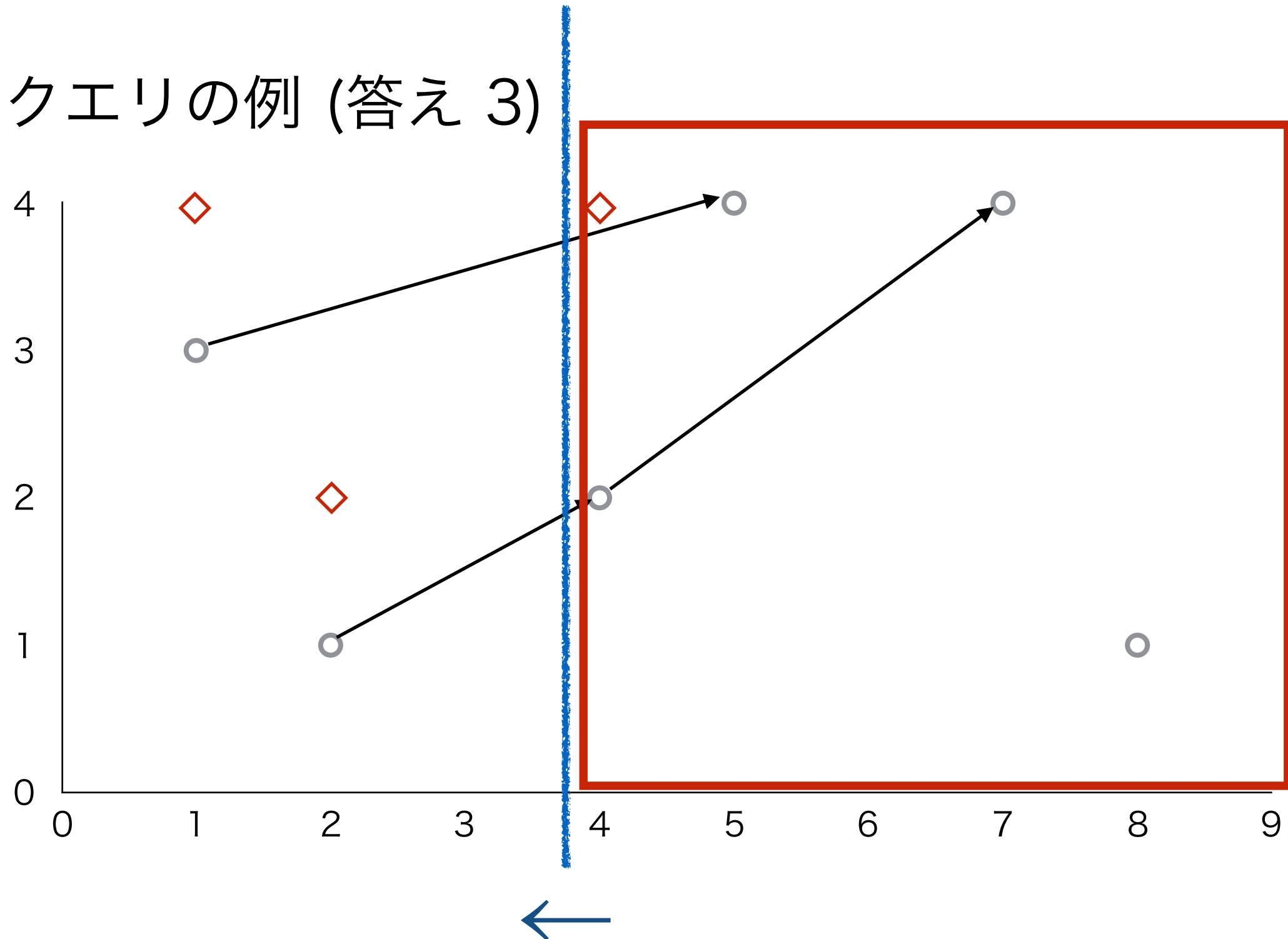
小課題4 例

- 平面の例 (赤い点が -1 の重み)



小課題4 例

・ クエリの例 (答え 3)



Greedy 高速化 まとめ

- ・ クエリと点の x 座標が大きい順 (同じ時は点の追加が先) にソートしておく
- ・ ある範囲の値の合計を求める
- ・ ある場所に値を足す
- ・ という 2 種類ができれば良い →

Greedy 高速化 まとめ

- ・ クエリと点の x 座標が大きい順 (同じ時は点の追加が先) にソートしておく
- ・ ある範囲の値の合計を求める
- ・ ある場所に値を足す
- ・ という 2 種類ができれば良い → **座圧 BIT**



上手な解法

- ・ 実は LIS 解法で LIS に特有な $O(N \log N)$ 解法 (二分探索するやつ) を使うと、`std::upper_bound` だけで満点を取ることができます

クエリはソート
します



得点分布



神經衰弱 (Memory2) 解説

三谷庸

問題概要

- $2N$ 枚のカードがあり、書かれている整数 $A_0, A_1, \dots, A_{2N-1}$ は $0, 0, 1, 1, \dots, N-1, N-1$ の並べ替えである
- $\text{Flip}(i, j)$ を呼び出すと、 A_i, A_j のうち「覚えやすい方」を教えてもらえる
 - 「覚えやすさ」は $0, 1, \dots, N-1$ の並べ替え $P_0, P_1, \dots, P_{N-1}$ として与えられる
- カードに書かれている整数を特定せよ

前置き (1)

- これは Reactive 型課題です
- 与えられる関数の仕様などは注意深く読みましょう
- 必要なら grader を書きかえてテストしましょう

前置き (2)

- 以下のようにしておくとお余計な Flip 呼び出しはしなくて済みます

```
int memo[MAX_N * 2][MAX_N * 2]; //-1 で初期化
```

```
int flip(int i, int j){  
    if(memo[i][j] != -1) return memo[i][j];  
    return memo[i][j] = memo[j][i] = Flip(i, j);  
}
```

小課題 1

- $P_i = i \ (0 \leq i < N)$
- $K = 10,000 \ (= (2N)^2)$

小課題 1

- すべてのペア (i, j) について $\text{Flip}(i, j)$ を呼ぶことができる
- I を固定した時各 j について $\text{Flip}(I, j)$ を呼んで A_I の値を特定できないか

小課題 1

- $\text{Flip}(I, j)$ の値は
 - $A_I < A_j$ のとき、 A_I
 - $A_I \geq A_j$ のとき、 A_j
- つまり、どの j についても $\text{Flip}(I, j) \leq A_I$ が成り立ち、等号が成立する j が存在する
- A_I の値はありえる $\text{Flip}(I, j)$ の値の最大値
- $2N(2N - 1)$ 回の Flip 呼び出しで全部わかる

小課題 2

- $P_i = i \ (0 \leq i < N)$
- $K = 400 \ (= 8 N)$

小課題 2

- 各カード i について、 $\text{Flip}(i, j)$ を呼び出す j の個数を定数に抑えたい

小課題 2

- 最も覚えにくいカードを特定しようとしてみる
- 順番にカードを見ていき、覚えにくいカードを定数個だけ残す

小課題 2

- カード $0, 1, \dots, i$ から 2 つ選んで Flip を呼んだときの戻り値の最大値を x とする
- x は $A_0, A_1, \dots, A_i$ のうちで 2 番目に大きい値である

小課題 2

- 以下の条件を満たすように集合 s_i を更新していく
 - カード $0, 1, \dots, i$ のうちで大きい方から 2 つは集合 s_i に含まれている
 - 大きい方から 2 番目よりも小さい値のカードは含まれない
- さらに、 s_i から 2 つとって Flip を呼んだときの戻り値も覚えておく
- 集合 s_i の大きさは 3 以下になる

小課題 2

- 初期状態

- $i = 2$ のとき $s_i = \{0, 1, 2\}$ または $i = 1$ のとき $s_i = \{0, 1\}$

小課題 2

- i のときわかっているとして、 $i + 1$ のときを計算する
- 各 $j \in s_i$ について $\text{Flip}(i + 1, j)$ を呼ぶ
- これらの値と s_i の内部での Flip の値から何がわかるか

小課題 2

- これらの値のうち最大のものを x とする
- x はカード $0, 1, \dots, i + 1$ のうちで大きい方から 2 番目の値である
- 例えば $\text{Flip}(a_i, i + 1) == x$ のとき、カード a_i とカード $i + 1$ はともに x 以上なので、 a_i と $i + 1$ は集合 s_{i+1} に含める

小課題 2

- これを繰り返していくと、カード $2N - 1$ まで処理が終わったとき、大きい方から 2 つ、すなわち $N - 1$ が書かれたカード 2 つの番号がわかる
- ここまで最大で $6N$ 回程度 Flip を呼んでいる
 - 初期化の部分の影響で実際にはこれよりも定数個少ないはずです

小課題 2

- 集合 s_i の大きさは最大 3 になりうる
 - 2 番目に大きい値のカードが 2 つあるとき

小課題 2

- $A_I = N - 1$ であったとする
- このとき、各 j について、 $\text{Flip}(I, j) = A_j$ となる
- 合計 $2N$ 回くらいの Flip 呼び出しによって A_j の値がすべてわかる

小課題 2

- 合計 $8N$ 回くらいの Flip 呼び出しで解ける

小課題 3

- $P_0, P_1, \dots, P_{N-1}$ は任意
- $K = 300 (= 6N)$

小課題 3

- Flip の戻り値のうち「大きい方」という考え方が意味を持たなくなった
- K が少し小さくなった
- 実は、小課題 2 の解法を少し変更するだけで良い

小課題 3

- 小課題 2 と同様に、覚えにくいカードを含むように 3 つ残していく
- 4 枚のカード i, j, k, l について $\text{Flip}(i, j)$ などの値がすべてわかっているとき何がわかるか詳しく考察

小課題 3

- A_i, A_j, A_k, A_l のうちで最も覚えやすい値を x とする
- 例えば $A_i = x$ であったとすると、 $\text{Flip}(i, j) = \text{Flip}(i, k) = \text{Flip}(i, l)$ となる

小課題 3

- 逆に、 $\text{Flip}(i, j) = \text{Flip}(i, k) = \text{Flip}(i, l)$ のとき、4 枚のカードのうちで最も覚えやすいものはカード i である
- さらに、 A_i の値は $\text{Flip}(i, j)$ の値に確定する

小課題 3

- これをふまえて小課題 2 と同様のことをやる
 - Flip の戻り値の大小とは関係なく覚えにくいカードを残していけるようになった

小課題 3

- a_i, b_i, c_i をカード $0, 1, \dots, i$ のうちで覚えやすい方から 3 つであるようにする
- 先ほどの議論から、カード $a_i, b_i, c_i, i + 1$ のうちで最も覚えやすいものはどのカードであるかと、その値がわかる
- 最も覚えやすいものをカードの値が確定したとして記録し、それ以外のカード番号を改めて $a_{i+1}, b_{i+1}, c_{i+1}$ とする

小課題 3

- 小課題 2 の最後のステップの、 $2N$ 回 Flip を呼んでそれぞれのカードの値を特定していく部分はやらなくて良くなった
- Flip 回数は合計 $6N$ 回

Flip 回数の改善 (1)

- 小課題 3 と同様のことをやる
- 「カード $0, 1, \dots, i$ から 3 つ残して、 $i + 1$ と比べる」ことを繰り返していた
- 3 つも残す必要はあるか？

Flip 回数の改善 (1)

- カードを 2 つしか残さないと、新たに追加したカードと合わせた 3 枚のカードでどれを残せばいいかわからないことがある(小課題 2 で説明したとおり)

Flip 回数の改善 (1)

- カードを $0, 1, \dots$ の順に見ていくのではなく、ランダムな順に見ていくとこのような「都合の悪い」ことはほぼ起こらない
- Flip 回数は $4N$ くらいになる
 - 最悪回数は $6N$
 - Flip が呼ばれたときこれまでの Flip の戻り値に矛盾しないように戻り値の値を変えることができれば $6N$ 回かかるようになる

Flip 回数の改善 (2)

- 小課題 3 の解法を少し改善すると Flip 呼び出しの最悪回数が $5N$ になる

Flip 回数の改善 (2)

- 以下の条件を満たすように各 i について 2 つのカード番号 a_i, b_i または 3 つのカード番号 a_i, b_i, c_i を持つ
 - $\text{Flip}(a_i, b_i) (= \text{Flip}(b_i, c_i) = \text{Flip}(c_i, a_i)) =$ カード $0, 1, \dots, i$ のうちで 2 番目に覚えない値
- i まで計算できているとして、 $i + 1$ のときをどのように計算するか考える
 - Flip を何回呼べば良いか?

Flip 回数の改善 (2)

- i までのときに覚えている番号が 2 つのときは 2 回 Flip を呼ぶだけで良い
- 覚えている番号が 3 つあるときも 2 回 Flip を呼ぶだけで結構いんなことがわかる

Flip 回数の改善 (2)

- $\text{Flip}(a_i, b_i) = \text{Flip}(b_i, c_i) = \text{Flip}(c_i, a_i) = x$ とする
- $\text{Flip}(a_i, i + 1) = y, \text{Flip}(b_i, i + 1) = z$ を求める

Flip 回数の改善 (2)

- (場合分け 1) $x \neq y$ かつ $x \neq z$ のとき
 - $y = z$ であり、カード $i + 1$ の値はこの値に等しいことがわかる
 - $a_{i+1}, b_{i+1}, c_{i+1}$ は a_i, b_i, c_i をそのまま使えばよい

Flip 回数の改善 (2)

- (場合分け 2) $x = y$ かつ $x \neq z$ のとき
 - カード a_i, c_i の値は x であり、 $a_{i+1} = b_i$, $b_{i+1} = i + 1$ (c_{i+1} は存在しない) とすればよいことがわかる

Flip 回数の改善 (2)

- (場合分け 3) $x = y = z$ のとき
 - カード a_i, b_i の値は x であり、 $a_{i+1} = c_i$, $b_{i+1} = i + 1$ (c_{i+1} は存在しない)とすればよいことがわかる

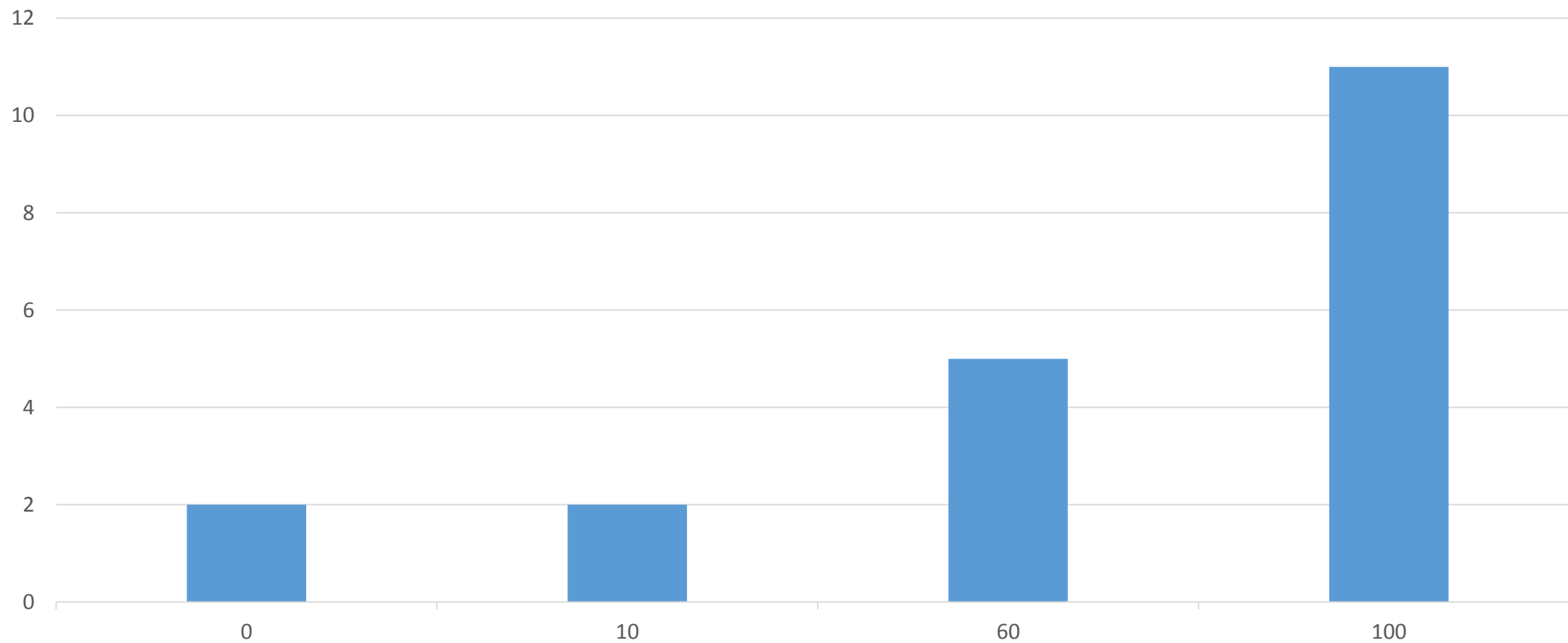
Flip 回数の改善 (2)

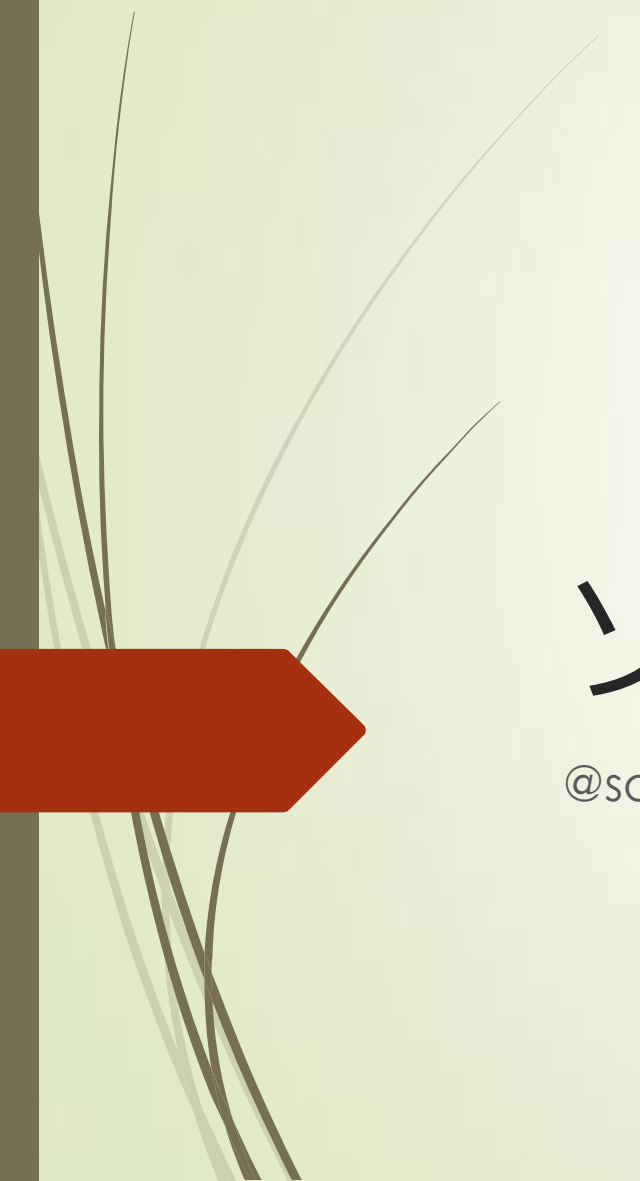
- 3 通りの場合分けとも、なぜそうなるかは考えてみてください
- i までの情報から $i + 1$ までの情報を計算するときの Flip の回数を num_i とする
- num_i は 2 または 3 であり、3 が連続することはないので Flip 回数の合計は $5N$ 回以下となる

別解

- 例えば、値が未確定のカードから 1 枚選び、他の未確定のすべてのカードと Flip することを繰り返す
 - 結果が x であるカード番号がちょうど 2 つのとき、 x が書かれたカード番号が定まる
- Flip 回数の期待値は $4N$ くらい
 - N 組 $2N$ 枚のときの期待値を $f(N)$ とすると $f(N) = 2N - 1 + \frac{1}{N}(\sum_{1 \leq i \leq N} f(i))$ となるから
- 他にもいろいろあるかもしれません

得点分布





ソリティア

@sortreew

問題概要

- $3 \times N$ マスの盤面があり、いくつかのマ스에コマが置かれている。 ($N \leq 2000$)
- 左右両方もしくは上下両方にコマが置かれている、コマのおかれていないマスにコマを置く。
- この操作をコマが置けなくなるまで繰り返す。
- すべてのコマを置くまでの手順は何通り存在するか。

例

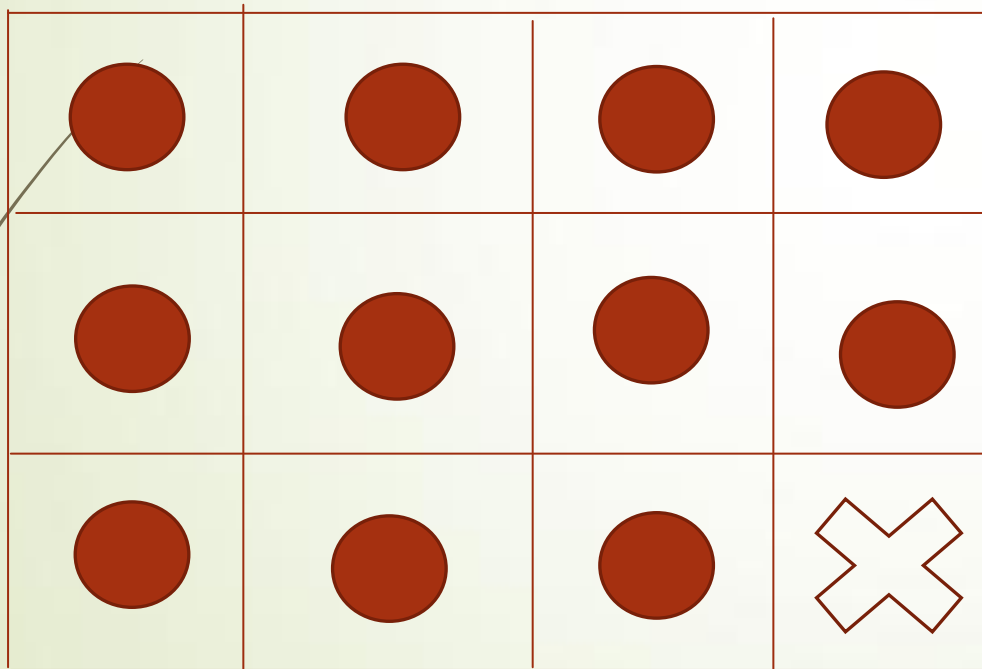
●	●	1	●	5	●	●	●
●	3	2	●	6	7	8	●
●	4	●	10	●	●	9	●

置ける場所

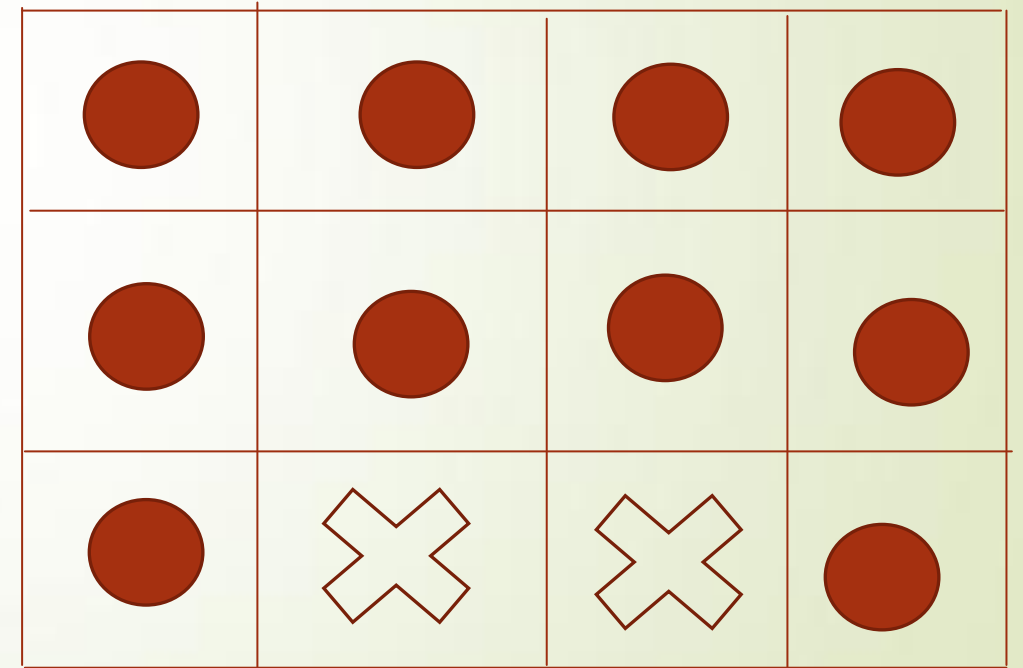
●	○	●	×
○	×	×	×
●	×	×	●

0通りの場合

→ すべての石を置けない場合が存在！



四隅に空白



1 or 3行目に横に連続した空白



0通りの場合

これらの場合を取り除いて考える。



ちよい考察

0通りの場合でないとき、
1行目および3行目の空白の左右はコマが置かれている
(それはそう)

●		●		●	●		●
●	●	●	●	●	●	●	●
●		●		●		●	●

小課題1

■ 空白が16個以下

盤面の状態は高々 $2^{16} = 65536$ 通り

というわけでDP[盤面の状態]でbitDPすれば間に合う

$O(N \cdot 2^N)$ でも間に合うので余裕

全探索は $16!$ がきびしいのできびしい

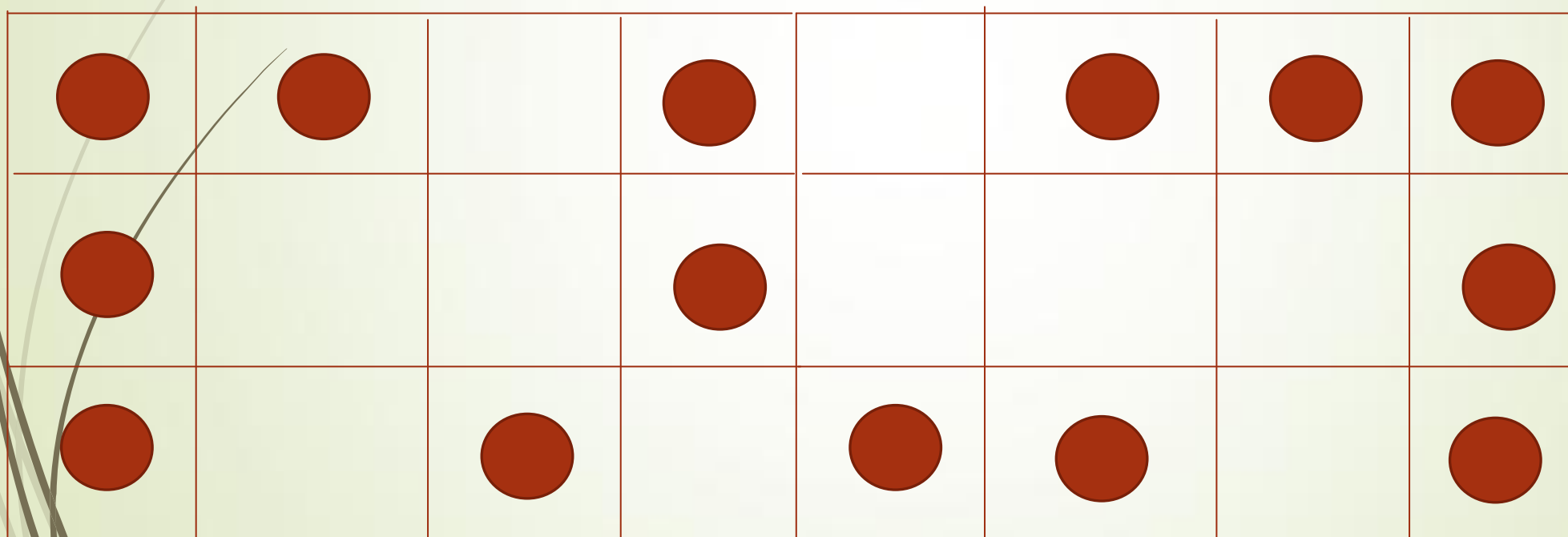


小課題2のための考察

連結でない空白は置けるかどうかに関与しない

小課題2のための考察

連結でない空白は置けるかどうかに関与しない





小課題2のための考察

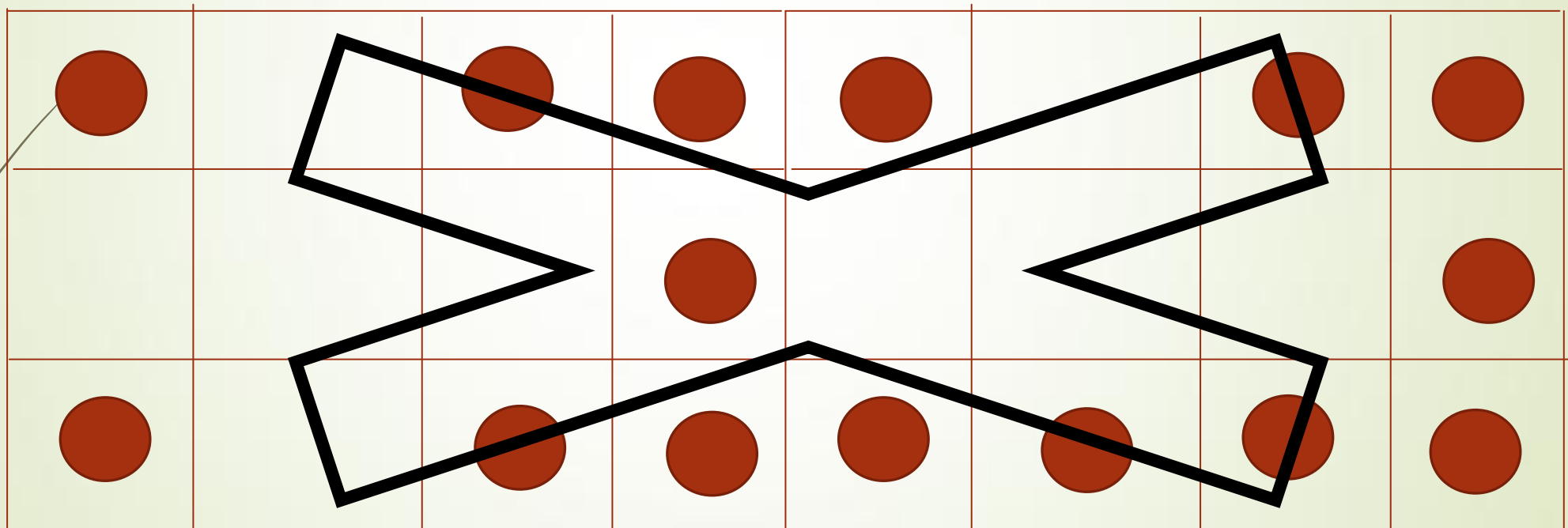
連結でない空白は置けるかどうかに関与しない

なので、それぞれの連結した空白について考えられる
(答えはさておき)

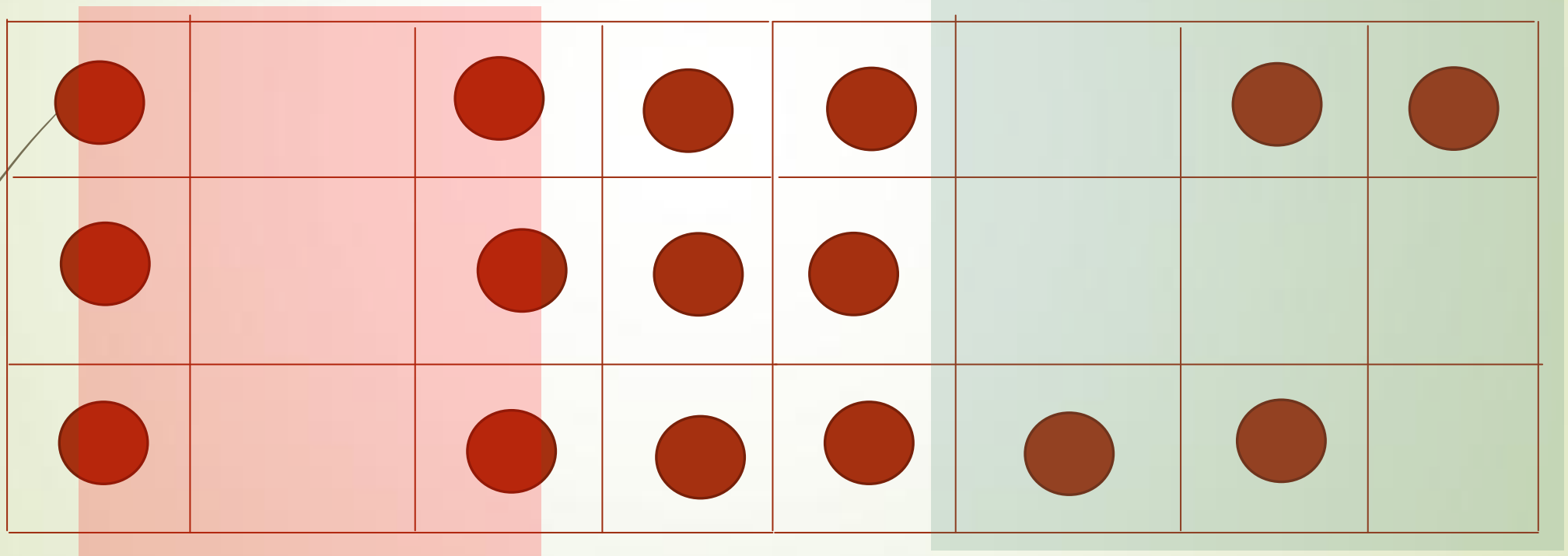
あと、石を置いても他に影響が出ない場所は連結して
いないものとする

小課題2

■ 小課題2では空白に隣接する空白は高々2つ

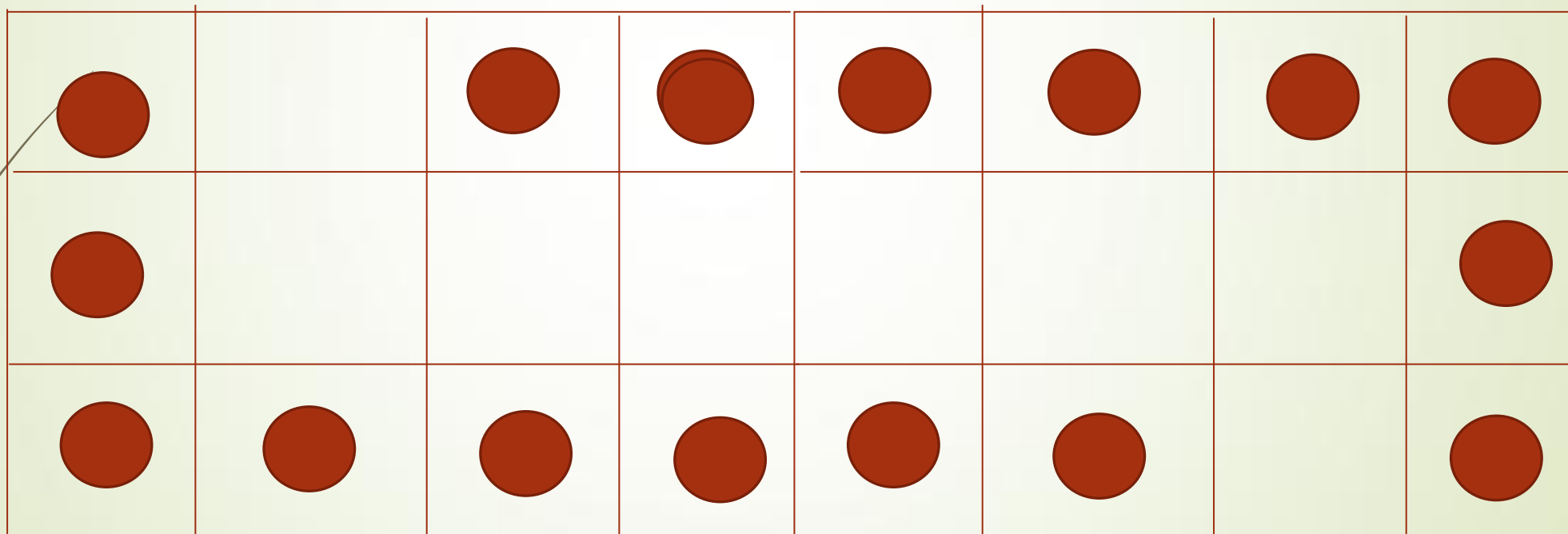


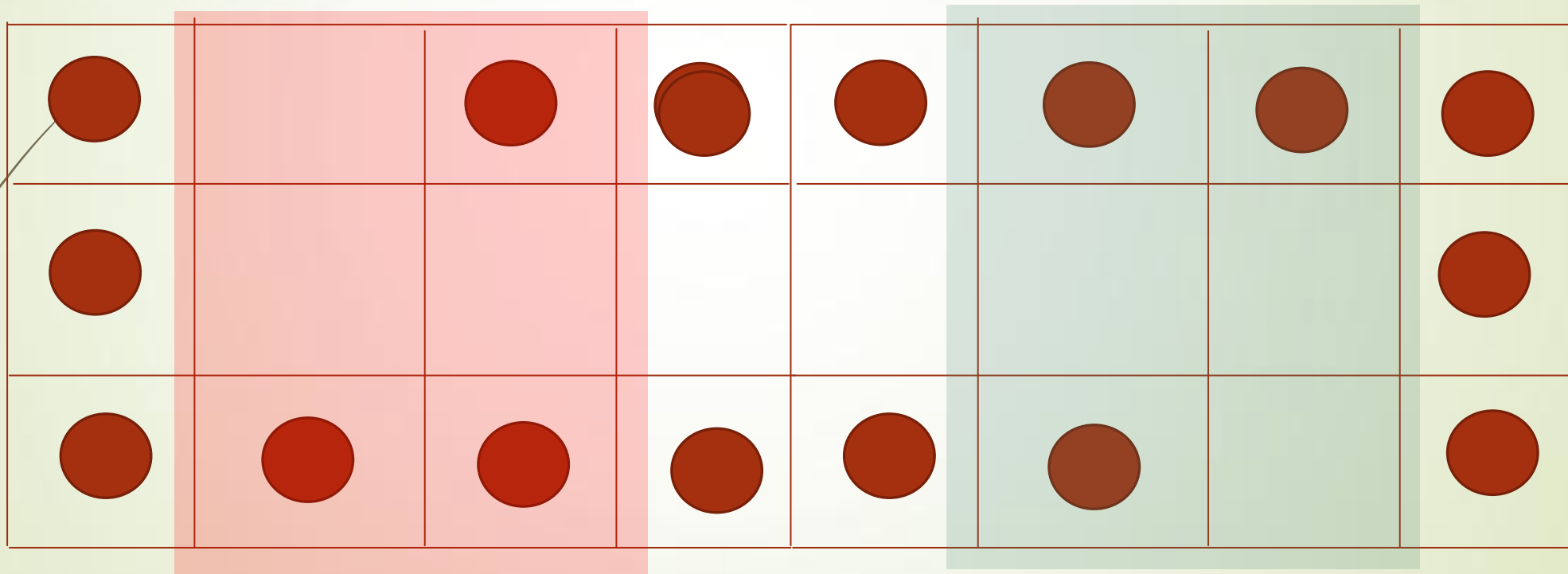
小課題2でありうるパターン



小課題2でありうるパターン

すごい広いように見えるが・・・？



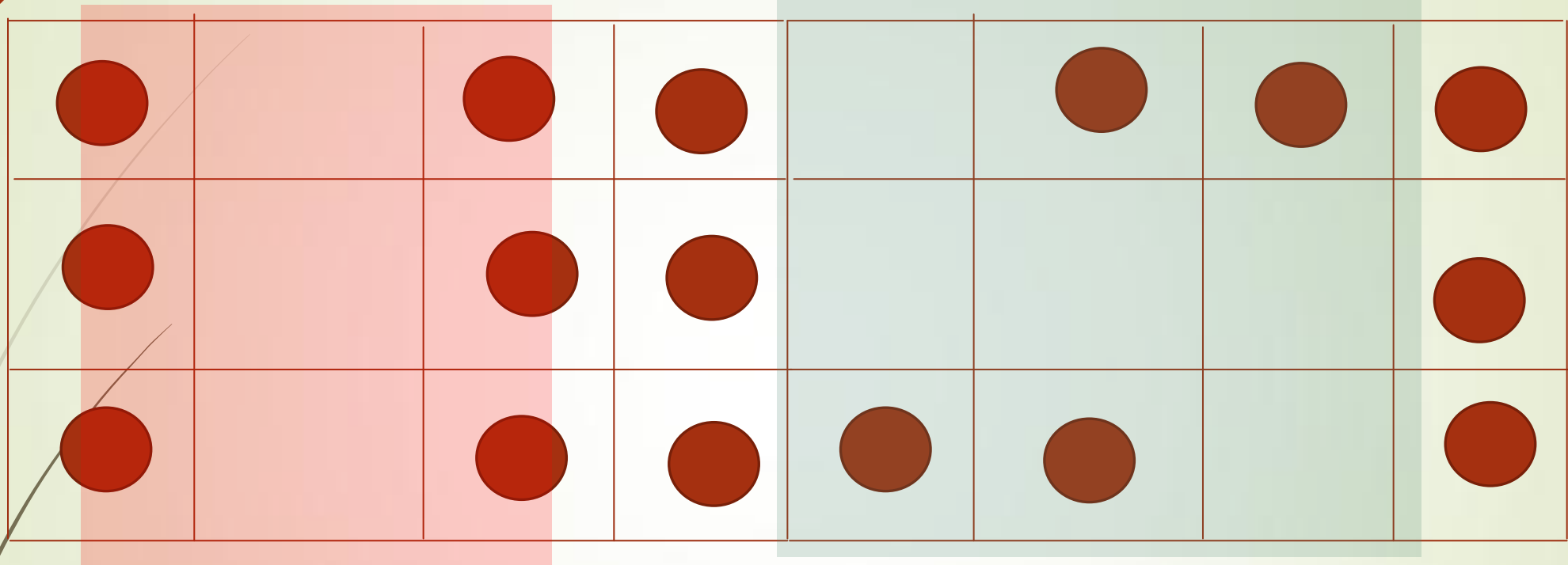




小課題2

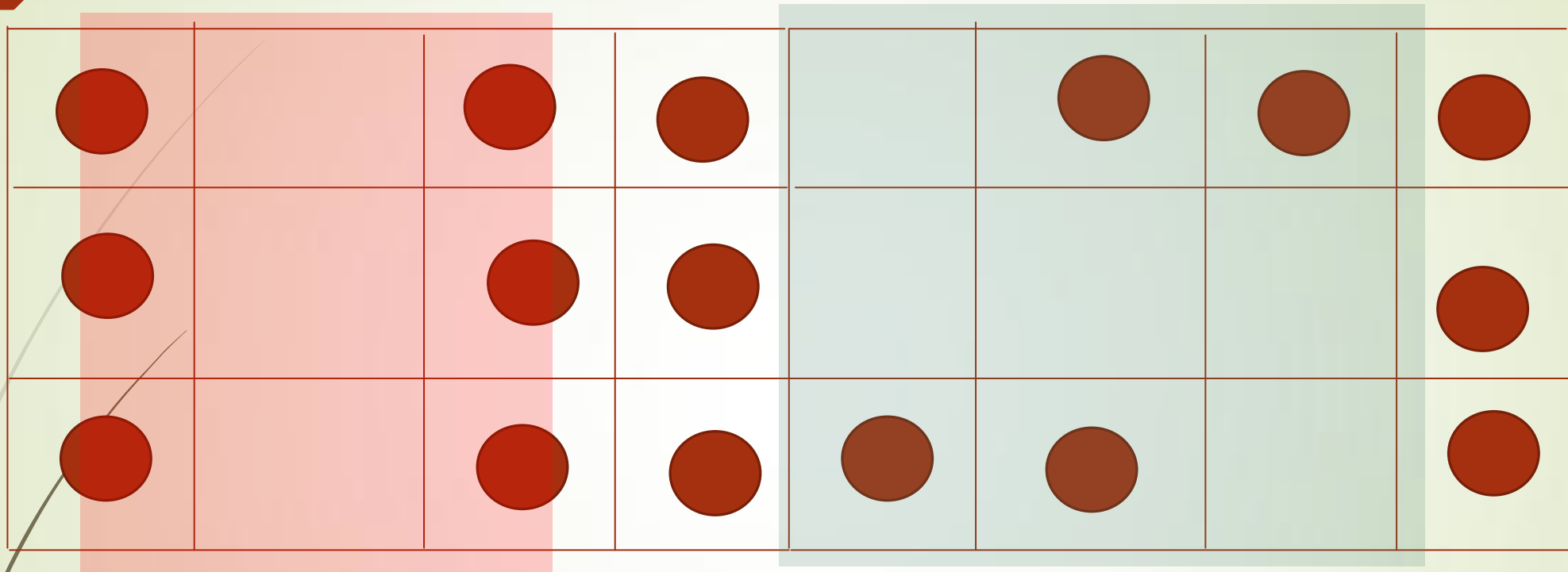
- 小課題2では、依存関係にある領域はそんなに大きくならない（高々5箇所）ので、それぞれ別個に考えてやればよい

計算



全体の操作回数は8回、赤い領域に石を置く回数は3回、青い領域には5回

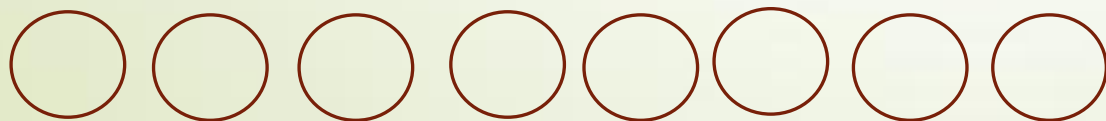
計算



8回のうち赤い領域に置く番を決める。

赤い領域の中で順番を決める

青い領域で同様のことを行う





計算

- 実際にはもっと領域の数が多いが、一つの領域について手番の選び方を数える（組み合わせ）→領域の中での順番の数を数える（全探索とかbitDPとか）を繰り返して掛けあわせていけばよい
- 組み合わせはパスカルの三角形を用いたDPで問題ない

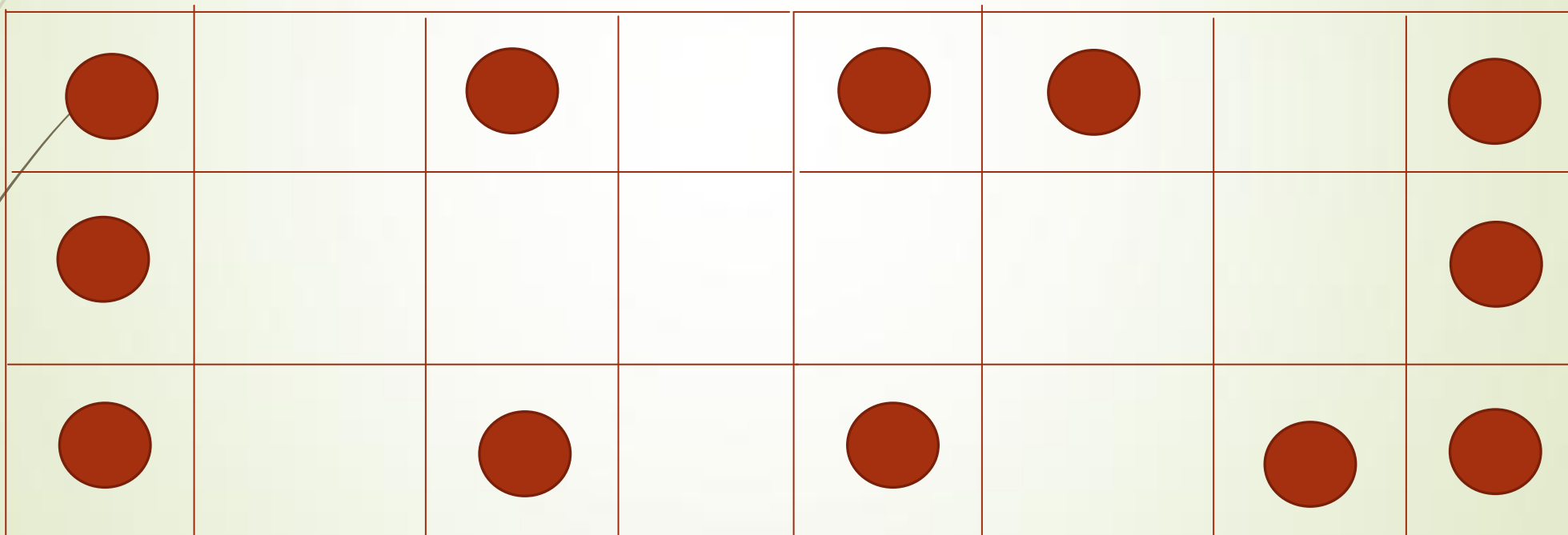


小課題4,5

- 小課題5では、連結領域が

小課題4,5

➡ 小課題5では、連結領域がこんなものになる





ちよい考察

連結でない空白は置けるかどうかに関与しない
これはそのまま

しかし、連結した空白の数は小課題2と比べ膨大な大きさになる
(数千とか)

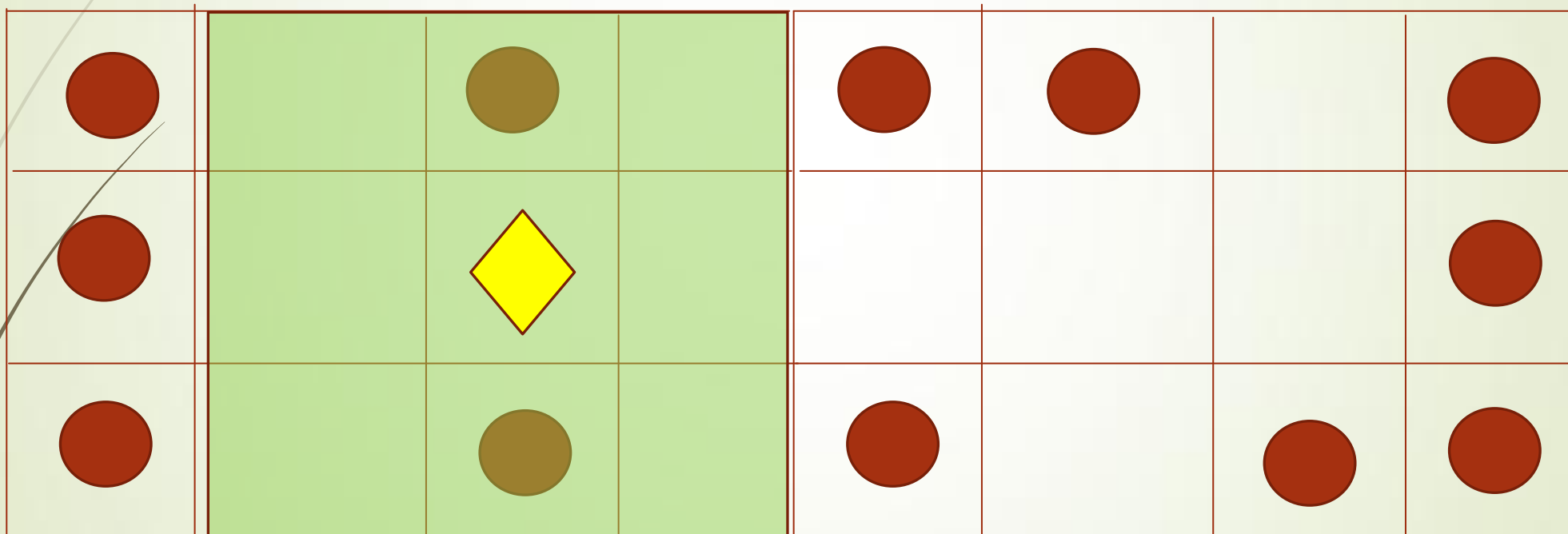
→全探索やbitDPではきびしい！



考察 1 2行目の空白について

2行目の空白はタテヨコ両方どちらでも挟められる
しかし、横に連続した2個以上の空白を両方ヨコで挟める場合は存在しない。

考察2 置く順の依存について

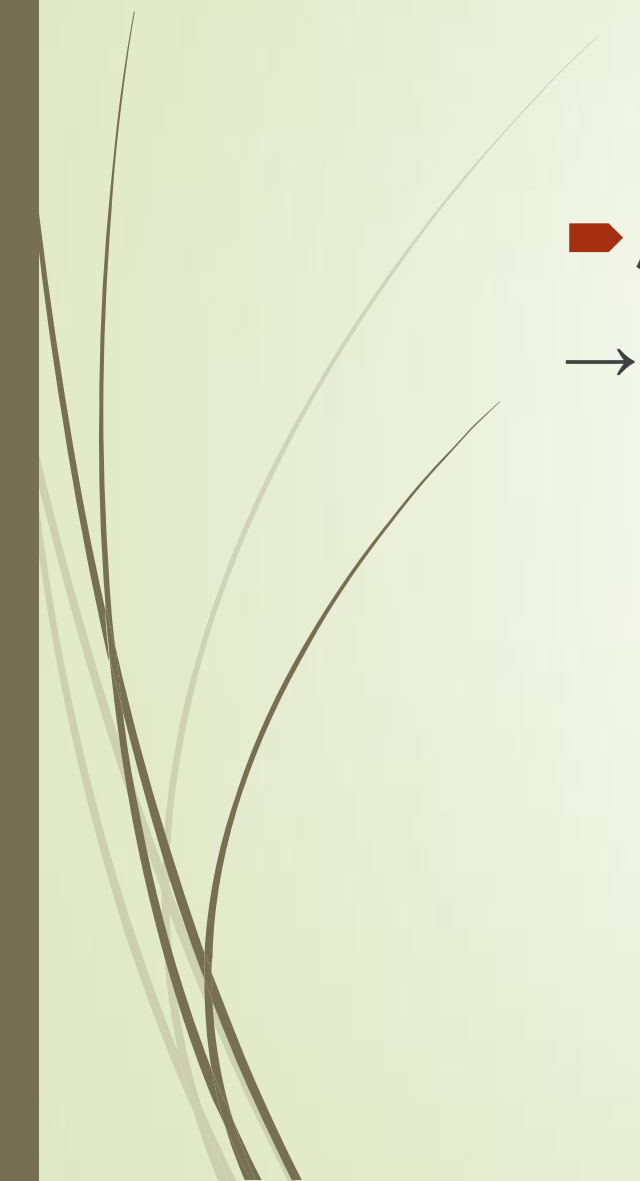


あるマスの置き方に影響する・されるマスは
高々そのマスの周囲1マス



■ 周囲の情報をうまく持てばDPできそう？



- 
- 
- 周囲の情報をうまく持てばDPできそう？
→ 実際できる

実際の解法

- 大まかな方針は小課題2と同じ
一つの領域について手番の選び方を数える（組み合わせ）→領域の中での順番の数を数える（全探索とかbitDPとか）を繰り返して掛けあわせる。

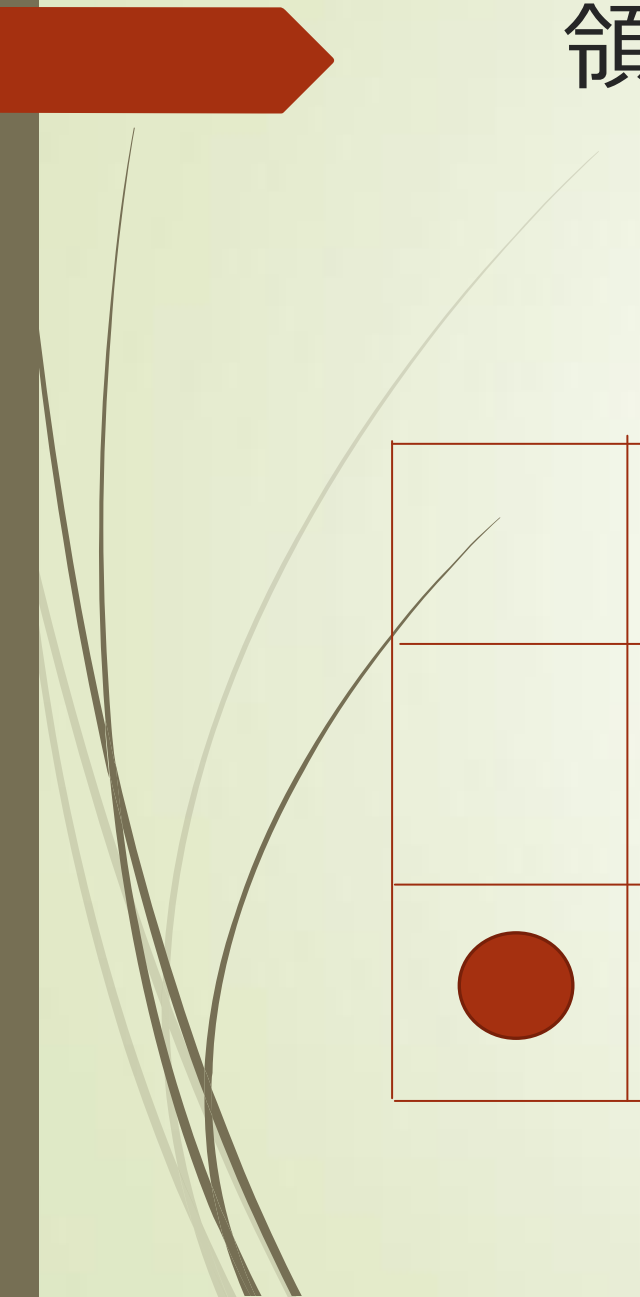


実際の解法

組み合わせがNが数千でも問題ない
順番の数については考える必要がある

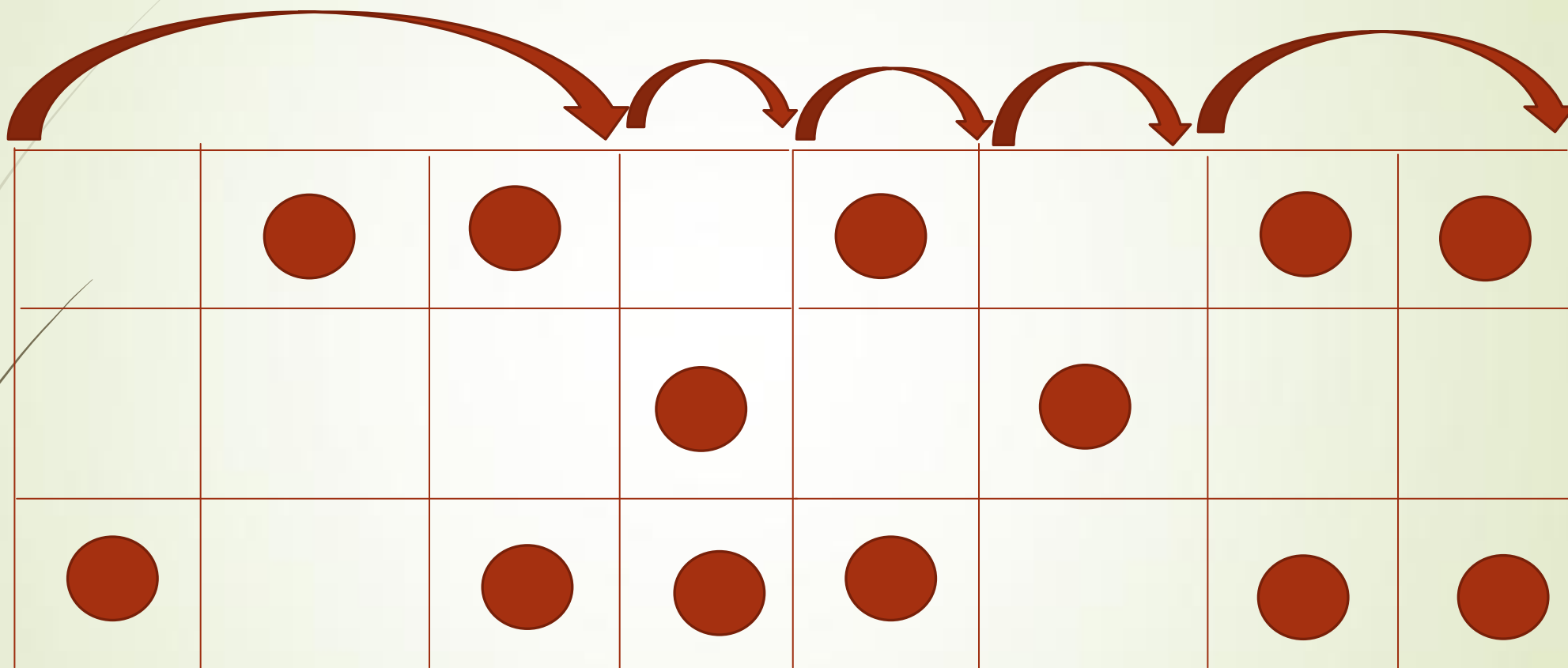


領域の分け方



	●	●		●	●	●	●
			●		●		
●		●	●	●	●	●	●

領域の分け方



2行目の空白が連続している列によって領域を区切る
2行目の空白が存在しない列は1列で独立している



2行目にコマがある列について

- 2行目にコマがある場合、その列単独で考えればよい。
- 列の空白が0,1つのとき、領域内での順番の数は1通り
- 列の空白が2つのとき、領域内での順番の数は2通り



2行目にコマがない列（領域）について

動的計画法を用いる

$DP[i][j][k]$

領域ははじめから*i*列目までの盤面としたとき、

*i*列目の2行目のコマが列の中で

(*k*?最後以外:最後)かつ全体*j*番目に置かれる場合の数(*i*+1列目に空白はないとする)

(最後以外・最後は横・縦で考えるとよい)

初期条件

- a列目からb列目までの領域について
a列目の空白がc個あるとき
DP[a][j][1]は $1 \leq c < j$ なら cPc 通り
DP[a][j][0]は $c=j$ なら cPc 通り
それ以外は0通り



遷移

2行目が最後るとき、

2行目 左右について特に制約はない

1,3行目 2行目よりも前にコマを置く必要がある

1つ前の2行目 最後以外なら今の2行目より後に置く必要がある



遷移

2行目が最後以外するとき、

2行目 左の2行目のコマよりも後である必要がある

1,3行目 両方とも2行目より前はだめ（片方だけならOK）

1つ前の2行目 最後以外の場合は不可

遷移

```
for (i, a...b) {  
    c = 領域内のi列目の空白の数;  
    d=領域内のi-1列目までの空白の数;  
    for (j, 0...d) {  
        for (k, 1...c) {  
            if (k == c)  
                DP[i][j + k][0] += nPr (c - 1, c - 1)*nCr (j + k - 1, k -  
1)*(DP[i-1][0...d][0] +DP[i-1][j+1...d][1]);  
            else  
                DP[i][j + k][1] += nPr (c - 1, c - 1)*nCr (j + k - 1, k -  
1)*nCr (c+d - (j + k), c - k)*DP[i-1][0...j][0];  
        }  
    }  
}
```



そのほか

modはちゃんと取ろう

DPの総和を取ってくる部分を高速化しないと $O(N^3)$ となってしまう、満点が取れなくなります。

小課題3はよくわかりません

雇用計画 (Employment)

@joisino_

March 21, 2016

目次

問題概要

小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

目次

問題概要

小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

問題概要

- ▶ 数列が与えられる
- ▶ 以下のクエリを処理せよ
 - ▶ B_i 以上の数のみを考えたとき連続する部分の数を求めよ
 - ▶ 数列の値を変更せよ

サンプル入出力

8

6

3

5

4

サンプル入出力

8

採用

6

採用

3

不採用

5

採用

4

不採用

- ▶ 評価値が 5 以上の候補者を採用する
- ▶ 連続部分は 2 個

サンプル入出力



- ▶ 4 番目の候補者の評価値を 1 に変更する

サンプル入出力

8

採用

6

採用

3

不採用

1

不採用

4

不採用

- ▶ 評価値が 5 以上の候補者を採用する
- ▶ 連続部分は 1 個

サンプル入出力



- ▶ 評価値が 3 以上の候補者を採用する
- ▶ 連続部分は 2 個

目次

問題概要

小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

小課題 1 : 10 点 (累計 10 点)

▶ $N \leq 2000$

▶ $M \leq 2000$

小課題 1 : 10 点 (累計 10 点)

- ▶ シミュレーションをやるだけ
- ▶ クエリごとに誰が採用されるかを求めて，連続部分の個数を求める
- ▶ $O(NM)$ でできる

目次

問題概要

小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

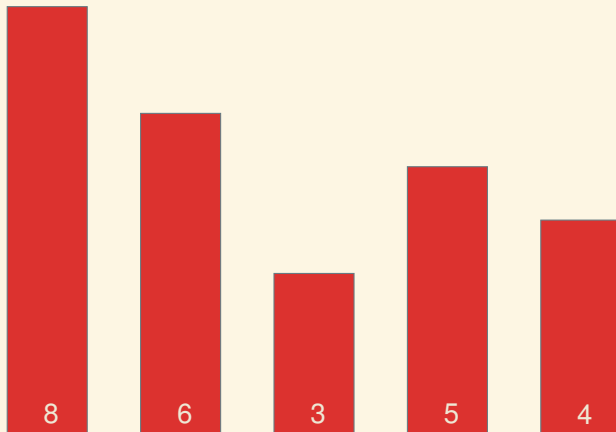
小課題 2 : 30 点 (累計 40 点)

- ▶ $T_i = 1$ (解答クエリのみ)
- ▶ $N \leq 200\,000$
- ▶ $M \leq 200\,000$

小課題 2 : 30 点 (累計 40 点)

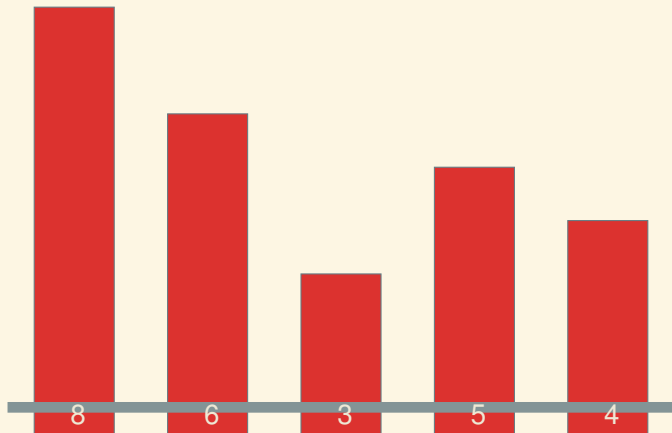
- ▶ 解答クエリを観察してみる

小課題 2 : 30 点 (累計 40 点)



- ▶ わかりやすい図
- ▶ 高さが評価の大きさを表す

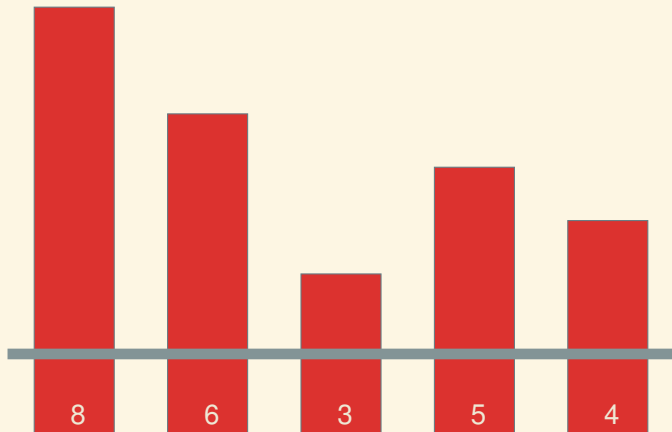
小課題 2 : 30 点 (累計 40 点)



▶ $B = 1$ のとき

▶ $ans = 1$

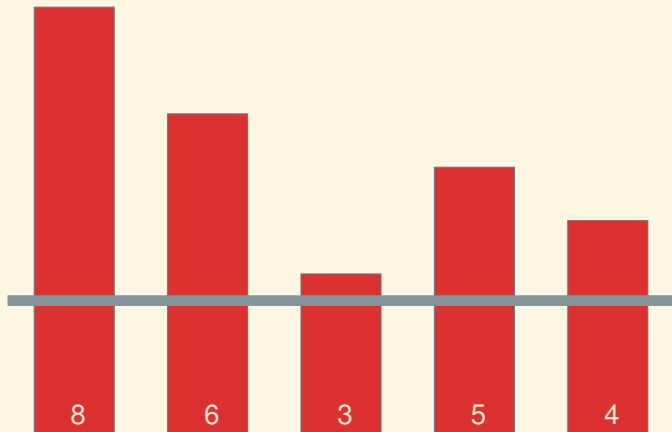
小課題 2 : 30 点 (累計 40 点)



▶ $B = 2$ のとき

▶ $ans = 1$

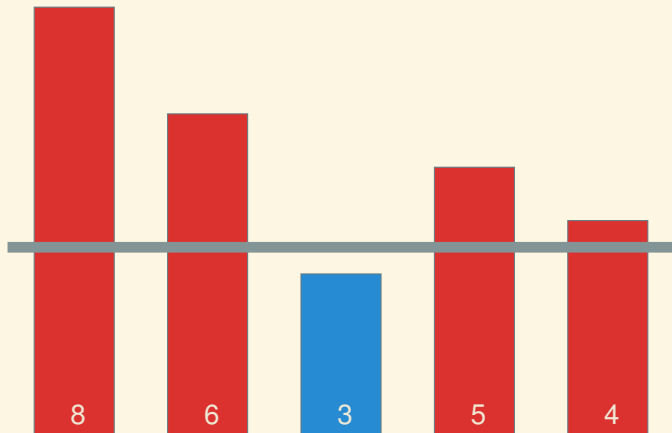
小課題 2 : 30 点 (累計 40 点)



▶ $B = 3$ のとき

▶ $ans = 1$

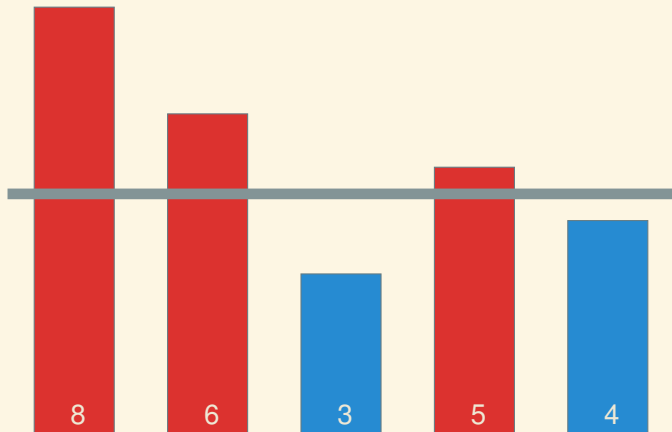
小課題 2 : 30 点 (累計 40 点)



▶ $B = 4$ のとき

▶ $ans = 2$

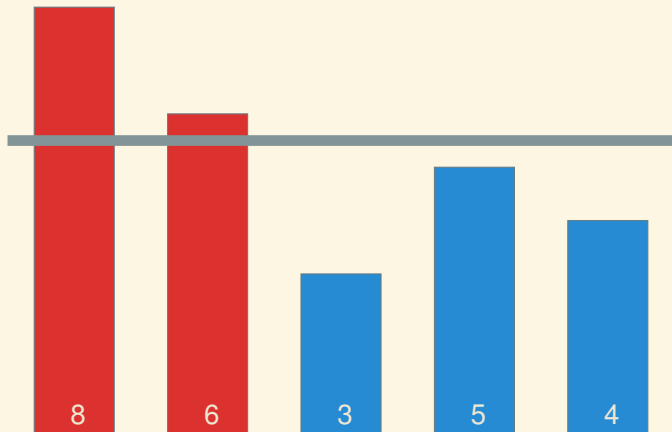
小課題 2 : 30 点 (累計 40 点)



▶ $B = 5$ のとき

▶ $ans = 2$

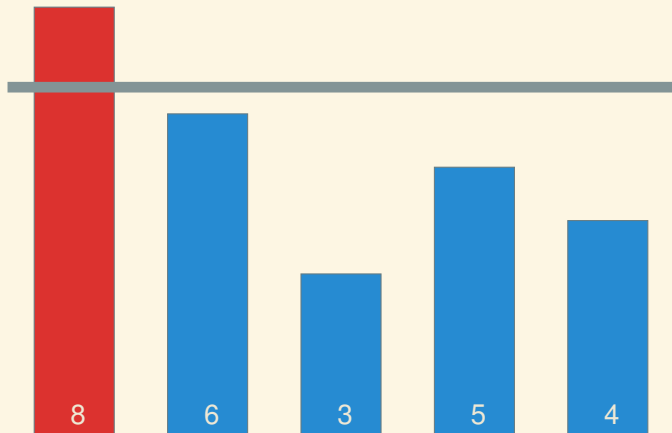
小課題 2 : 30 点 (累計 40 点)



▶ $B = 6$ のとき

▶ $ans = 1$

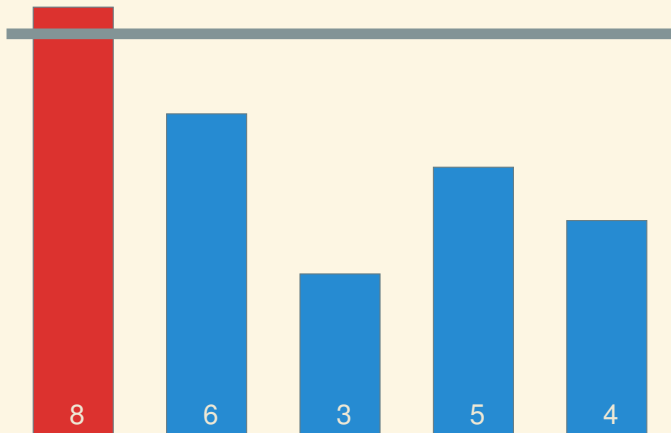
小課題 2 : 30 点 (累計 40 点)



▶ $B = 7$ のとき

▶ $ans = 1$

小課題 2 : 30 点 (累計 40 点)



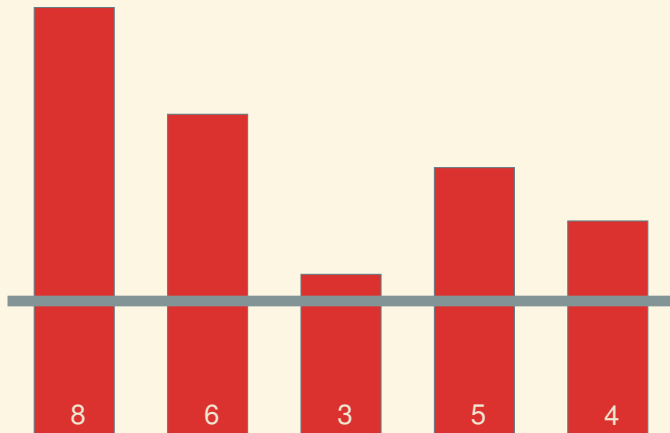
▶ $B = 8$ のとき

▶ $ans = 1$

小課題 2 : 30 点 (累計 40 点)

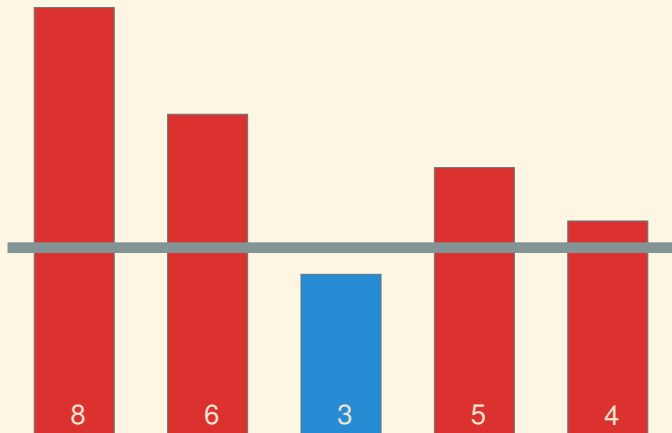
- ▶ 採用基準の小さい方からクエリをみていくと，採用される人は減っていく
- ▶ 採用される人が減ったときに答えがどう変わるかを計算すればよい

小課題 2 : 30 点 (累計 40 点)



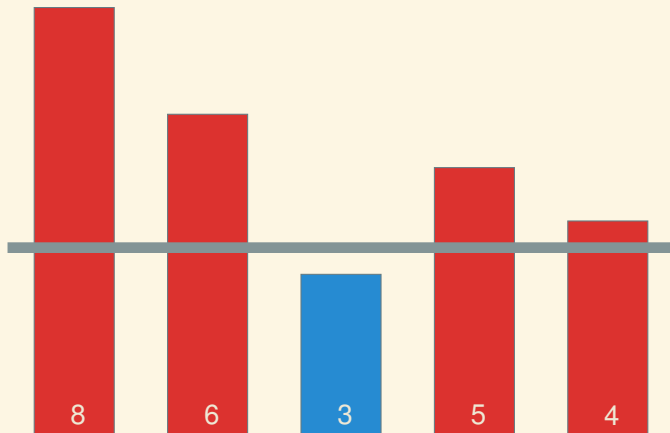
▶ 答えが増えるタイミング

小課題 2 : 30 点 (累計 40 点)



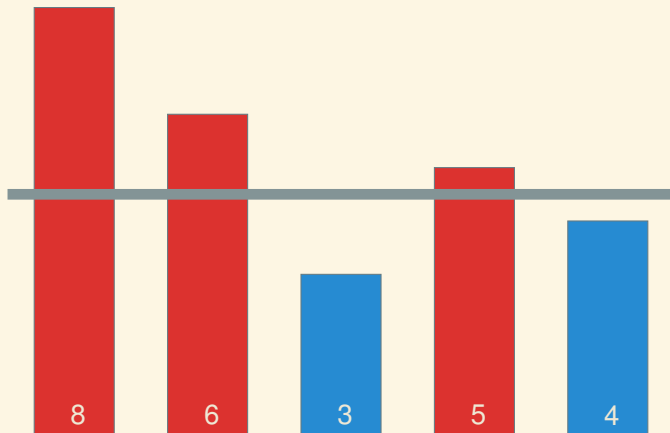
- ▶ 3 人目が不採用になることで答えが 1 増えた

小課題 2 : 30 点 (累計 40 点)



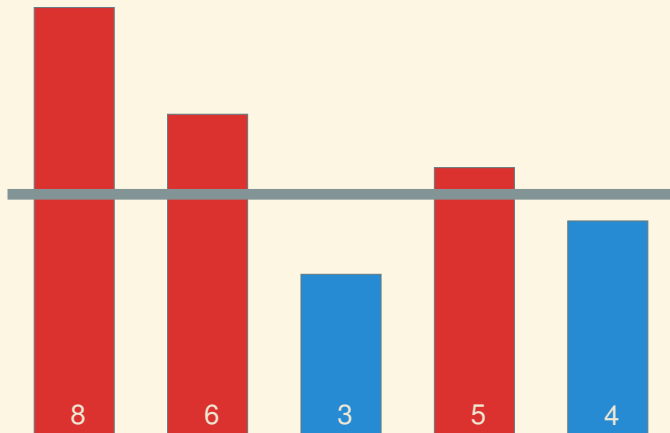
▶ 答えが変わらないタイミング

小課題 2 : 30 点 (累計 40 点)



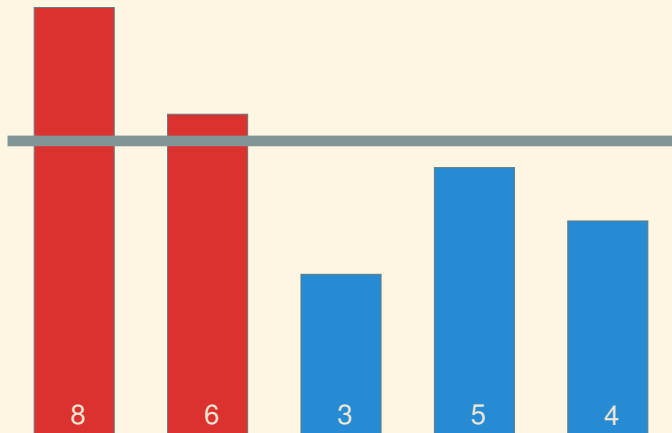
- ▶ 5 人目が不採用になっても答えは変わらなかった

小課題 2 : 30 点 (累計 40 点)



▶ 答えが減るタイミング

小課題 2 : 30 点 (累計 40 点)



- ▶ 4 人目が不採用になることで答えが 1 減った

小課題 2 : 30 点 (累計 40 点)

- ▶ 両隣が採用されているときに不採用になると，答えが 1 増える
- ▶ 両隣が不採用なときに不採用になると，答えが 1 減る
- ▶ 両隣のうち一方が採用，一方が不採用のときに不採用になると，答えは変わらない
- ▶ ただし 0 番目と $N + 1$ 番目には不採用の人がいるとする

小課題 2 : 30 点 (累計 40 点)

- ▶ クエリと候補者のソートに $O((N + M) \log(N + M))$
- ▶ 不採用になるのは合計 N 回 で毎回 $O(1)$ で処理できる.
- ▶ 答えが計算できているとクエリに答えるのは $O(1)$ でできる.
- ▶ よって, 全体の計算量は $O((N + M) \log(N + M))$
- ▶ 小課題 2 が解ける

目次

問題概要

小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

小課題 3 : 60 点 (累計 100 点)

- ▶ 追加の制約はない

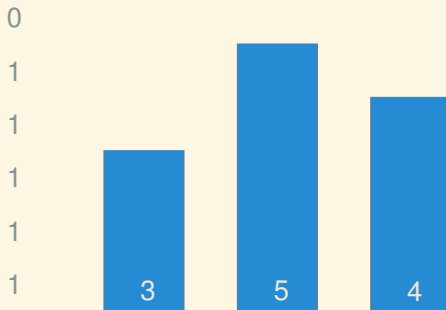
- ▶ $N \leq 200\,000$

- ▶ $M \leq 200\,000$

小課題3 : 60 点 (累計 100 点)

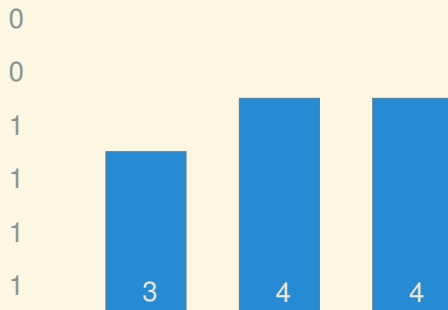
- ▶ 更新クエリを観察してみる

小課題3 : 60 点 (累計 100 点)



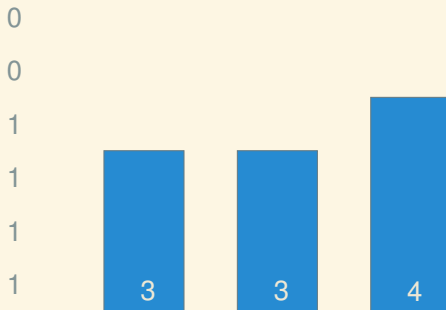
- ▶ 左側の数字はそこでの解答クエリの値
- ▶ 少しずつ真ん中の人の評価を減らしていきましょう

小課題3 : 60点 (累計100点)



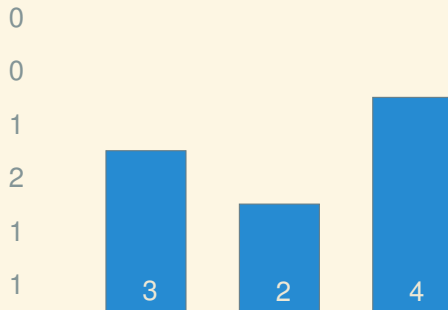
- ▶ 左側の数字はそこでの解答クエリの値
- ▶ 少しずつ真ん中の人の評価を減らしていきましょう

小課題 3 : 60 点 (累計 100 点)



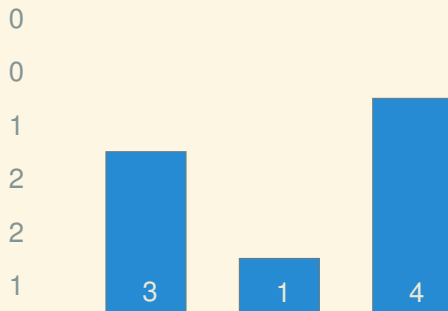
- ▶ 左側の数字はそこでの解答クエリの値
- ▶ 少しずつ真ん中の人の評価を減らしていきましょう

小課題3 : 60 点 (累計 100 点)



- ▶ 左側の数字はそこでの解答クエリの値
- ▶ 少しずつ真ん中の人の評価を減らしていきましょう

小課題3 : 60 点 (累計 100 点)

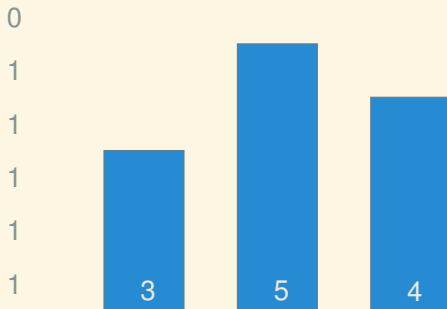


- ▶ 左側の数字はそこでの解答クエリの値
- ▶ 少しずつ真ん中の人の評価を減らしていきましょう

小課題 3 : 60 点 (累計 100 点)

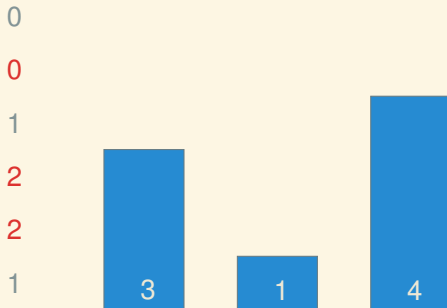
- ▶ 一人の評価を変更しても，解答クエリの値は高々 1 しか変わらない

小課題3 : 60点 (累計100点)



▶ 一気に減らすとこんな感じ

小課題3 : 60点 (累計100点)



▶ 一気に減らすとこんな感じ

小課題 3 : 60 点 (累計 100 点)

- ▶ どのように変化するのか
- ▶ 小課題 2 の考察と同様に考えると分かる
- ▶ 評価値が増加するときには、左右が共に存在するときに答えが 1 減少する
- ▶ 評価値が増加するときには、左右がどちらも存在しないときに答えが 1 増加する
- ▶ 評価値が減少するときはこの逆

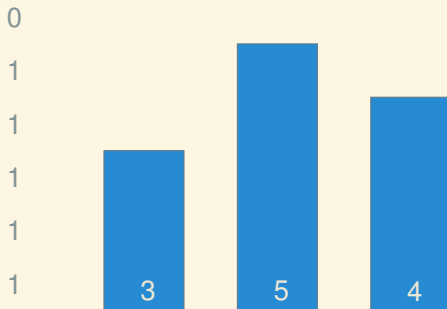
小課題 3 : 60 点 (累計 100 点)

- ▶ 初期状態の答えを計算しておいて，更新クエリのたびにその答えをうまく変化させればよい
- ▶ 初期状態の答えを計算するのは，小課題 2 の方法でもできるし，全員の初期評価が 0（このときの答えはどの位置でも 0）で，それぞれの本当の初期評価になるように最初に更新クエリが飛んできたと考えると手間が省ける

小課題 3 : 60 点 (累計 100 点)

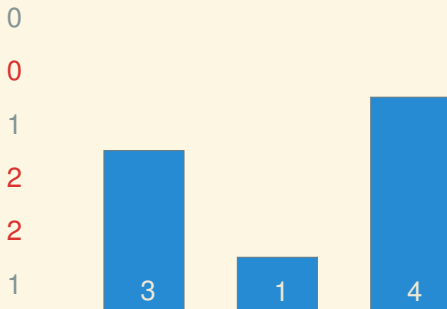
- ▶ 答えをうまく変更させるにはどうすればよいか
- ▶ 少し考えると，更新クエリのたびに 1 増加する場所と 1 減少する場所はそれぞれ高々 1 つの区間であることが分かる
- ▶ 両隣の max より上と，min より下の部分

小課題3：60点(累計100点)



▶ 再掲

小課題3：60点(累計100点)



▶ 再掲

小課題 3 : 60 点 (累計 100 点)

- ▶ 区間に 1 足したり 1 引いたりさえできれば...

小課題 3 : 60 点 (累計 100 点)

Segment tree

小課題 3 : 60 点 (累計 100 点)

- ▶ いろいろな方法で実装できる
- ▶ それぞれのノードに一様に加算された値を持ったり，BIT を 2 つ使ったり（蟻本 163 ページ付近を参照）

小課題 3 : 60 点 (累計 100 点)

- ▶ 今回は一点での値がわかれば良いので BIT 1 つでできる
- ▶ $[l, r)$ への x の加算は $\text{add}(l, x)$, $\text{add}(r, -x)$ を呼んで
- ▶ a での値を求めるには $\text{sum}(a)$ を呼べば良い
- ▶ 累積和を計算する感じ
- ▶ ただし、評価値の最大は 10^9 なので、最初に座標圧縮をする
か、動的に segtree を構築する必要がある

小課題 3 : 60 点 (累計 100 点)

- ▶ 座標圧縮に $O((N + M) \log(N + M))$
- ▶ 初期計算に $O(N \log(N + M))$
- ▶ クエリの処理に $O(M \log(N + M))$
- ▶ よって、全体の計算量は $O((N + M) \log(N + M))$
- ▶ 小課題 3 が解ける
- ▶ 満点

小課題 3 : 60 点 (累計 100 点)

- ▶ 別解
- ▶ クエリの平方分割
- ▶ 更新クエリが溜まったら答えを再計算し直す
- ▶ 解答クエリでは、計算しておいた答えに溜まっている更新クエリを加味して答えを出す
- ▶ 全体の計算量は $O((N + M)(\log(N + M) + \sqrt{M}))$
- ▶ この解法だと少し実行時間制限が厳しいので、いろいろバケットサイズを試したり高速化する必要があるかもしれない

目次

問題概要

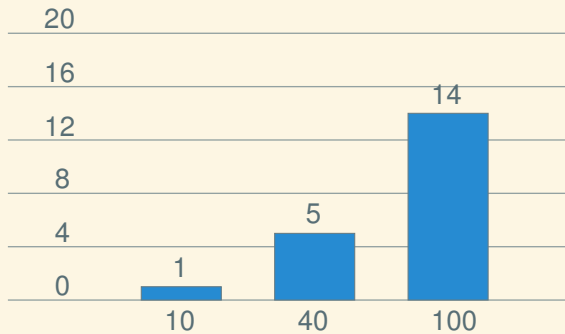
小課題 1 : 10 点 (累計 10 点)

小課題 2 : 30 点 (累計 40 点)

小課題 3 : 60 点 (累計 100 点)

得点分布

得点分布



▶ 優秀

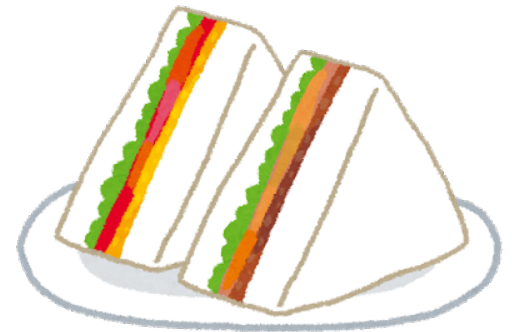
小課題3：60点(累計100点)

- ▶ ご清聴ありがとうございました

サンドイッチ (Sandwich)

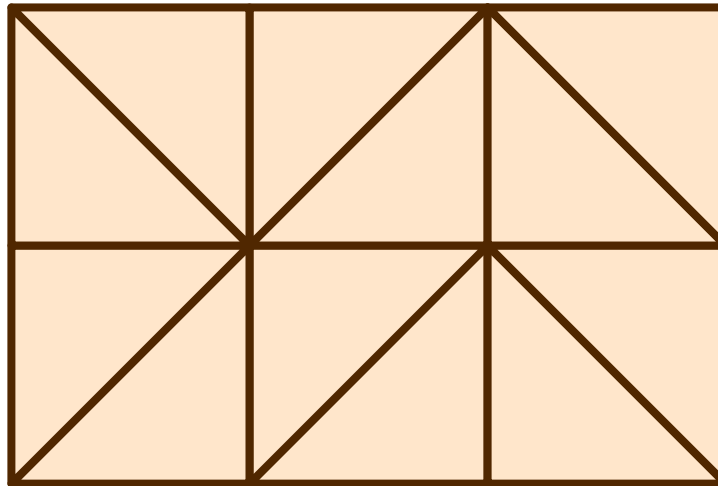
解説

JOI 2016 春合宿 day2
snuke



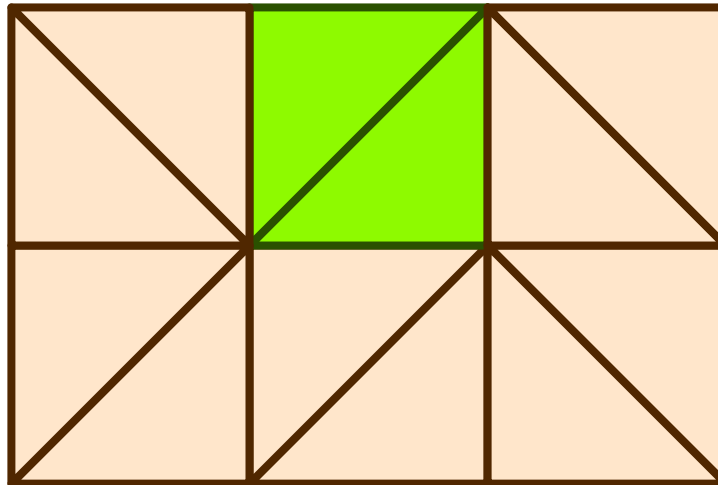
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



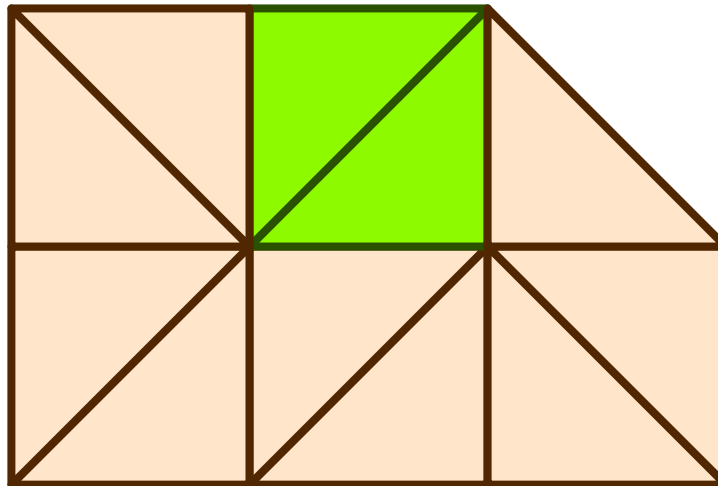
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



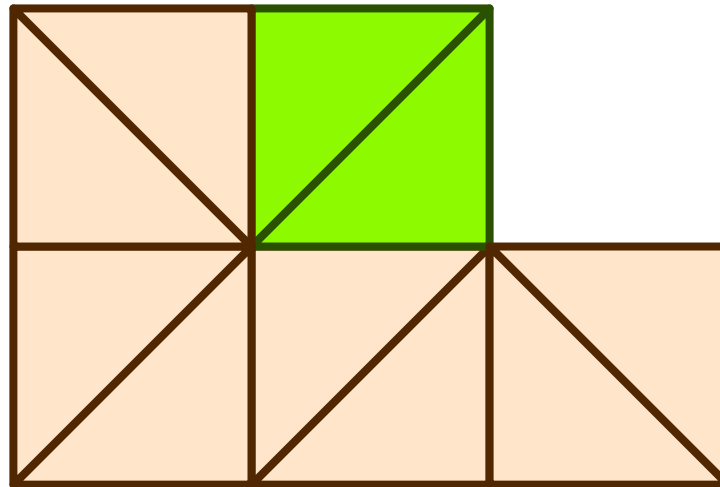
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



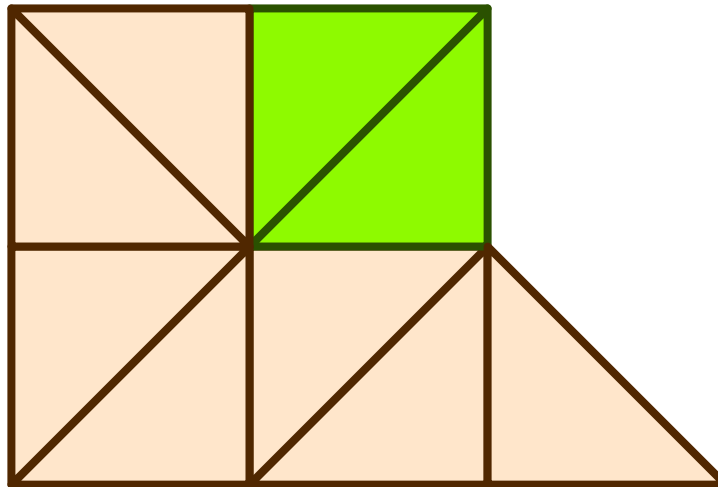
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



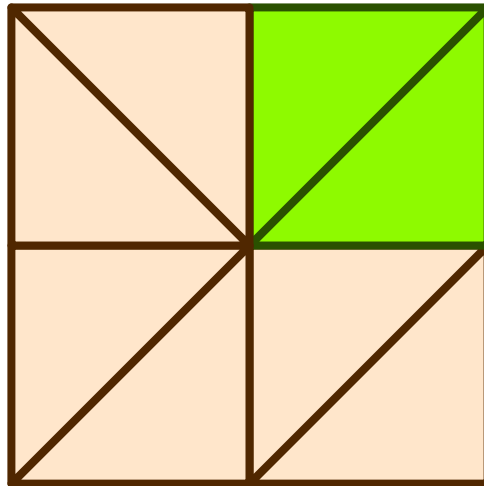
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



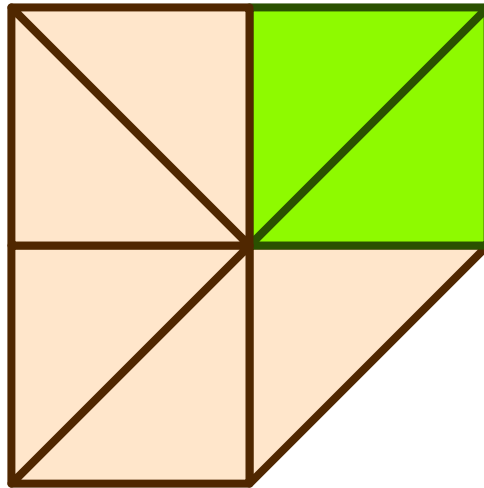
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



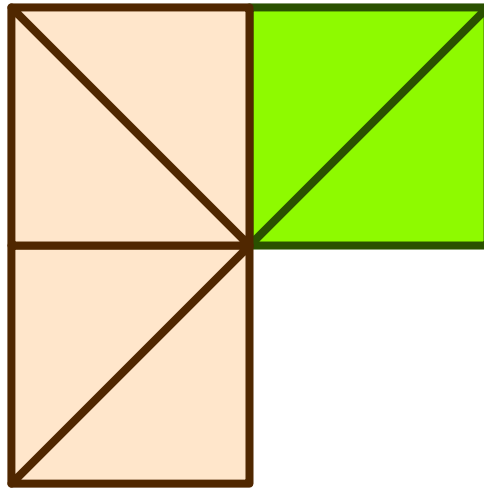
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



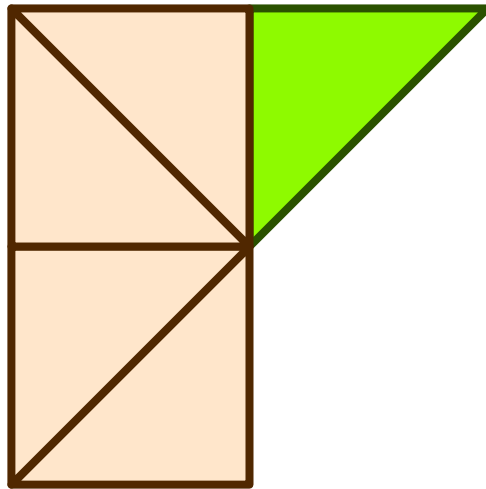
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



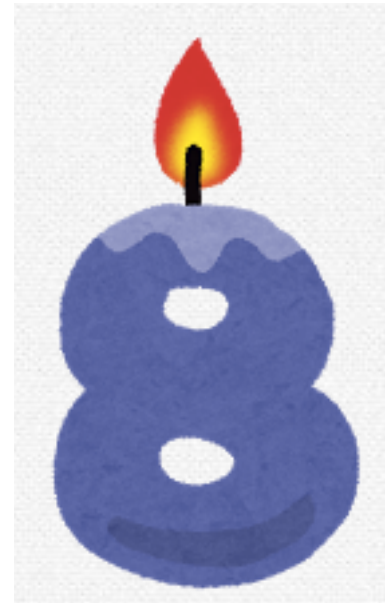
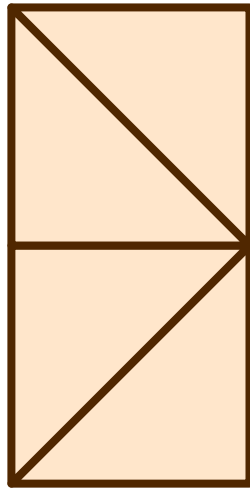
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



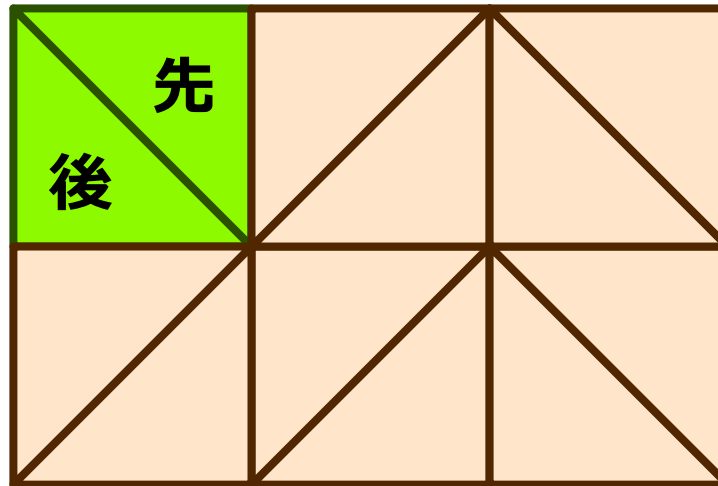
問題概要

- R行C列のマス目にサンドイッチが並んでいる
- 斜辺または他の二辺が空いてるサンドイッチは取れる
- 各マスについて、そのマスのサンドイッチを両方取るために取る必要のあるサンドイッチの個数の最小値は？



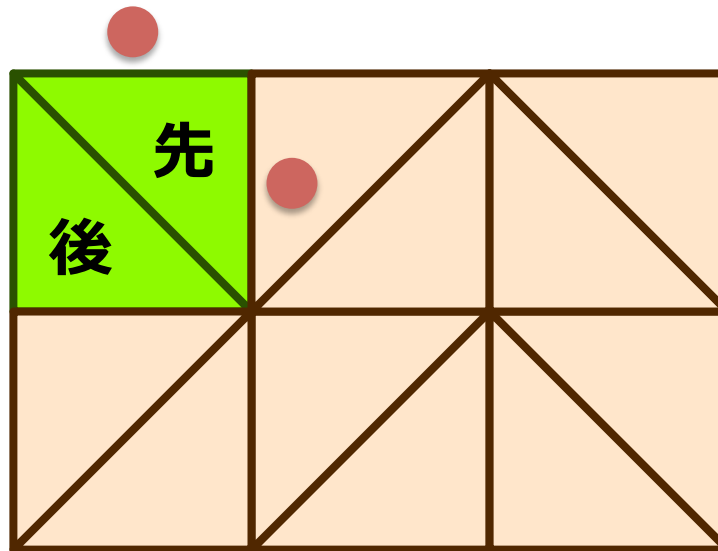
小課題 1 (35点)

- 目的の2個のサンドイッチの取り方は2通りある
 - (A) 上側のサンドイッチを先にとって下側を次を取る
 - (B) 下側のサンドイッチを先にとって上側を次を取る
- (A)についての最小値が求められれば、(B)についても同様な方法で求めることができるので、(A)のみを考える



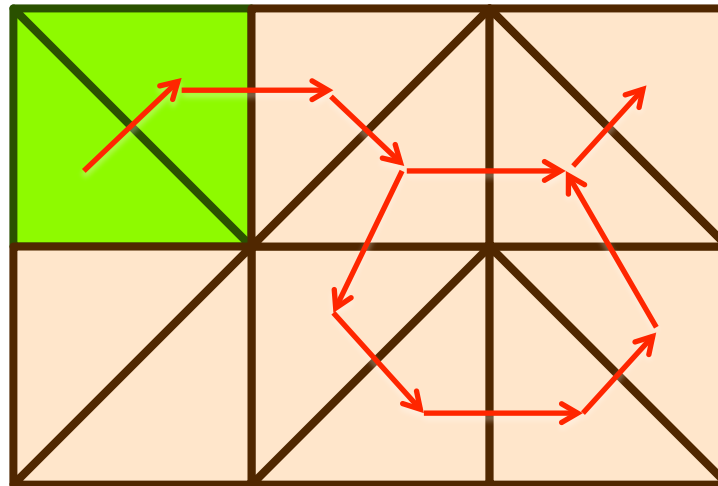
小課題 1 (35点)

- 上側のサンドイッチを取るためには、赤マルの位置のサンドイッチを先に取りなければならない
- このように「Xを取るためには先にYを取らなければいけない」という状況を、「XがYに依存している」と呼ぶことにする



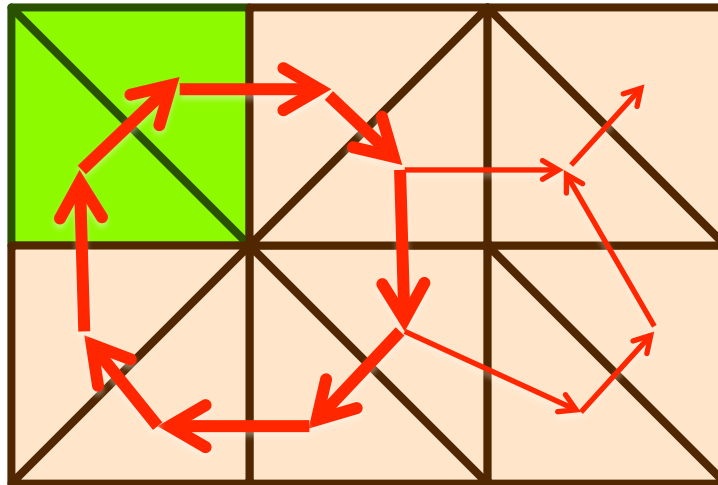
小課題 1 (35点)

- XがYに依存しているとき、 $X \rightarrow Y$ という辺を貼ってみる
- 辺が貼られたものを逆順に、取れるものから取っていけば緑のサンドイッチを取ることができる



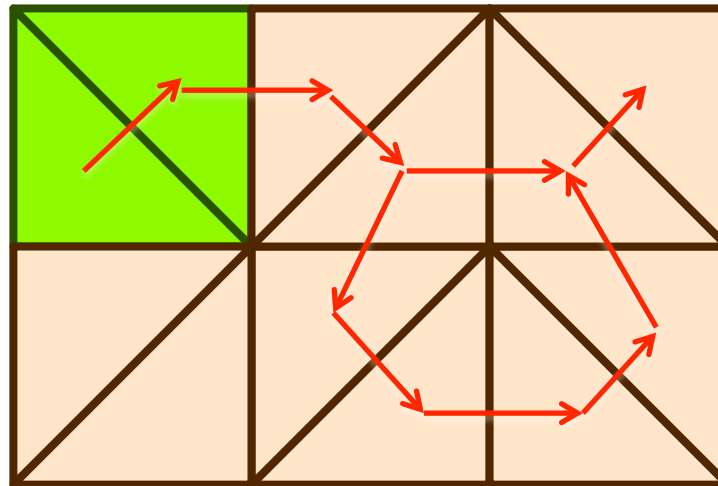
小課題 1 (35点)

- このようにサイクルができる場合は取ることが出来ない



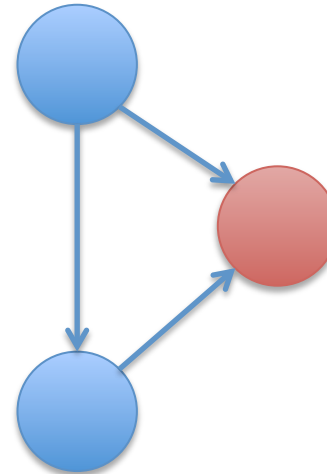
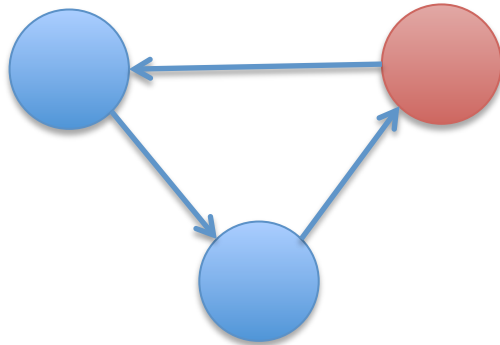
小課題 1 (35点)

- 各マスについて、(A),(B)パターンを試す
 - 依存関係を列挙していき、辺を貼る
 - 辺が貼られたサンドイッチの個数を求める
 - ただし、サイクルが存在する場合は取れないと判定する
- 計算量： $O((RC)^2)$



小課題 1 (35点)

- 具体的にどう計算するか
 - 単にbool visited[MAX_V]のような配列で状態を管理していると、「サイクル」なのか「単に2回訪れただけ」なのかが判定できない



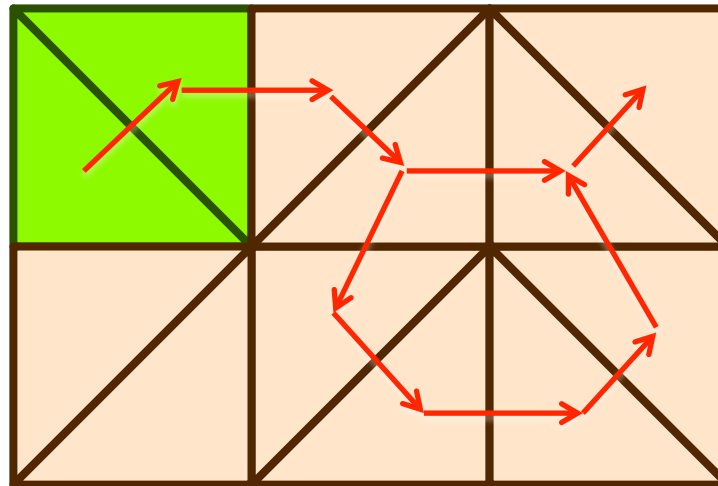
小課題 1 (35点)

- 具体的にどう計算するか
 - DFSを用いると簡潔に実装できる

```
int state[MAX_V]; // 0 で初期化
int dfs(int v) {
    if (state[v] == 1) return INF;
    if (state[v] == 2) return 0;
    state[v] = 1;
    int sum = 1;
    for (int u : to[v]) {
        int res = dfs(u);
        if (res == INF) return INF;
        sum += res;
    }
    state[v] = 2;
    return sum;
}
```

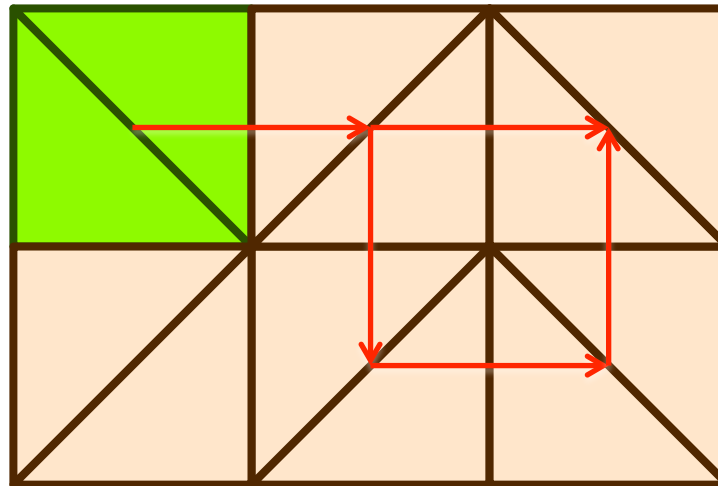
実装の簡略化

- サンドイッチが三角形なのは扱いにくいので、マス単位で考えることにする



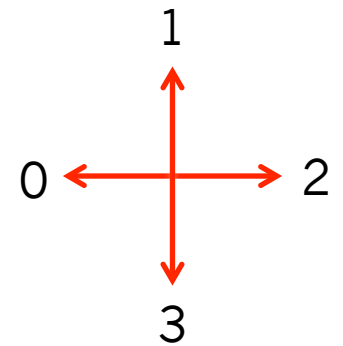
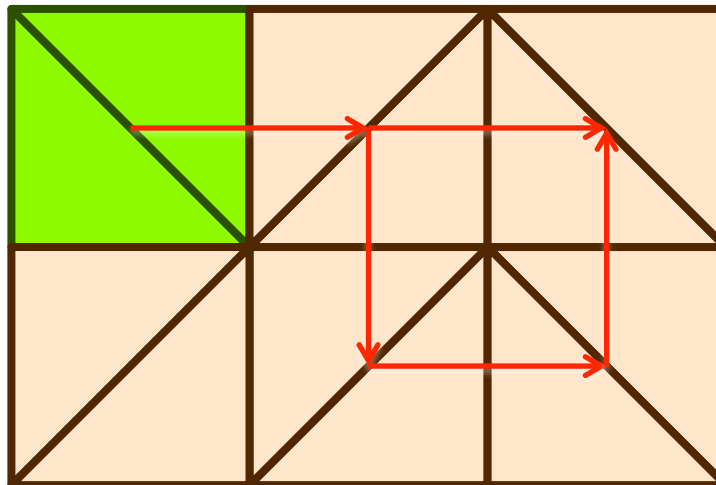
実装の簡略化

- サンドイッチが三角形なのは扱いにくいので、マス単位で考えることにする



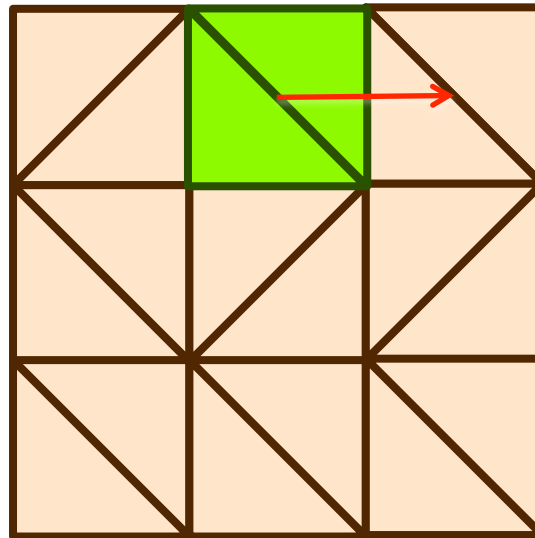
実装の簡略化

- マス目の隣接関係を扱うときのテクニック
 - $dx[] = \{-1, 0, 1, 0\}$, $dy[] = \{0, -1, 0, 1\}$
 - for($v=0\sim 3$) $nx = x+dx[v]$, $ny = y+dy[v]$
- v という方向で'Z'のマスに入ってきた時、そのマスから貼る辺は v と $v \text{ xor } 1$ の方向のマス
 - 'N'のマスなら v と $v \text{ xor } 3$



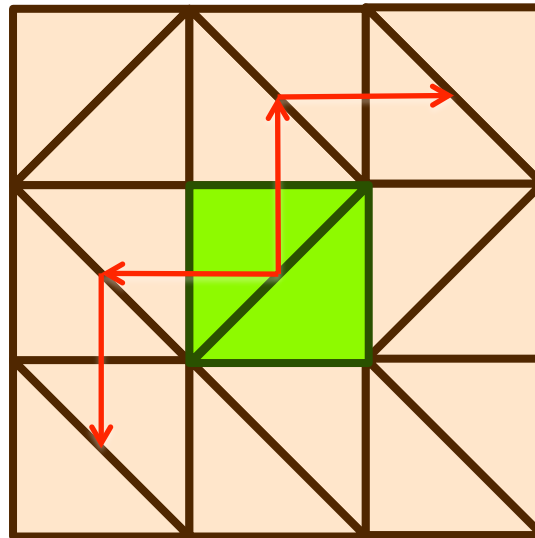
小課題 2 (満点)

- 列ごとに考える
- 列の上から順に依存関係を計算していく



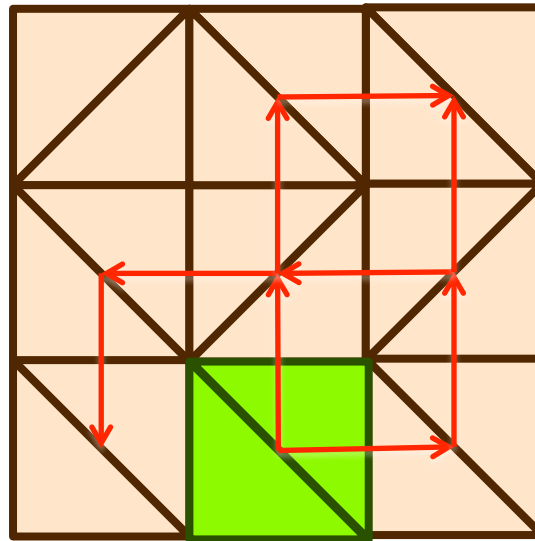
小課題 2 (満点)

- 列ごとに考える
- 列の上から順に依存関係を計算していく



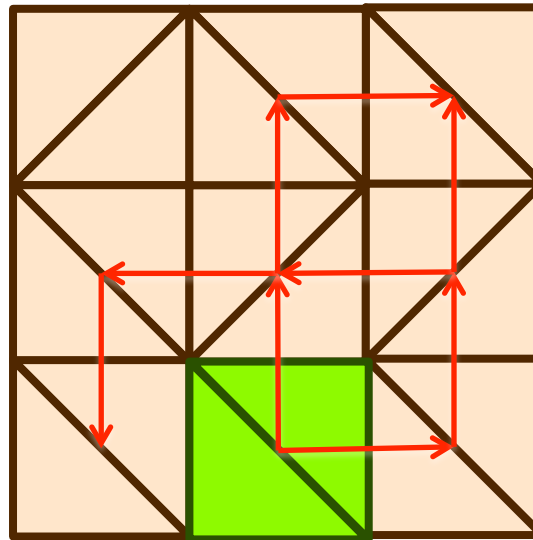
小課題 2 (満点)

- 列ごとに考える
- 列の上から順に依存関係を計算していく



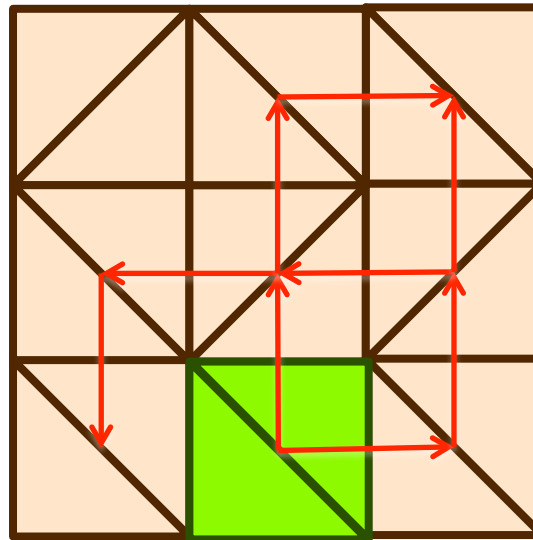
小課題 2 (満点)

- 列ごとに考える
- 列の上から順に依存関係を計算していく
- **依存関係の辺は単調増加！**



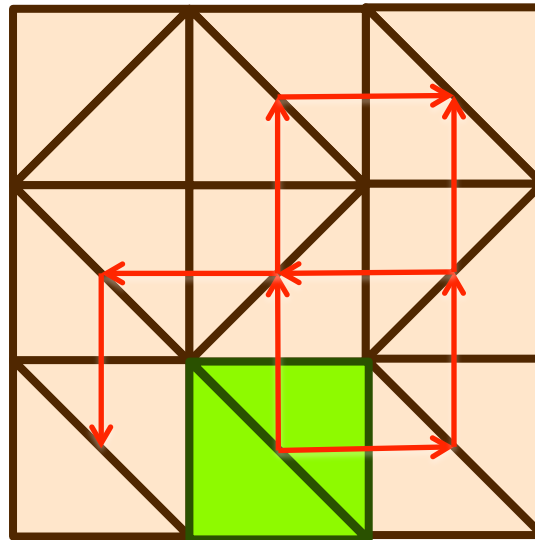
小課題 2 (満点)

- 列ごとに考える
- 列の上から順に依存関係を計算していく
- **依存関係の辺は単調増加！**
 - 上側のサンドイッチは必ず上方向に依存しており、
上方向に依存されたマスは上方向に依存しているため



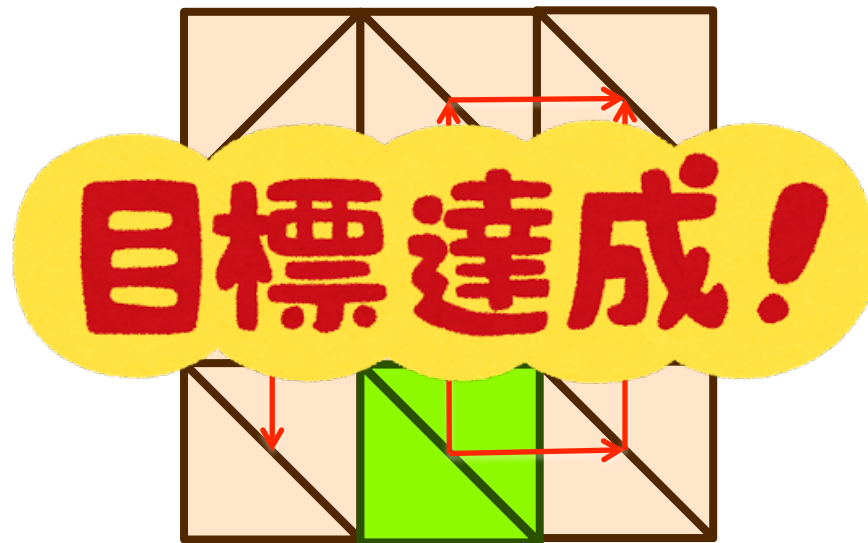
小課題 2 (満点)

- 列ごとに、上から順に依存関係を計算する
- 同じ列内では依存関係を初期化をせずに計算していく
- 依存関係の個数は高々 $O(RC)$ なので、列ごとに $O(RC)$ で計算でき、全体で $O(RC \cdot C)$ となる



小課題 2 (満点)

- 列ごとに、上から順に依存関係を計算する
- 同じ列内では依存関係を初期化をせずに計算していく
- 依存関係の個数は高々 $O(RC)$ なので、列ごとに $O(RC)$ で計算でき、全体で $O(RC \cdot C)$ となる



統計

0



15

35



4

100



1

統計

菩提樹の木の下で悟りを開くブッダのイラスト

0



15

巫女さんのイラスト

35



4

おからのイラスト

100



1

トイレ (toilets)

JOI2015-2016 春合宿 Day2

wafrelka

問題概要

- 2 つのトイレに $2N$ 人の選手が並んでいます
- 男性は男女共用トイレしか使えません
- 女性はどちらのトイレも使えます
- 列をうまく並べ替えてトイレをフル稼働させたいです



制約

- 小課題1: $2N \leq 20$
- 小課題2: $2N \leq 2 \times 10^5$
- 小課題3: $2N \leq 2 \times 10^{18}$



不可能な場合

- 男性が使えるトイレは 1 つ
 - 男性が $N + 1$ 人以上 $\rightarrow$ どう頑張っても無理
 - 男性が N 人以下 $\rightarrow$ 男性を先頭に並べれば OK
-
- 以降は男性が N 人以下の場合のみ考える

自明な探索解 – 解法

- ありえる並べ替え方をすべて試す
- $(2N)!$ 通りの並べ替え方がある
- 実際に並べ替えて OK かどうか判定する
- OK ならば不満度の最大値を計算する

自明な探索解 – 実装

- 並べ替え方は `next_permutation` などを用いてお好みで
- 不満度の計算のことも考えて列挙する
- `queue` などを使って頑張ると判定が $O(N)$ になる
- 頑張らないと判定が $O(N^2)$ になる

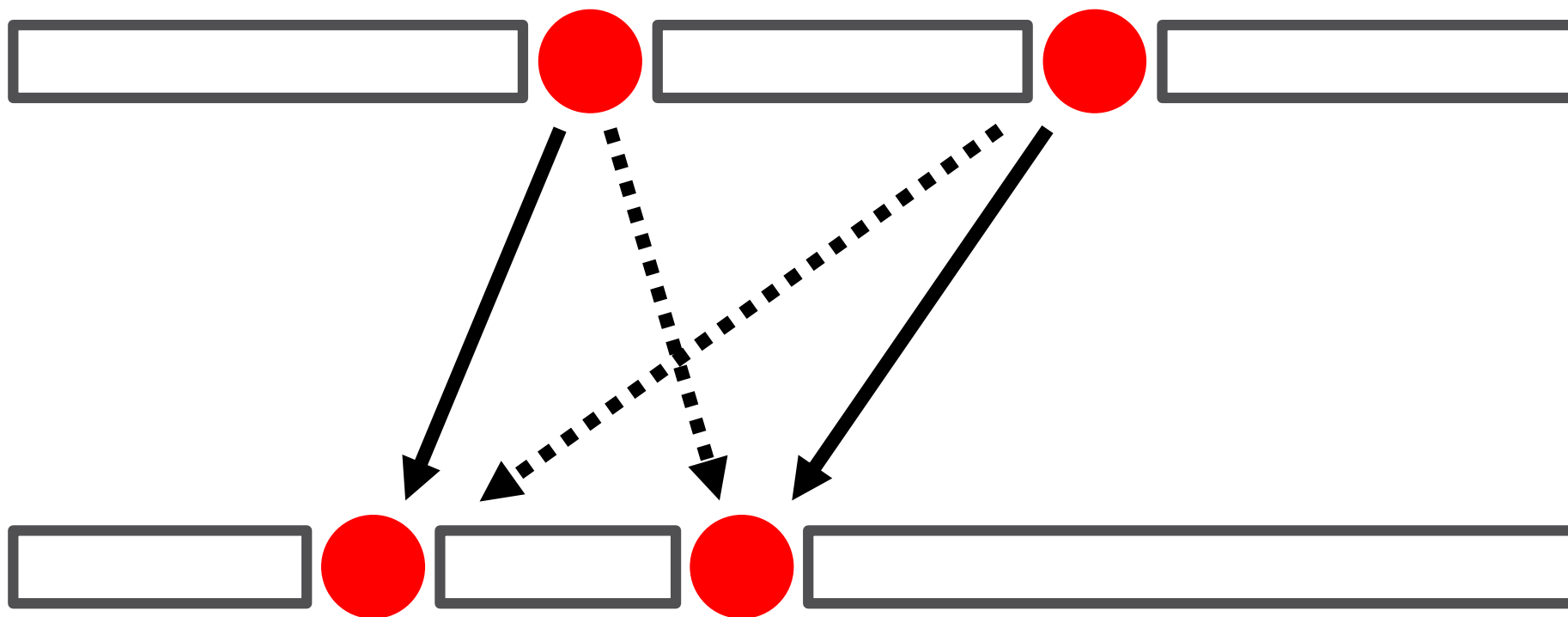
自明な探索解 – 解析

- 判定に $O(N^2)$ または $O(N)$
- 不満度の計算に $O(N^2)$
- 時間計算量 $O((2N)! N^2) \rightarrow$ 小課題 1 が解けない

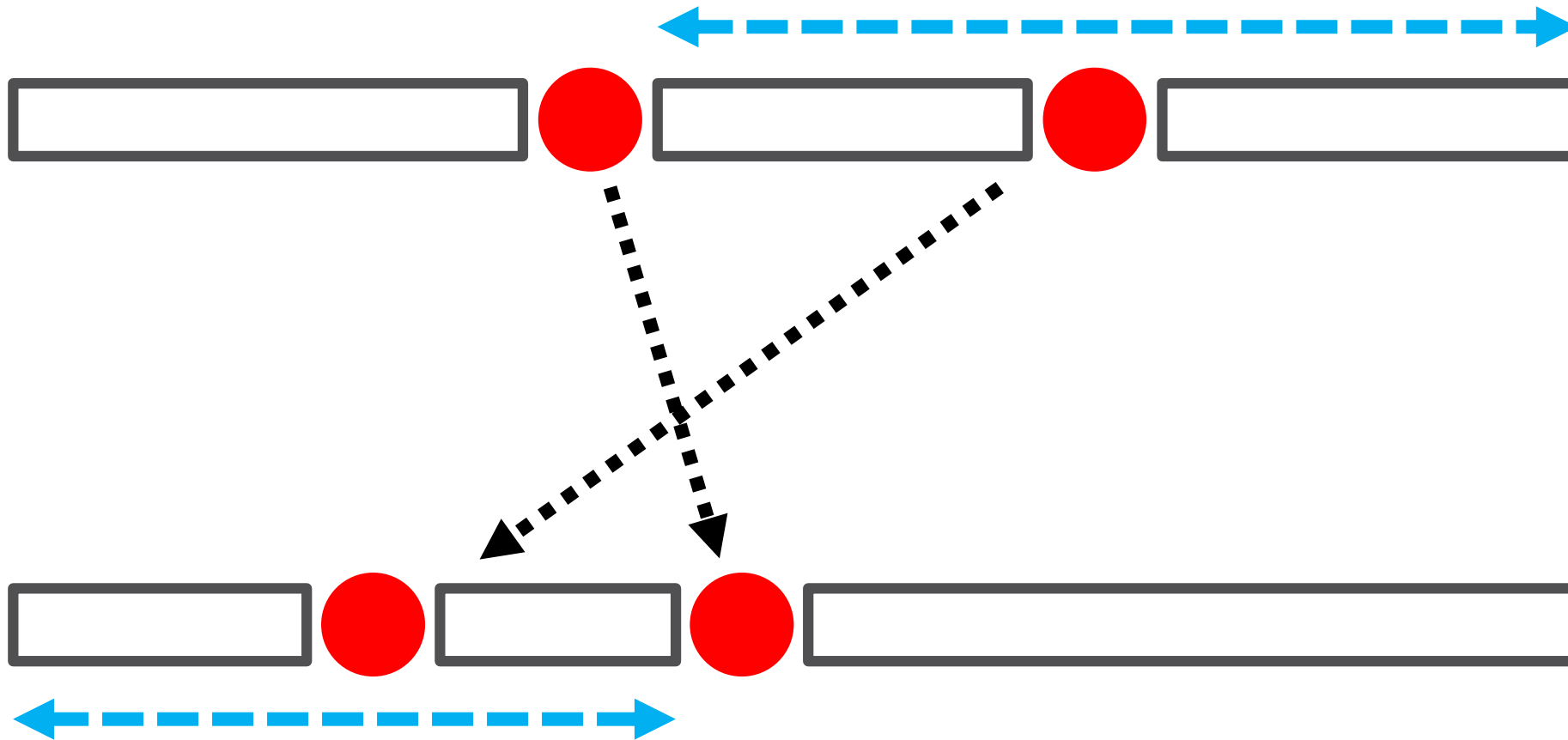
同性での順番入れ替え

- 男性だけ注目したとき：
男性同士での順番の入れ替えは必要ない
- 女性についても同様

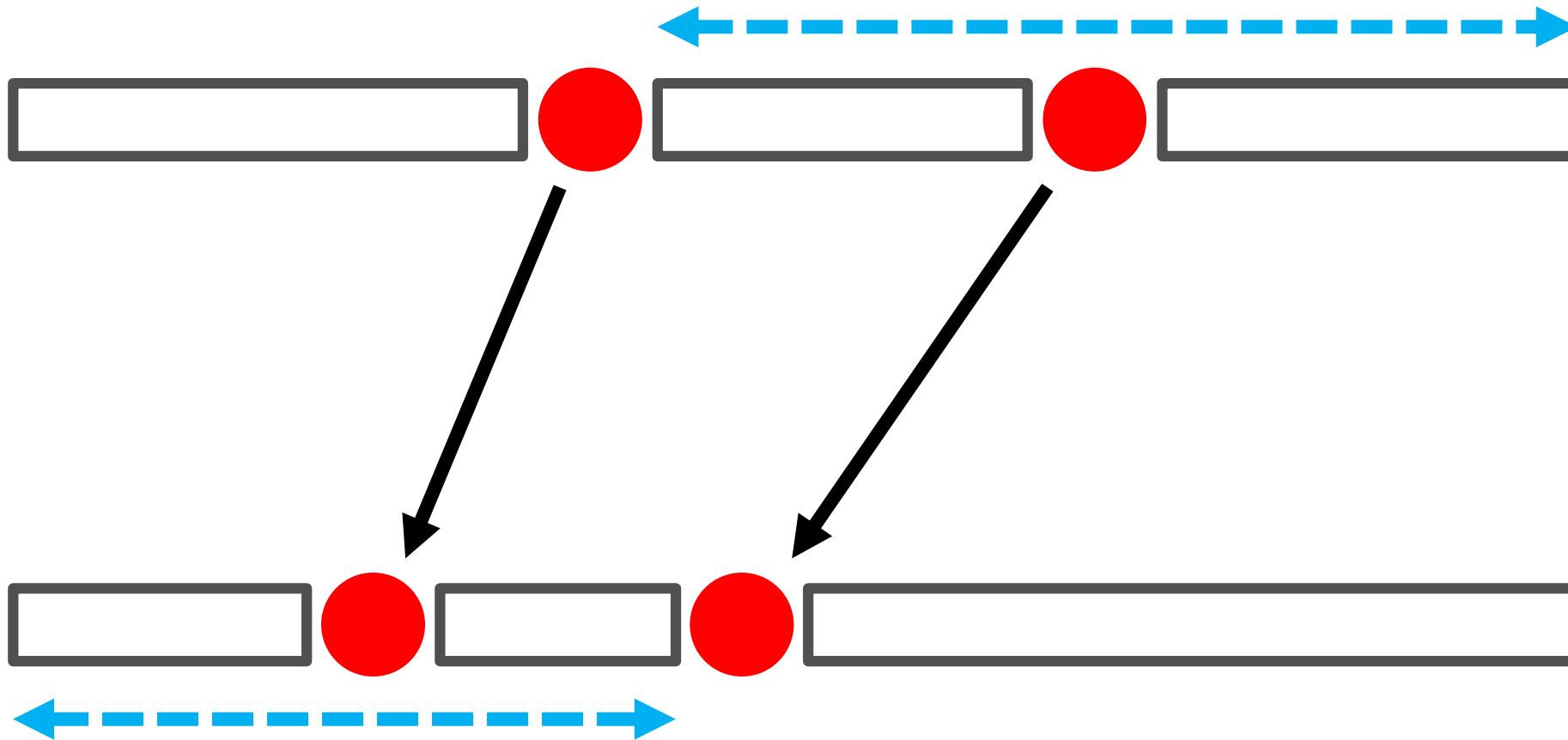
同性での順番入れ替え



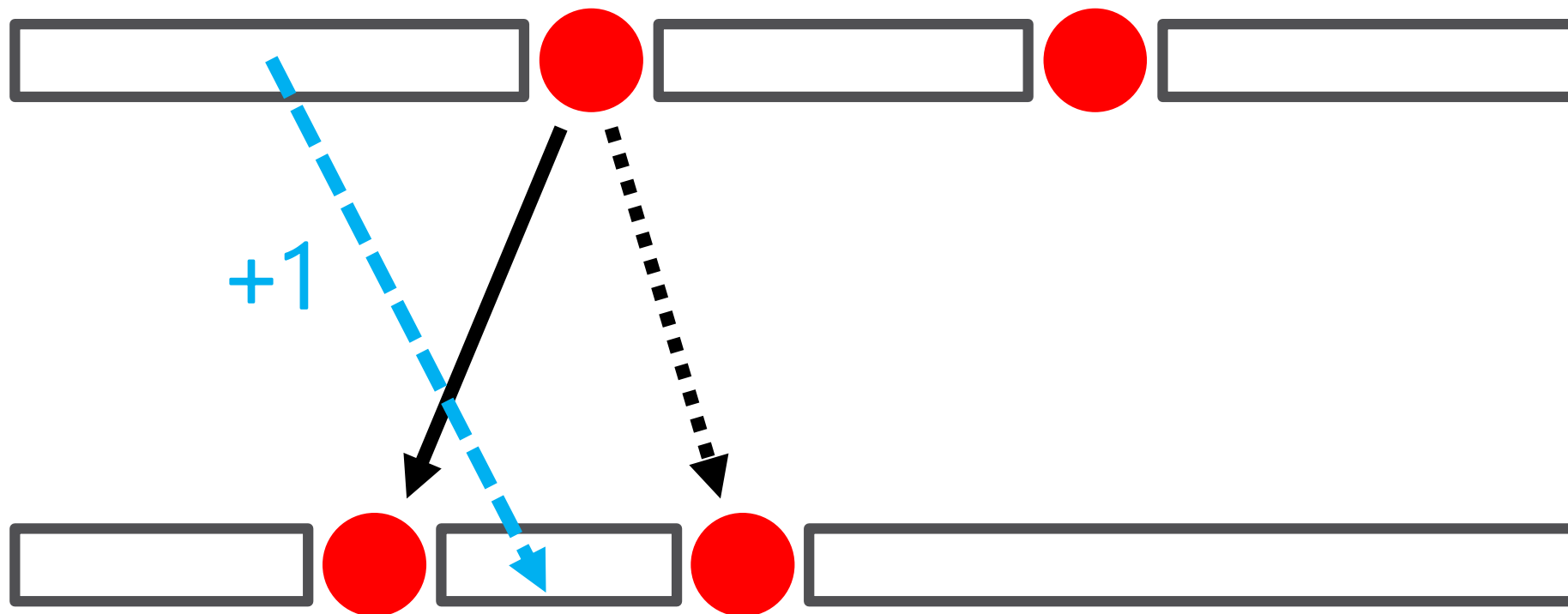
●の不満足度 – before



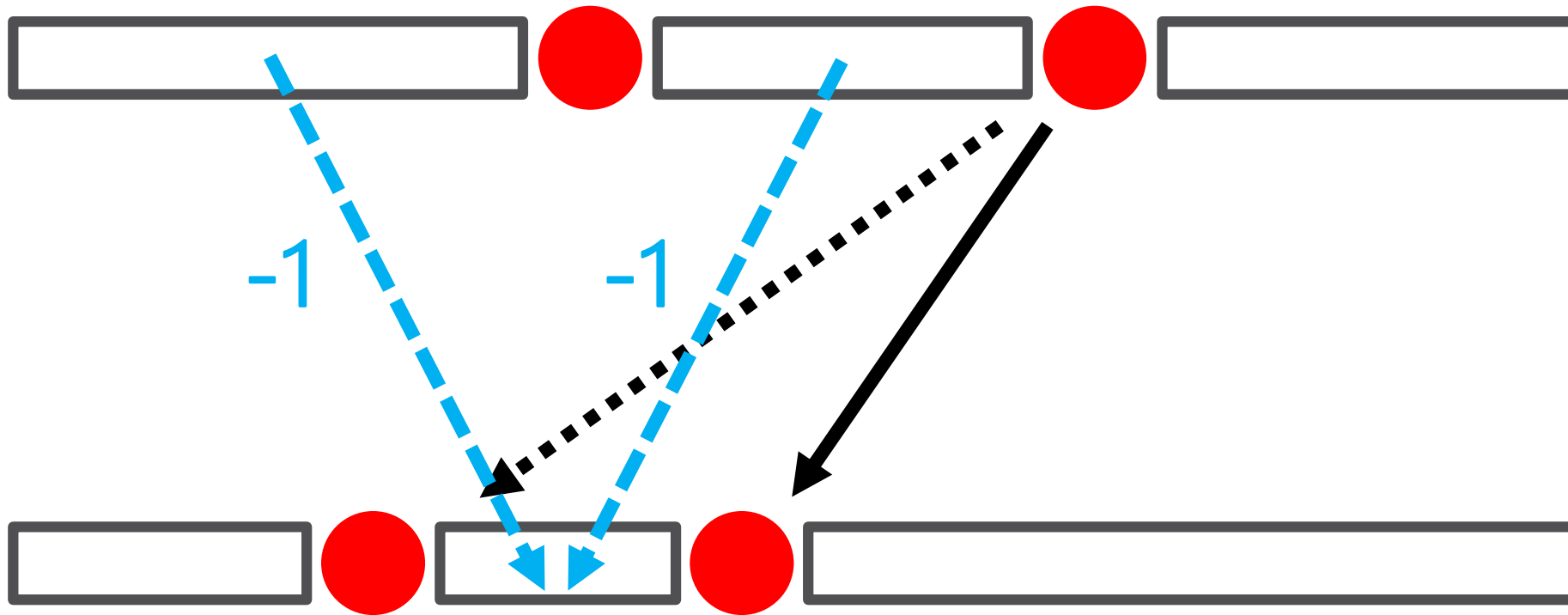
●の不満足度 – after



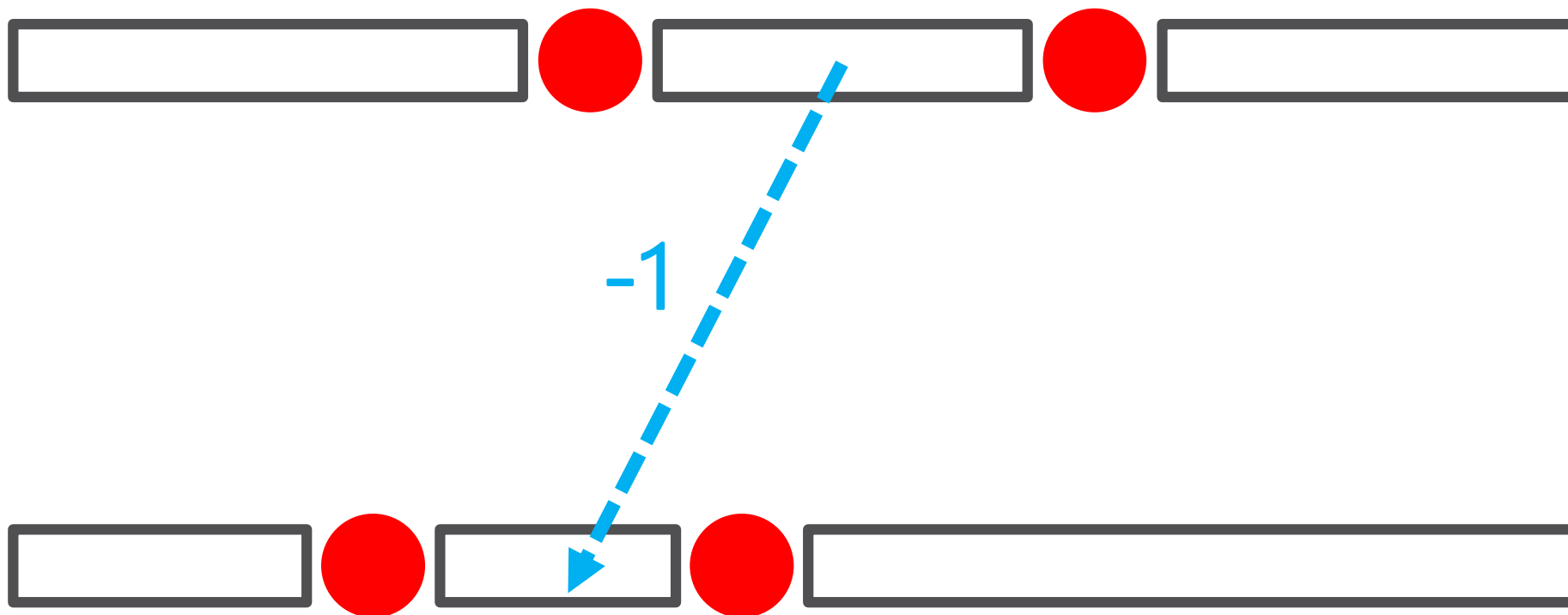
□の不満足度 - 1人目



□の不満足度 – 2人目



□の不満足度 - 合計



同性での順番入れ替え

- 入れ替えが起こらない並べ替えの方が不満度の最大値が小さい (または等しい)
- 不満度の最大値がもっとも小さい並べ替え方のうち同性での順番入れ替えがないようなものが少なくとも 1 通り存在する
- 同性での順番入れ替えが起こらないような並べ替え方だけを考えれば良い

非自明な探索解 – 解法

- 同性での順番入れ替えが起こらないような並べ替え方をすべて試す
- 男性同士,女性同士を区別しないで並べる
- 並べ替え方は 2^{2N} 通り以下になる
- あとは 自明な探索解 と同じ

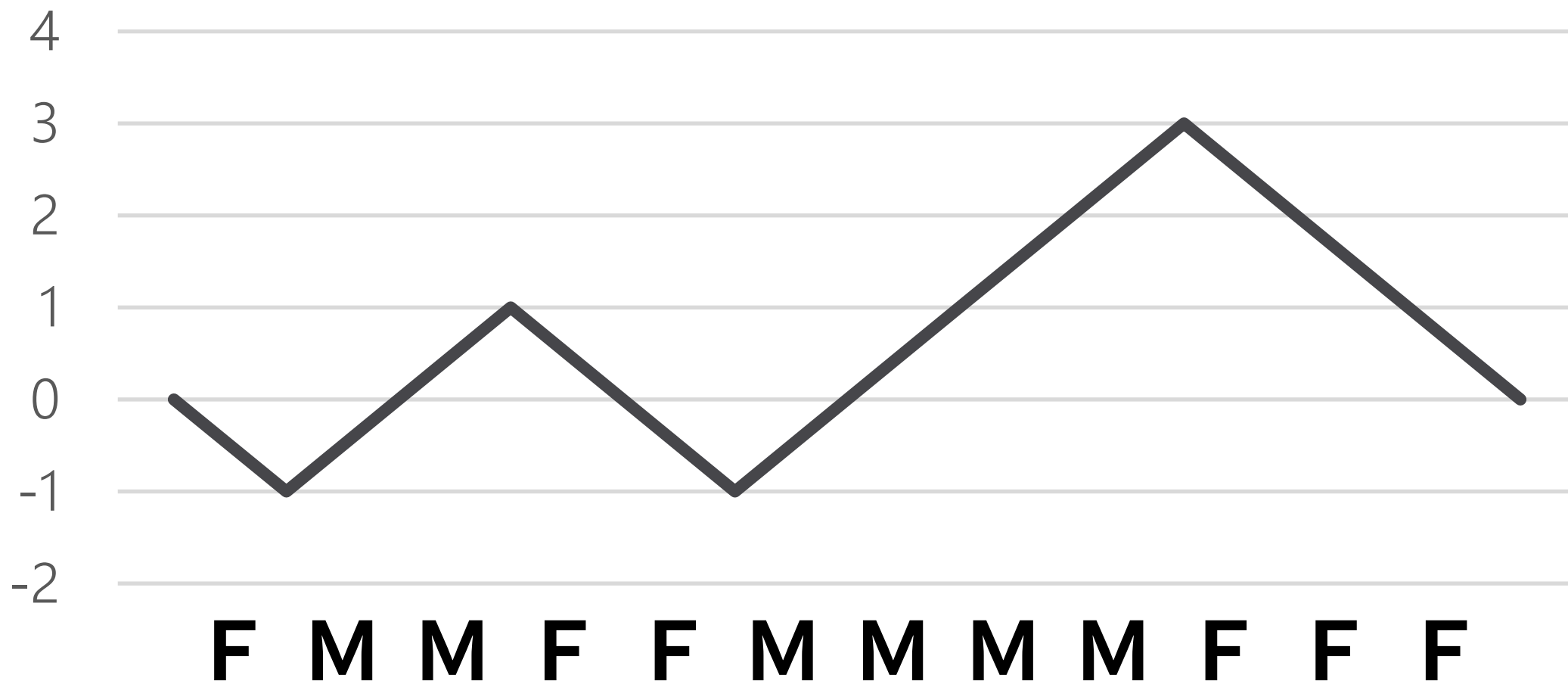
非自明な探索解 – 解析

- 判定に $O(N^2)$ または $O(N)$
- 不満度の計算に $O(N^2)$
- 時間計算量 $O(2^{2N}N^2) \rightarrow$ 小課題 1 が解ける
- (真面目に考えるともう少し抑えられるが
いずれにせよ小課題 2 は無理そう)

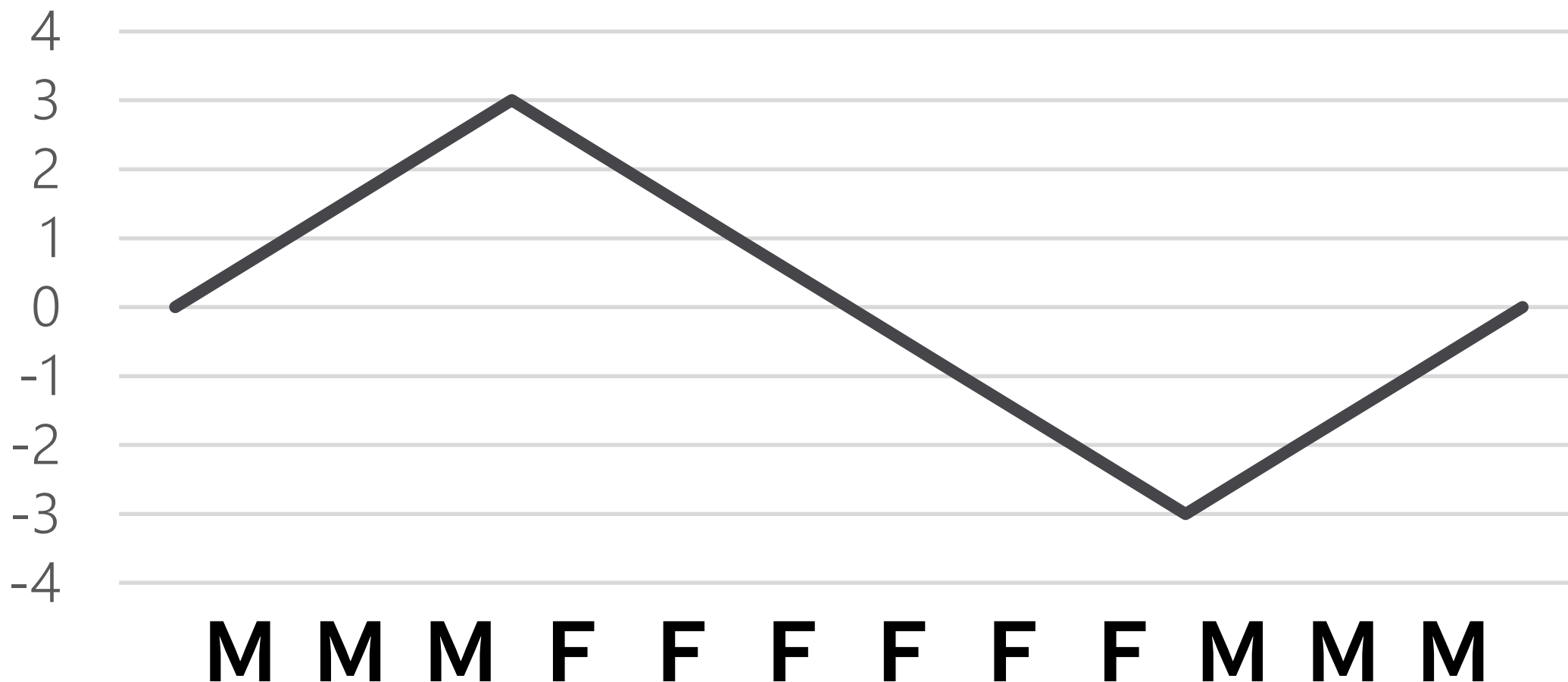
高速化への道

- 並べ替え後の列を考える
- この列が OK な列かどうか楽に判定する方法を考える
- ここで最後尾からの累積和を取ってみる
- 男性がきたら -1 , 女性がきたら $+1$

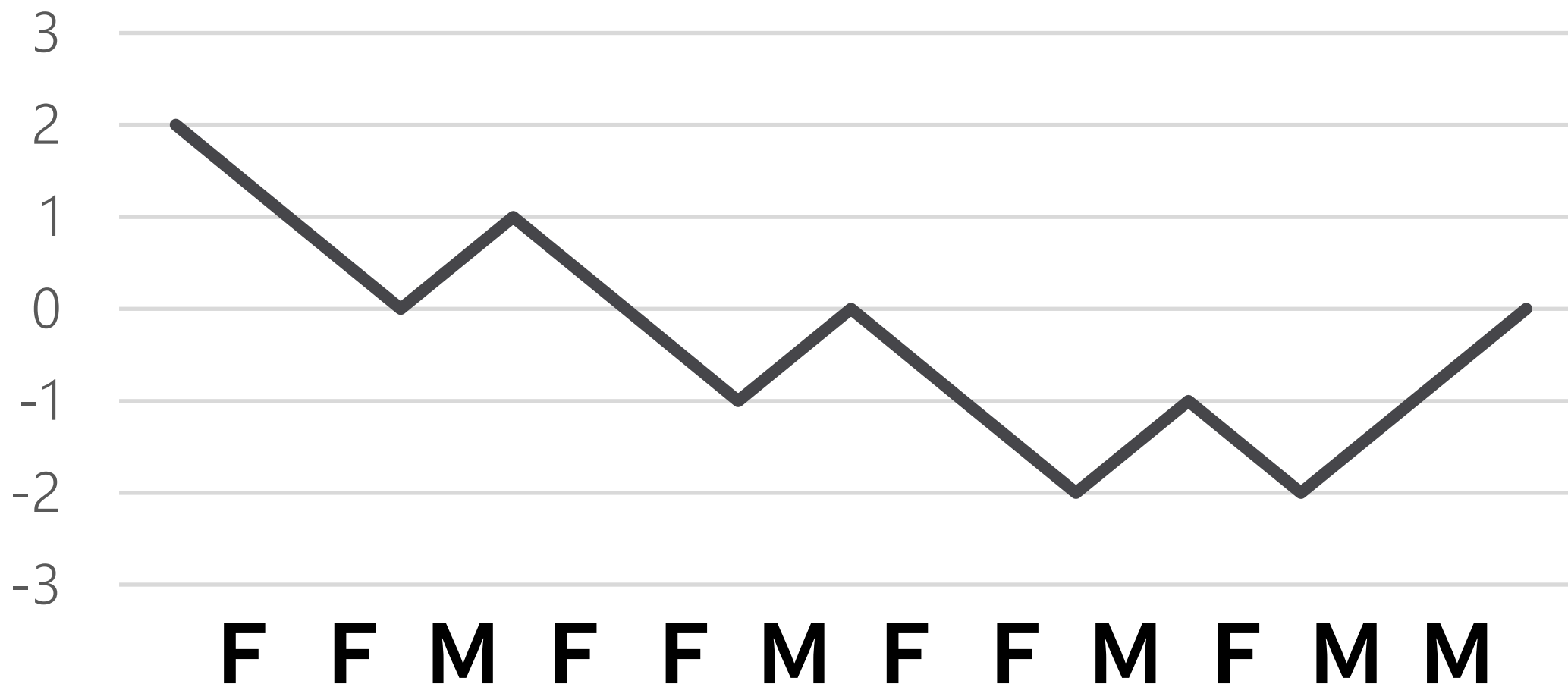
累積和 – OK



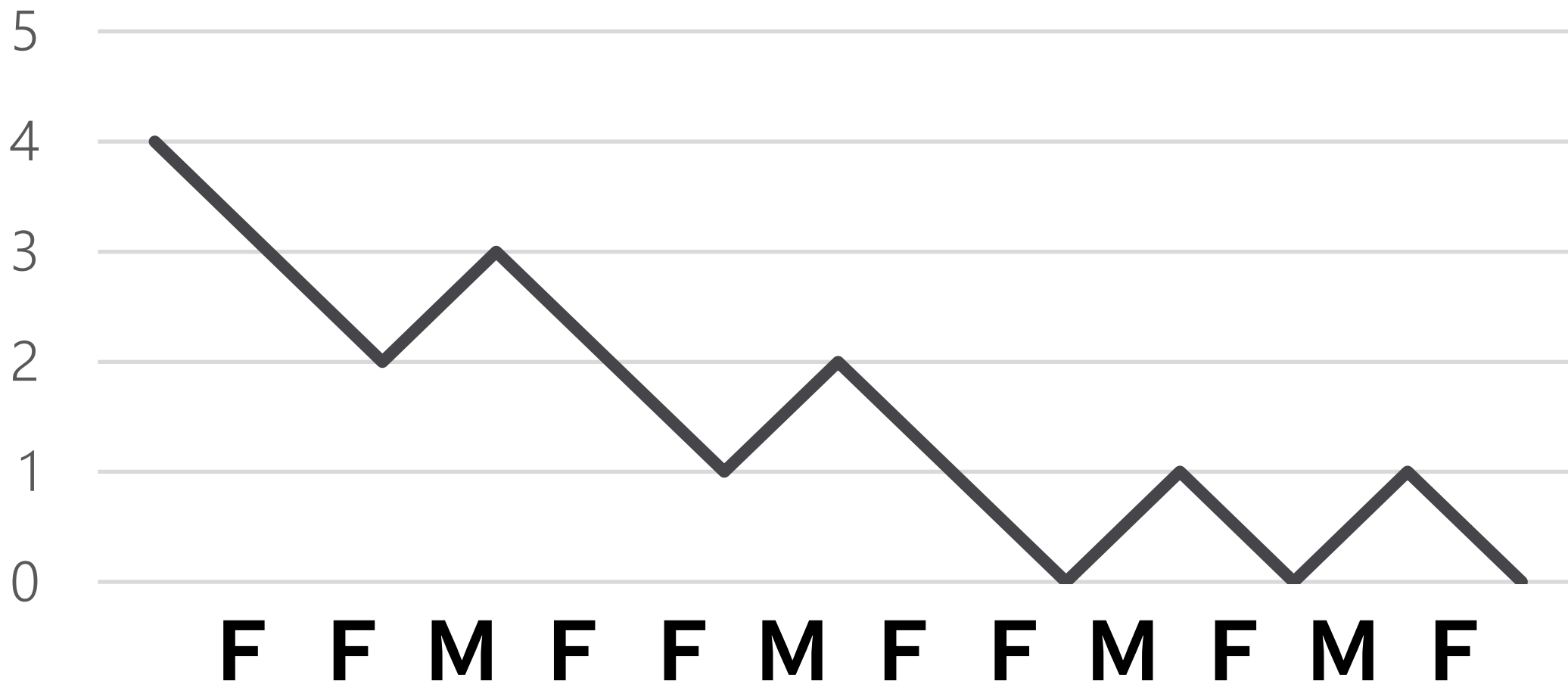
累積和 – NG



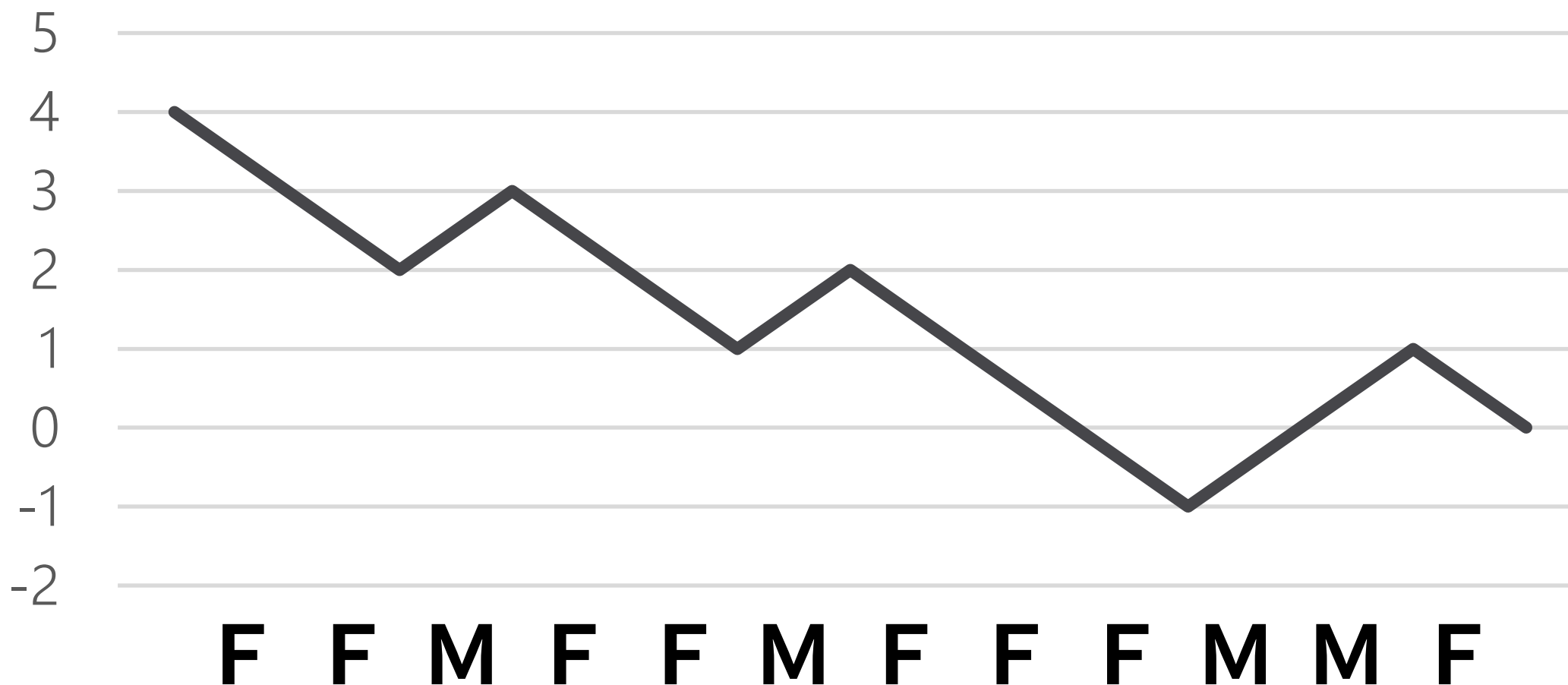
累積和 – NG



累積和 – OK



累積和 – OK



重要な性質

後ろから x ($1 \leq x \leq 2N$) 人取り出したときに
常に (女性の人数) - (男性の人数) ≥ -1

$\Leftrightarrow$

N 分で全員がトイレに入れる

重要な性質 – 証明

- 証明は列全体の長さに関する帰納法による
- $N = 1$ は明らか
- $x = 2N$ で NG のとき：どう頑張ってもダメ
- $x = 2N$ で OK のとき：
 - 先頭が 女性, 男性 $\rightarrow$ $3 \sim 2N$ 人目を取り出して再び判定
 - 先頭が 男性, 女性 or 女性, 女性 $\rightarrow$ おなじ

重要な性質 – 証明

- $x = 2N$ で OK & 先頭が 男性, 男性 のケース
- いちばん前にいる女性の位置を $2N - t$ とする
(女性がいないとするとすべて男性となり
 $4 \leq 2N$ なので $x = 2N$ で NG になる)
- 現在の列での $x = 1, \dots, t$ の判定と
1 分後の列での $x = 1, \dots, t$ の判定は一致する
- $x = 2N$ で OK ならば
どちらの列も $x = t + 1, \dots, 2N$ では大丈夫

重要な結果

- 女性,男性 と並んでいるところを入れ換えると
累積和の最小値は小さくなる (または等しいまま)
- 不満度が大丈夫ならば
男性をできるだけ前に動かした方がよい
- 最大の不満度を決めれば最適な並べ替え方が決まる

最適な並べ替え方

- 「この並べ替え方でも無理なら不満度 C ではどう頑張っても無理」という並べ方が存在する (→最適)

最適な並べ替え方

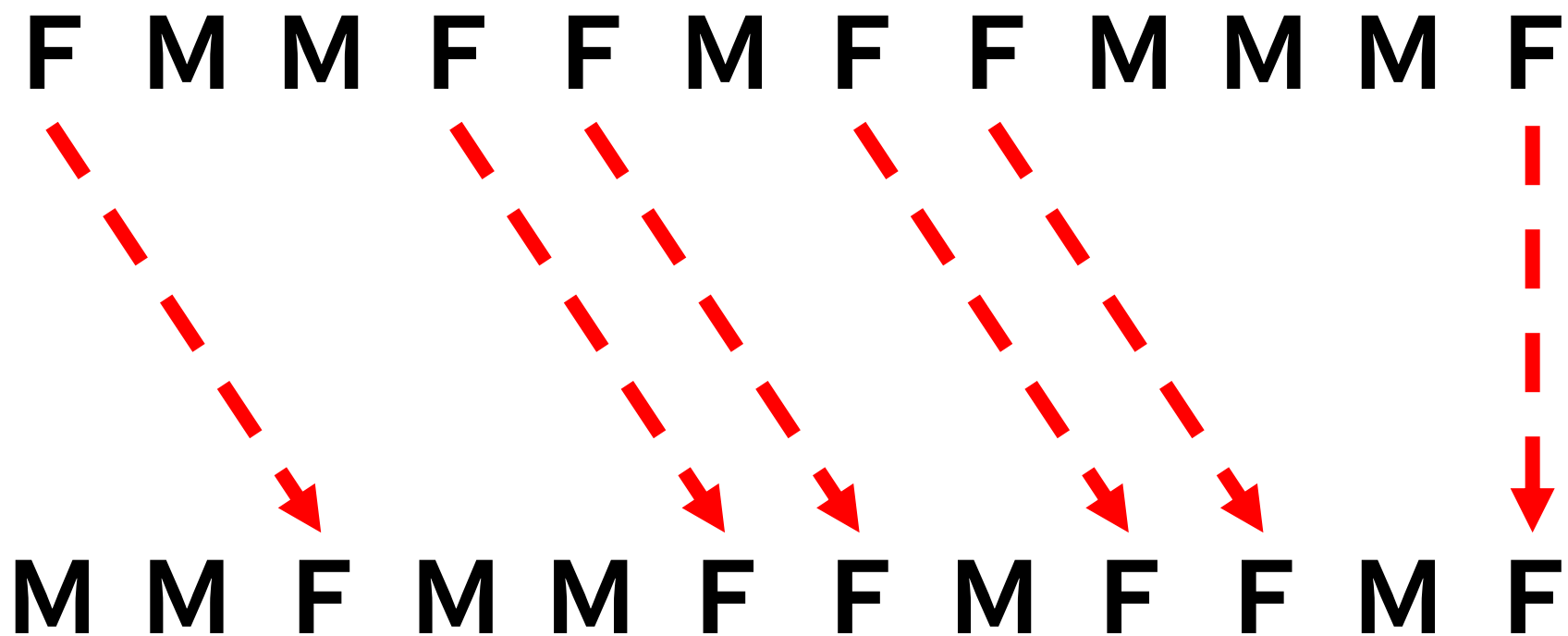
F M M F F M F F M M M F



M M F M M F F M F F M F

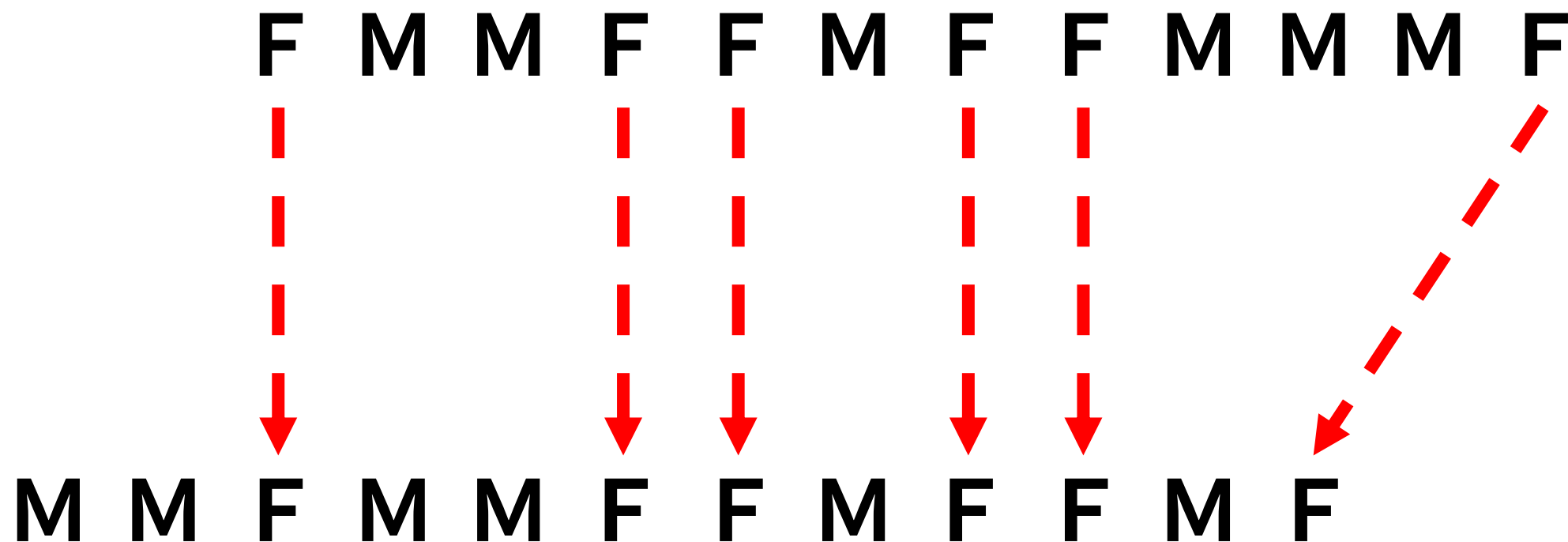
($C = 2$)

最適な並び替え方



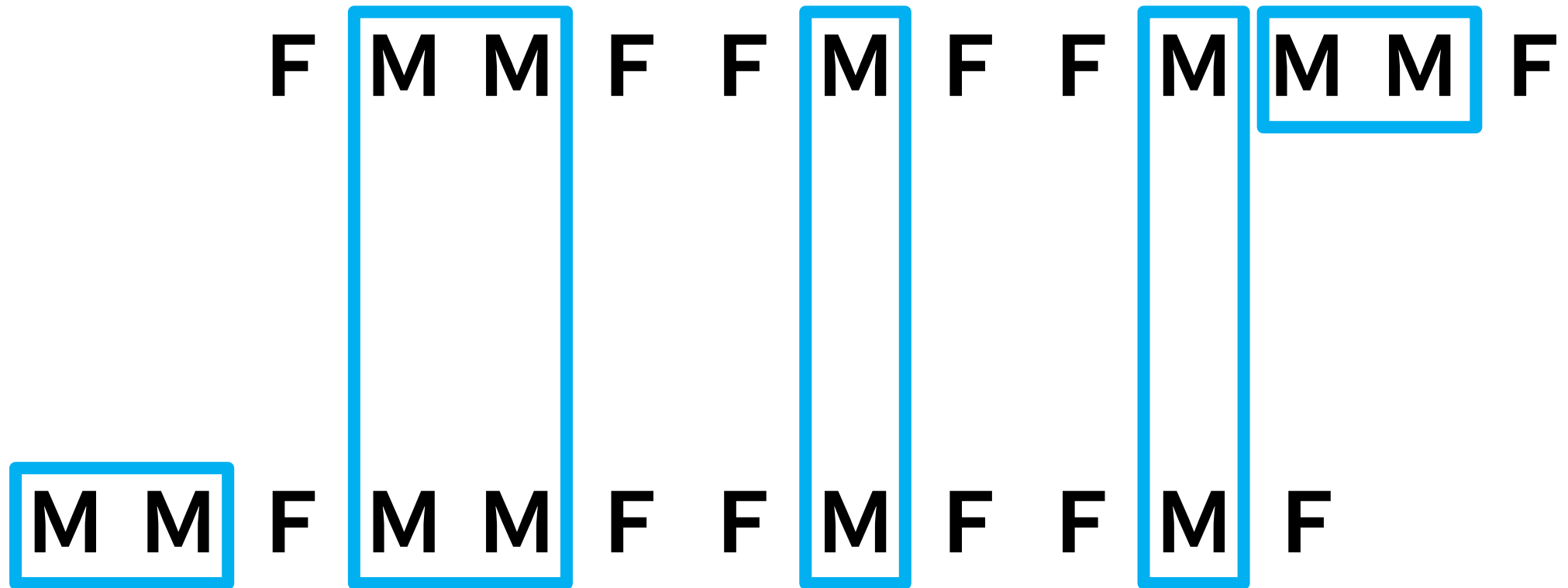
($C = 2$)

最適な並べ替え方



($C = 2$)

最適な並び替え方



($C = 2$)

最適な並べ替え方

- 先頭に新たに c 人の男性がやってくる
- 残りの男性は c 人分だけ前にシフト

二分探索解 – 解法

- 不満度を定めると最適な並べ替え方は簡単に求まる
- 最大の不満度の値で二分探索ができる

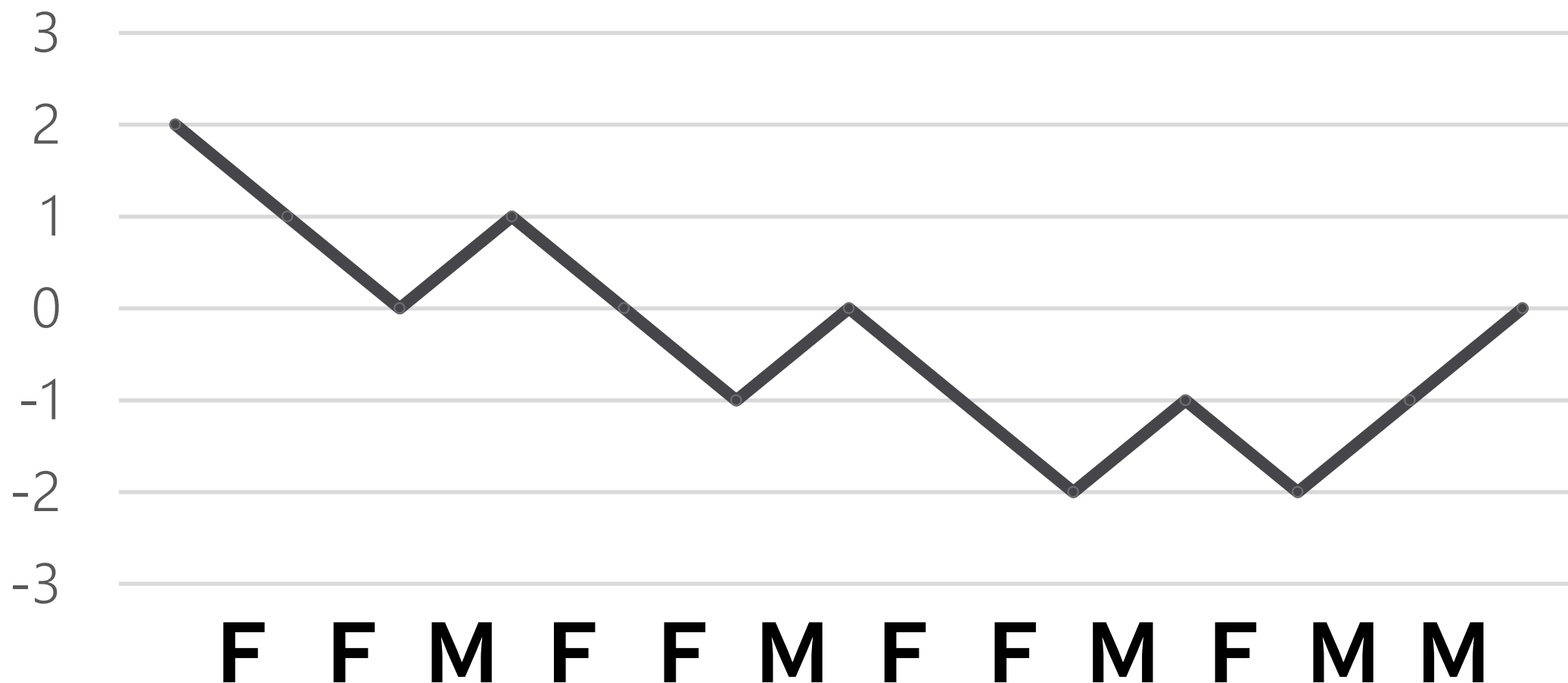
二分探索解 – 解析

- 最適な並べ替え方を求めるのに $O(n)$
- 最後尾からの累積和で OK かどうか判定できる
- よって二分探索の 1 回のチェックが $O(n)$
- 時間計算量 $O(n \log n)$ → 小課題 2 が解ける

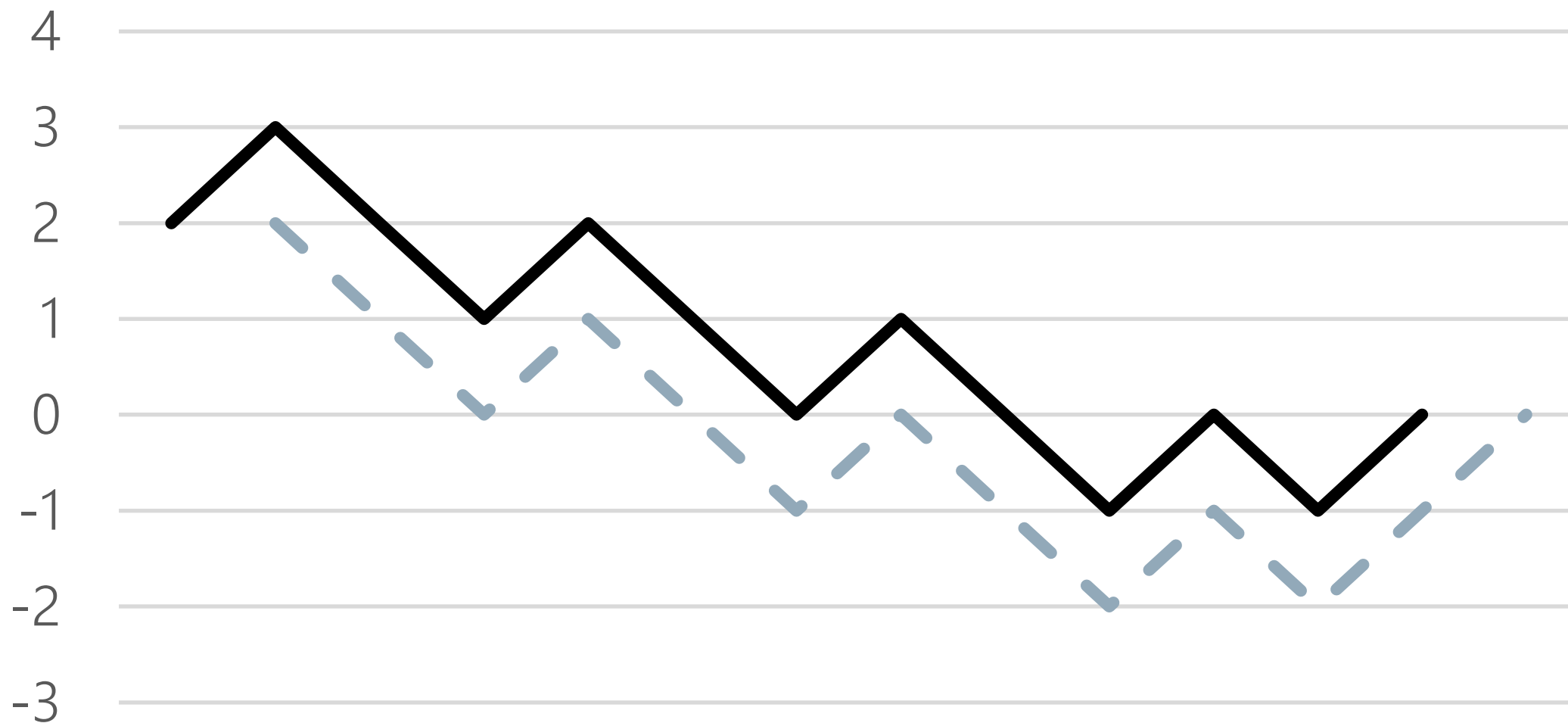
二分探索をやめる

- 最大の不満度を $C + 1$ にすると
 - 先頭に $C + 1$ 人の男性がやってくる
 - 残りの男性は $C + 1$ 人分だけ前にシフト
- → 最大不満度 C のときの列で一番後ろにいた男性が先頭にやってきただけ

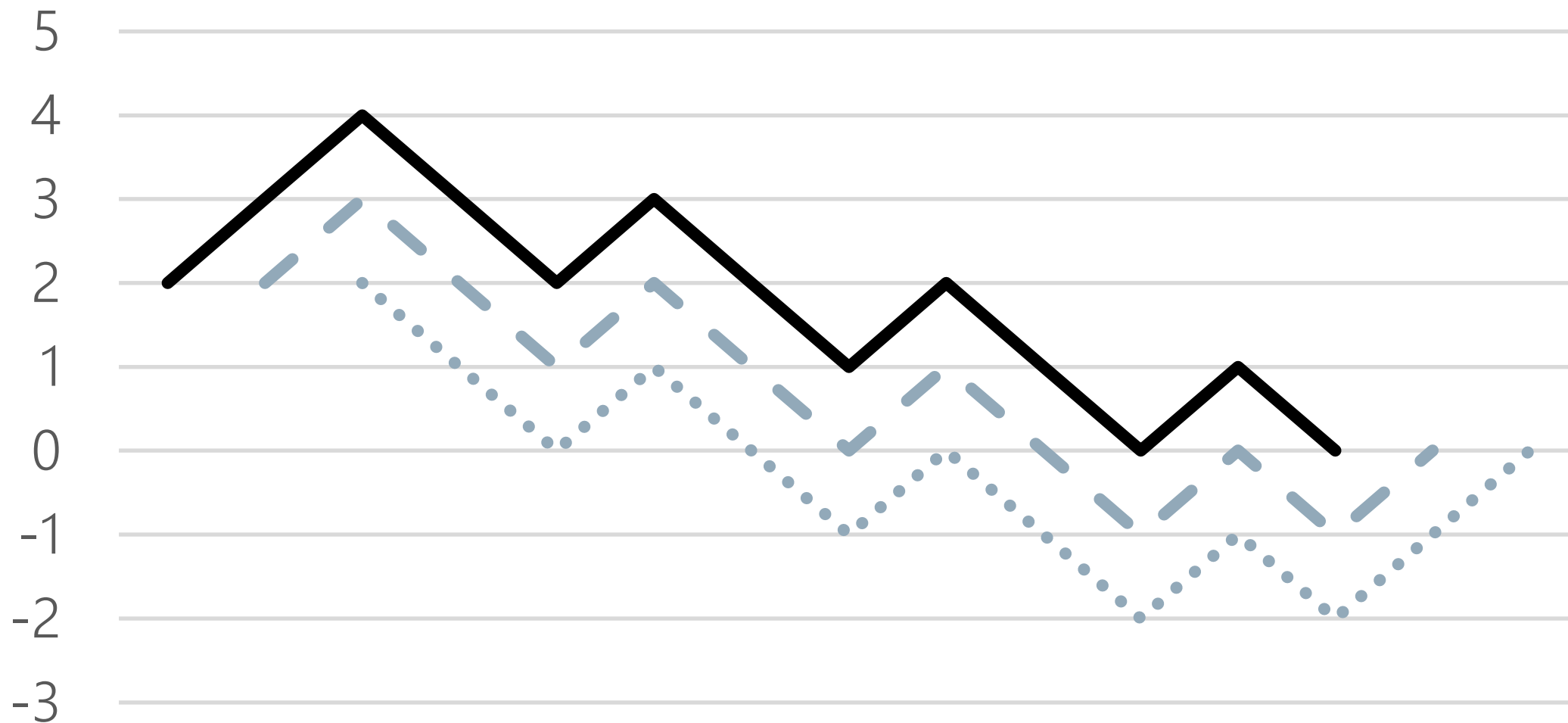
累積和 – NG (再掲)



1人シフト



2人シフト



累積和の最小値

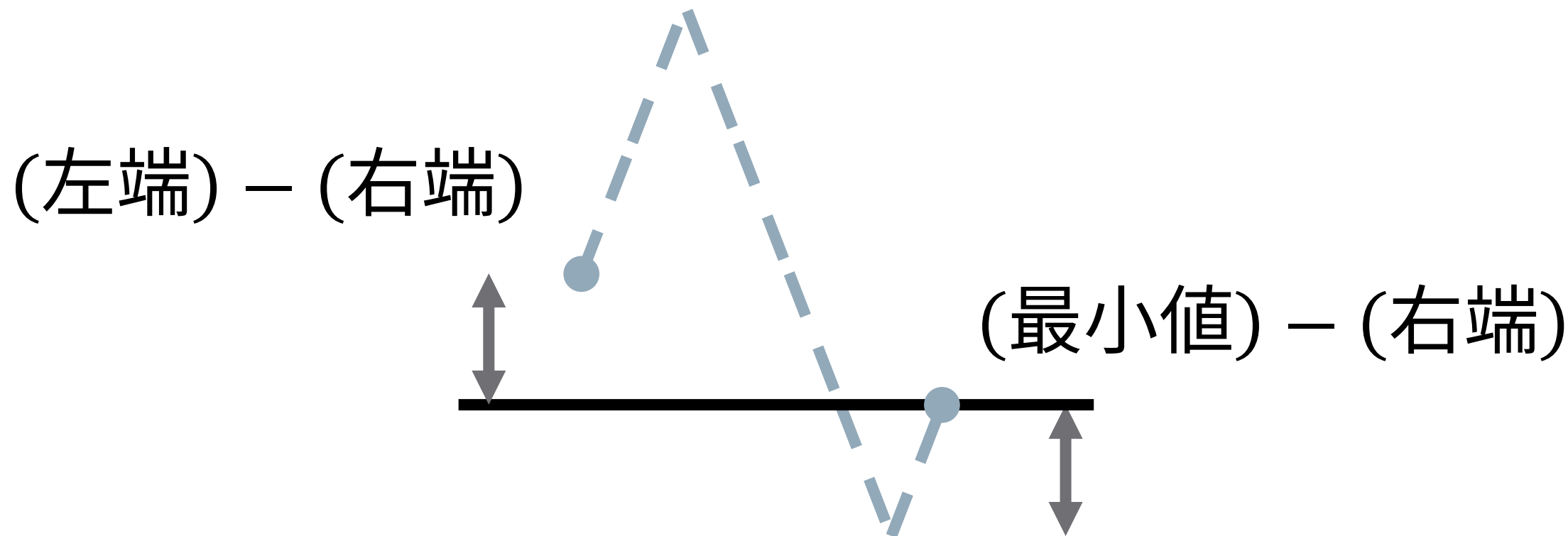
- 最大の不満度を 1 増やす
- 累積和の最小値が 1 減る
- 累積和の最後 ($x = 2N$ のとき) の値は必ず非負になる
($\because$ (男性の人数) $\leq N$)

満点解 – 解法

- 初期状態での累積和の最小値を T とする
- T が -1 以上ならなにも問題ない
- そうでないなら $-(T + 1)$ 人の男性を先頭に持っていく
- 累積和の最小値が -1 になる
- 最大の不満度は $-(T + 1)$ になる

満点解 – 実装

- 各 S_i, K_i について S_i を 1 つの区間として次の値を計算



満点解 – 解析

- 各区間ごとに $O(|S_i|)$
- 全区間で $O(|S|)$
- 時間計算量 $O(|S|) \rightarrow$ 小課題 3 が解ける

おしまい



Dungeon2 解説

問題概要

- 無向グラフがあります
- グラフを探索して、構造を決定してください
 - 各頂点からは、各辺に番号が付けられている
- 探索するために使えるものは、
 - 現在地から出ている辺の本数がわかる
 - 辺をたどって移動することができる
 - 最後にたどった辺が、現在地から何番目の辺かわかる
 - x 色の宝石を使って、頂点に情報を持たせておくことができる

構造決定？

- 別に必ずしもグラフの構造を決定する必要はない
- が、求める値は「最短距離が D の頂点对が何個あるか？」
- 構造を決定せずにこの値を求めるのは大変そう
- なので、グラフの構造を決定することを考える

「最短距離が D の頂点对が何個あるか？」

- 最短路問題
- 全点間の最短路を求めましょう
- $N \leq 200$ と小さいので大抵の方法が使えます
- Warshall-Floyd が実装簡潔，定数倍高速でおすすめです
 - $O(N^3)$ ，余裕

使える情報

- X 色の宝石
 - かなり重要そう
- 頂点から出ている辺の数(次数)
 - 使える??
 - どうせ「全頂点の次数が同じ」みたいな悪質なケースが入ってるんだろなあ

深さ優先探索

- 頂点に値を持たせられる(しかも初期値が決まっている)ので,「その頂点をすでに訪問した」というのは判定できるようになる
- 「どの辺からやってきたか」も常にわかる
 - 移動元へ戻ることは可能
- 深さ優先探索 (DFS) ができそう？

小課題 1

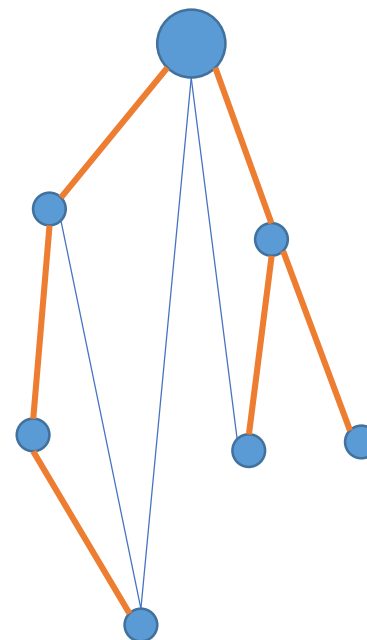
- DFS を行う
- 頂点の値は次のようにする
 - 1: まだ訪れていない (初期値)
 - それ以外: すでに訪れた (異なる頂点には異なる値を割り当てる)
- 各頂点では, 戻る方向以外のすべての辺をとりあえずたどってみて,
 - 未訪問だったら再帰的に探索
 - 訪問済みだったら, 割り当てられてる値を見て, 辺を張る
 - すべての辺をたどったら, 戻る方向の辺を使って元の頂点に戻る
- $X=100$ なので余裕
- 小課題 1 が解けて, 17 点 that 得られる

DFS 木

- この探索を行うときに、辺は 2 種類に分かれる
 1. 探索中の移動時に、移動先が未訪問となる辺
 2. 探索中の移動時に、移動先が訪問済みとなる辺（後退辺）
- 1. の辺だけを見ると全域木になっている

DFS 木の性質

- DFS の開始点を根として根付き木にして考える
- すると, 2. の辺は, ある頂点からその先祖の頂点を結ぶ辺になることがわかる



DFS 木の構成

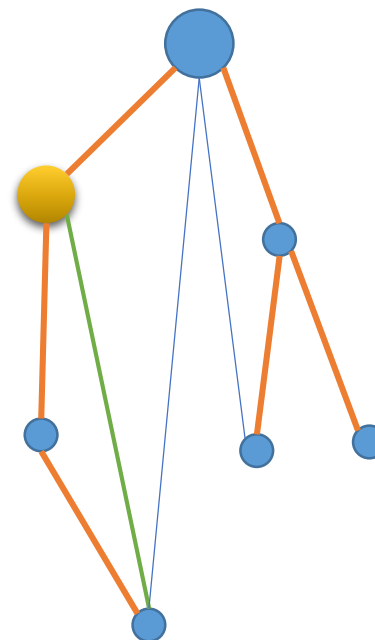
- 小課題 2 以降では $X=3$ なので, 3 種類の状態しか頂点に持たせられない
- が, 訪問済みかを覚えるだけで十分
- 小課題 1 と同様の方法で行える
 - ただし, 辺をたどった先がすでに訪問済みだった場合でも, 具体的にどの頂点につながっているのかは分からない
 - 後退辺が何かある, ことだけ覚えておく

小課題 2

- 後退辺の行先が決定できればよい
- 後退辺があったら、後退辺の行先の頂点にマークをつける
- 全域木を根まで遡って、マークが出てくるかどうか探す
 - マークがあれば、そこが後退辺の行先
 - マークがなければ、後退辺は葉方向へ延びているので、後退辺を逆向きに見るのに任せる
- 遡った後、マークを消すため再び後退辺をたどる
- 状態は 3 つあれば足りる
 1. 未訪問, マークなし
 2. 訪問済, マークなし
 3. 訪問済, マークあり (未訪問, マークなしは DFS をしている限り現れない)

マーク

- 緑の辺を下の頂点から見る場合は、上の頂点にマークをつけて、木の上をたどってマークを探索



小課題 2

- 移動回数を見積もる
 - DFS 木の移動のために $2(N - 1)$ 回
 - 後退辺は $M - N + 1$ 本ある
 - 各後退辺ごとに,
 - マークして戻るのに 2 回
 - マークを探すのに最大 $2(N - 1)$ 回
 - マークを解除するのに 2 回
 - よって最大 $2N + 2$ 回
 - よって $2(N - 1) + (M - N + 1)(2N + 2)$ 回以内で終了
- 小課題 2 は解けて合計 44 点が見られる

後退辺の行先の決定

- 各辺について、最悪根まで戻るというのはあまりに効率が悪い
- もっと効率よく決定したい
- ところで、後退辺の行先は、行先の「根からの深さ」さえ分かれば十分
 - 自分より深いところにあることが判明したら、逆向きで見るのに任せる
- 小課題 2 の解法の「マーク」がなければ、使う状態は 2 通りのみで余っている
 - 「訪問済み」のために 2 種類の状態を覚えられる

深さ情報の分散

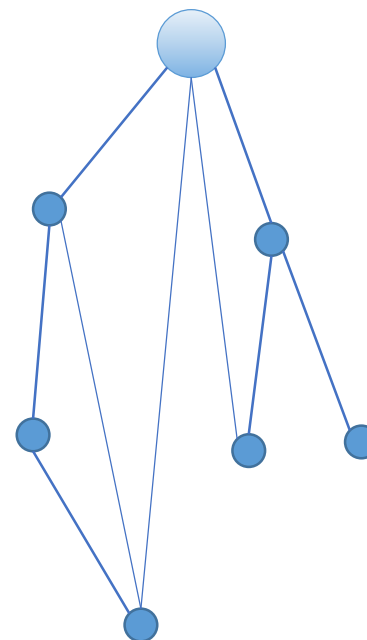
- 深さ情報は 8bit で表現できる
- 「訪問済み」のために使える 2 状態で、深さ情報を少しずつ伝える
 - DFS を 8 回行う
 - i 回目には、深さ情報の i ビット目を覚えておく

小課題 3 ($L = 64$)

- DFS を 8 回行い, i 回目には根からの深さの i ビット目を計算する
- 具体的には,
 - DFS で新たな頂点を訪問したときには, 深さに応じてその頂点に与える値を計算
 - 辺の行先がすでに訪問済みだったら, 行先の値を見て, 行先の深さ情報を追加
 - DFS の帰りがけにも頂点の値は変更しない
- DFS を行くと頂点の値がめちゃくちゃになる (すべて初期値 1 以外になる)
 - DFS 後に, 値消去用の DFS を走らせて, すべて初期値 1 にする
- 複数回 DFS を行うけど, 構造の一貫性は大丈夫?
 - 同じ方法で DFS を行えば大丈夫

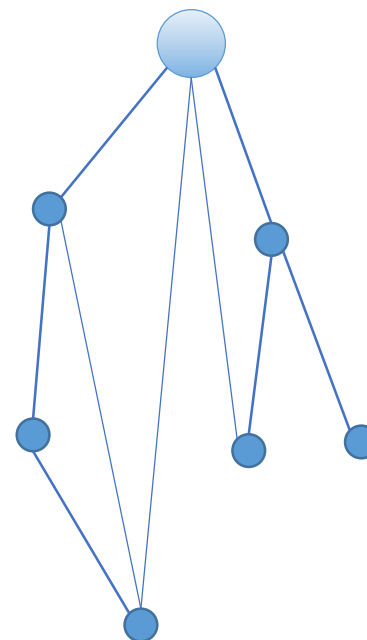
DFS の様子

- 最下位ビットを決定する DFS
- 現在一番上の頂点にいます



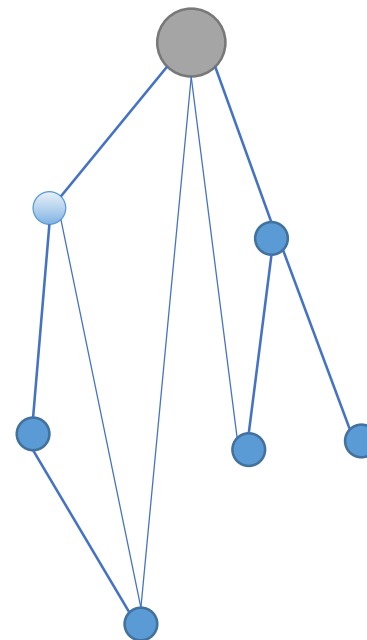
DFS の様子

- 出ている 3 本の辺のうち 1 本選んで移動
- 移動する前に, 頂点を「深さ情報:0」の色に更新



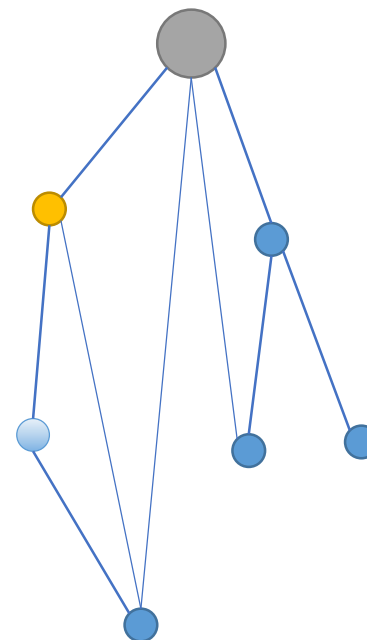
DFS の様子

- 今度は、深さが 1 なので 1 の色にする



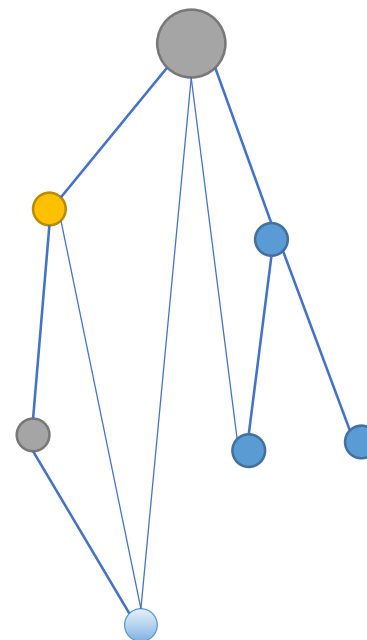
DFS の様子

- 今度は、深さが 2 なので 0 の色にする



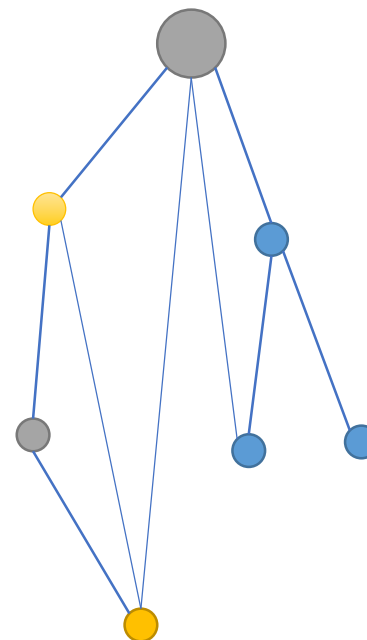
DFS の様子

- 今度は、深さが 3 なので 1 の色にする



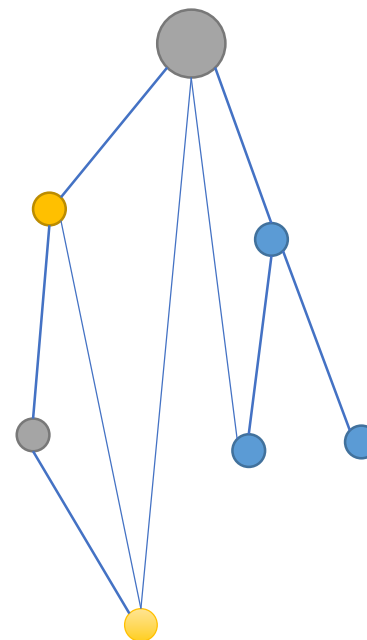
DFS の様子

- 深さが 4 なので, と思いきや
- 色が「深さ情報: 1」なので訪問済み
- 同じ色のまま引き返す



DFS の様子

- この頂点からの 2 本目の辺が,「深さの最下位ビットが 1」であると覚えておく



小課題 3 ($L = 64$)

- 移動回数を見積もる
 - 全域木の辺 $N - 1$ 本は各 DFS で 2 回 (1 往復) 通る
 - 後退辺 $M - N + 1$ 本は各 DFS で 4 回 (2 往復) 通る
 - よって各 DFS で $4M - 2N + 2$ 回移動
 - 値初期化用の DFS も同じ移動回数
- DFS は $8 + 8$ 回走るので, 合計 $64M - 32N + 32 < 64M$ 回の移動
 - 小課題 3 で $L = 64$ の場合が解け, 66 点が得られる

小課題 3 ($L = 60$)

- 値消去用の DFS は, 最後の DFS の後には不要
- DFS は $8 + 7$ 回走るので, 合計 $60M - 30N + 30 < 60M$ 回の移動
 - 小課題 3 で $L = 60$ の場合が解け, 68 点を得られる

値の消去？

- 値の消去ごときに 28M 回も使うのはバカバカしい
- うまく値を書き換えることで、値の消去を不要にできないか？

小課題 3 ($L = 32$)

- 帰りがけに, 値をすべて (深さの i ビット目: 1) に対応する状態に変更
- すると, DFS 終了時には「根を除いて」すべて同じ値になる
 - 次回の DFS では, 初期値を別な値と思って実行
- 後退辺のうち, 葉に向かう向きを見ると変なことが起きないか?
 - 深さが見かけ上, 必ず 255 (0b11111111) になっている
 - これは無視することにすれば OK
- すると, DFS の回数を 8 回に減らすことができる
- $L = 32$ の場合が解け, 82 点 that 得られる

全域木

- 1 回 DFS すると, 各頂点に勝手な番号が割り当てられる
- 各頂点から出ている各辺が次のいずれかなのかもわかる
 - 全域木の辺, 葉へ向かう方向
 - 全域木の辺, 根へ向かう方向
 - 後退辺
- 1 回 DFS してしまえば, 全域木をたどるのは頂点の値すら見ずに可能

小課題 3 ($L = 24$)

- 1 回 DFS して, 全域木の構造を把握
- 2 回目以降の DFS では, 頂点に深さ情報のために 3 通りの値を持たせ, 後退辺の情報を取得
- $3^5 > 200$ なので, DFS は 1 + 5 回で十分
- 各 DFS は 4M 回以下の移動で行えるので, 全体で 24M 回
- $L = 24$ の場合が解け, 90 点が得られる

後退辺 2 往復？

- 後退辺は，根へ向かう向きするとき以外は情報を得られない
- 無駄な後退辺の移動を減らしたい
- DFS 1 回使うと，各後退辺の方向もわかる

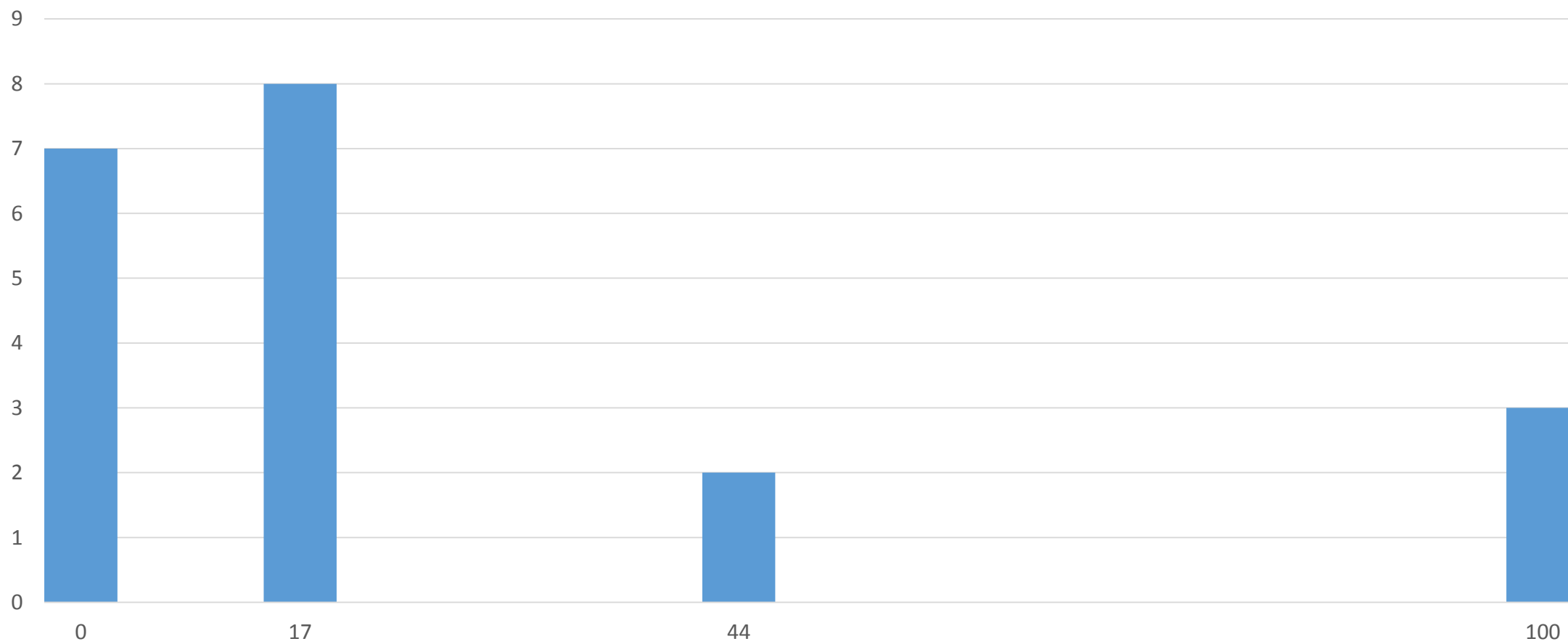
後退辺の向きを知る DFS

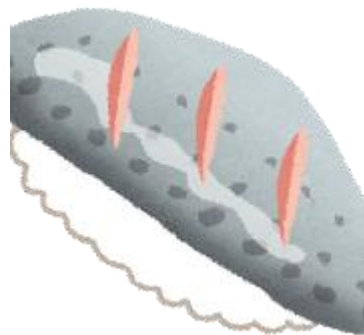
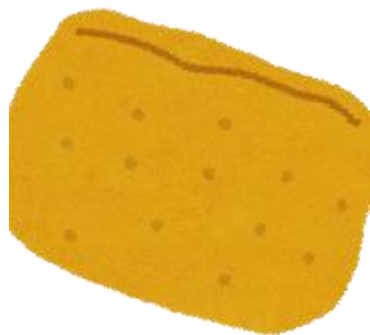
- 3通りの状態
 1. 未訪問
 2. 訪問済みだが、完了していない
 3. 訪問済み, しかも完了済み(帰りがけにこの状態に変更)
- DFS 中では, 行先の状態に応じて辺の種類が決まる
 1. 全域木の辺
 2. 後退辺(根へ向かう向き)
 3. 後退辺(葉へ向かう向き)

満点解法

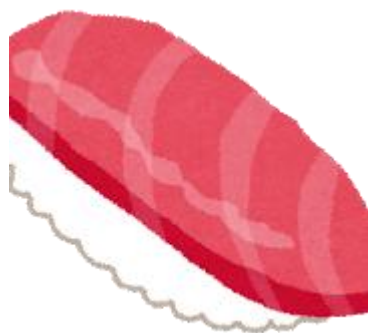
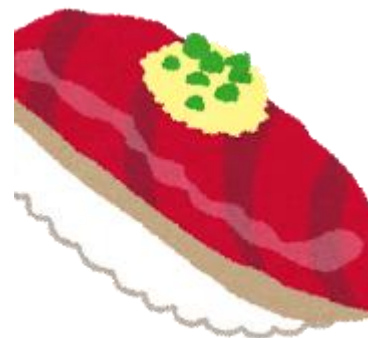
- まず, 前スライドの方法で, 各辺の種類を把握
 - 以降の DFS では, 後退辺も 1 往復ずつしか見ない
- 2 回目以降の DFS で, 各頂点に, 根からの深さについての 3 通りの状態を持たせる
- DFS は $1 + 5$ 回
 - 最初の 1 回は $4M$ 回以下の移動
 - あとの 5 回はそれぞれ $2M$ 回以下の移動
- $14M$ 回以下の移動で済み, 満点が得られる

得点分布

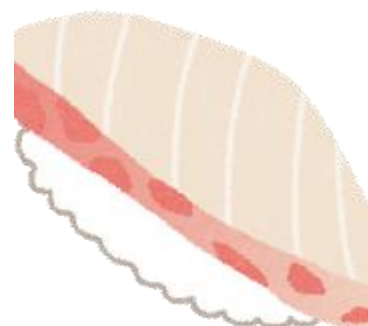




回転寿司 解説



DEGwer



問題概要

- 円環状に皿を持った人が N 人並んでいる
- i 人目の人の持っている皿の価格は最初 x_i
- 次の処理を行った時の変数 x の値を答えよというクエリ Q 個に答えよ
- $x = p_i$ (ベルトコンベア上の皿の価格を表す)
- $for(k: s_i, \dots, t_i) if(x < x_k) swap(x, x_k)$
– (より安い皿が目の前であれば交換)
- $N \leq 400000, Q \leq 25000$

問題概要

- 円環状に人が N 人並んでいて、皿を1枚ずつ持っている
- i 人目の持つ皿の価格は最初 x_i
- 各人は、ベルトコンベアの自分の目の前に皿がある場合、自分の皿の価格と比較して小さいほうを手元に置き、大きいほうをベルトコンベアに乗せる
- 人 s に皿を渡し、人 t から皿を回収したとき、回収
- $N \leq 400000, Q \leq 25000$

とりあえず

- 直線として考えます
- 皿の経路が0地点を通るときは、そのクエリを始点から N 番目までと1番目から終点までに分けて考えればいい

小課題1(5点)

- 書いてある通りに実装してください
- $O(NQ)$ となり、5点が得られる

考察

- まず、ベルトコンベアの気持ちになって考えてみる



考察

- 各クエリに対し、ベルトコンベアに最後に乗っている皿の価格は、操作に関係した皿の価格の最大値になっているはず
 - そうでなければ、交換されているはずなので矛盾
- 正確には、区間 $[s, t]$ に、 $x = p$ としてクエリの操作を行った場合、最後には $\max(x_s, \dots, x_t, p)$ が x に入っているはず

小課題2(15点)

- $s_i = 1, t_i = N$
- クエリごとに $\max(x_1, \dots, x_N, p)$ を求めればよい
 - そこで求めた価格最大の皿は消滅するので、それ以降操作にかかわらないので、適当にないことにしてやる

小課題2(15点)

- これはpriority_queueでできることそのもの
 - priority_queueに p をpush
 - priority_queueのmaxを出力してpop
- を各クエリに対してやればいい
- $O(Q \log N)$ で、15点が得られる
 - 適当に分岐をすれば20点(575)

満点解法

- $N \leq 400000, Q \leq 25000$



考察

- 今度は人の気持ちになって考えてみる
- 同じ区間にたくさんクエリが来たとき、各人は価格いくらの皿を持っているか？
- 正確には、同じ区間に $x = p_1, p_2, \dots, p_k$ の順にクエリの操作を施した時、残る皿の列はどうなっているか？

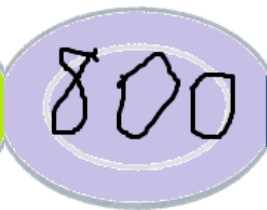
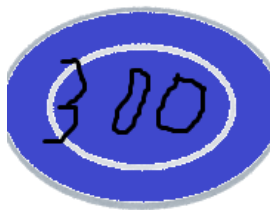
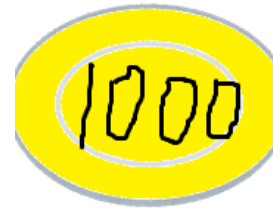
考察

- ここで、交換の様子を見てみる
 - 交換操作は、人の手元の皿とベルトコンベアの人の前にある位置の皿の価格を比べ、
 - 大きいほうをベルトコンベアに
 - 小さいほうを人の前に
 - 置く操作と言える
-
- これは人の列とベルトコンベアについて対称！

考察

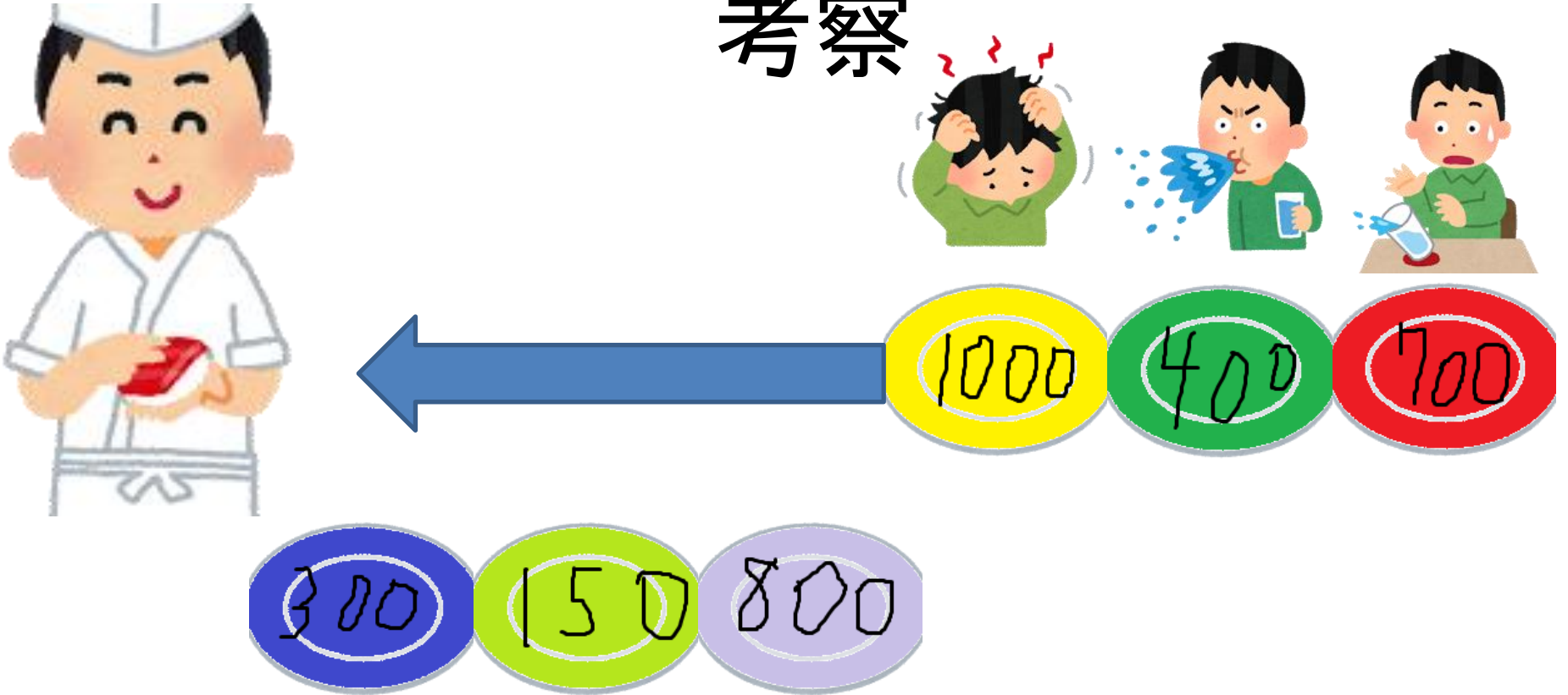
- つまり、同じ区間に対するクエリの操作は、
- 「人の列の前をベルトコンベア上の皿が通過する」とも
- 「ベルトコンベア上の皿の列の前を人が通過する」とも考えることができる

考察



皿がこの順にこの区間を通過するとする

考察



この操作は、人がこの順にベルトコンベアの前を通過すると考えても問題ない

考察

- つまり、同じ区間に対するクエリの操作において、小課題2と同じ操作によって、一連の操作後の各人が持っている皿の価格を高速に求めることができる
 - クエリで来る皿の列をpriority_queueに入れて同様の操作をすることで実現
 - ただし、符号だけ反転
- 具体的には $O((\text{区間長})\log(\text{クエリの数}))$

満点解法

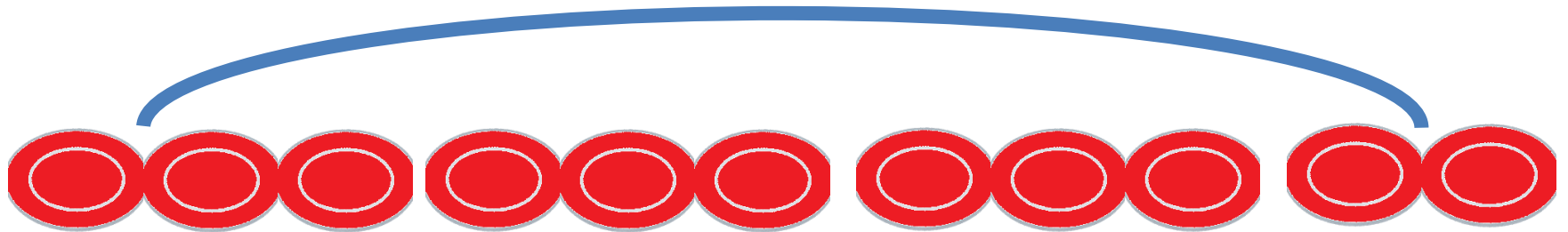
- いい性質がわかったので、ここで平方分割を考えます
- 2通りのやり方がある
 - ①列の平方分割
 - ②クエリの平方分割
- 両方紹介します
- 本質的にはだいたい同じ

満点解法①

- 列の平方分割
- 人の列を連続B個ごとのバケットに分割する
- 各区間についてpriority_queueを持ち、その区間の人のもつ皿の価格を全部入れておく

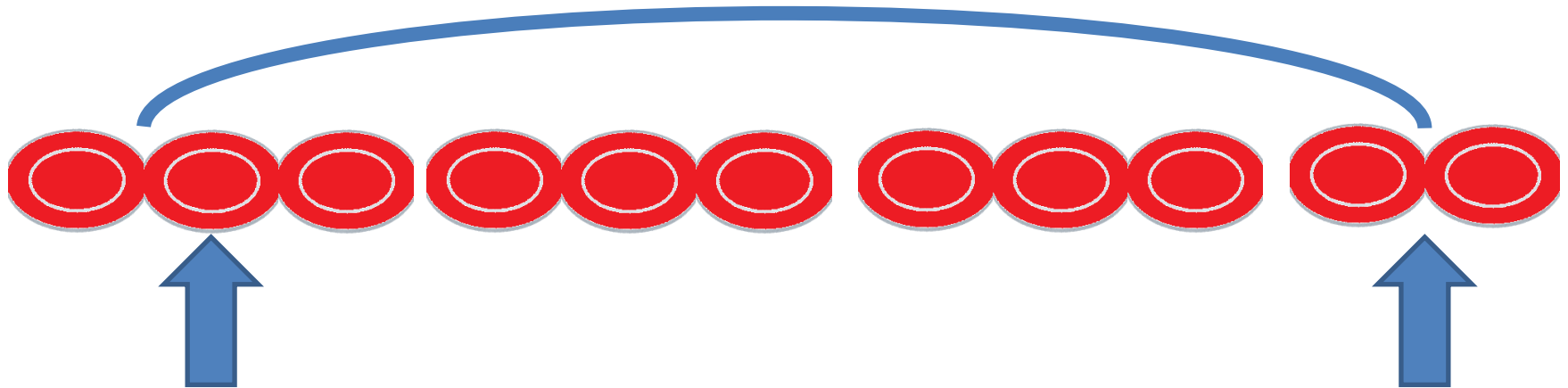
満点解法①

- この区間にクエリが来るとする



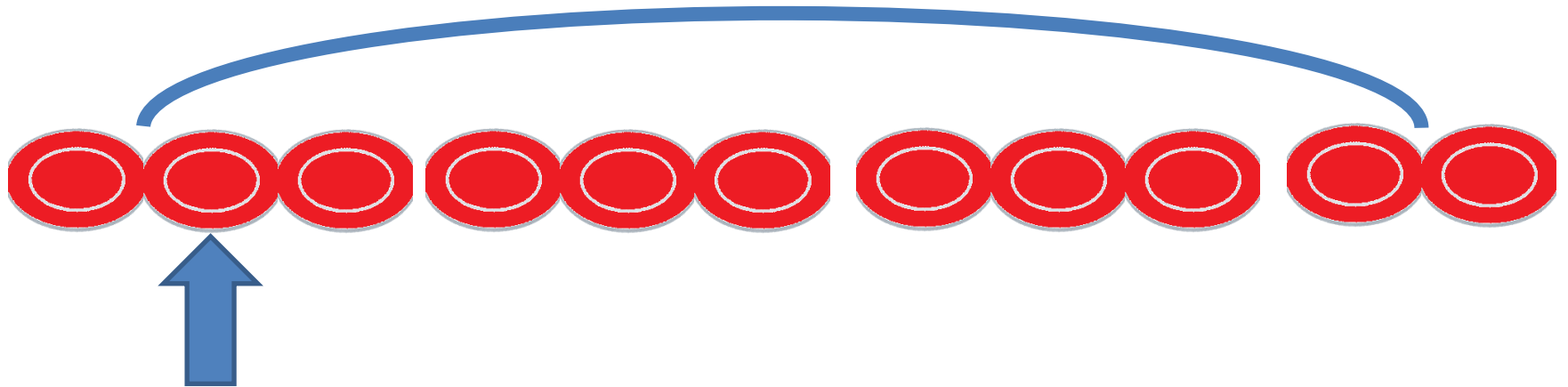
満点解法①

- 区間の端が存在するバケットでは、あらかじめ**後述の操作**をしておくことで、各皿の価格がわかるようにしておく



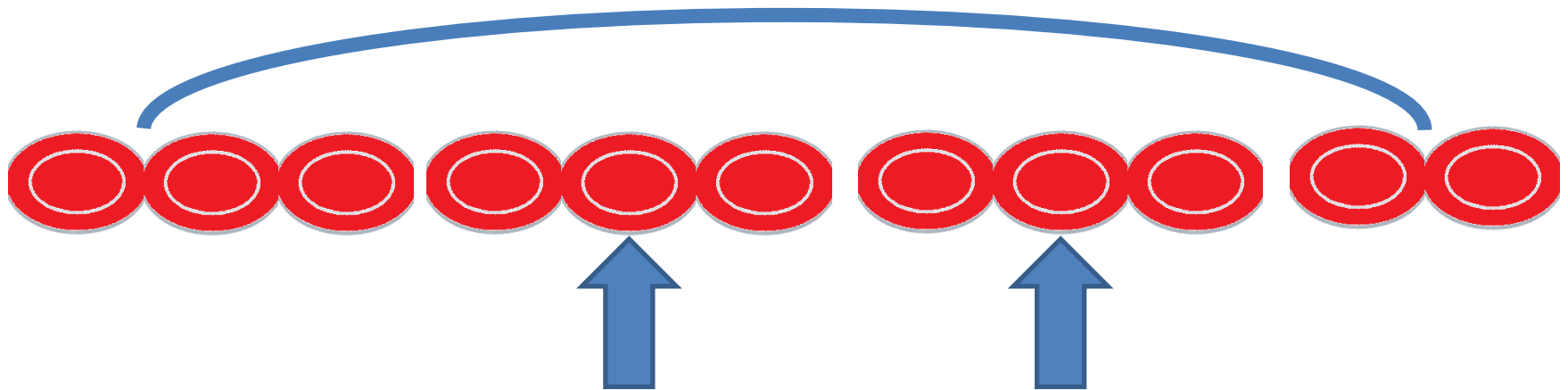
満点解法①

- このバケットについては、愚直に操作を行う



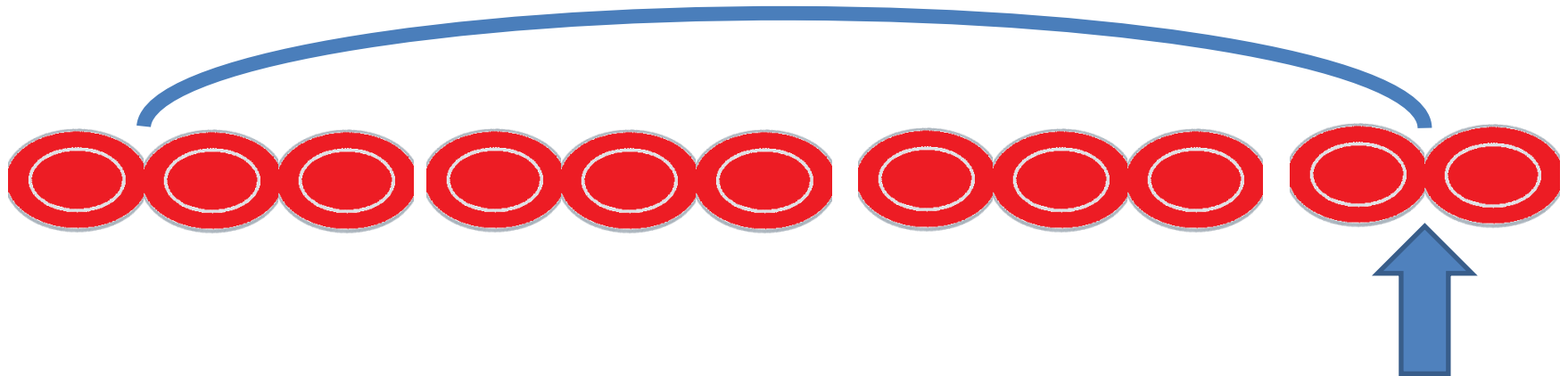
満点解法①

- これらのバケットについては、前のバケットから出てきた値をその区間の持つ priority_queue に入れ、最大値を取り出して次のバケットに渡す
 - ついでに諸事情によりその区間に入ってきた値を vector に入れておく



満点解法①

- このバケットについても、愚直に操作を行う



満点解法①

- 区間の皿の価格の列を復元する操作がまだ残っている
- いま、元の価格列の値と、その区間に入ってきた皿の価格の値をすべて持っておいているので、これは先ほど考察した状況と一緒
- よってこの操作はならし $O(B \log Q)$ でできる

満点解法①

- 以上をまとめると、
- バケットの端のある区間に対し、 $O(B \log Q)$
- そうでない区間に対し、 $O(\frac{N}{B} \log B)$
- で操作ができる
- $B = \sqrt{N}$ とすれば、クエリあたりならし
 $O(\sqrt{N}(\log N + \log Q))$ 、全体で
 $O(Q\sqrt{N}(\log N + \log Q))$ で満点が得られる

満点解法②

- クエリの平方分割
 - たぶんこっちのほうが実装が楽で、定数倍も速いです
 - ただしオフライン
- クエりを連続 B 個ごとのバケットに分割する
 - こうしておくと、このバケットに現れるクエリの区間の始点と終点の個数は高々 $2B$ 個

満点解法②

- クエリの区間の端点の組で座標圧縮をし、できた各区間にpriority_queueを持たせる
- バケット内の各クエリに対し、priority_queueに適切な操作を行う
- バケット内の各クエリには、 $O(\text{Blog } \frac{N}{B})$ で答えられる

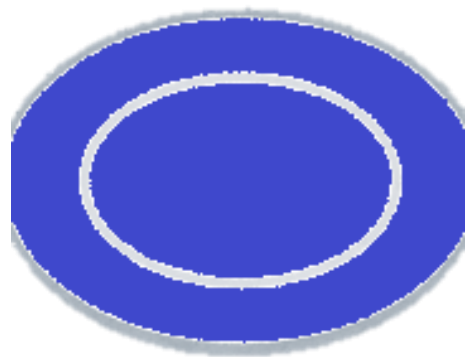
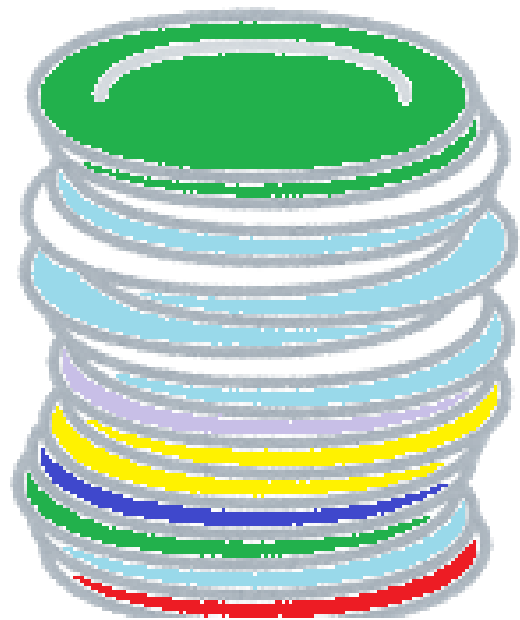
満点解法②

- 各バケットを処理したら、元の配列を復元する
- この操作は、先ほどと同様にやると $O(N \log B)$ でできる

満点解法②

- まとめると、各バケットについて
- $O(B^2 \log \frac{N}{B} + N \log B)$ でできる
- バケットは全部で $O(\frac{Q}{B})$ 個あるので、
- 全体で $O(QB \log \frac{N}{B} + \frac{QN}{B} \log B)$ でできる
- $B = \sqrt{N}$ とすれば、全体で $O(Q\sqrt{N} \log N)$ となり、満点が得られる

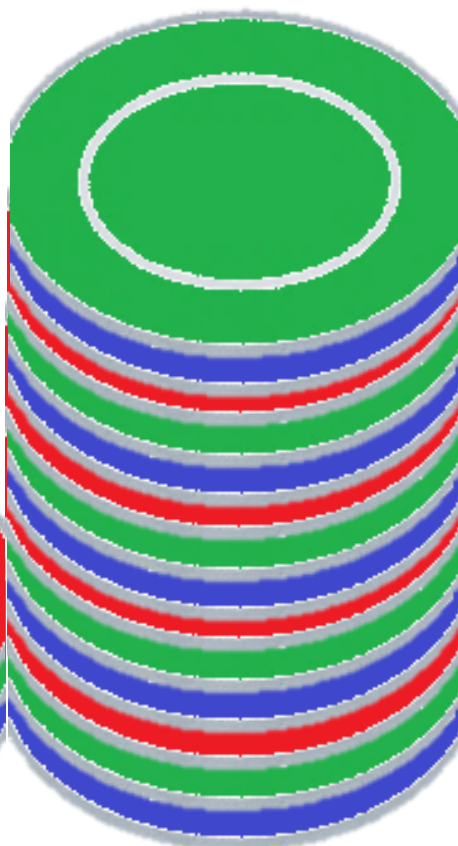
得点分布



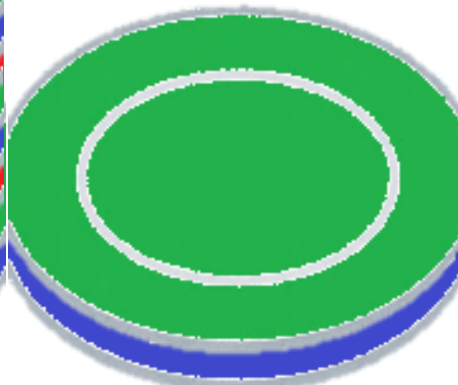
0点



5点



20点



100点

TELEGRAM

電報

海一浦箋

— • —

— — — • •

• • — •

— • — •

• • •

— • •

問題概要

- 全頂点の出次数が1の有向グラフがある(Functional Graph)
- コストを払えば辺の行先を変えられる
- 全体を強連結にしたい
- 必要な最小のコストは？



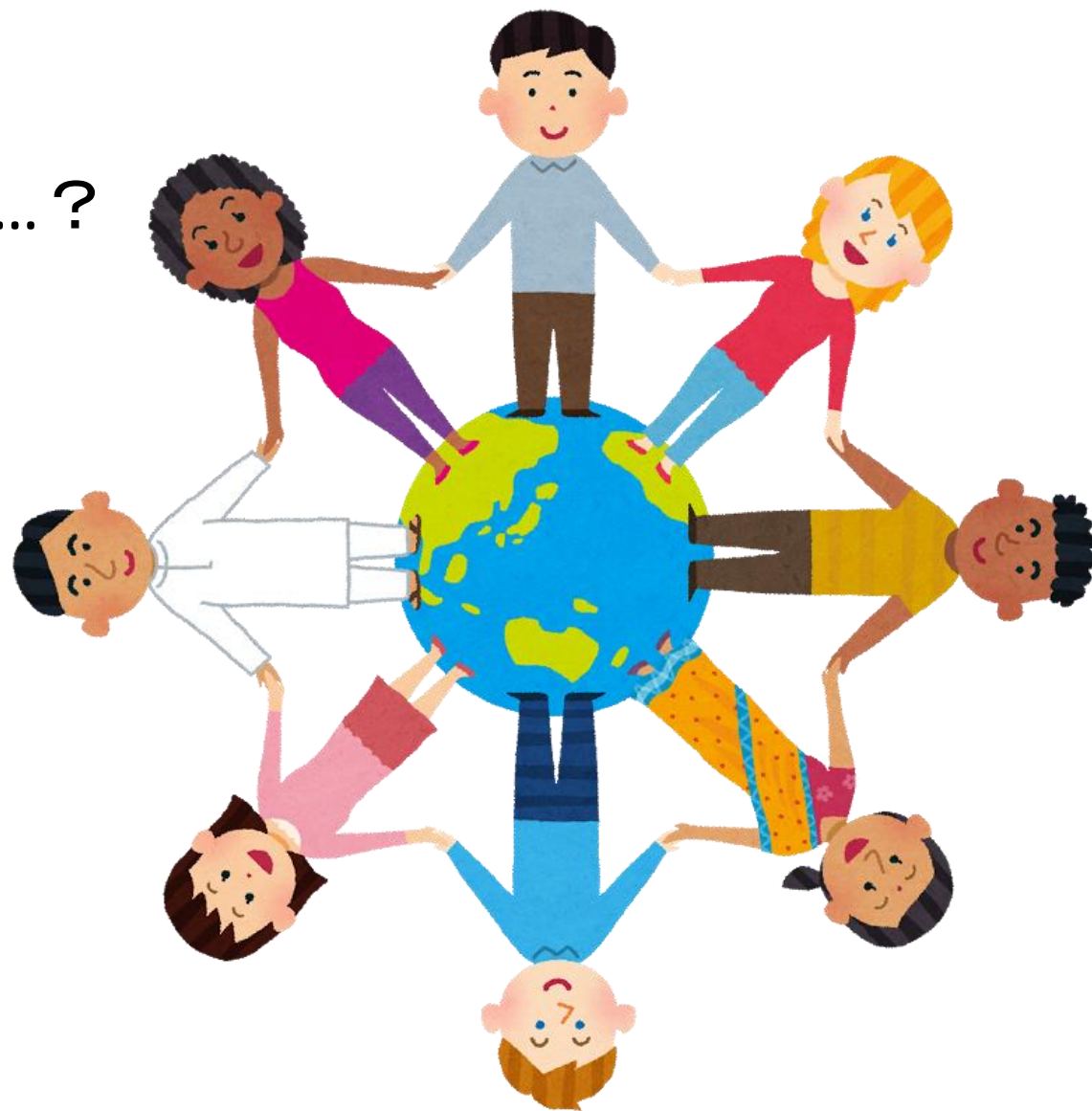
超重要考察

- 強連結なFunctional Graphとは.....？

超重要考察

- 強連結なFunctional Graphとは.....？

• 輪



全探索解法



- N 個の頂点を輪状に並べる方法は $(N-1)!$ 通り
- それぞれの並べ方について、その状態にするのにかかるコストを求める
- 行き先を変更する必要がある辺を調べてコストを足すだけ $O(N)$
- 全体で $O(N!)$
- Subtask 1 が解けた

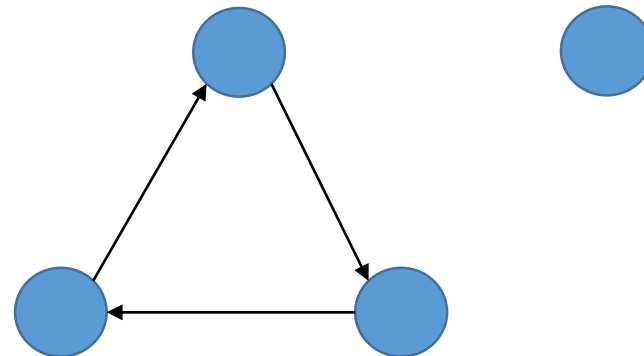
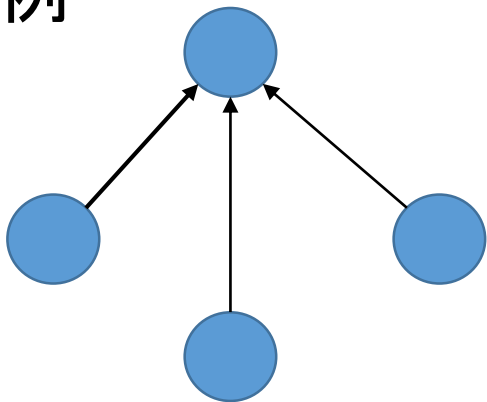
重要考察

- コストは「どの辺を行先を変更せずに残すか」のみによって決まる
- 変更にかかるコスト = (全辺のコストの合計) - (残す辺のコストの合計)
- ありえる「残す辺の集合」のうちコストの合計が最大のものを求めればよい。



「残す辺の集合」としてあり得るものは？

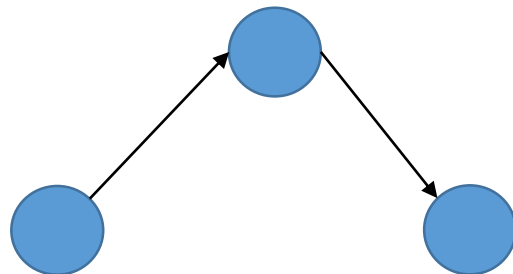
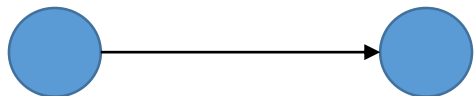
- サイクルグラフの部分グラフになっていればよい
- 駄目な例



- 入次数が2以上の頂点がある → 駄目
- サイクルがある(全体がサイクルの場合を除く) → 駄目

「残す辺の集合」としてあり得るものは？

- 逆に全頂点の入次数が1以下でサイクルがないとき
- (注意: 出次数は常に1以下)



- いくつかのパスからなるグラフ → 適当につなげれば輪になる
- 必要十分条件が求まった

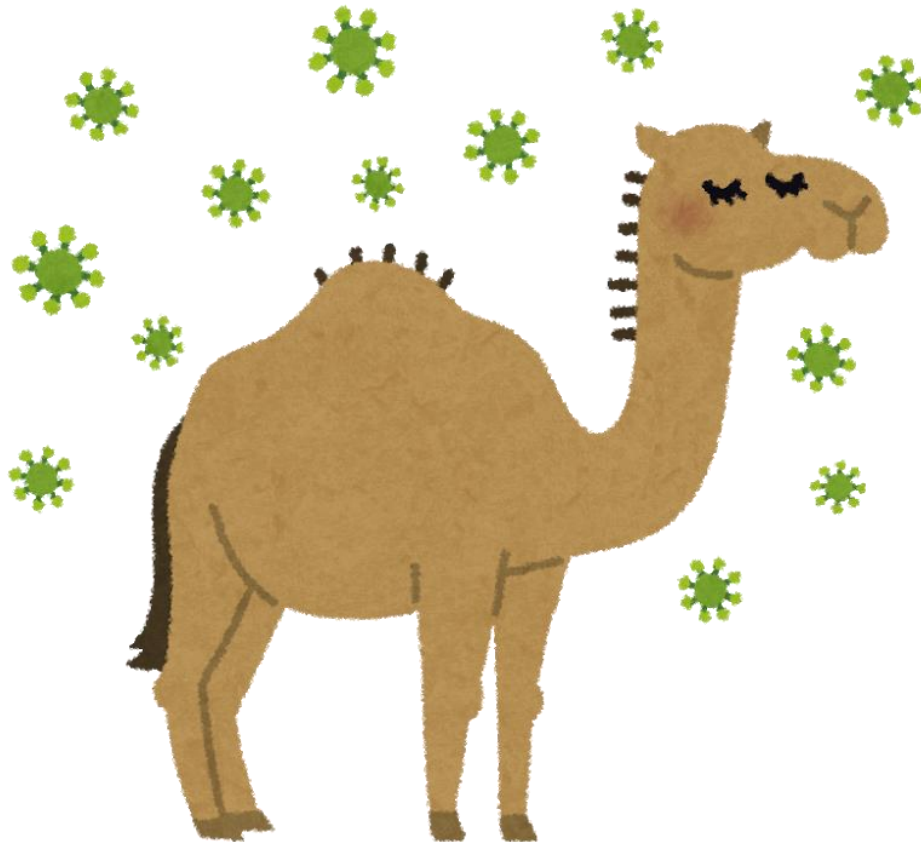
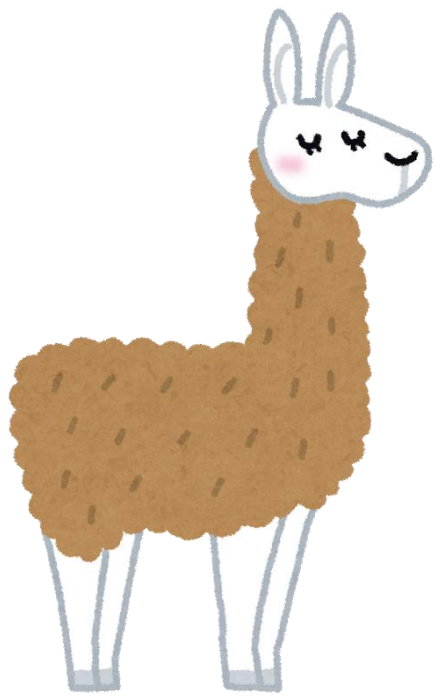
問題を言い換える

- 「全頂点の入次数が1以下で、サイクルを含まない部分グラフのうち、コストの合計が最大のものを求めよ」
- ただし初めから輪になっている場合は例外として処理
- 辺集合の部分集合の個数: 2^N
- 条件を確かめる: $O(N)$ (実装については後述)
- 全体で $O(2^N N)$
- Subtask 2 が解けた
- ※bit DP でもSubtask 2 は解ける



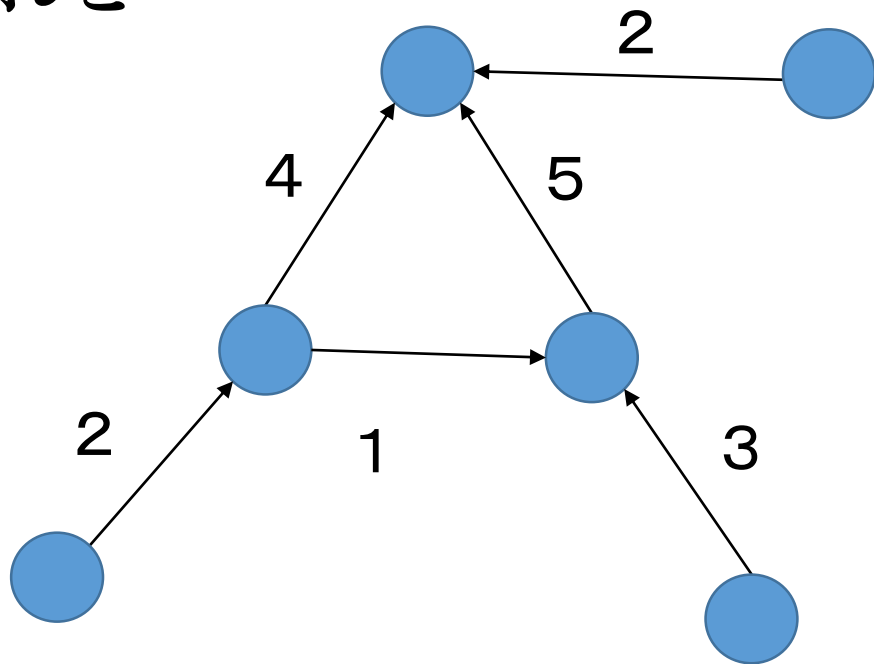
Greedy でできないか？

- 「サイクルをふくまない」という条件はとりあえず無視して「全頂点の入次数が1以下」だけ考える
- すると Greedy で簡単に解ける



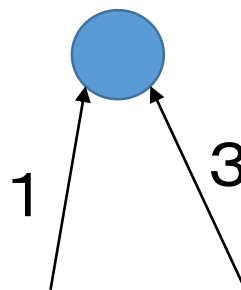
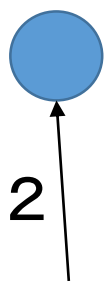
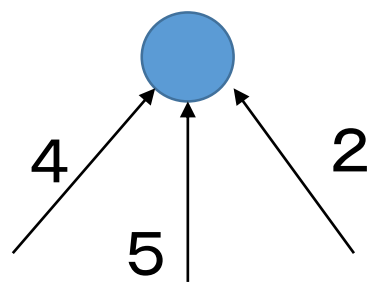
Greedy でできないか？

- 辺を行先ごとにグループ分け
- これを……



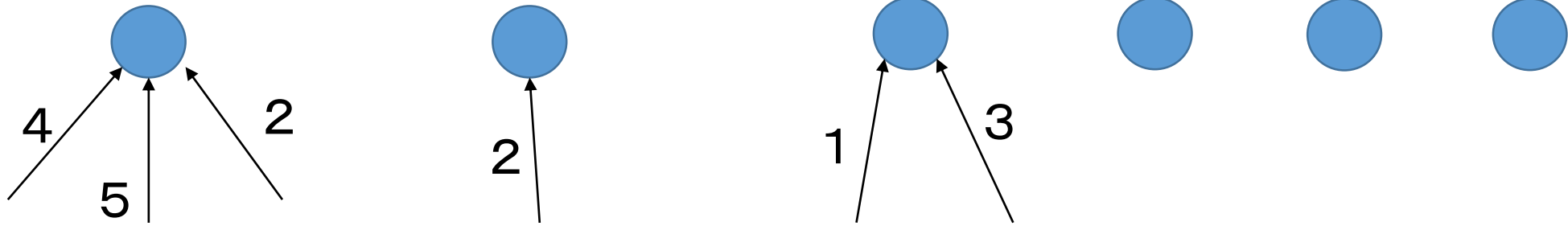
Greedy でできないか？

- 辺を行先ごとにグループ分け
- こうして……



Greedy でできないか？

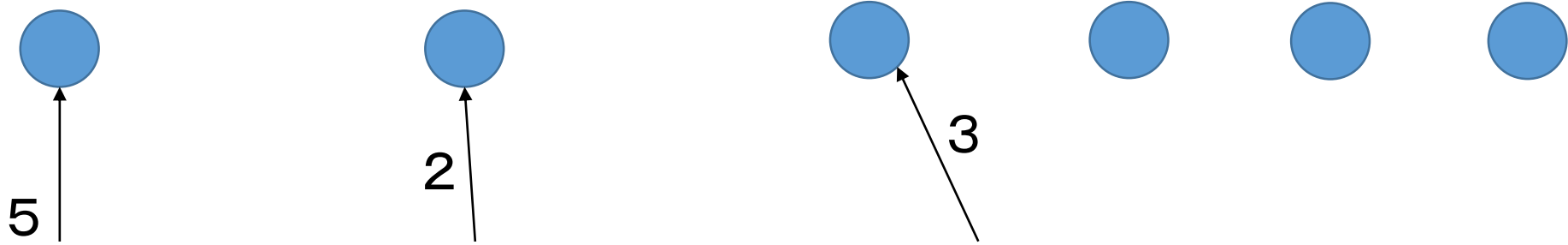
- 辺を行先ごとにグループ分け
- こうして……



- 各グループで最もコストの大きい辺をとる

Greedy でできないか？

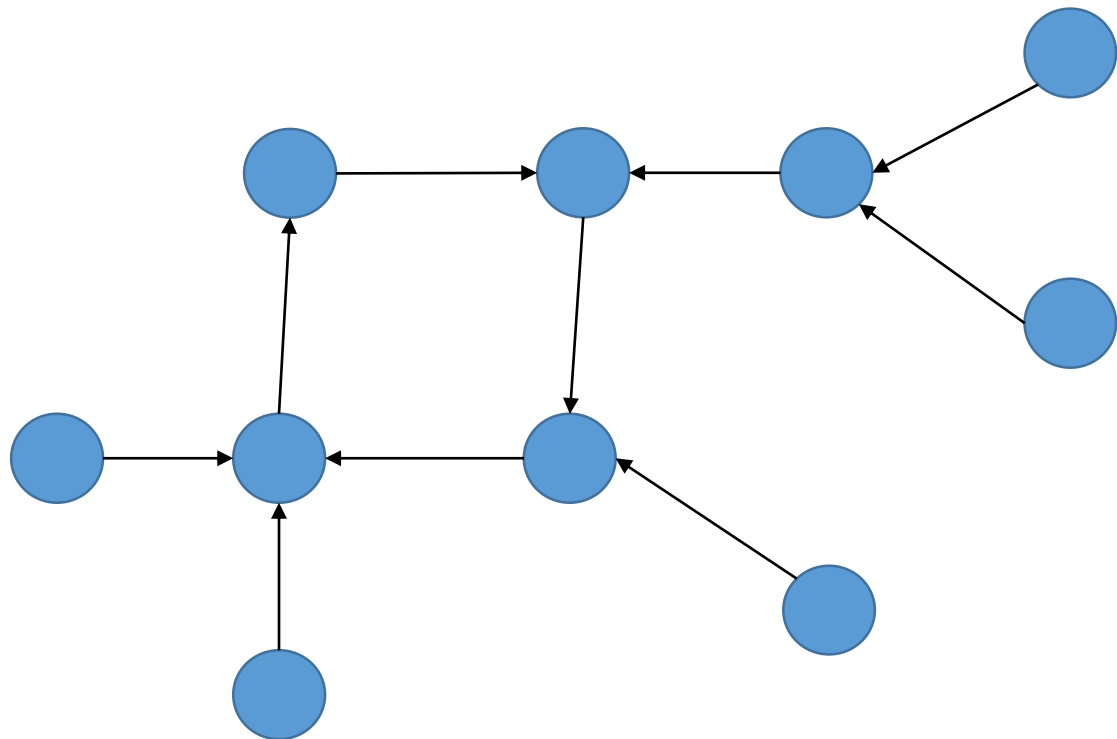
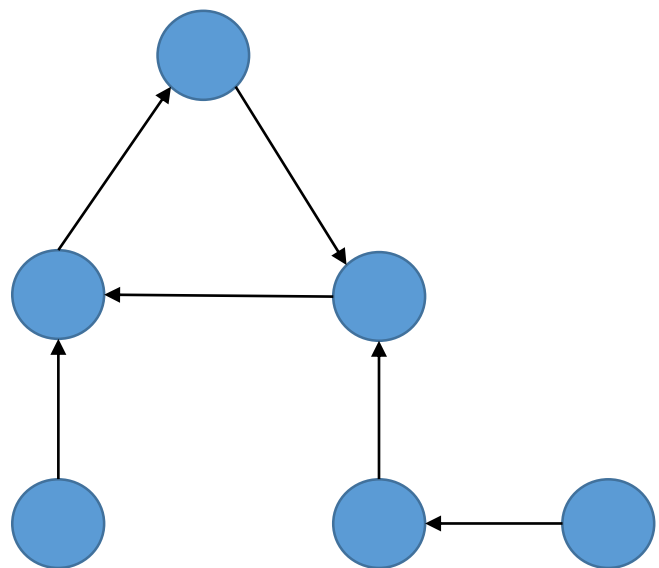
- 辺を行先ごとにグループ分け
- こうじゃ！



- 各グループで最もコストの大きい辺をとる

Functional Graph の形

- 「サイクルをふくまない」を考慮する
- Functional Graph における各弱連結成分は以下のような一つのサイクルに木がくっついた形である。サイクル同士が重なったりはしない。



サイクル上のどの辺を切るか？

- サイクル上の辺のうち少なくとも一つを切る（行先を変更する）必要がある
- とりあえず切る辺を全探索してみる
- 正確には各弱連結成分に対して、以下を行う
 - ① サイクルの辺から「必ず切る辺」をひとつ選ぶ
 - ② 残りの辺に対して先ほどの Greedy を行う
 - ③ これをサイクルの各辺について試し、最良の結果を記録



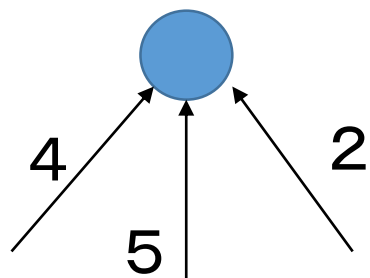
サイクル上のどの辺を切るか？

- 「必ず切る辺」の選び方は高々 N 通り
- Greedy は $O(N)$
- 計算量は $O(N^2)$
- → Subtask 3 まで解ける

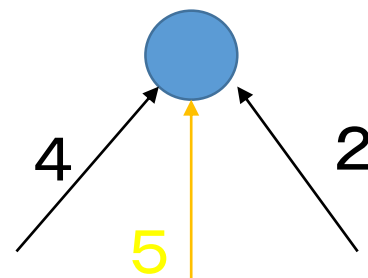


満点解法

- さっきの解法は明らかに無駄が多い
- 「必ず切る辺」を決めたことで影響を受けるのはその辺が属するグループの結果のみ



最大コスト:5



最大コスト:4

- (その辺を含む最大コスト) - (その辺以外の最大コスト)だけコストが減少

満点解法

- まずサイクルの条件を無視して Greedy
- 各サイクルの各辺に対し、その辺を「必ず切る辺」に決めたときに合計コストがいくつ減少するかを求める
- 各サイクルに対し、減少コストが最小の辺を選ぶ
- $\rightarrow O(N) \rightarrow$ 満点

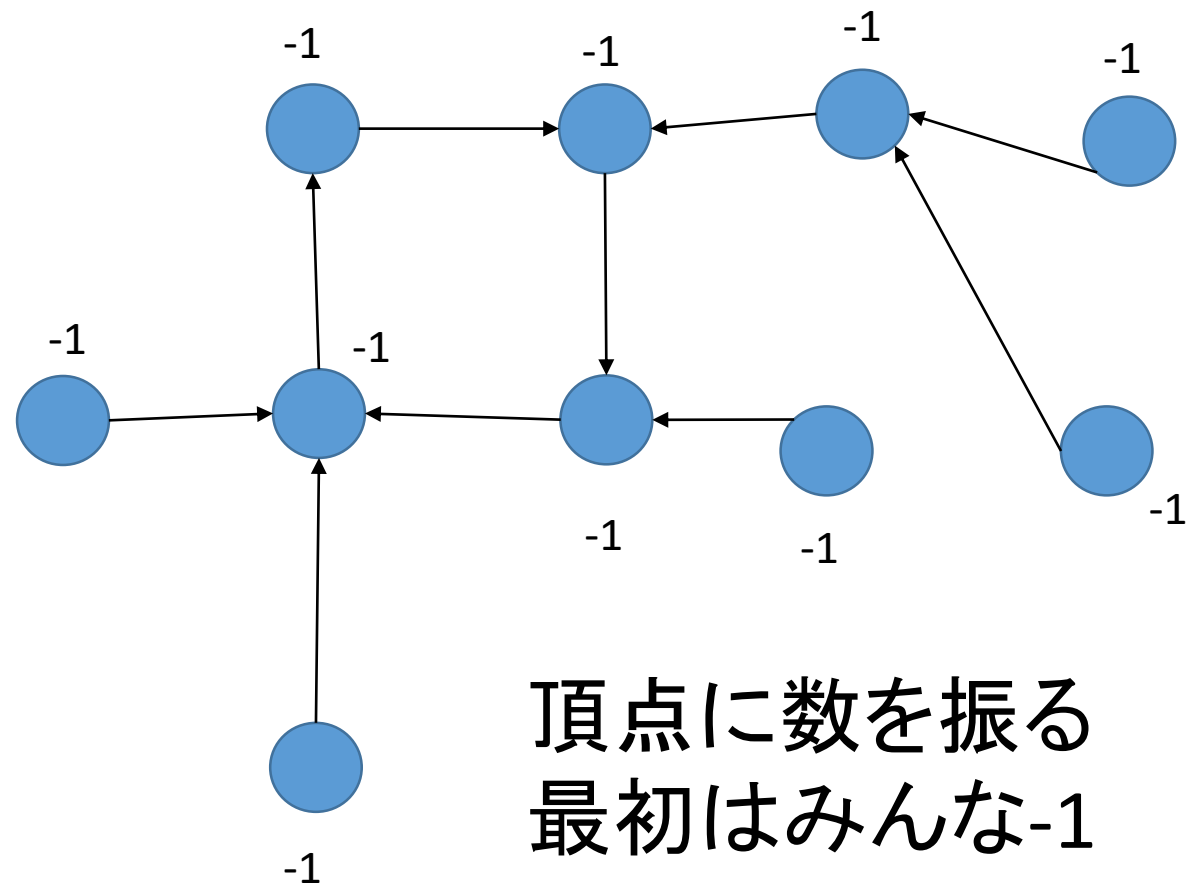
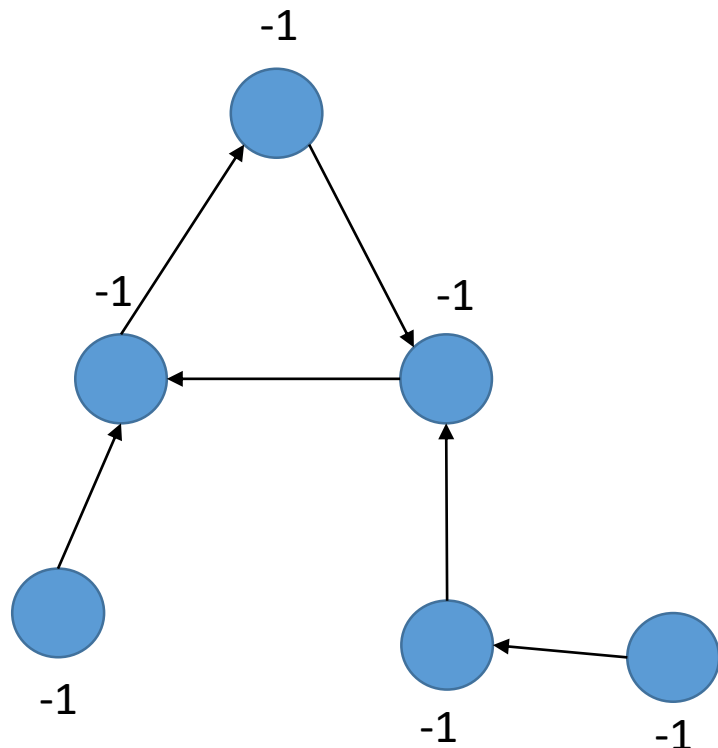


Functional Graph の閉路検出の実装

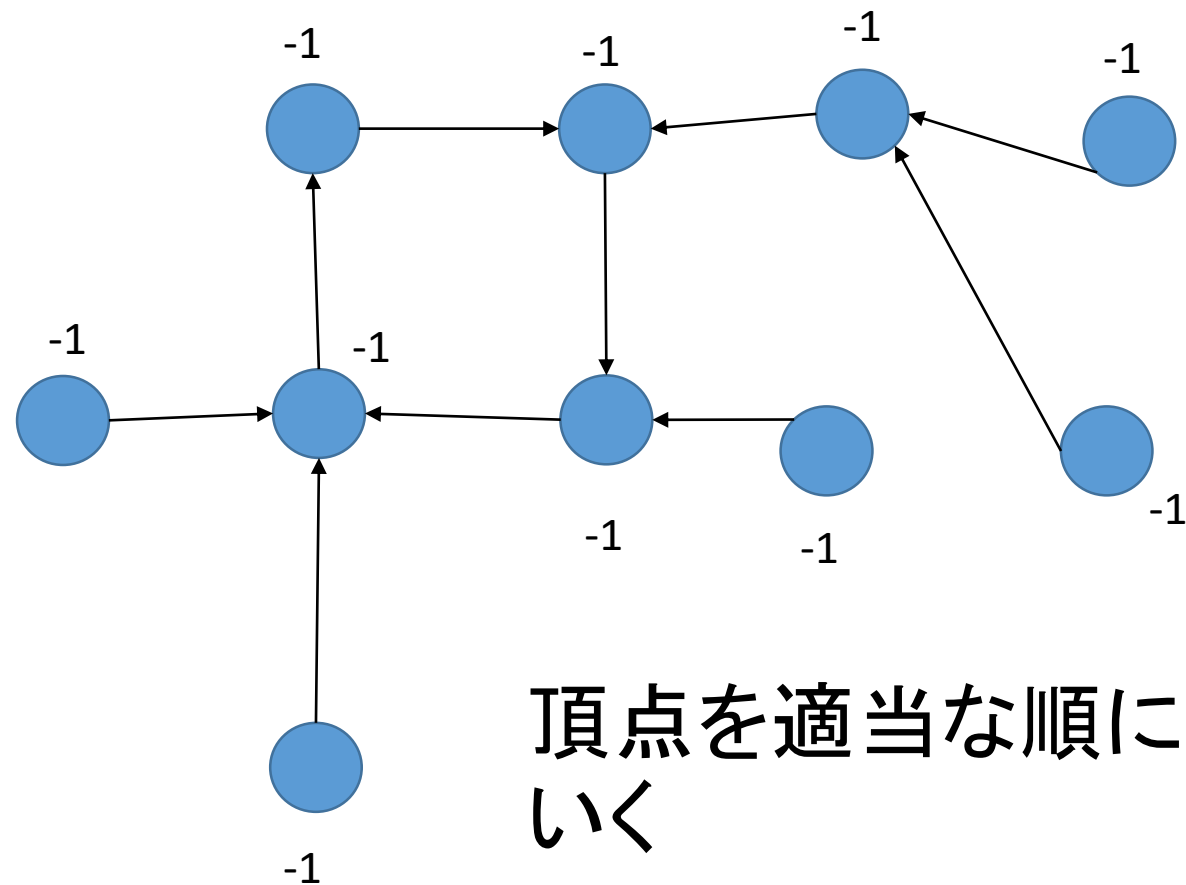
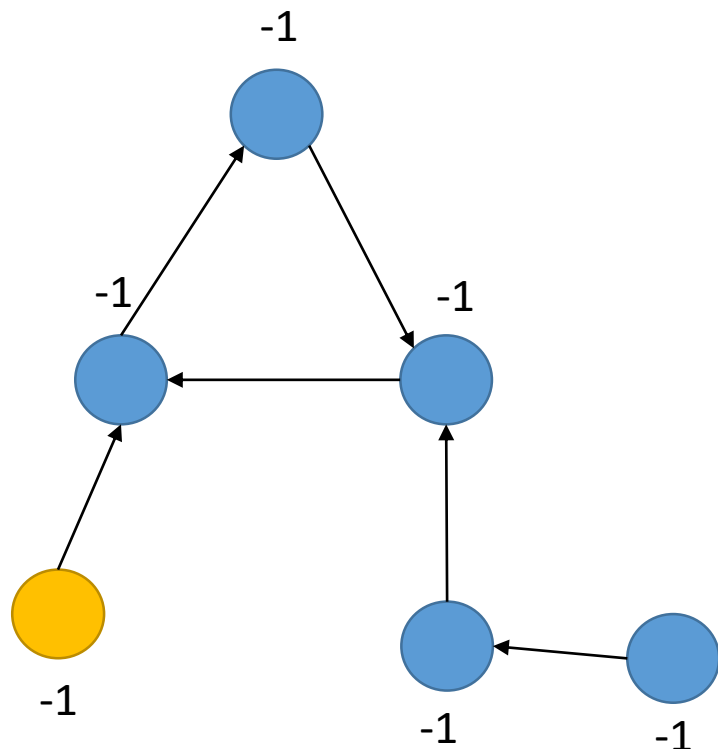
- JOI春合宿で定期的に出題されている
- 2011年春 報告 (Report)
- 2013年春 プレゼント (Presents)
- 「プレゼント」の解説スライドが詳しい



おすすめの実装

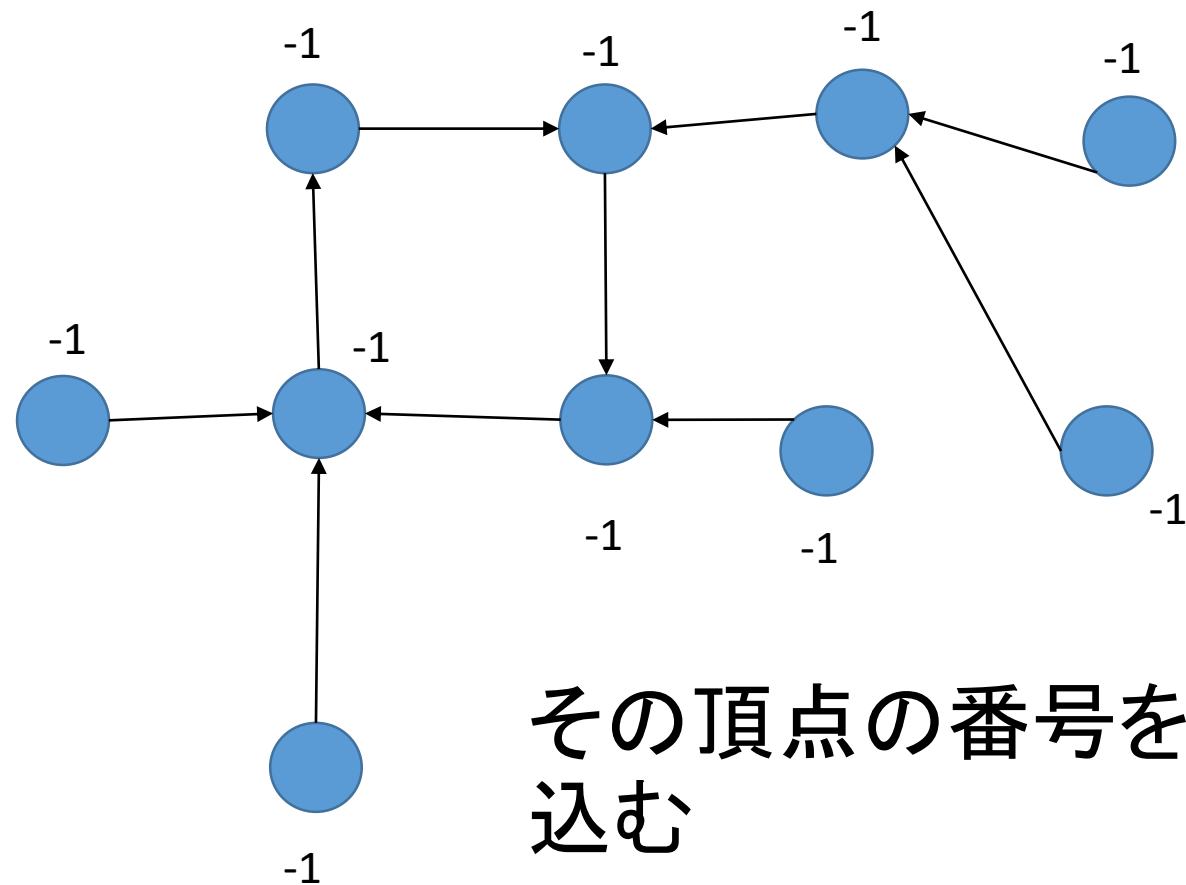
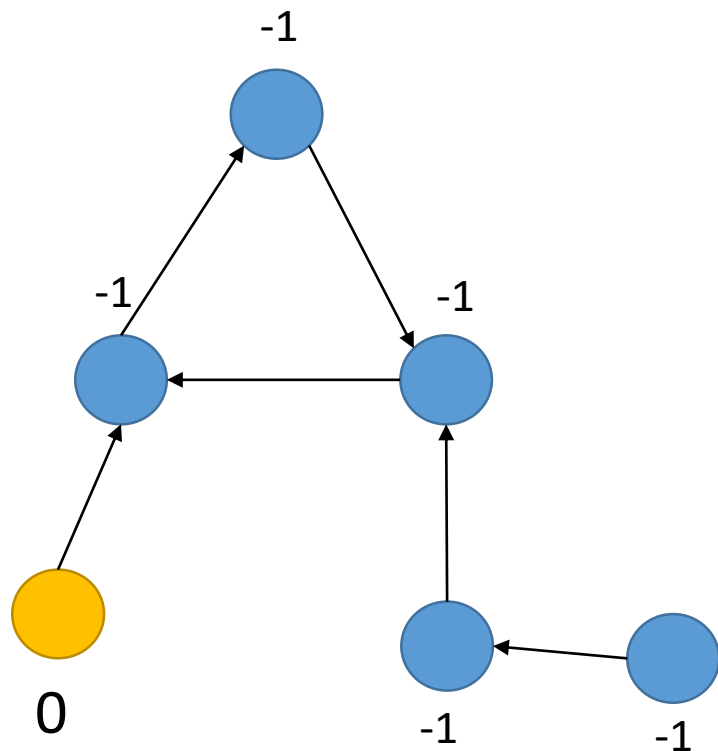


おすすめの実装



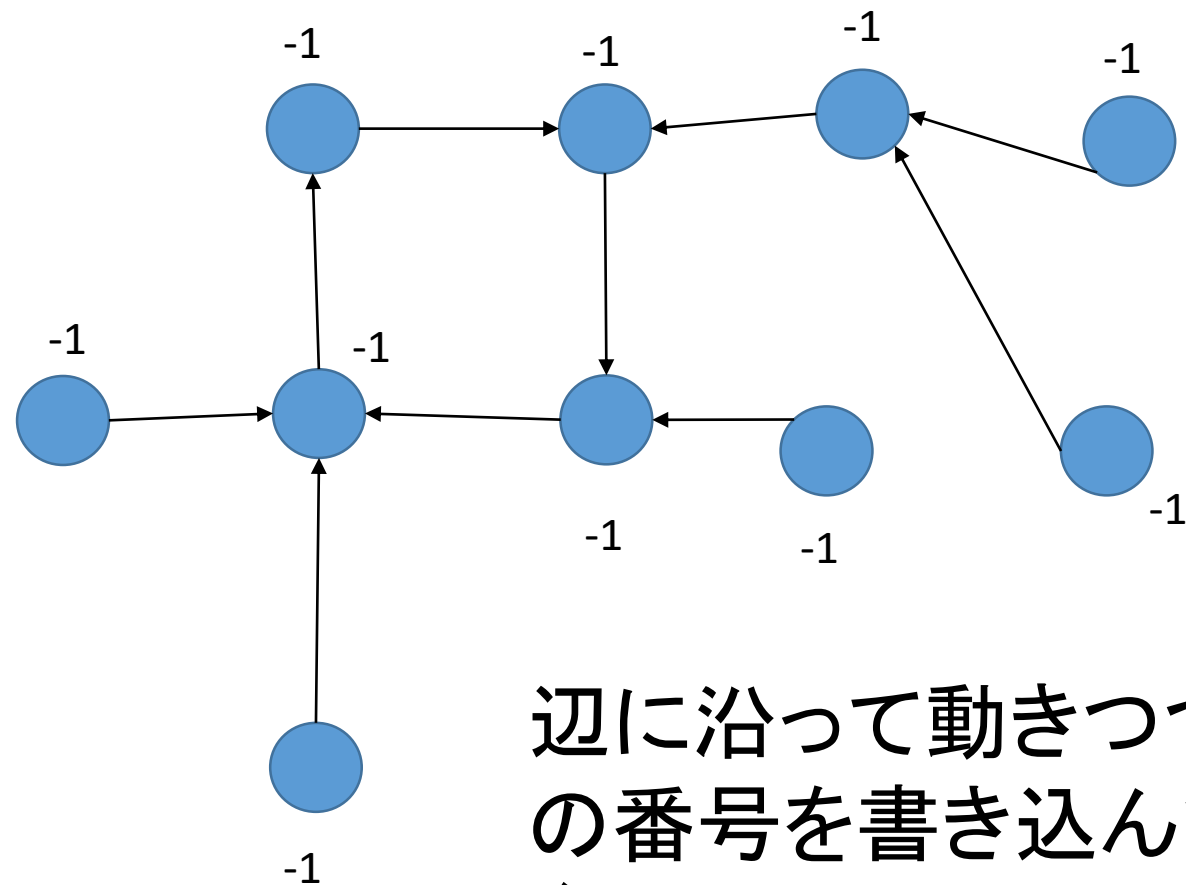
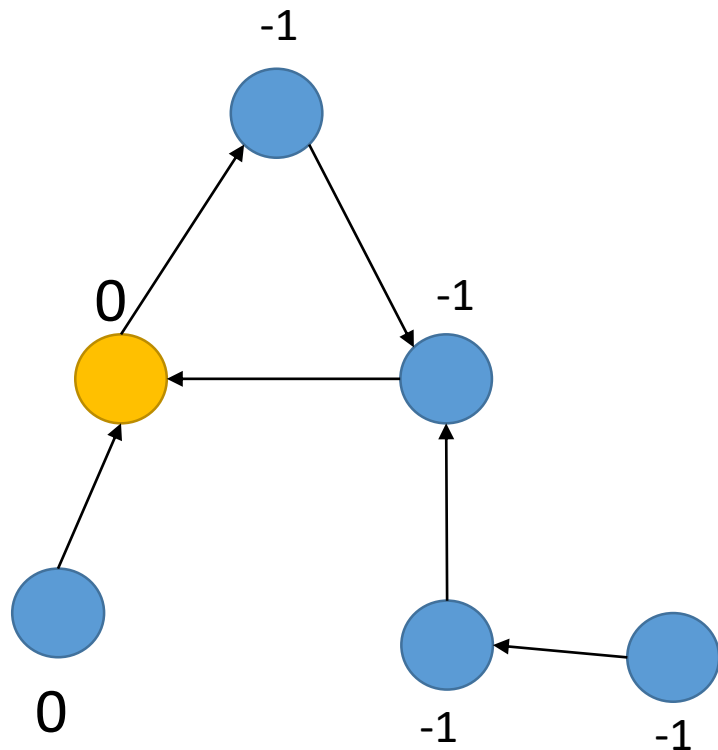
頂点を適当な順にみて
いく

おすすめの実装



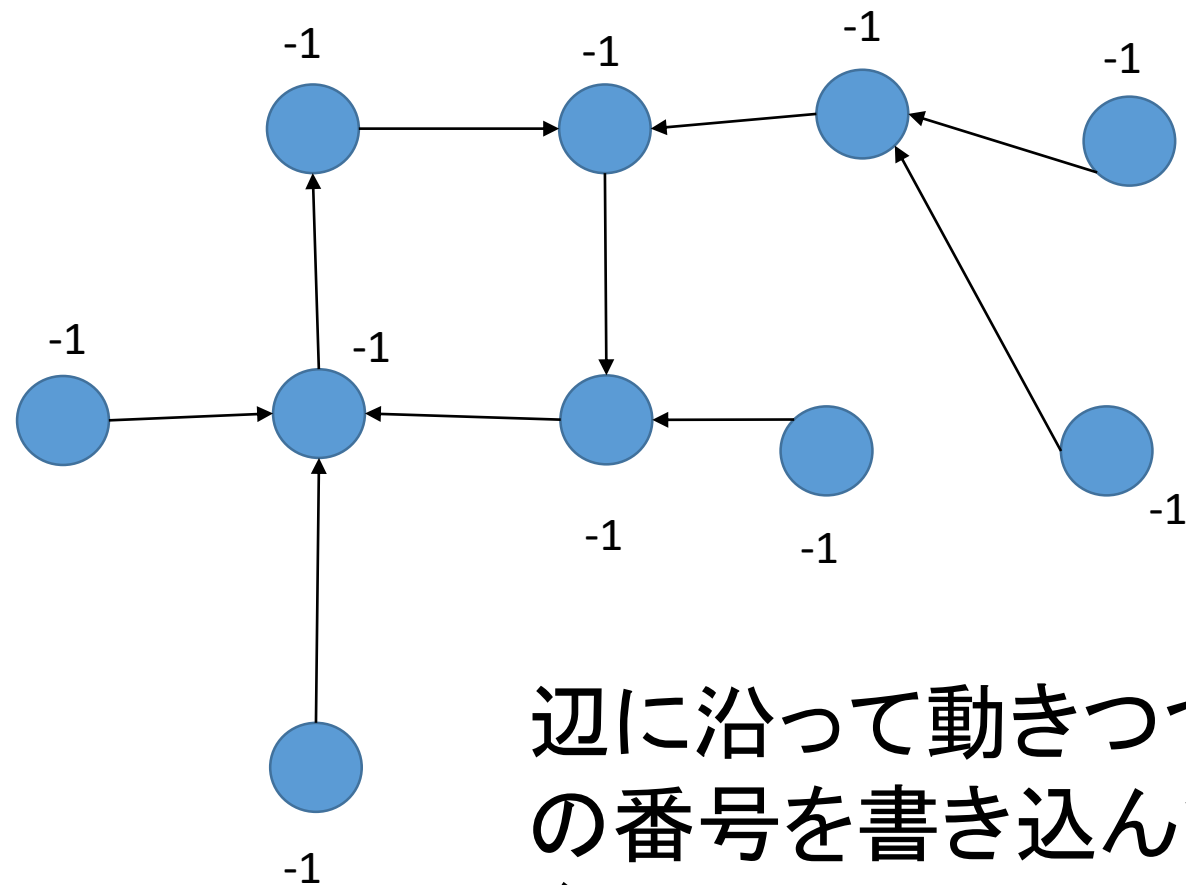
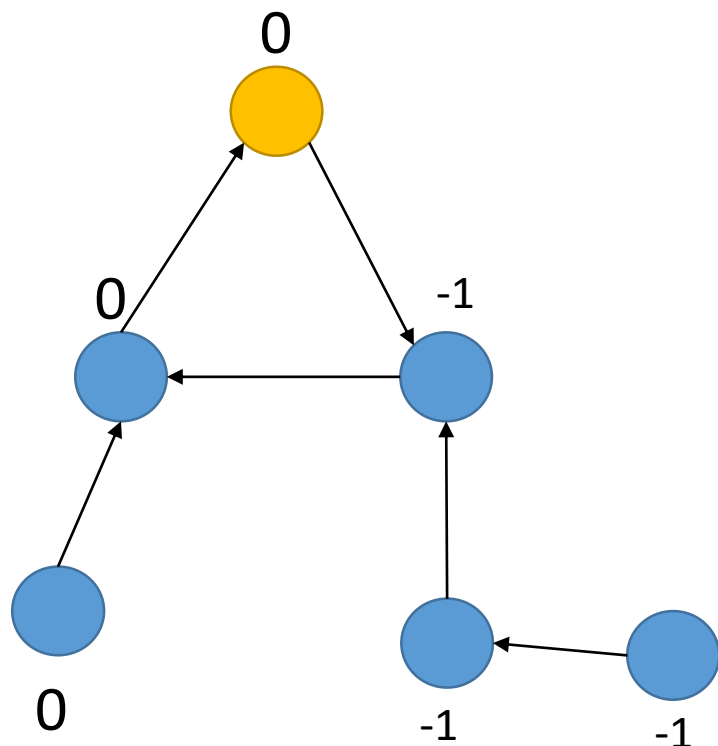
その頂点の番号を書き込む

おすすめの実装



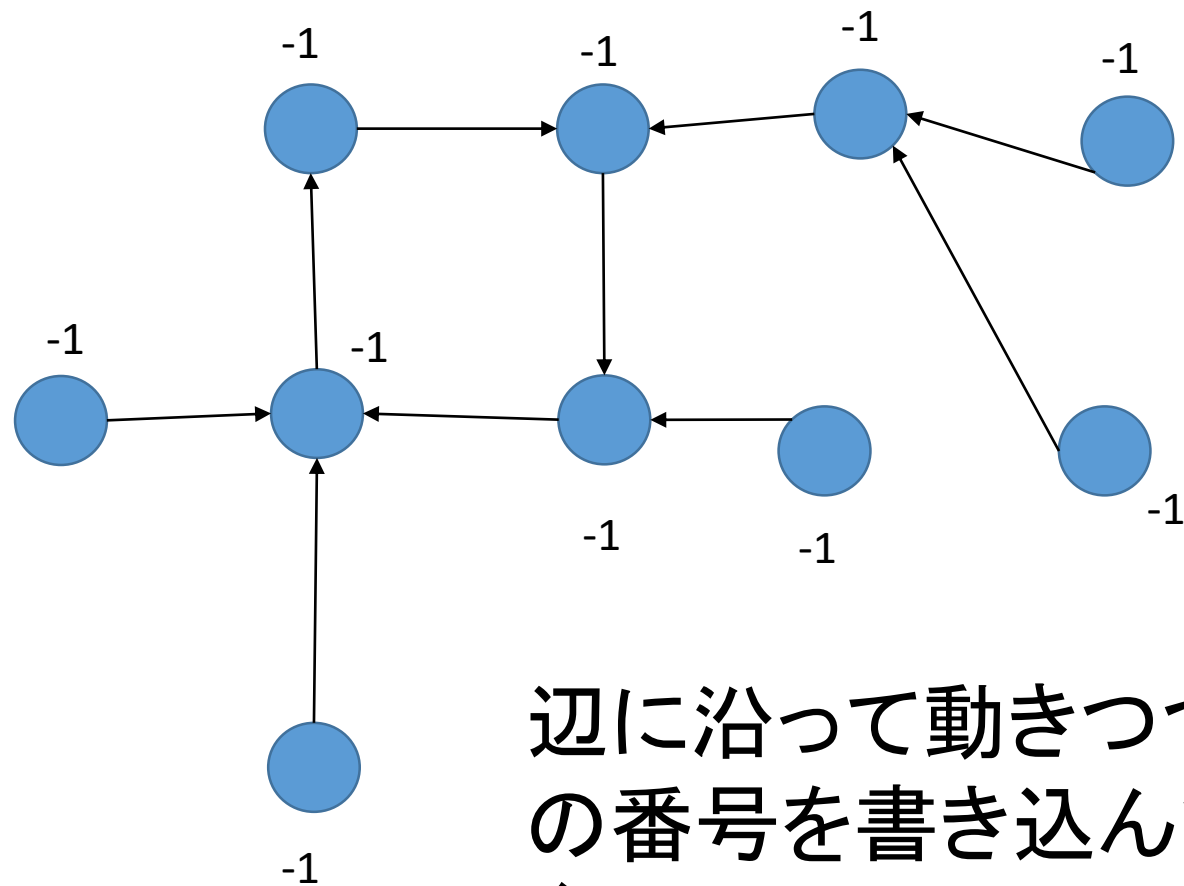
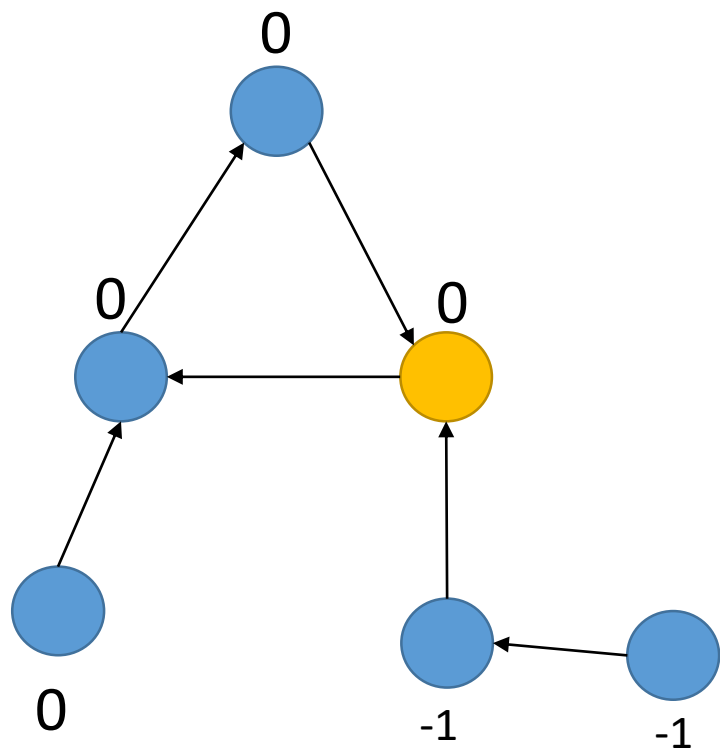
辺に沿って動きつつその番号を書き込んでいく

おすすめの実装



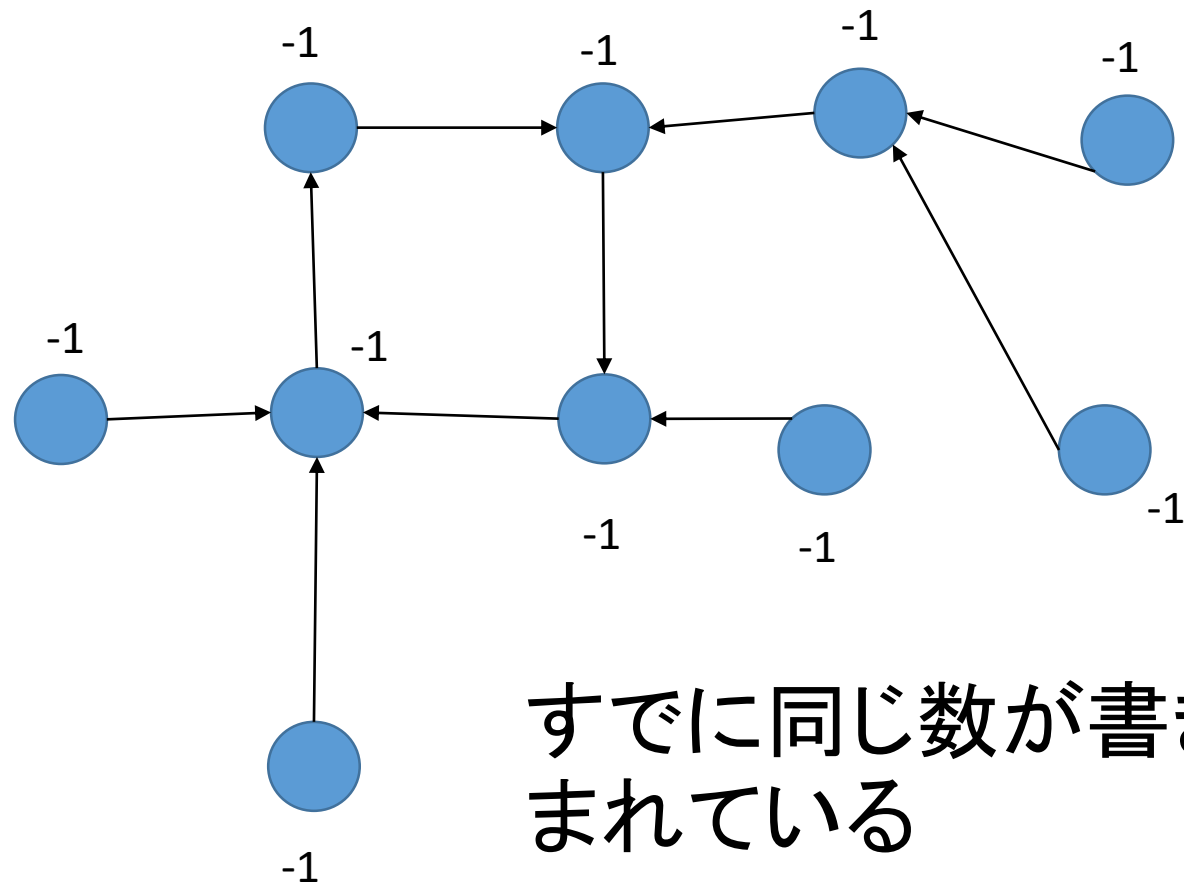
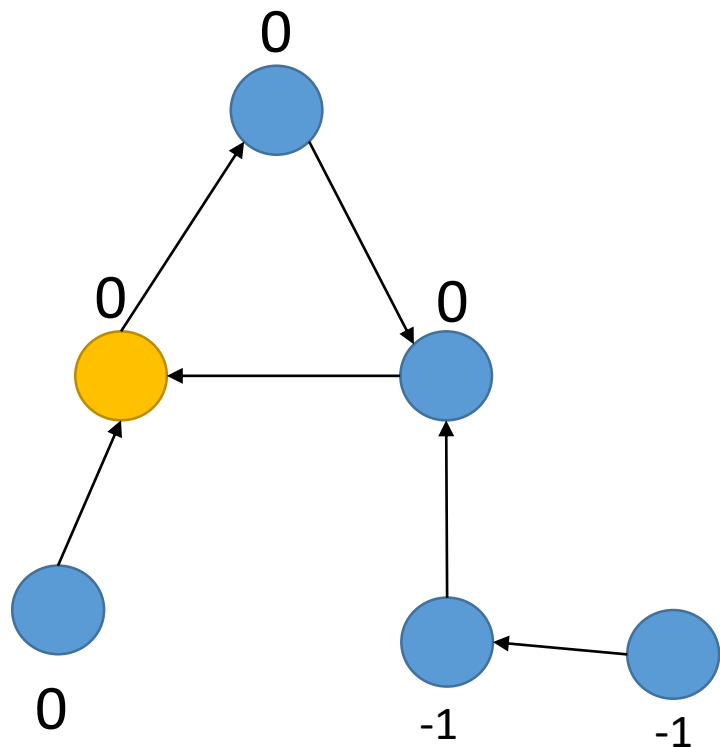
辺に沿って動きつつその番号を書き込んでいく

おすすめの実装



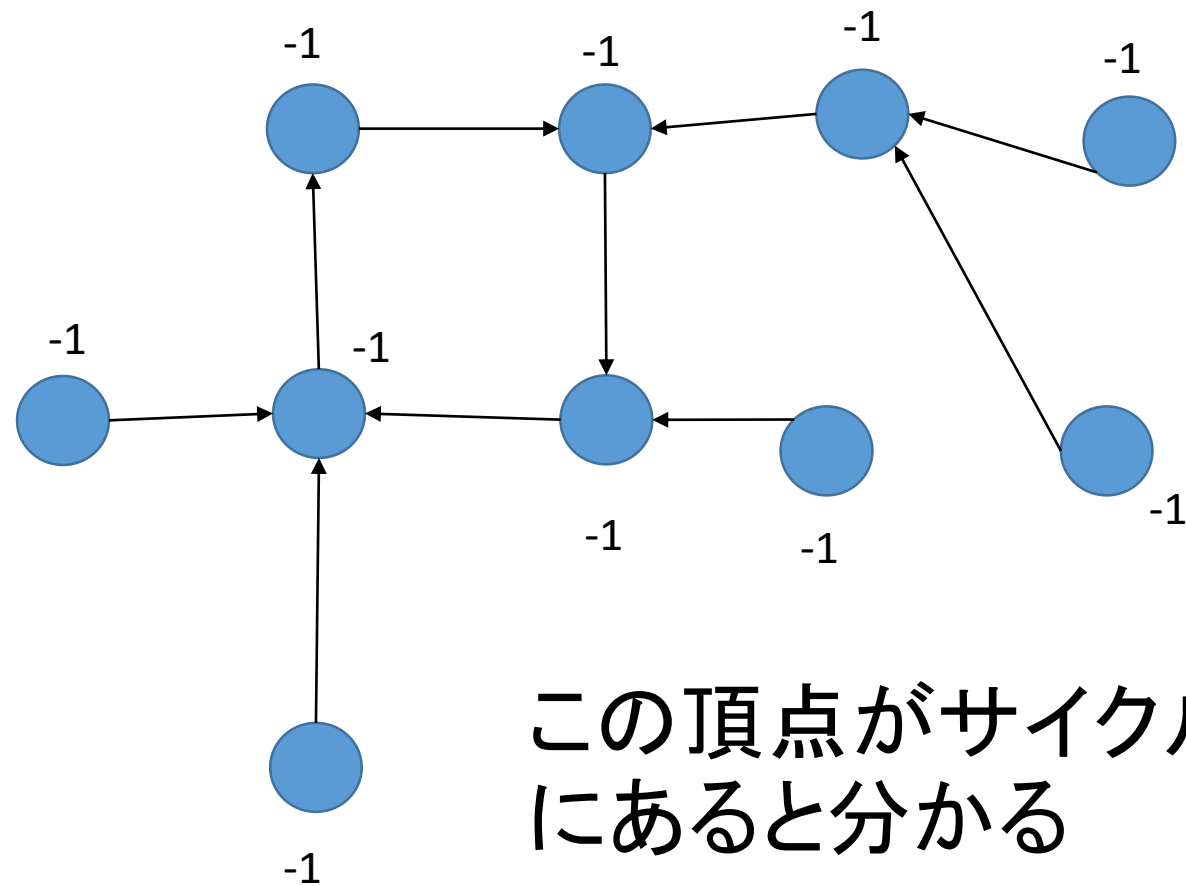
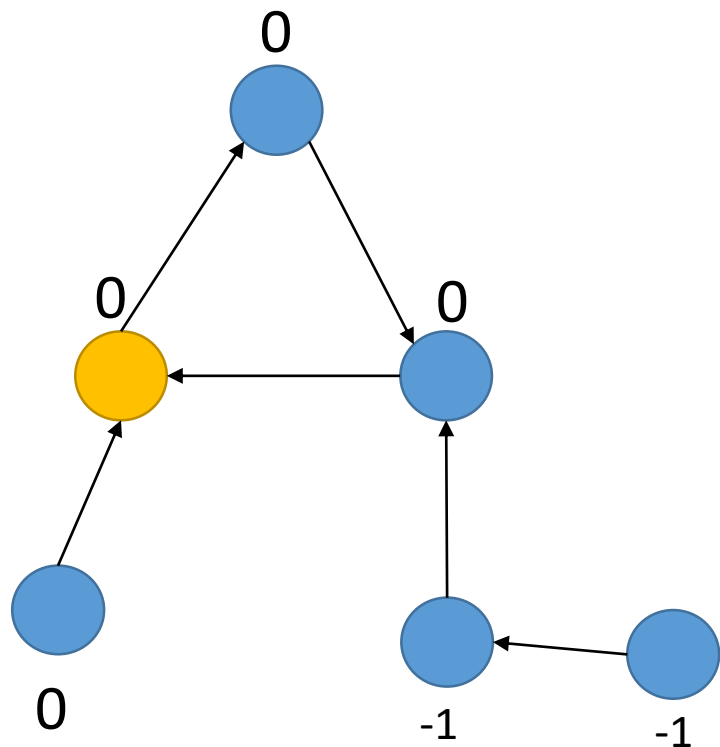
辺に沿って動きつつその番号を書き込んでいく

おすすめの実装



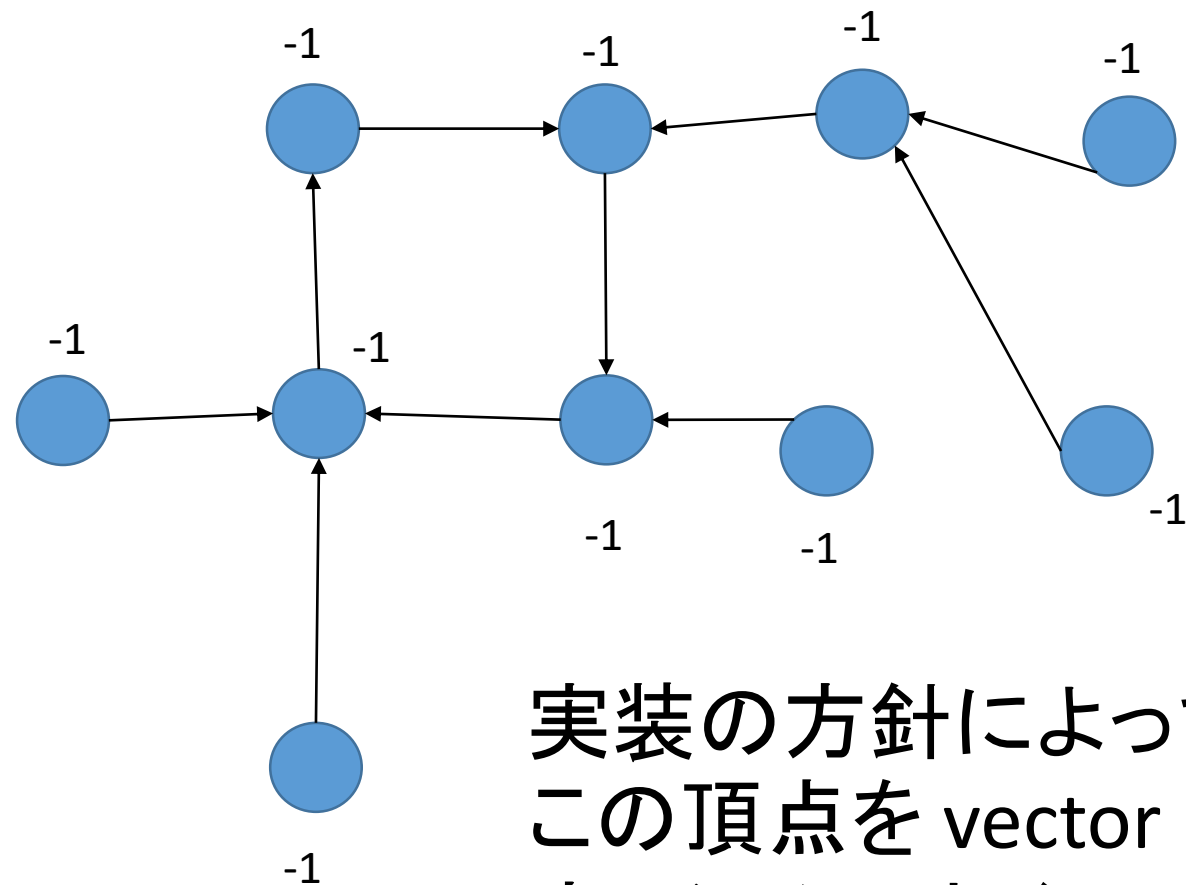
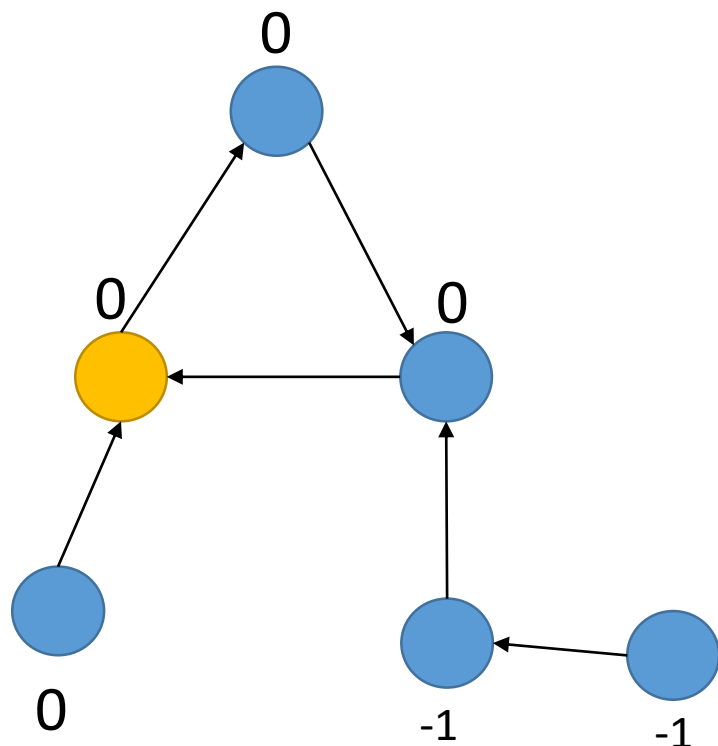
すでに同じ数が書き込まれている

おすすめの実装



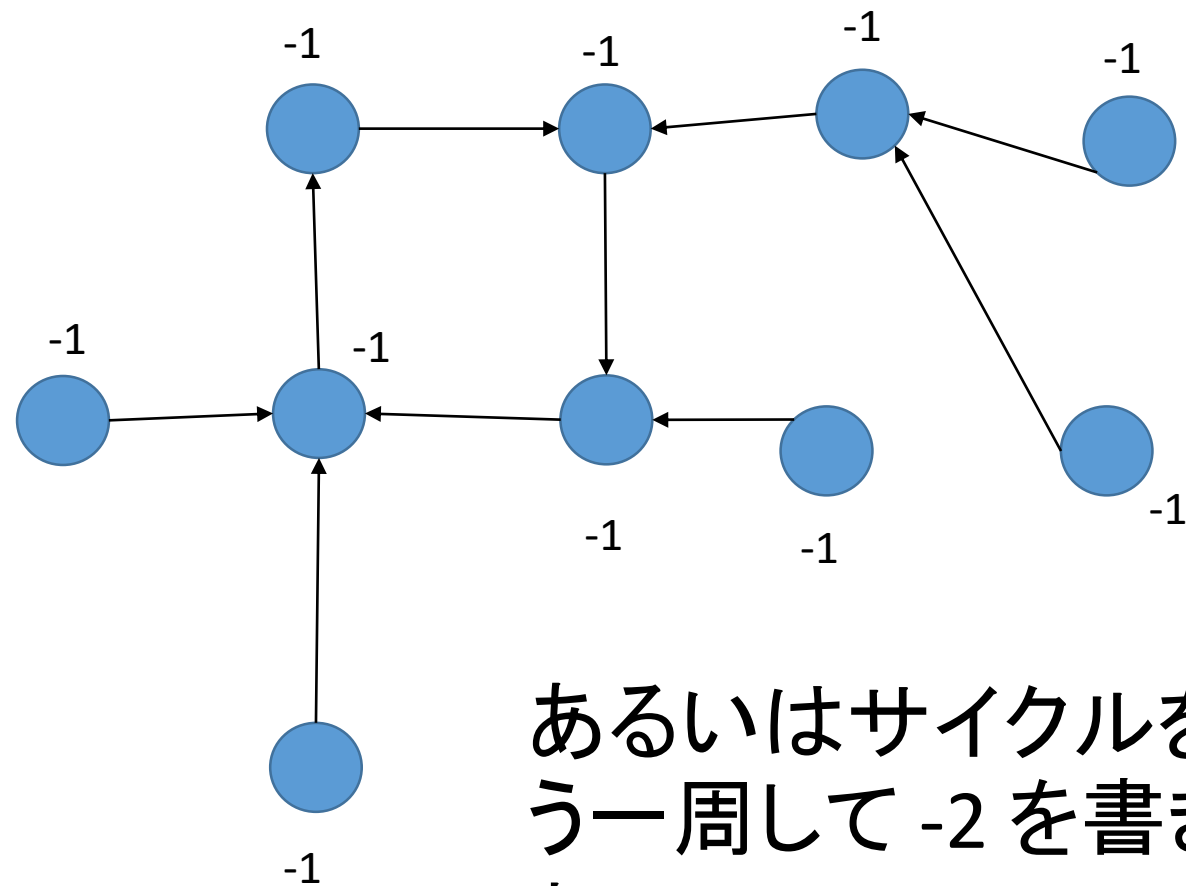
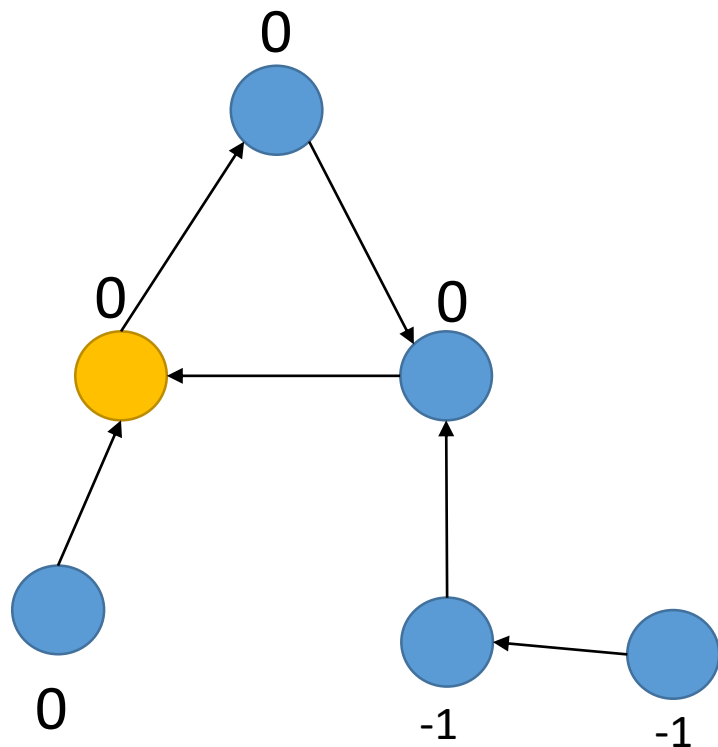
この頂点がサイクル上にあると分かる

おすすめの実装



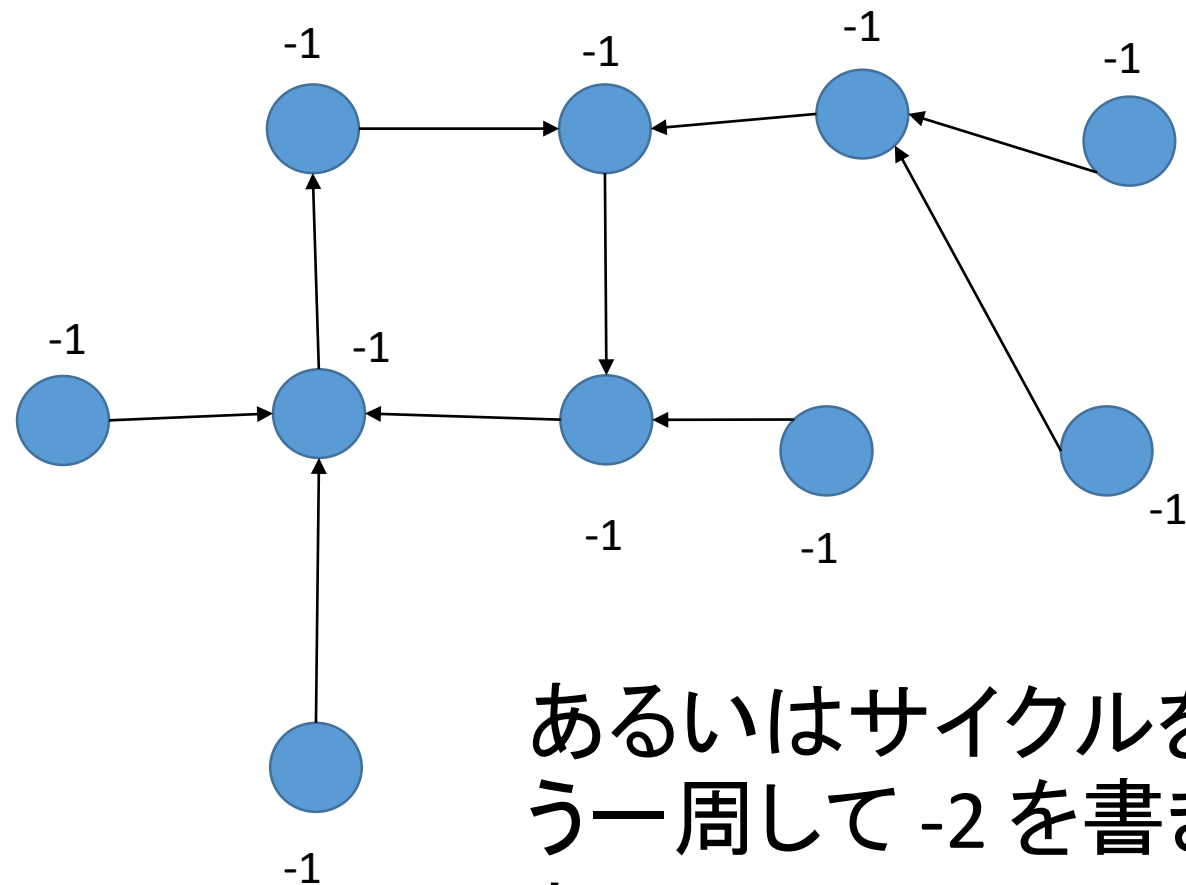
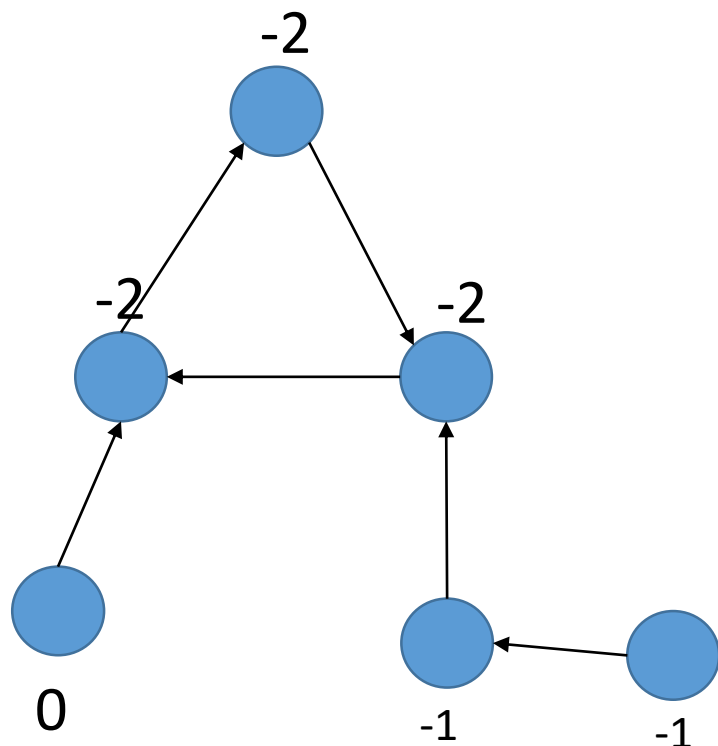
実装の方針によっては
この頂点を vector に
突っ込んでおく

おすすめの実装



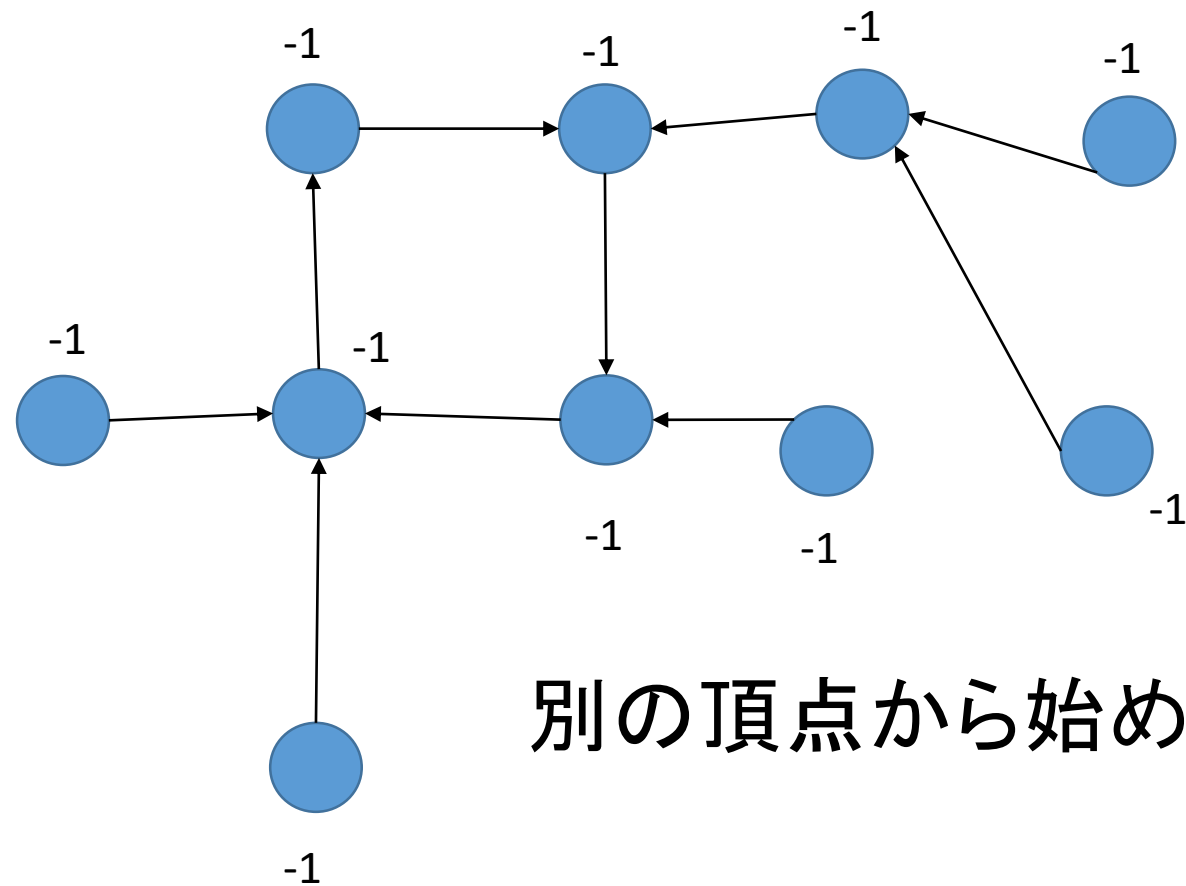
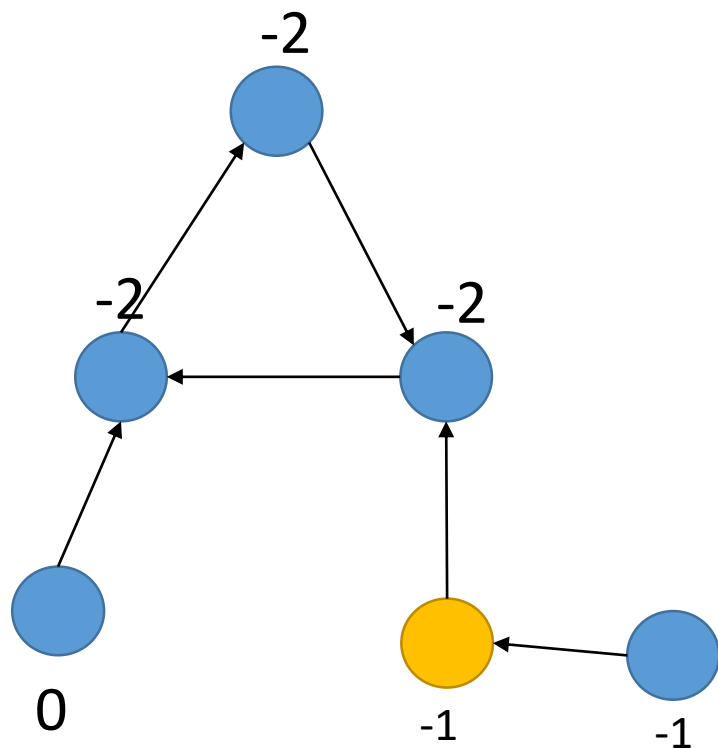
あるいはサイクルをも
う一周して -2 を書き込
む

おすすめの実装



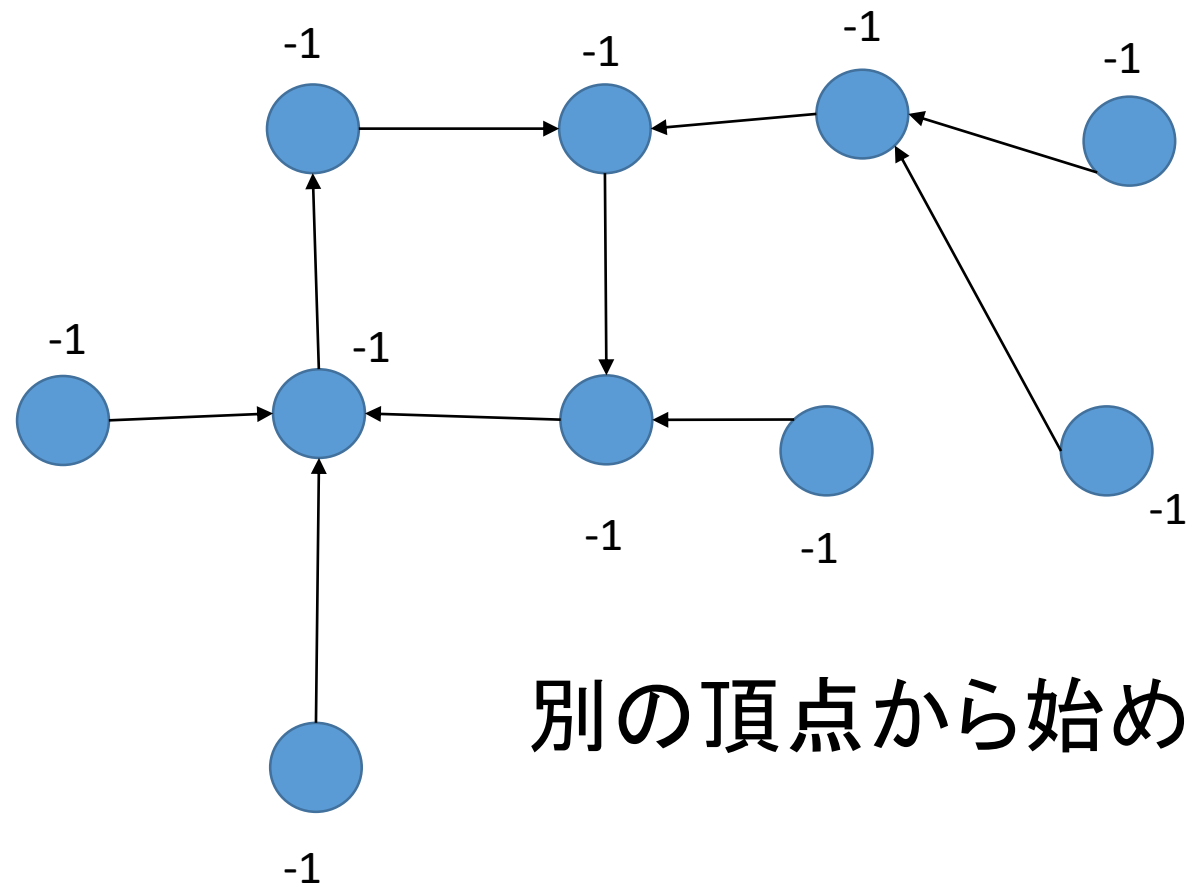
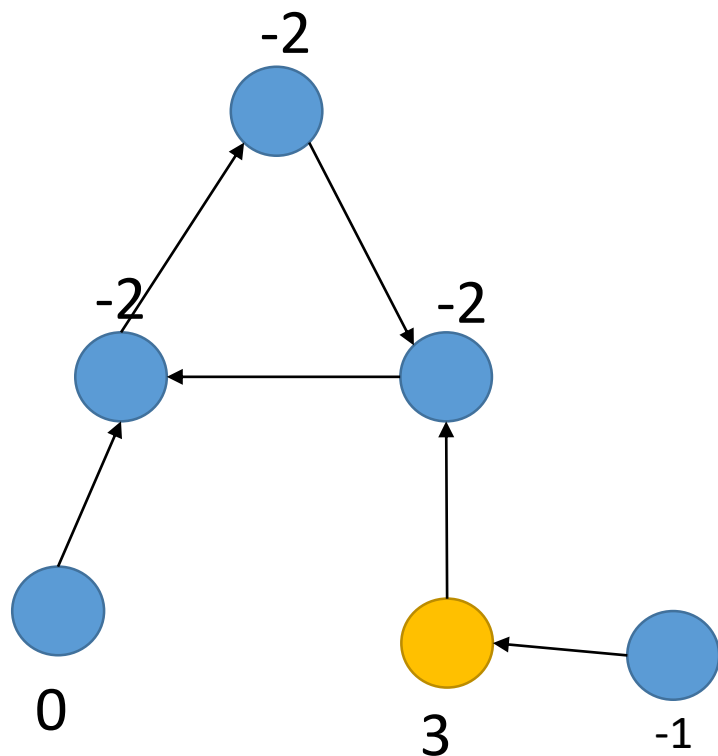
あるいはサイクルをもう一周して -2 を書き込む

おすすめの実装



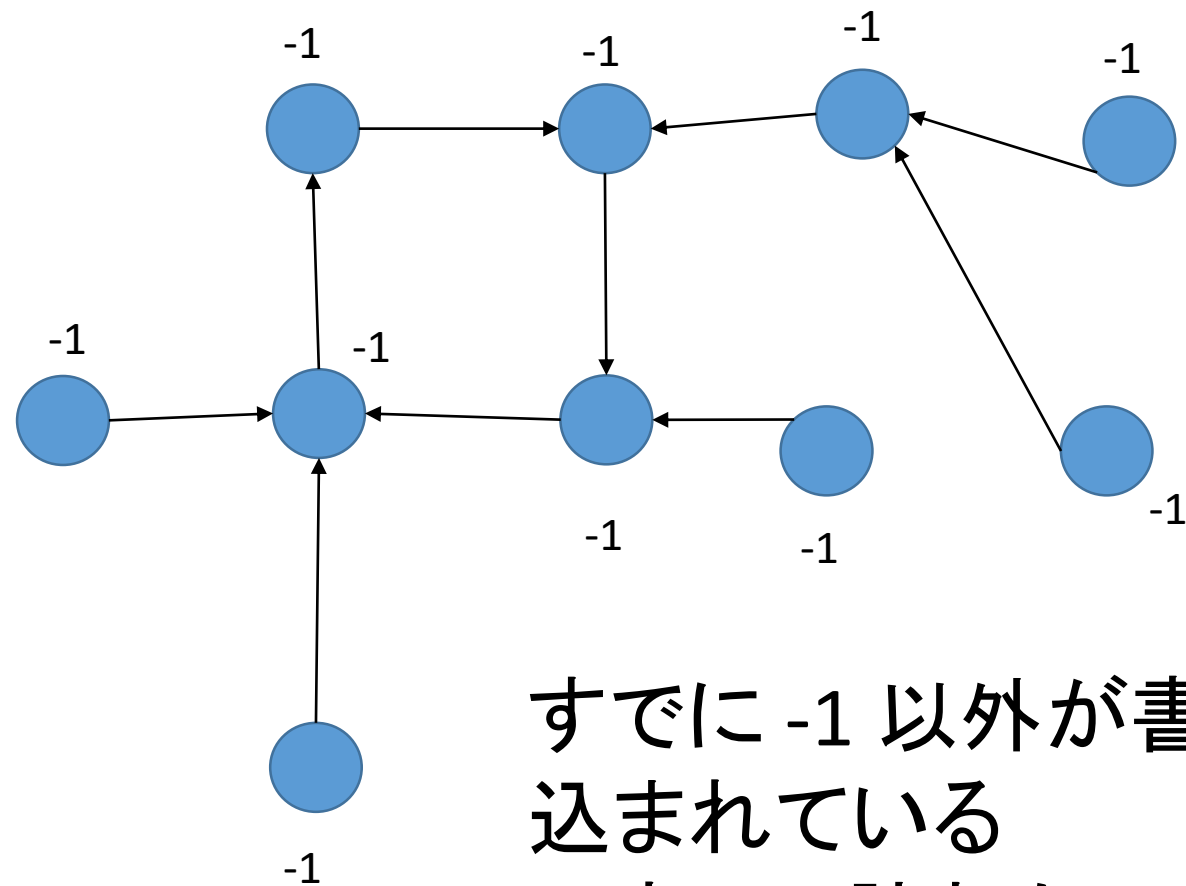
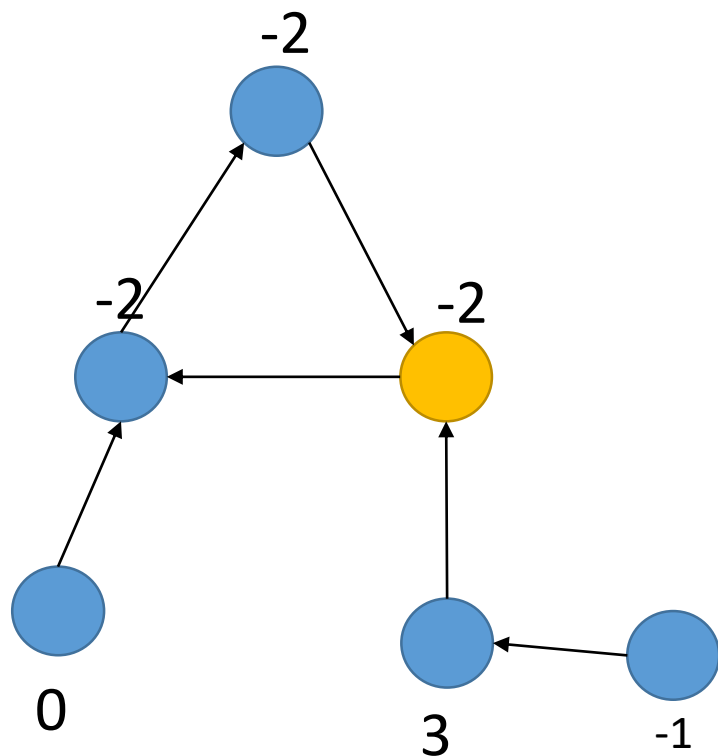
別の頂点から始める

おすすめの実装



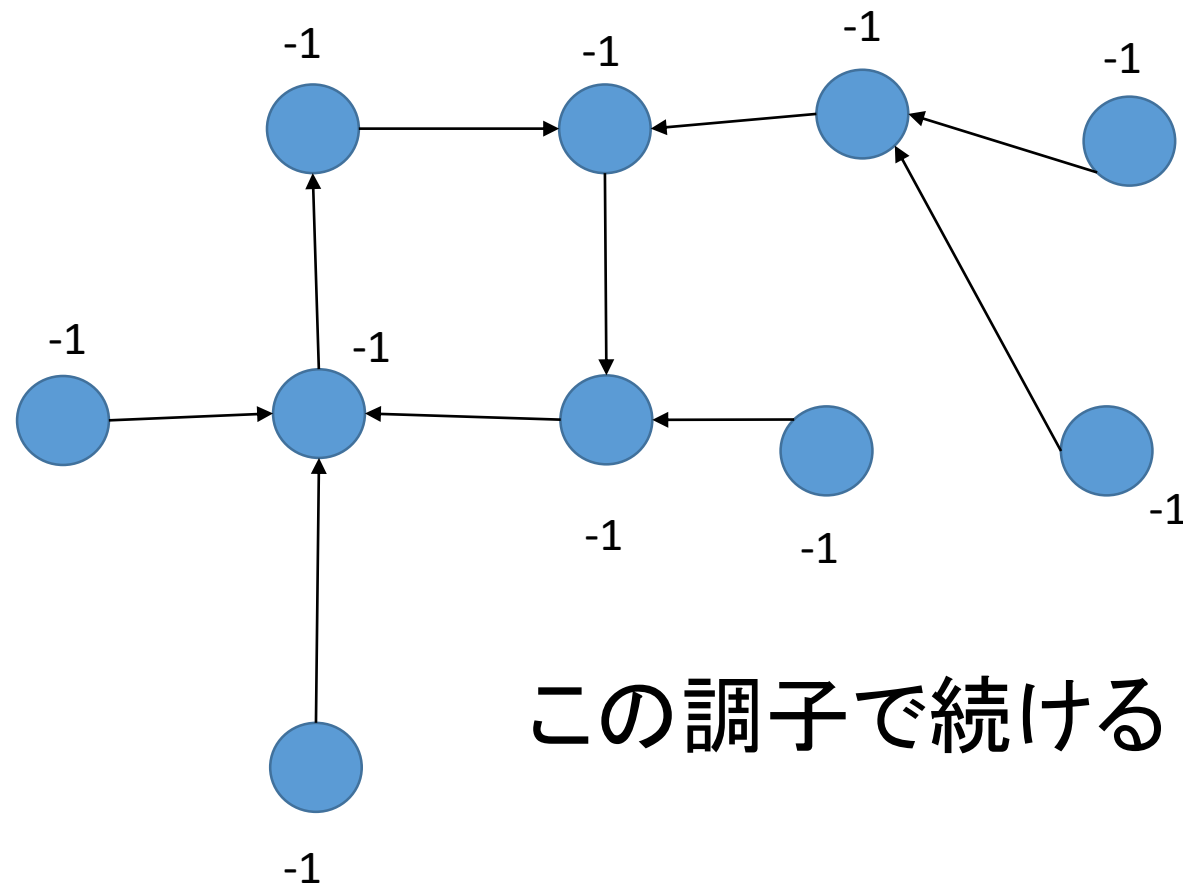
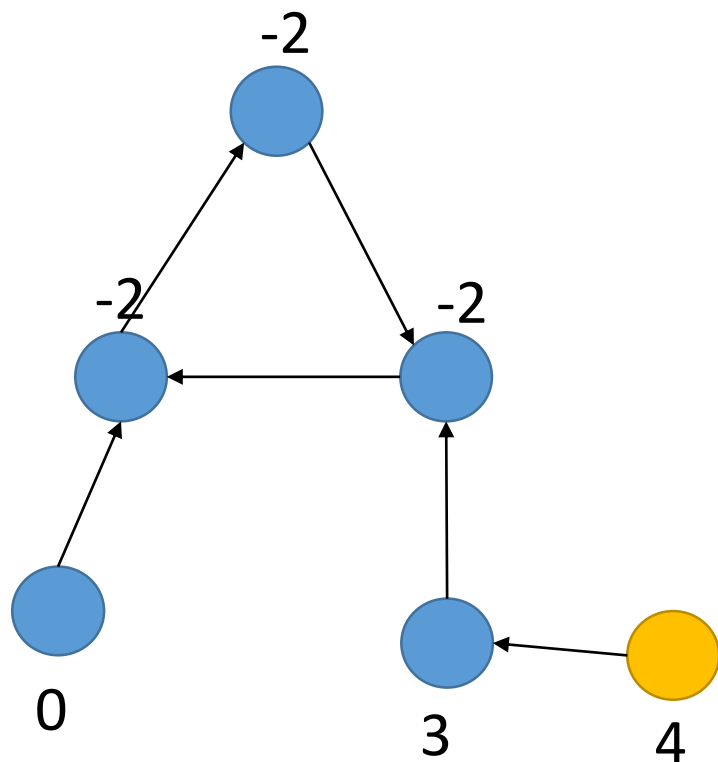
別の頂点から始める

おすすめの実装



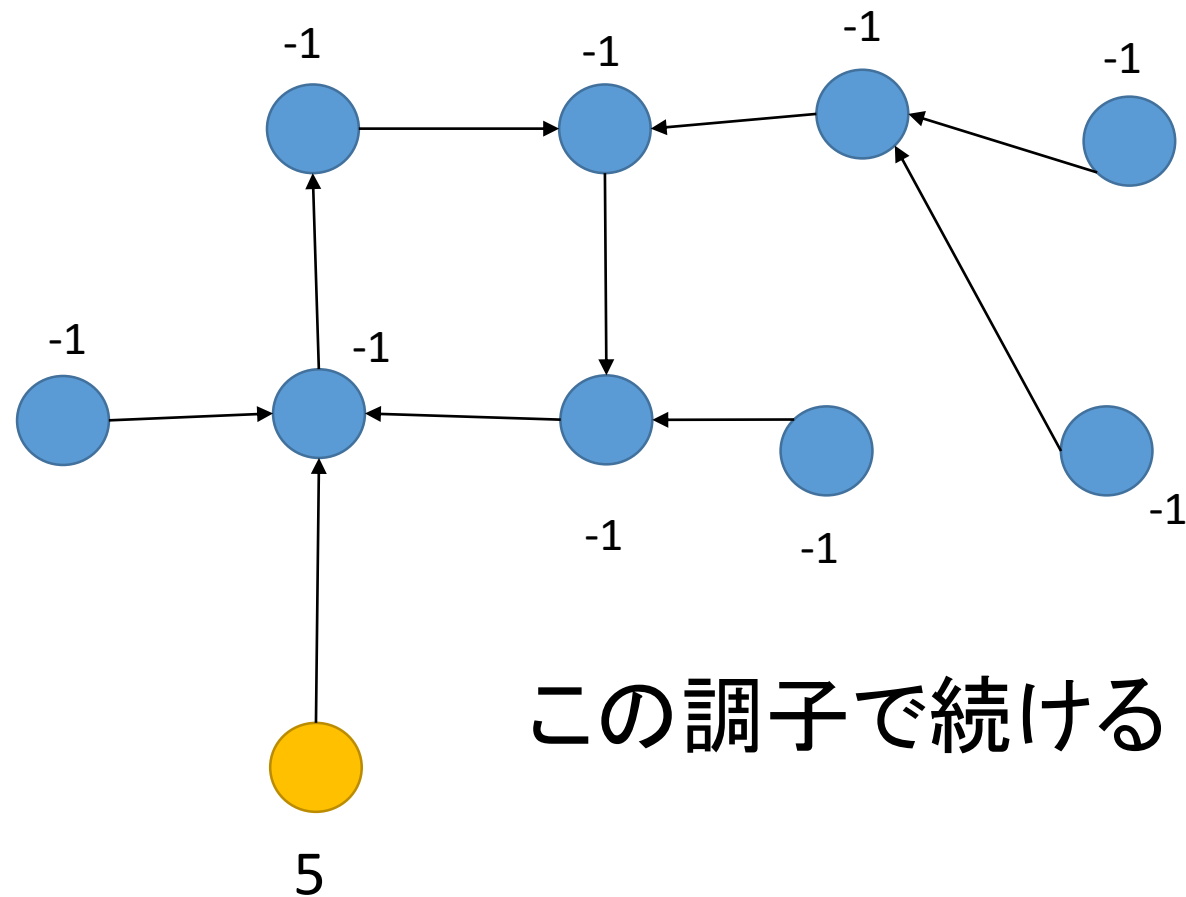
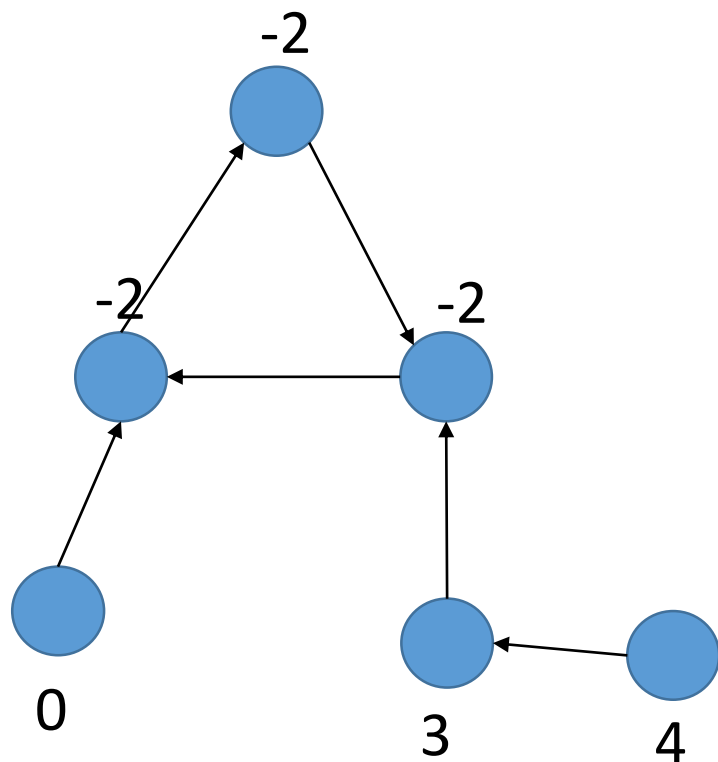
すでに -1 以外が書き
込まれている
→すでに訪れた

おすすめの実装

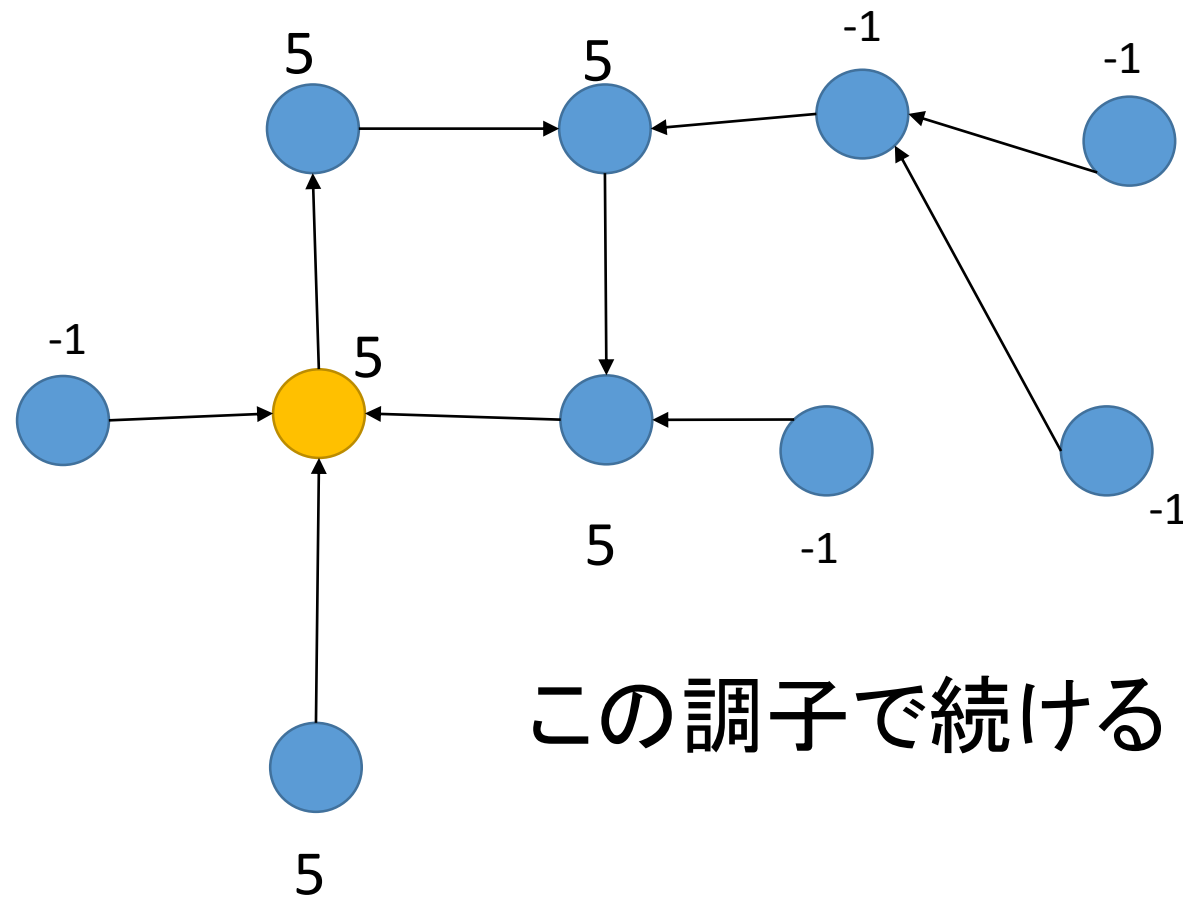
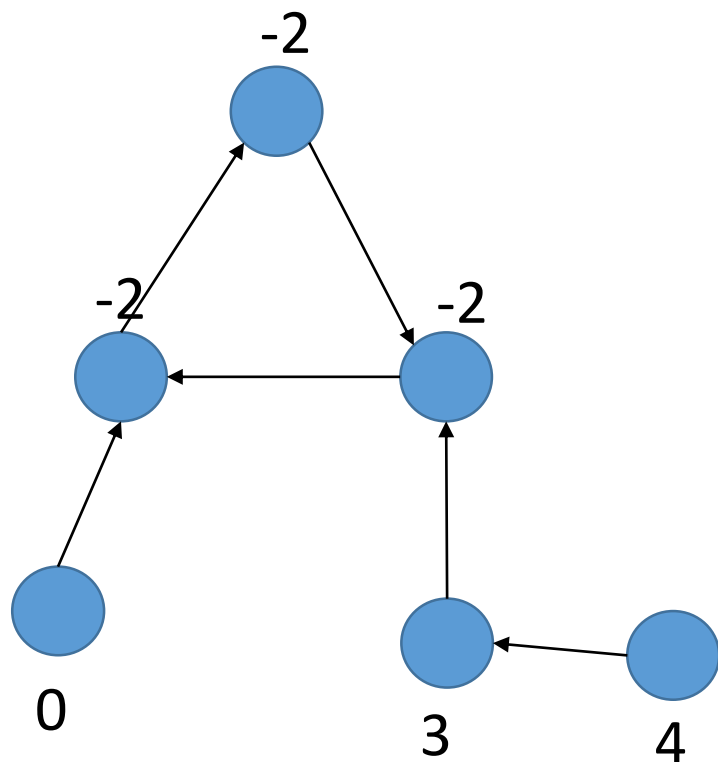


この調子で続ける

おすすめの実装

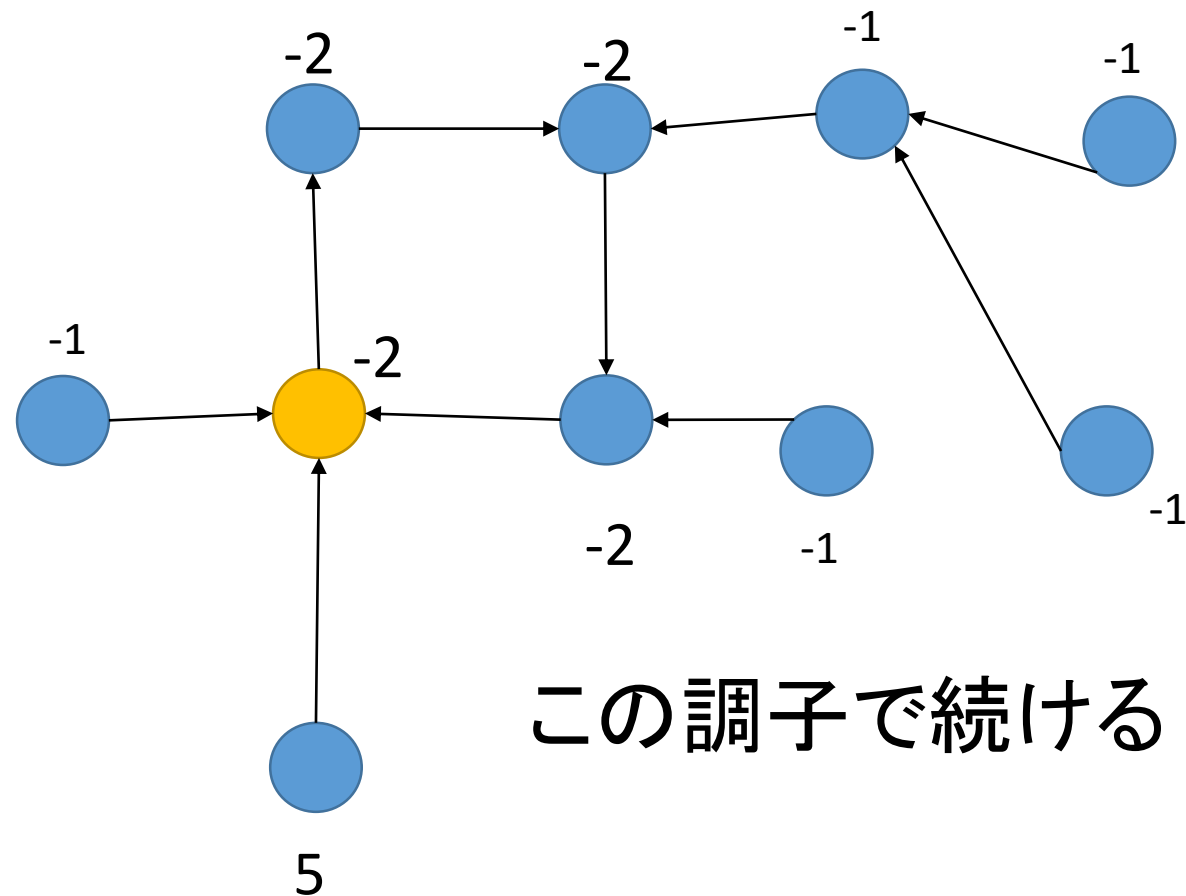
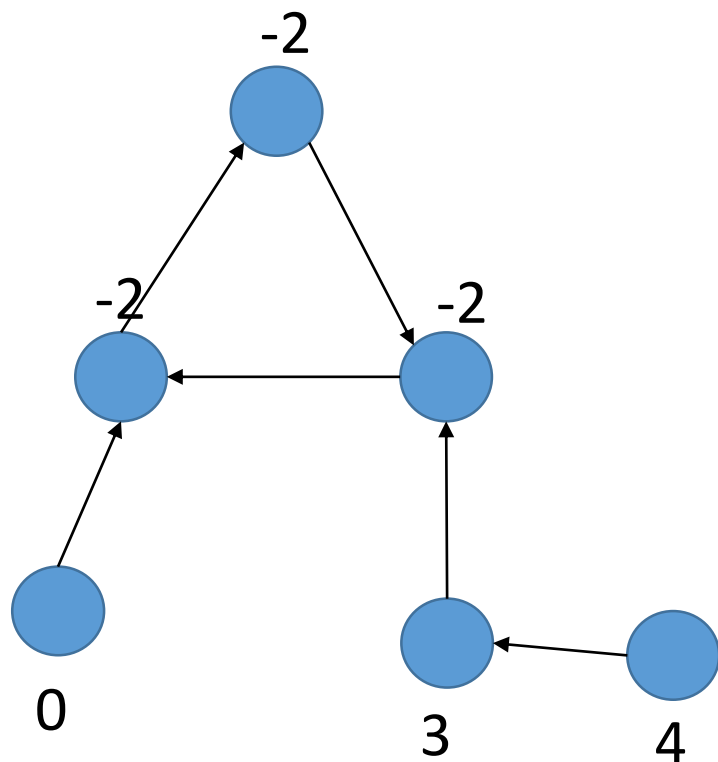


おすすめの実装



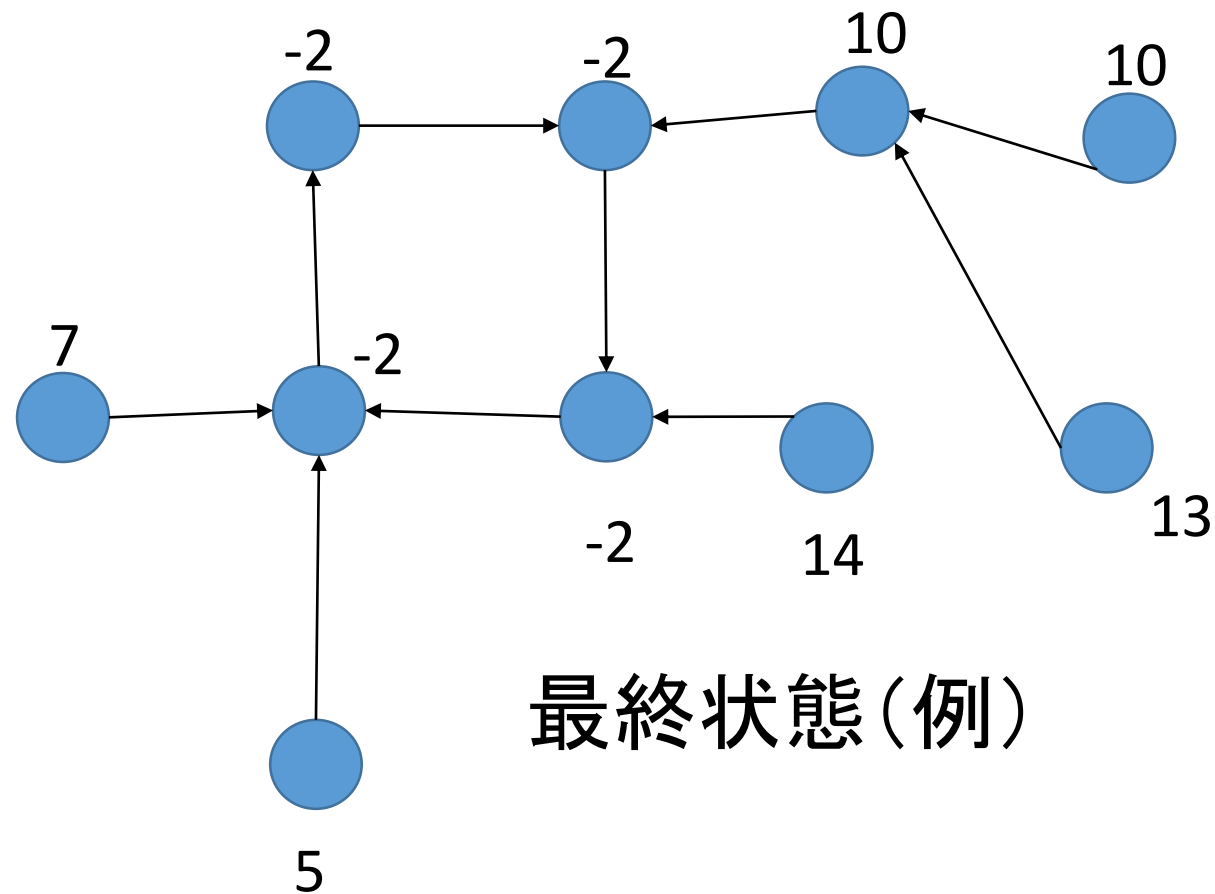
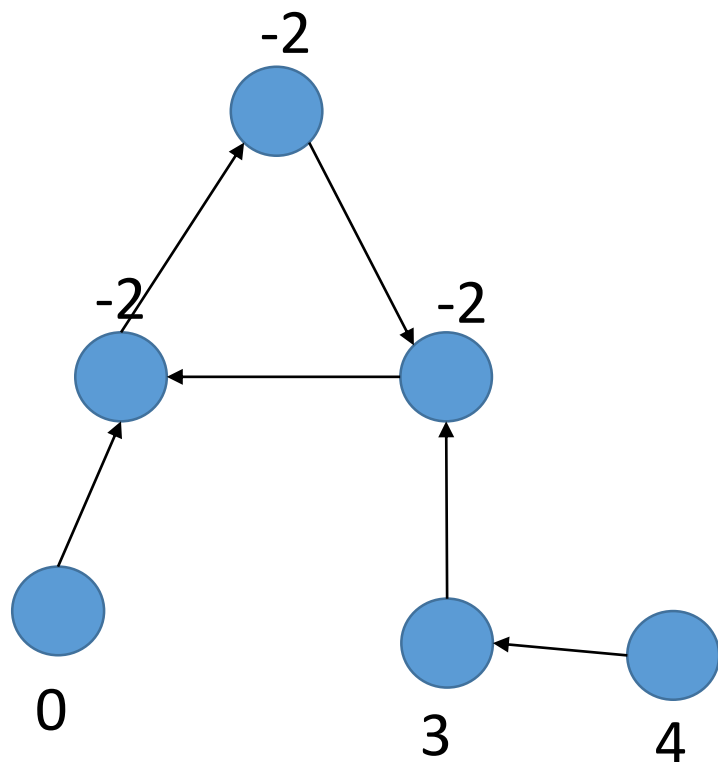
この調子で続ける

おすすめの実装



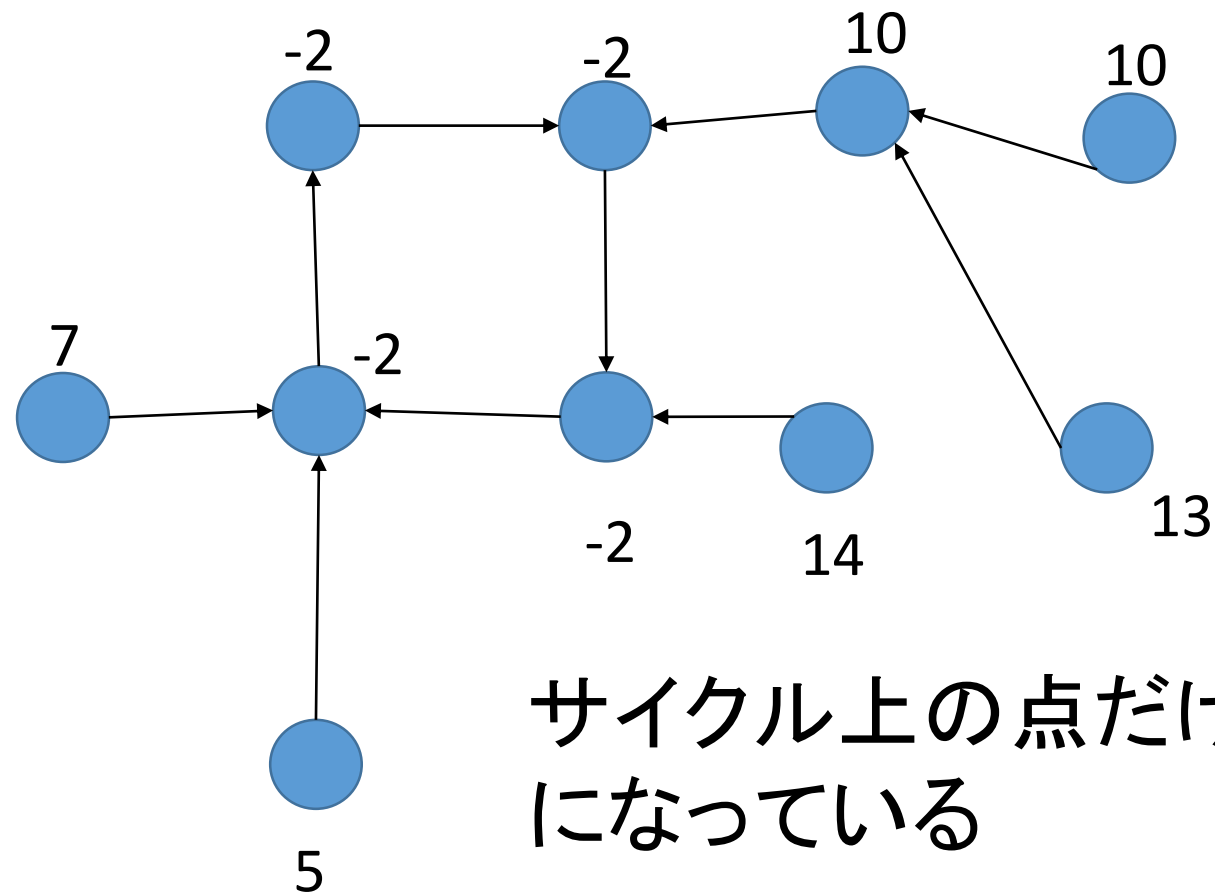
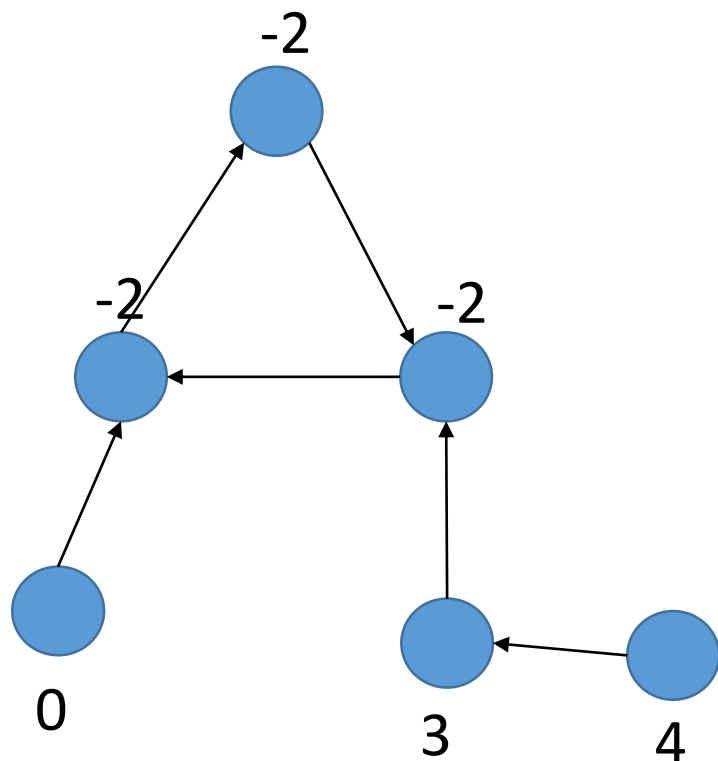
この調子で続ける

おすすめの実装



最終状態(例)

おすすめの実装



サイクル上の点だけ -2
になっている

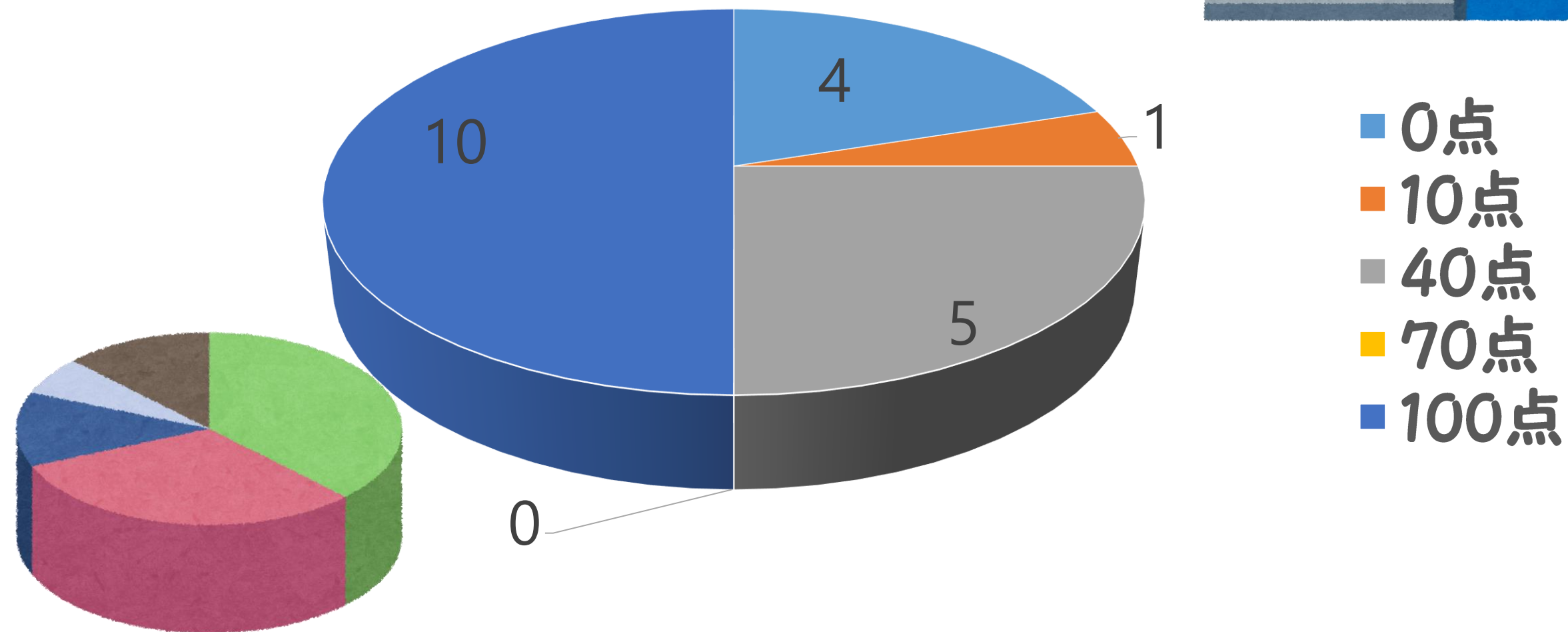
おすすめの実装



得点分布



得点分布



Dangerous Skating 危険なスケート 解説

今西 健介 (@japlj)

情報オリンピック 2015-2016 春季トレーニング合宿 競技日4

問題概要

グリッド上で JOI 君がスケートをする



問題概要

グリッド上で JOI 君がスケートをする



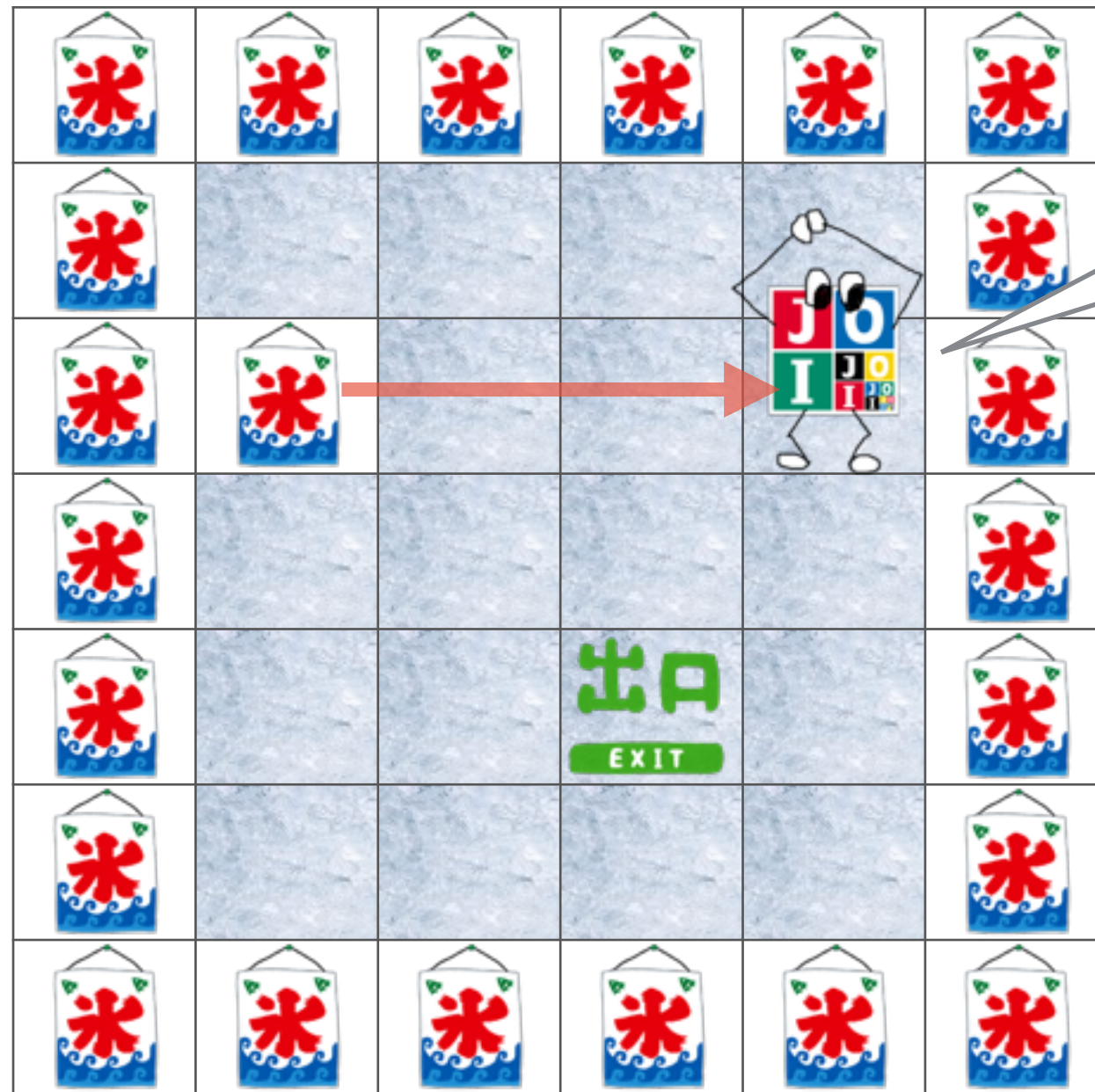
問題概要

グリッド上で JOI 君がスケートをする



問題概要

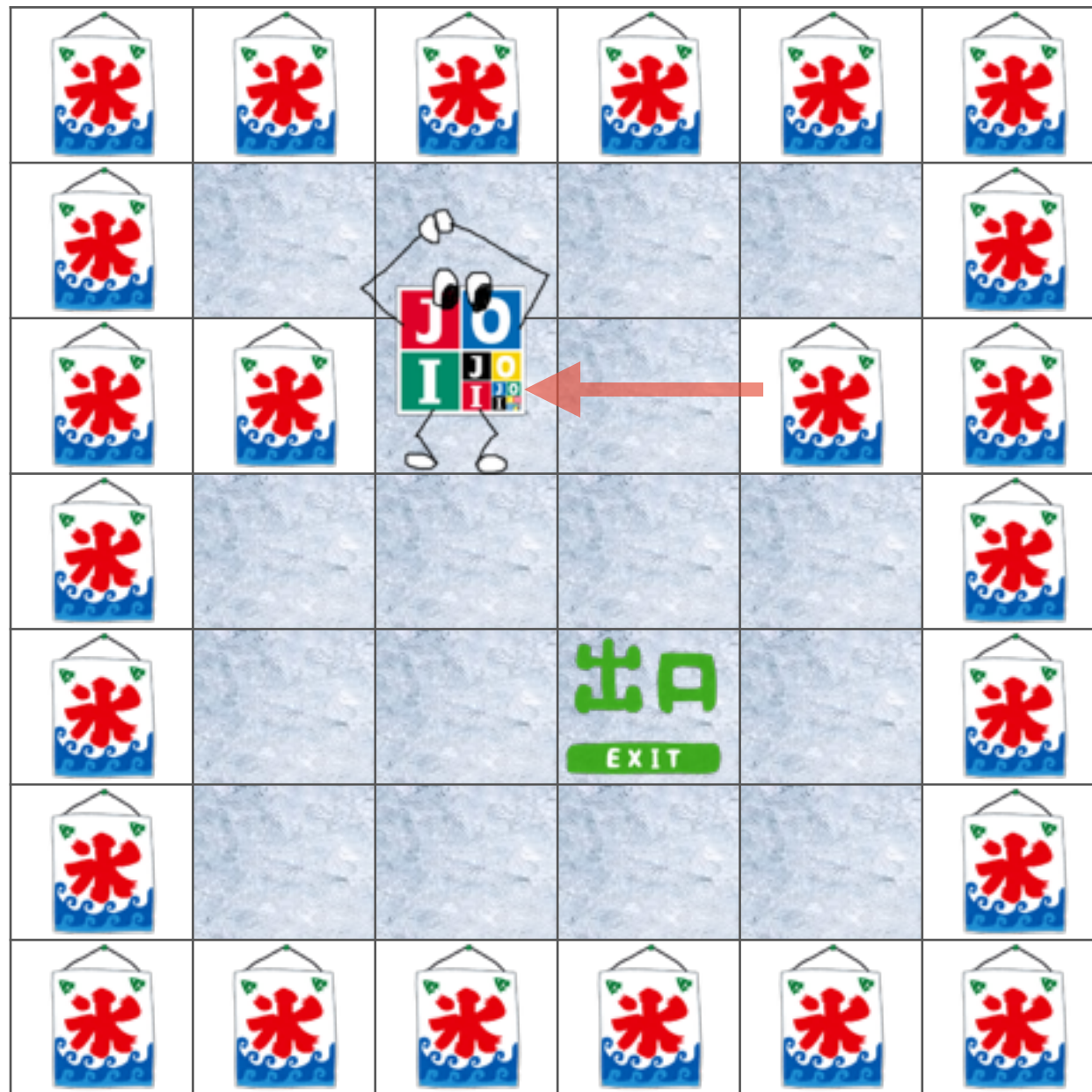
グリッド上で JOI 君がスケートをする



元いた場所に氷塊が！
コワイ！はよ帰る……

問題概要

グリッド上で JOI 君がスケートをする



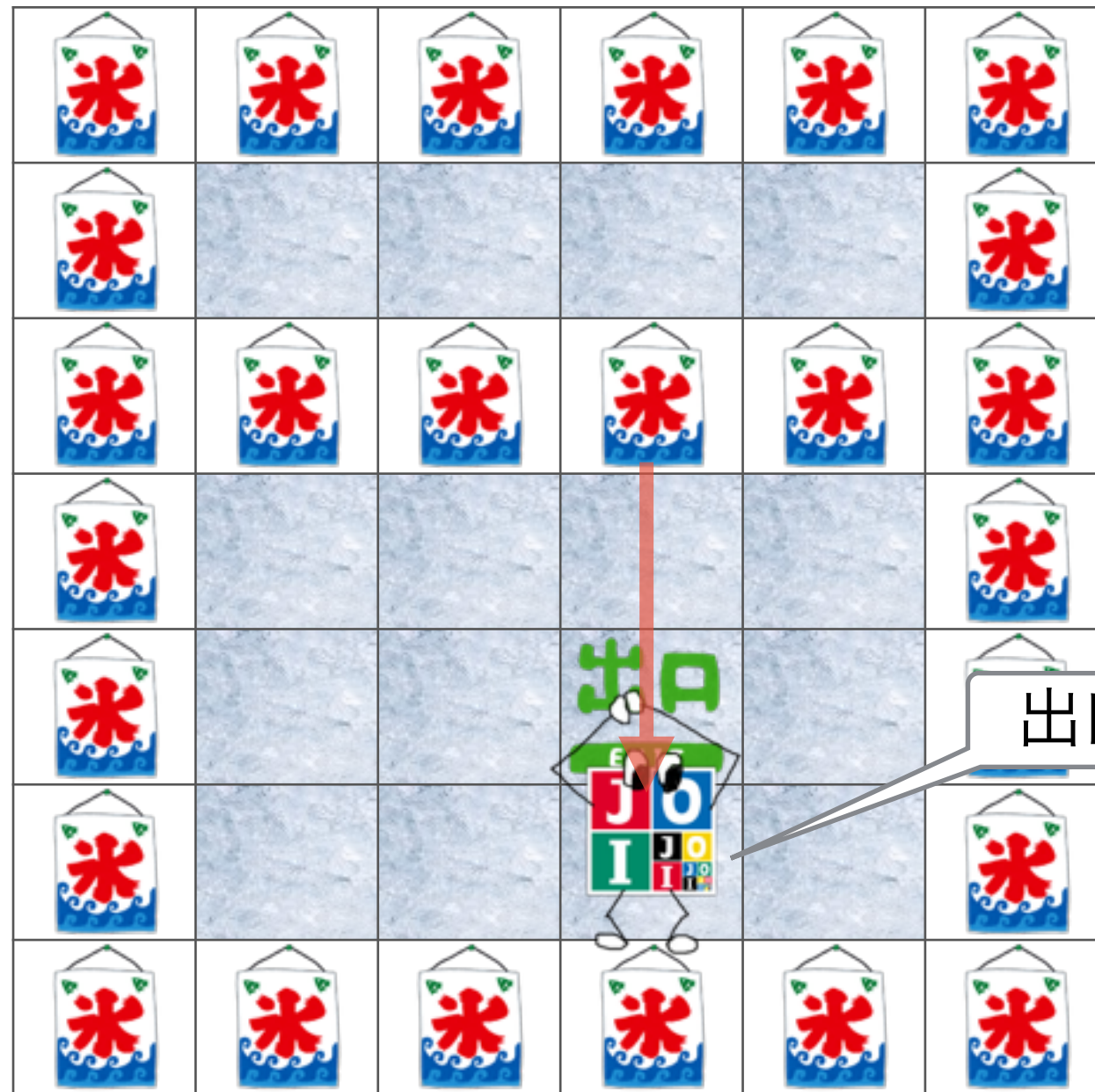
問題概要

グリッド上で JOI 君がスケートをする



問題概要

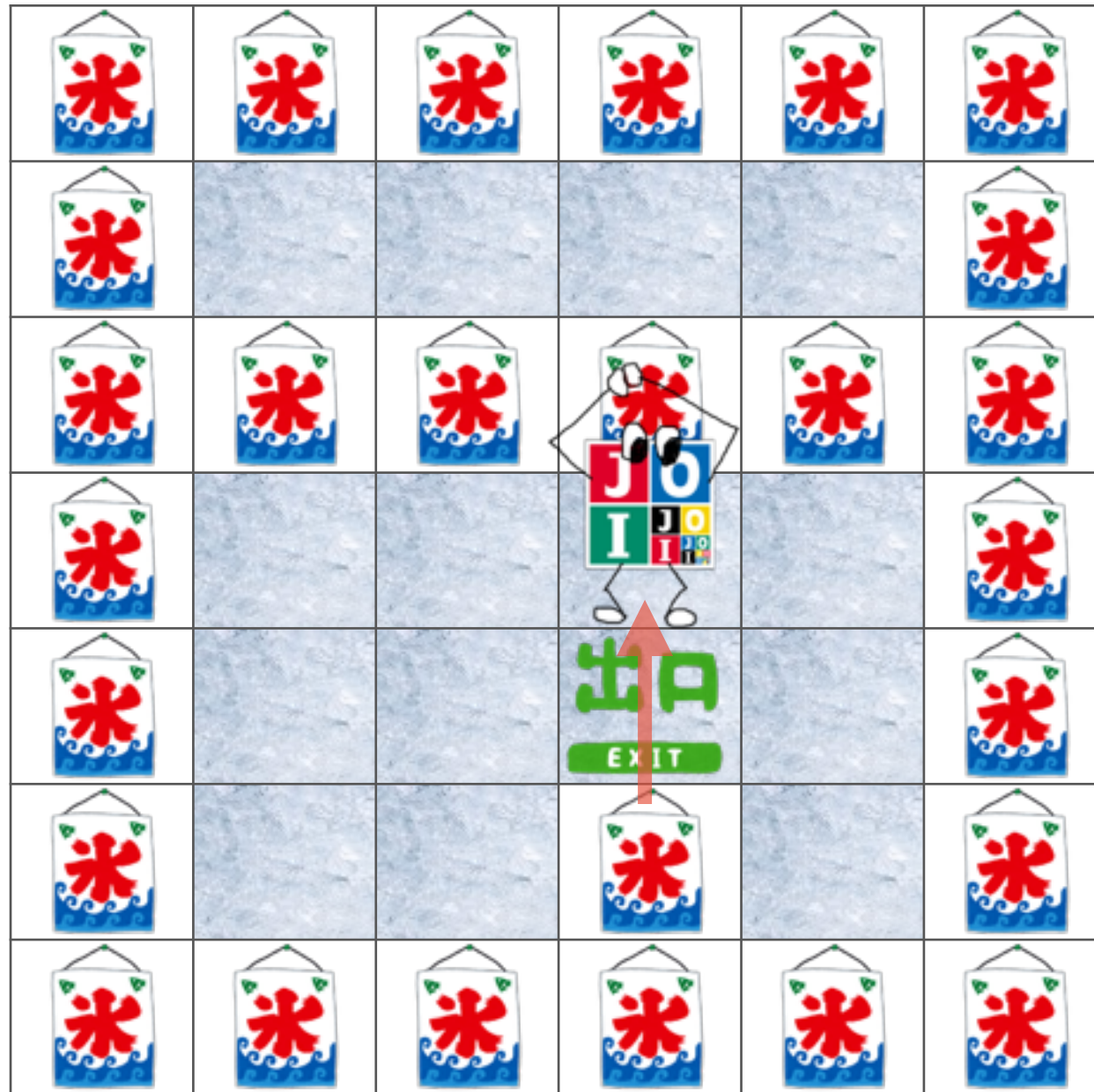
グリッド上で JOI 君がスケートをする



出口通りすぎてしもた……

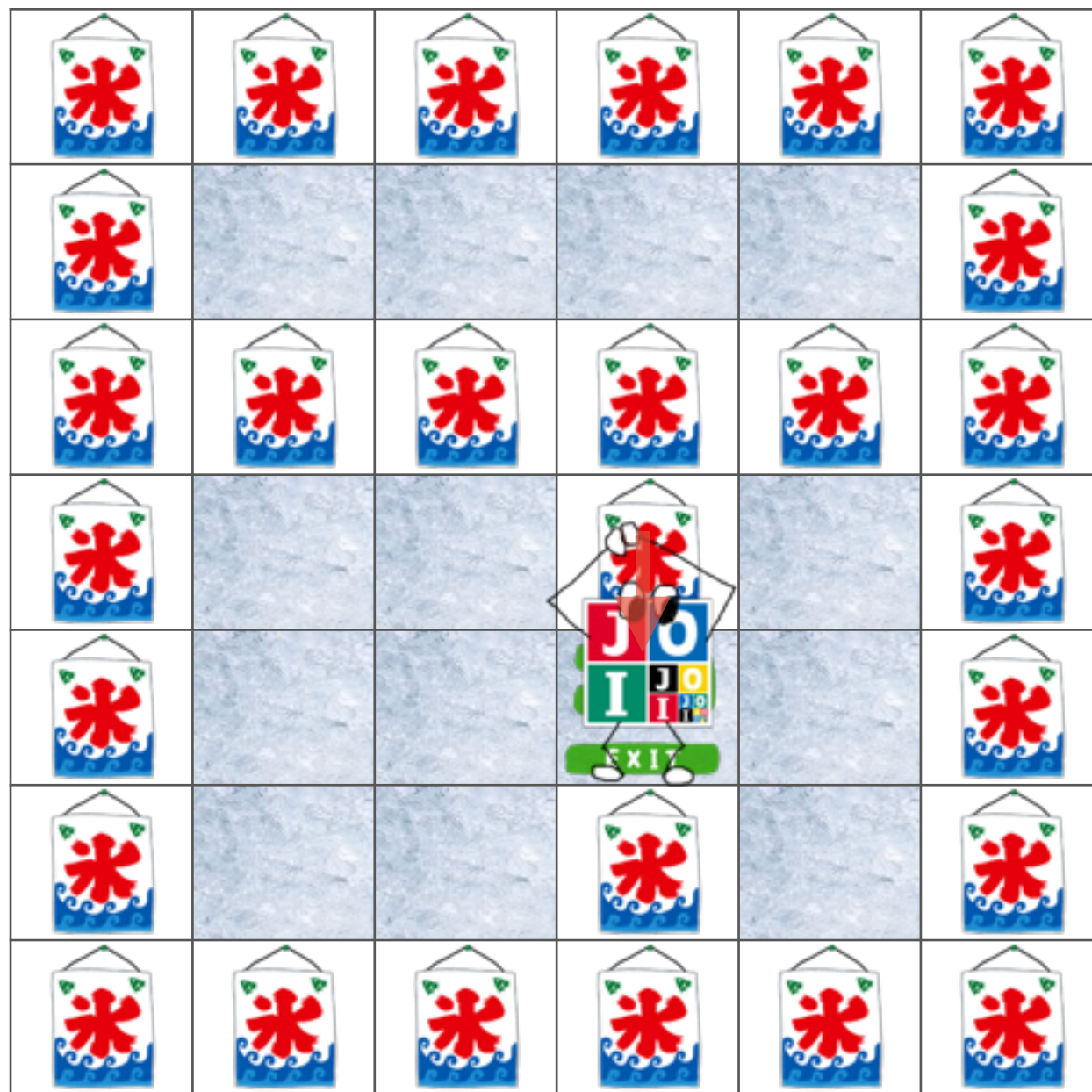
問題概要

グリッド上で JOI 君がスケートをする



問題概要

グリッド上で JOI 君がスケートをする



縦 R マス, 横 C マス
最短何ステップ?

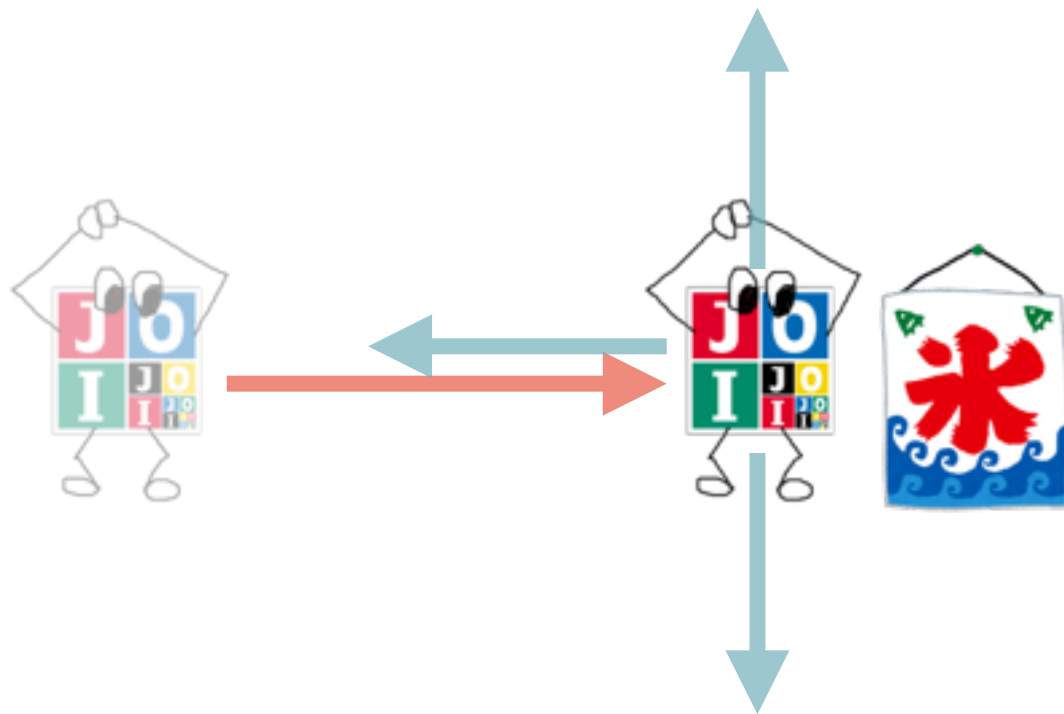
制約

- $R \leq 1,000$
- $C \leq 1,000$
- 最初から氷塊のマス有

あきらかな最短路？

最短路問題に見えるが、これまでの動作によって
氷塊の配置が異なる状態になりうる

= 位置だけで状態は確定しない



→ 直前の動き

→ 直後の動き

全探索しよう！！！！



$O^*(3^{\text{answer}})$ 時間で

小課題 1 が解ける (13 点)

何がやばいのか



何がやばいのか

氷塊が生える



何がやばいのか

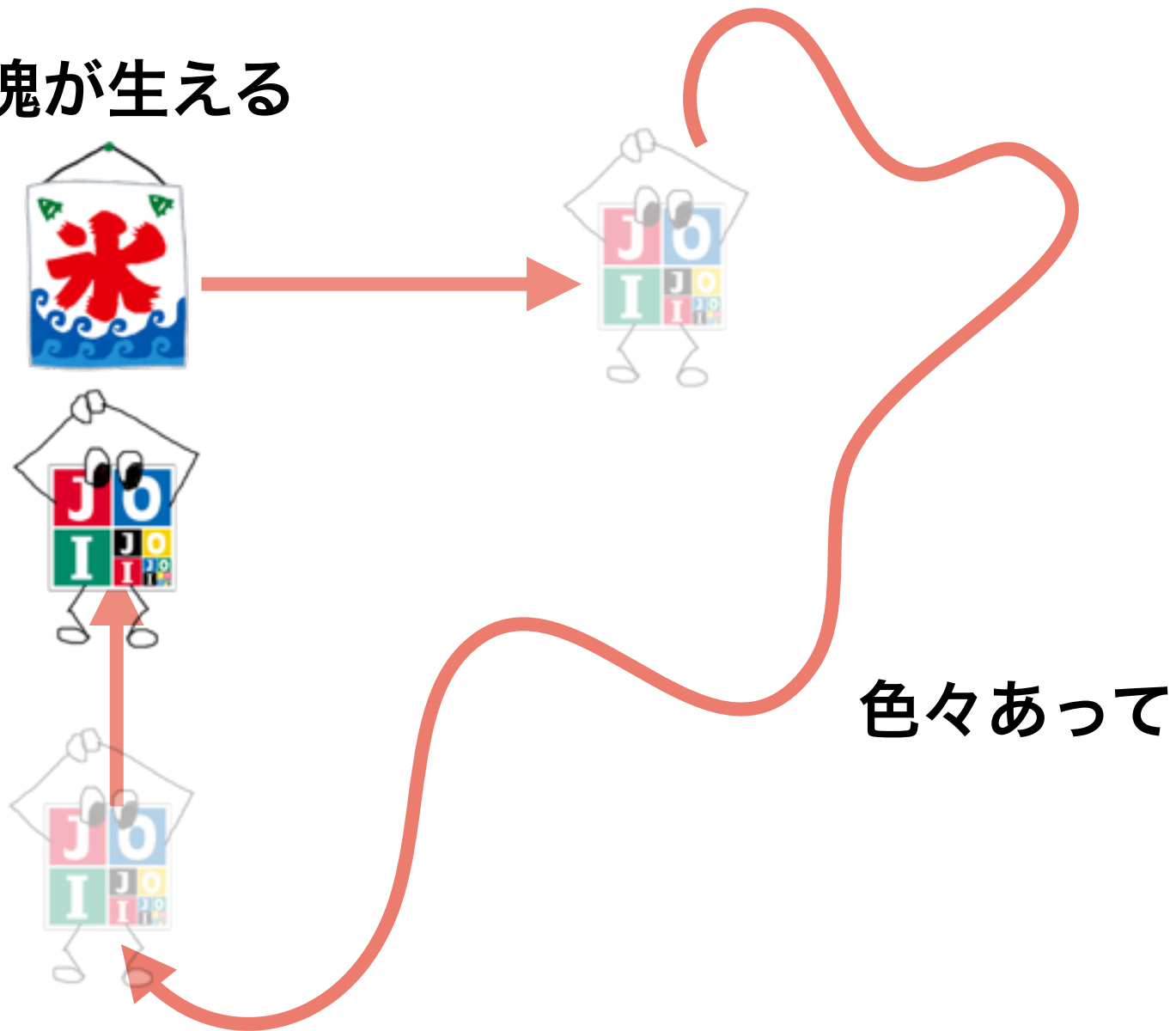
氷塊が生える



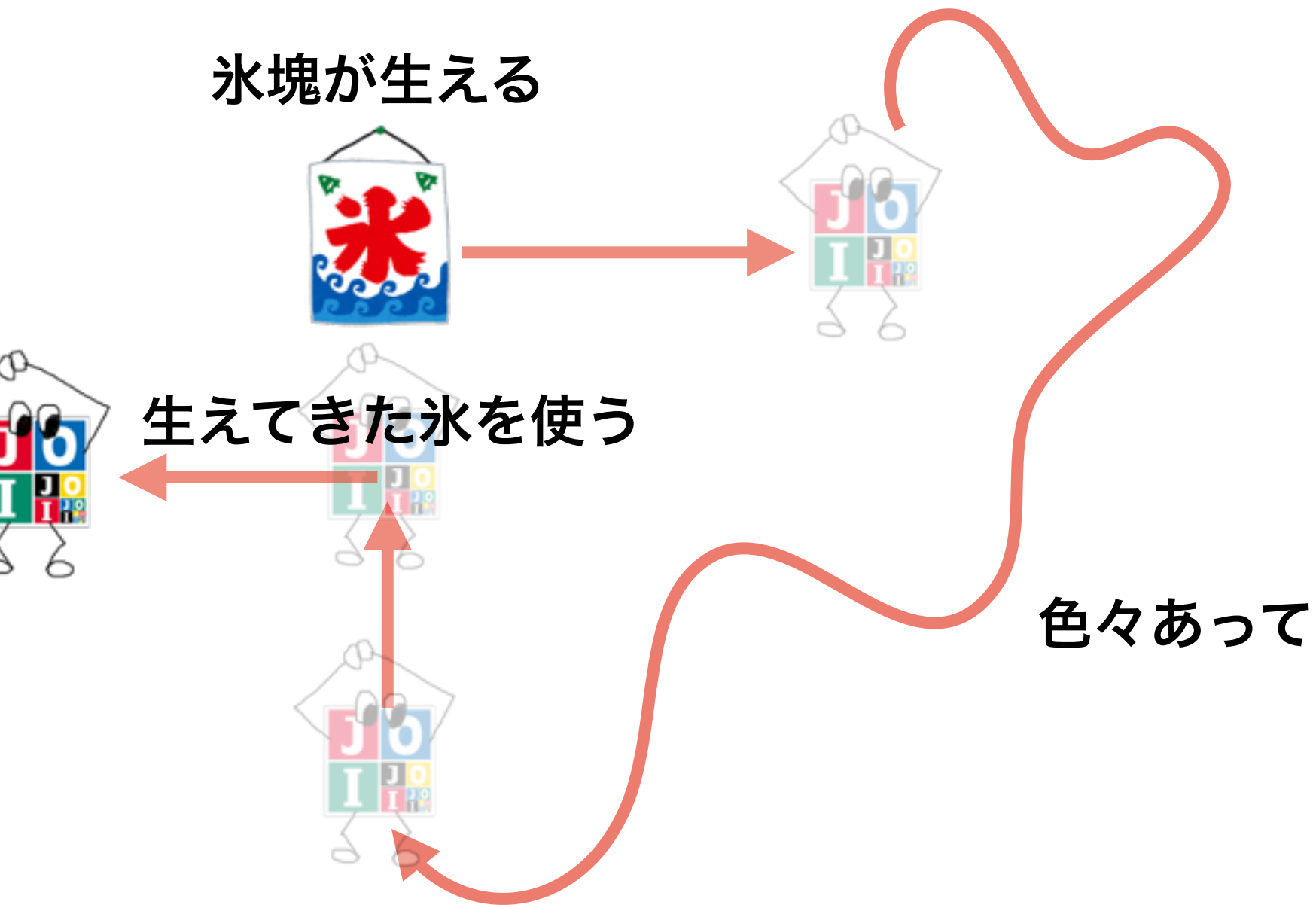
色々あって

何がやばいのか

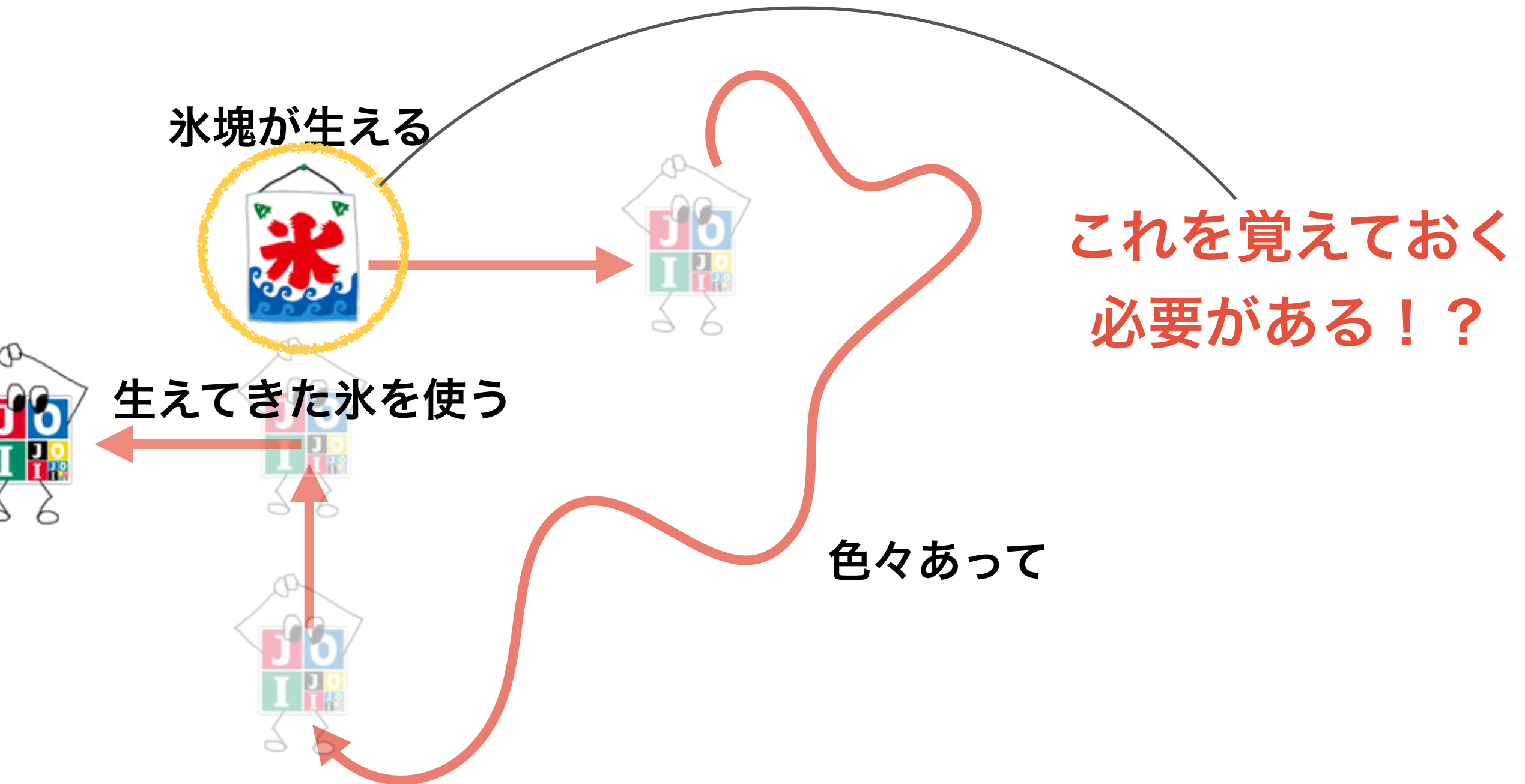
氷塊が生える



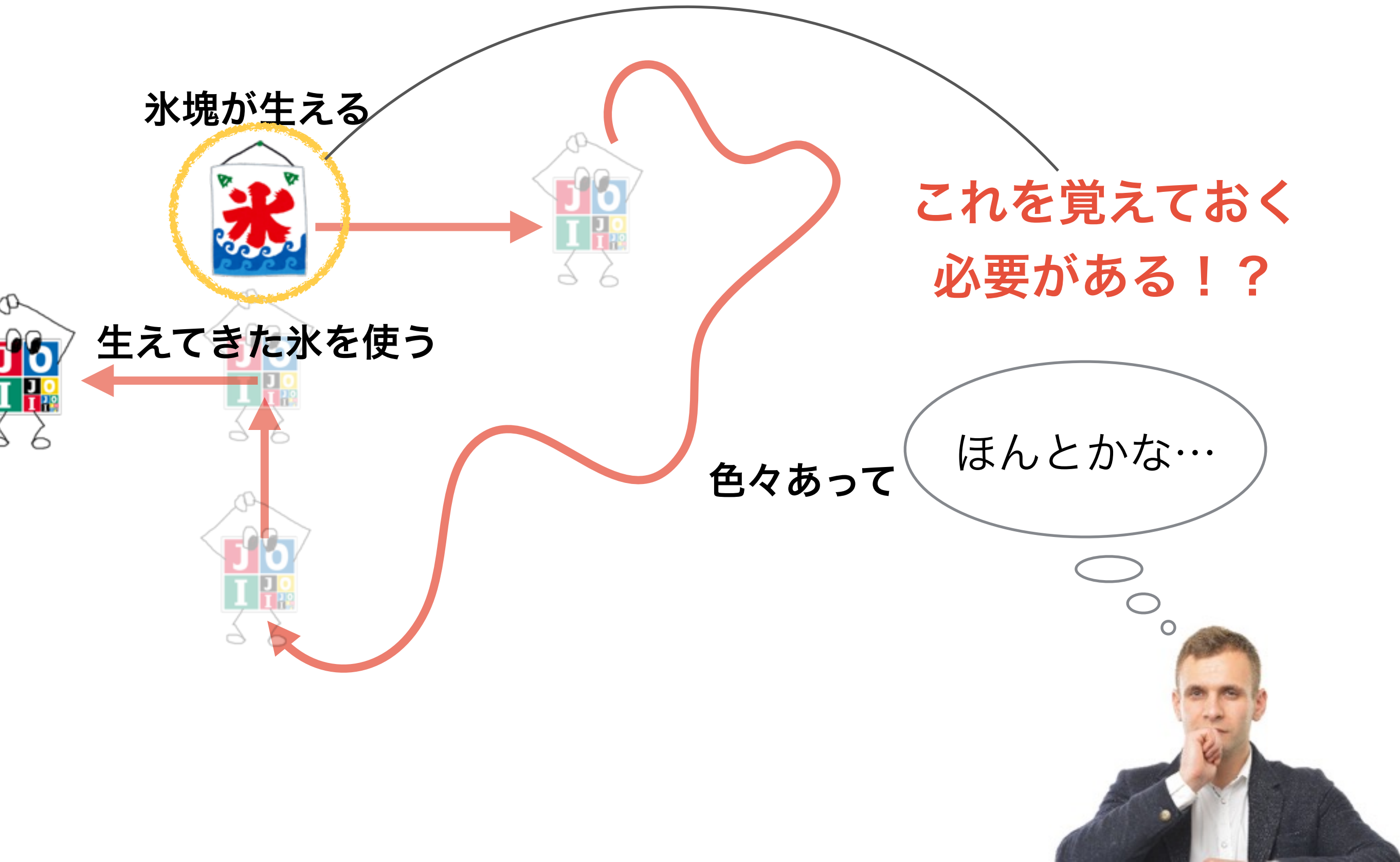
何がやばいのか



何がやばいのか



何がやばいのか



実はやばくない？

氷塊が生える



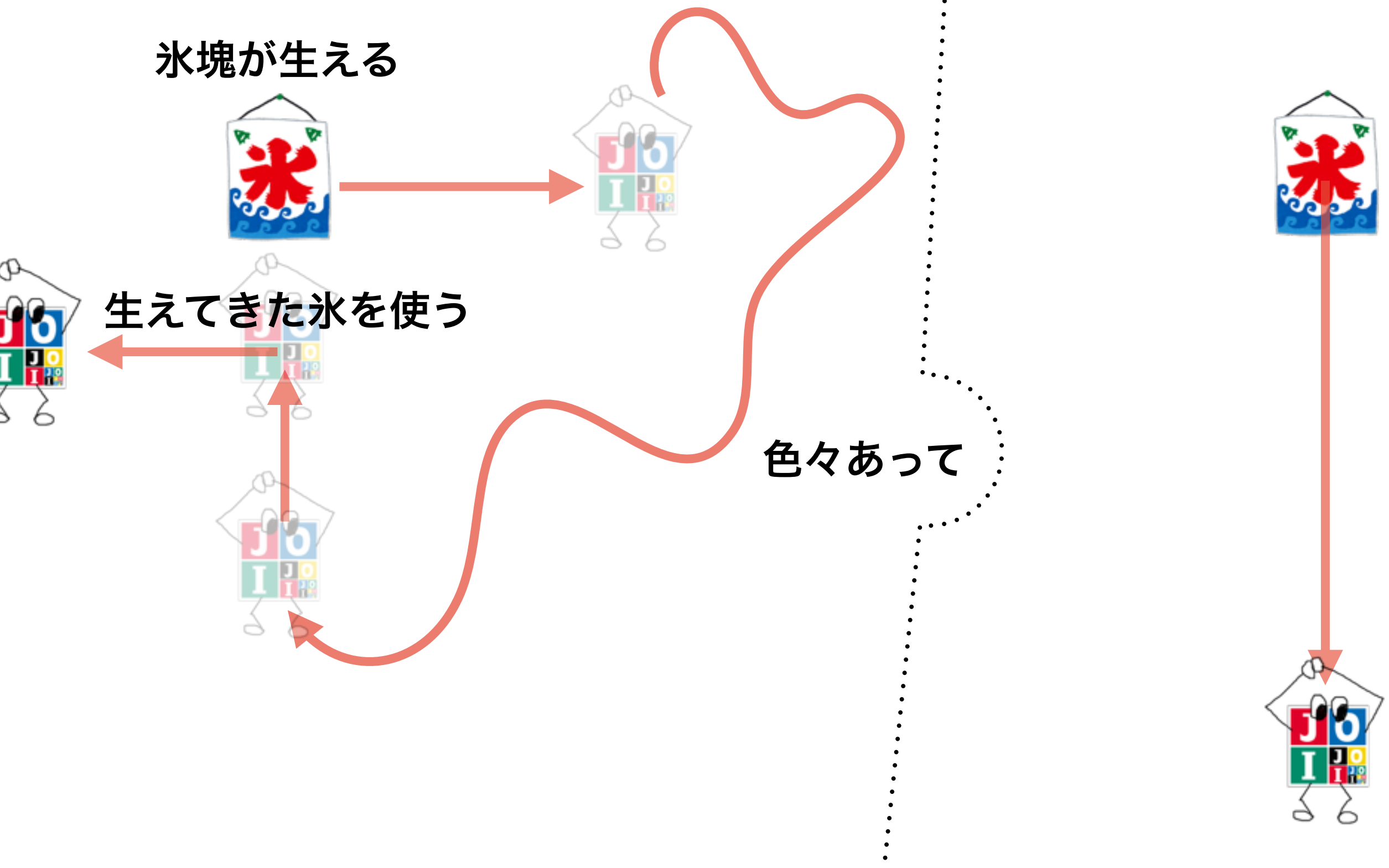
生えてきた氷を使う



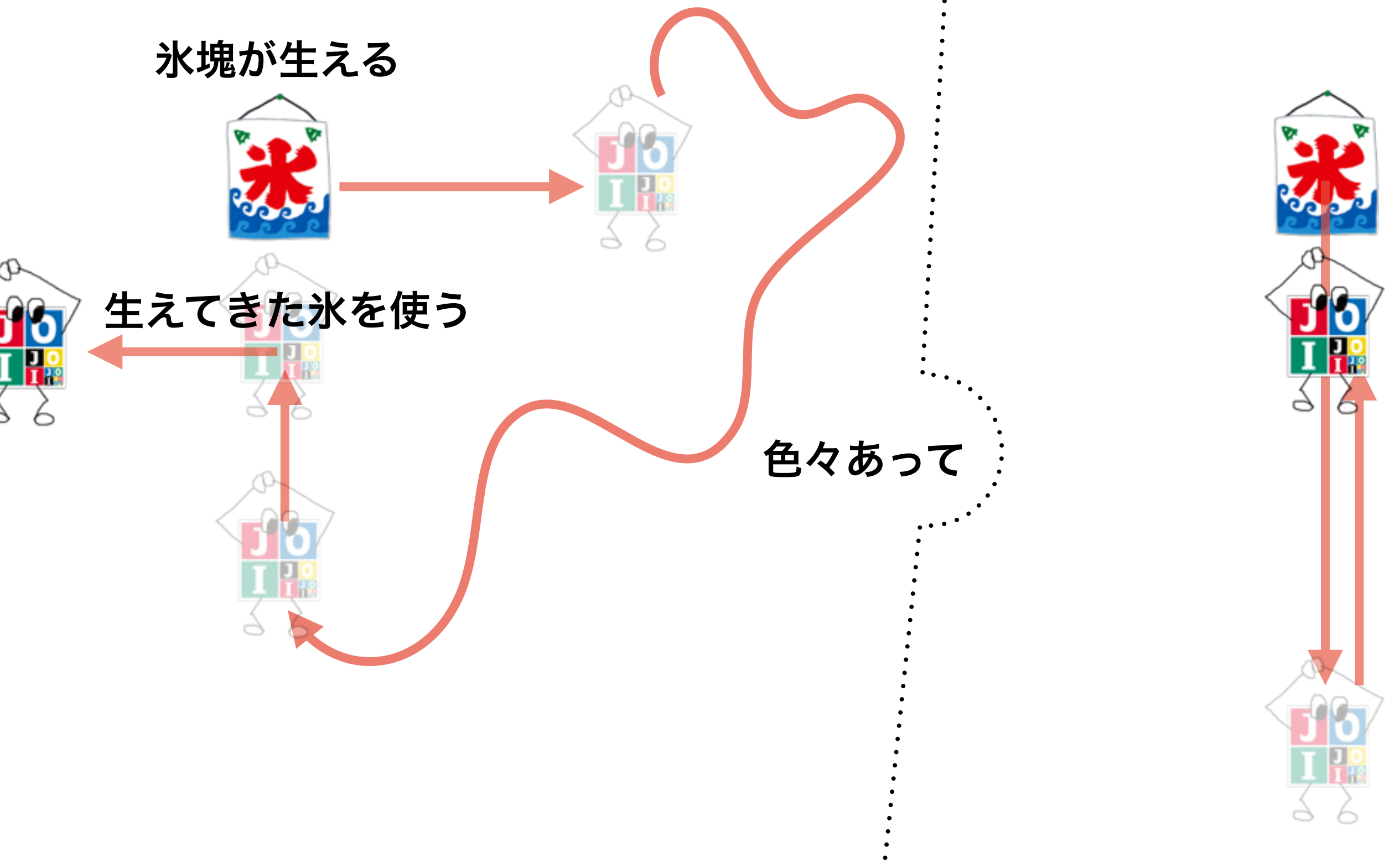
色々あって



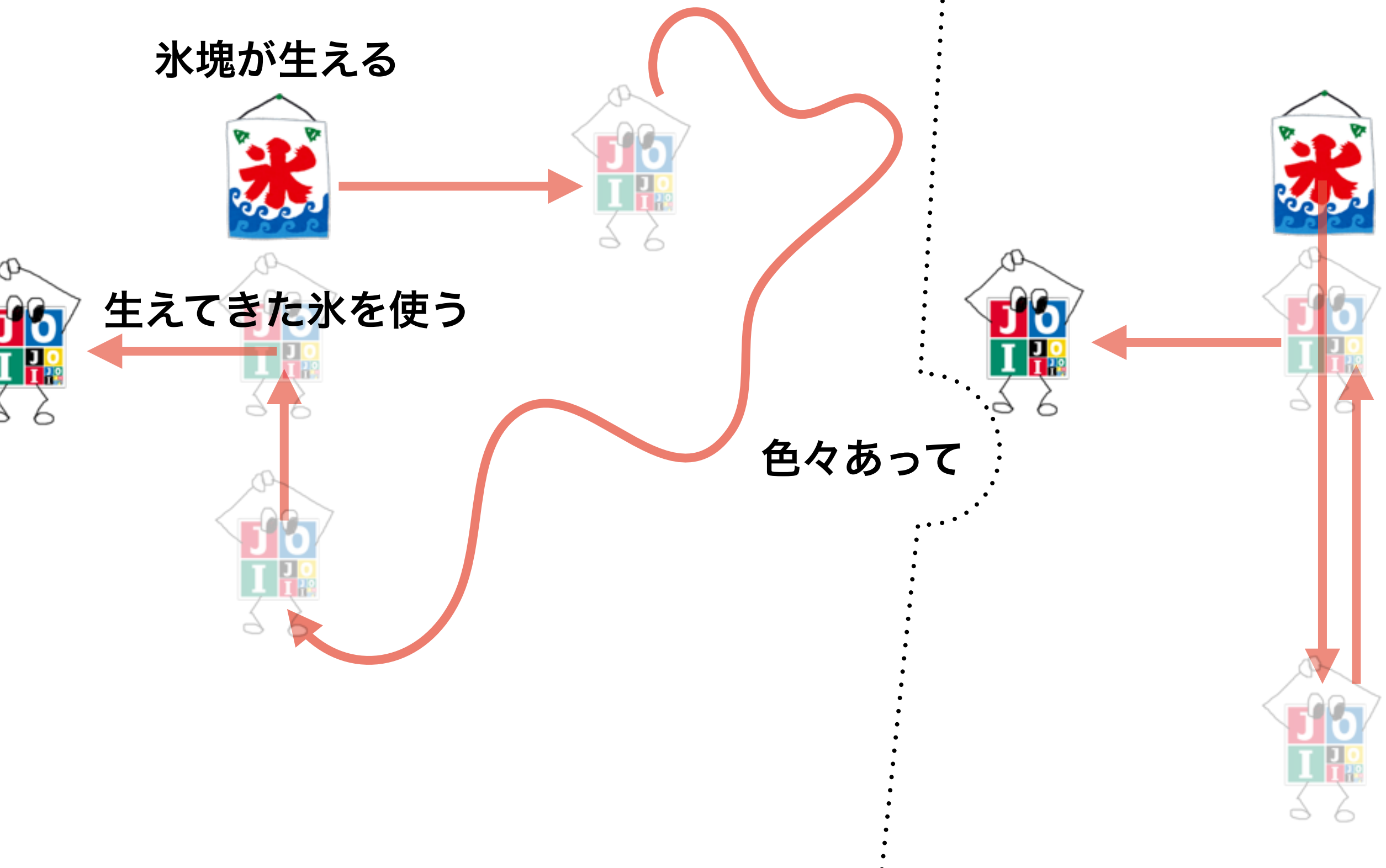
実はやばくない？



実はやばくない？



実はやばくない？



実はやばくない？

ショートカットできるし
左みたいな考えなくて
いいんでは

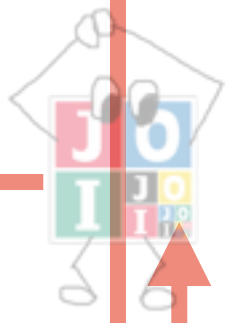
氷塊が生



生えてきた氷を使う



色々あって



実はやばくない？

ショートカットできるし
左みたいな考えなくて
いいのでは

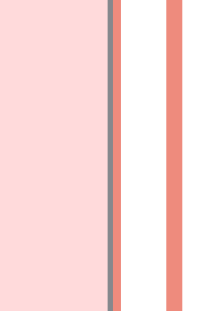
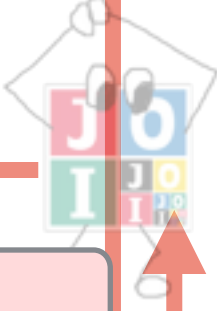
氷塊が生



生えてきた氷を使う



この辺で新しく氷作って
後で使う場合
やっぱりやばくない？



実はやばくない？

氷塊が生

ショートカットできるし
左みたいな考えなくて
いいのでは

実はやっぱりやばくない！！

この辺で新しく水作って

後で使う場合
やっぱりやばくない？

結局どういふことか



元からある氷塊



結局どういふことか

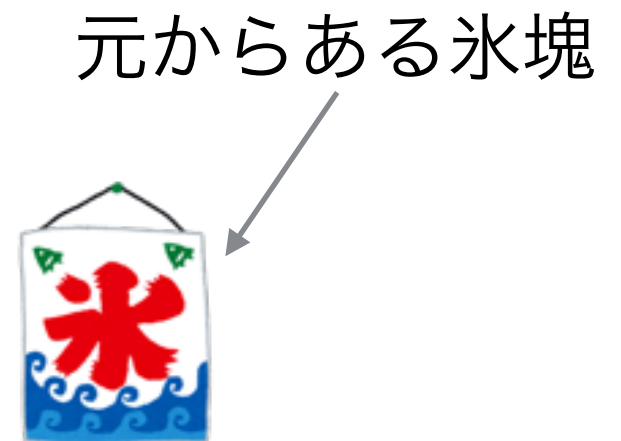
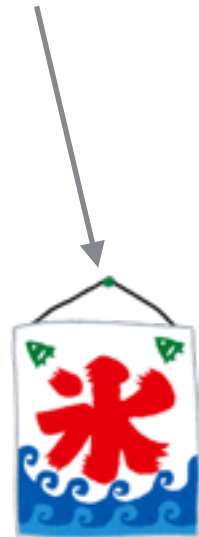


元からある氷塊



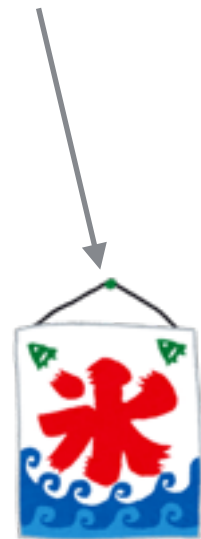
結局どういふことか

新しく生えた氷塊は



結局どういふことか

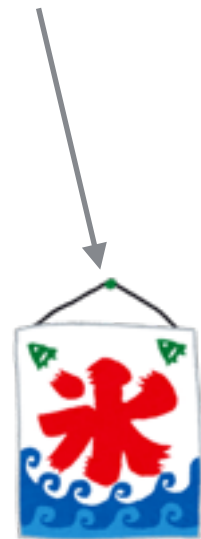
新しく生えた氷塊は



元からある氷塊

結局どういふことか

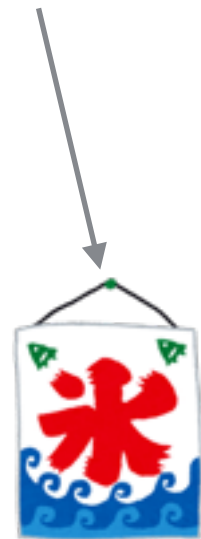
新しく生えた氷塊は



元からある氷塊

結局どういふことか

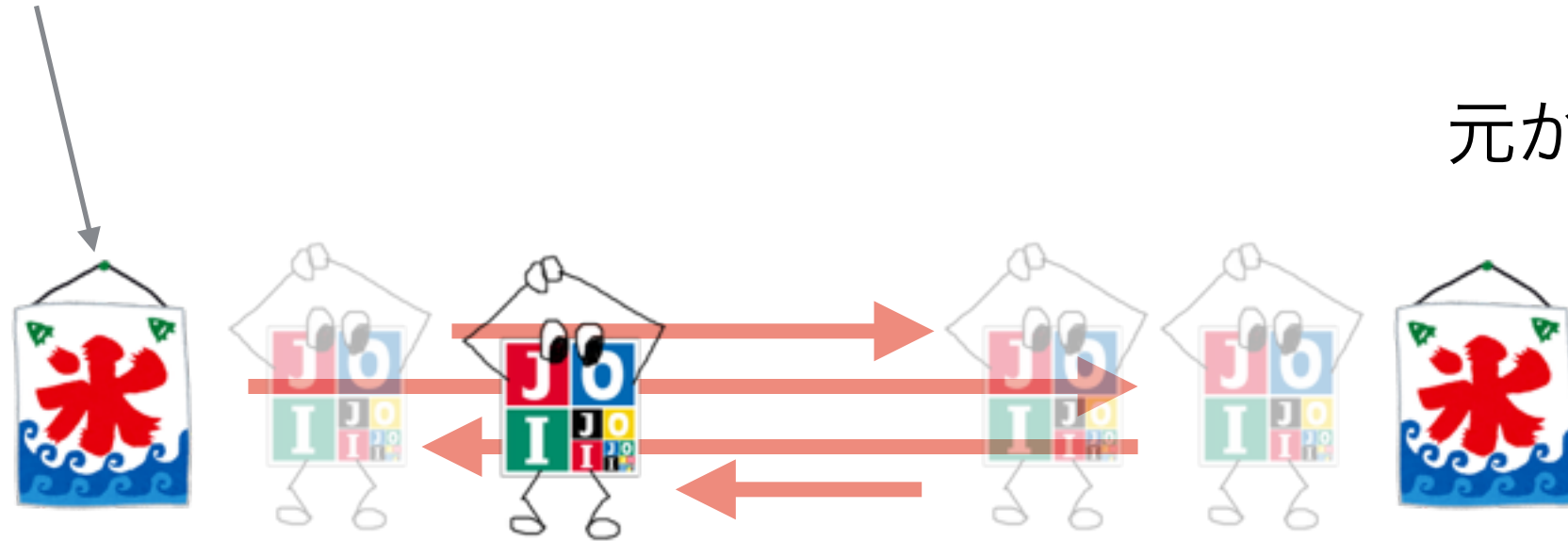
新しく生えた氷塊は



元からある氷塊

結局どういふことか

新しく生えた氷塊は

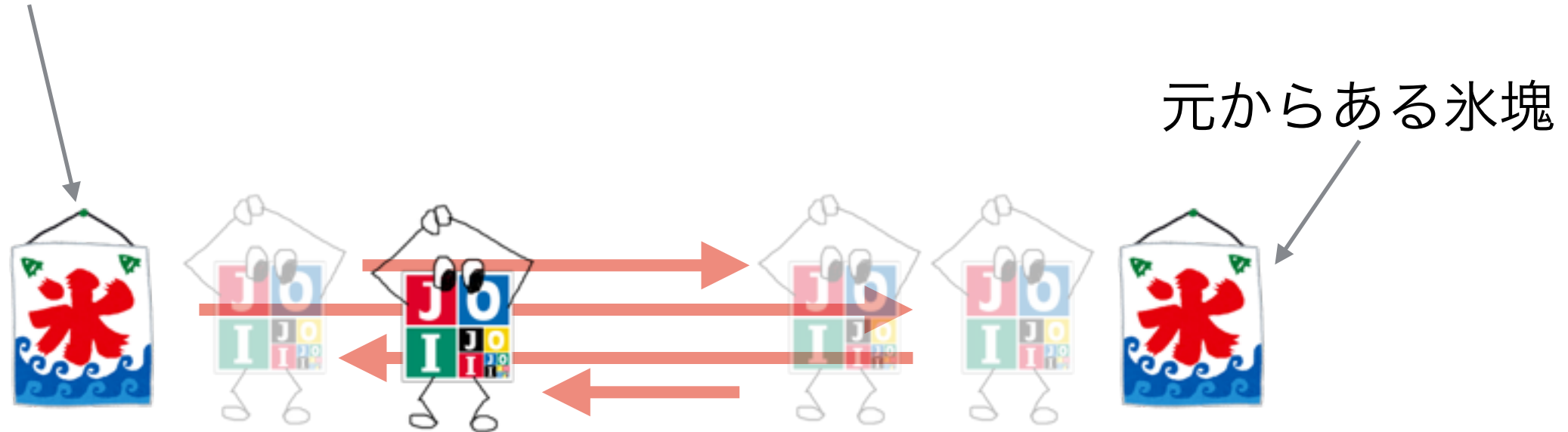


元からある氷塊

こういう往復運動でしか使わなくてもよいのでは？

結局どういふことか

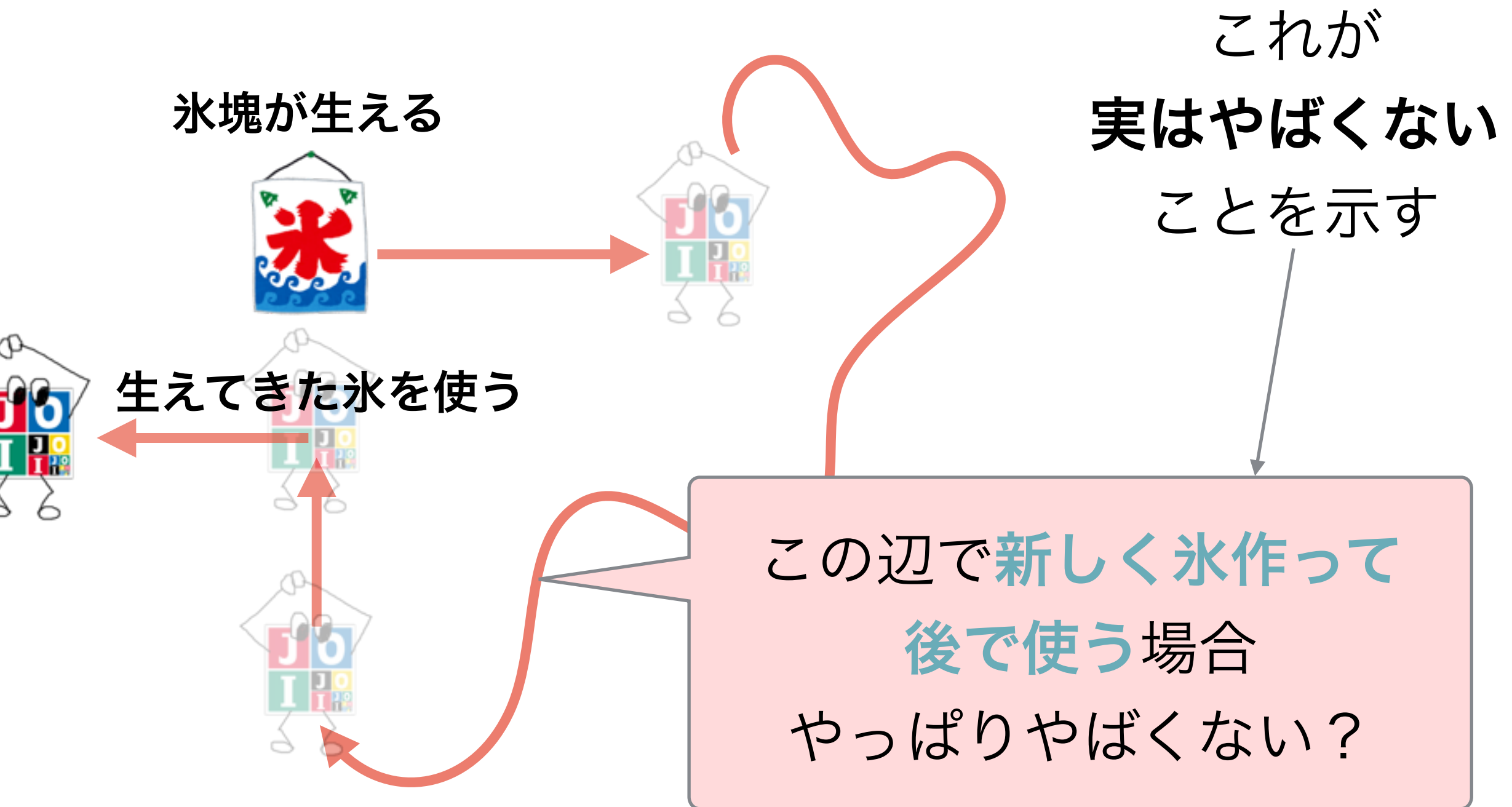
新しく生えた氷塊は



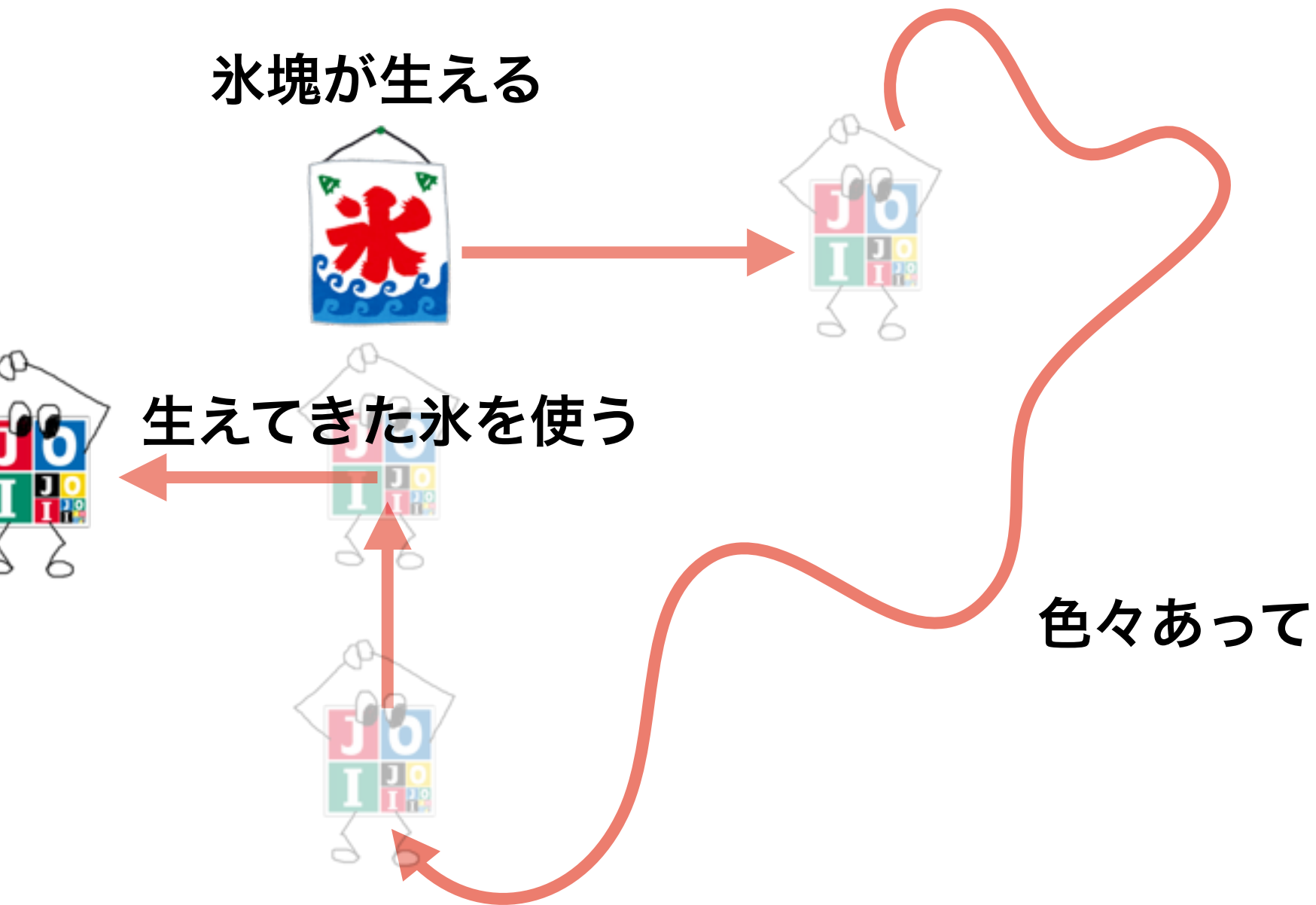
こういう往復運動でしか使わなくてもよいのでは？

証明しよう！

簡単な証明



簡単な証明



簡単な証明

これが最短路と仮定して

氷塊が生える



生えてきた氷を使う



色々あって



簡単な証明

これが最短路と仮定して

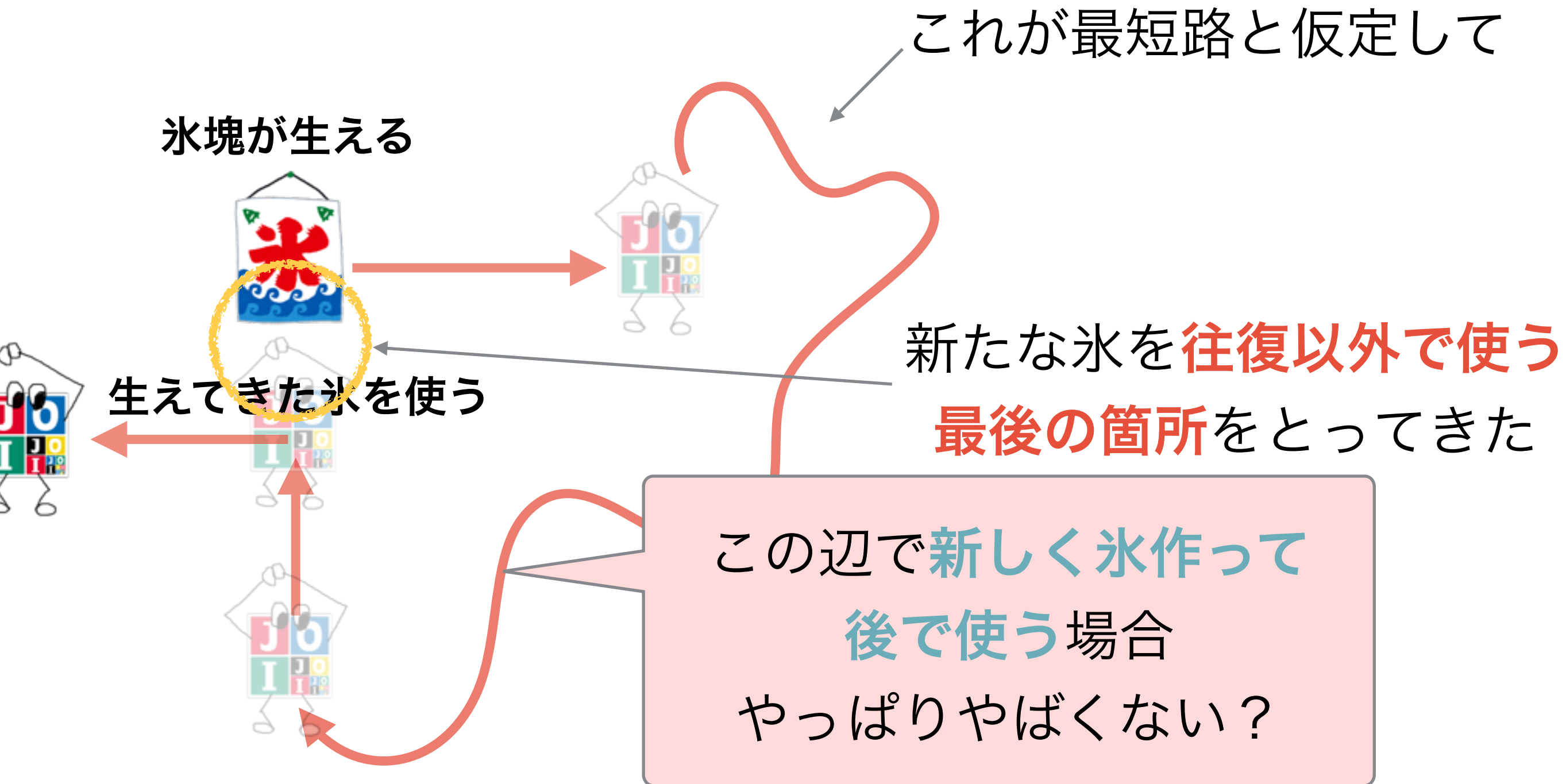
氷塊が生える

生えてきた氷を使う

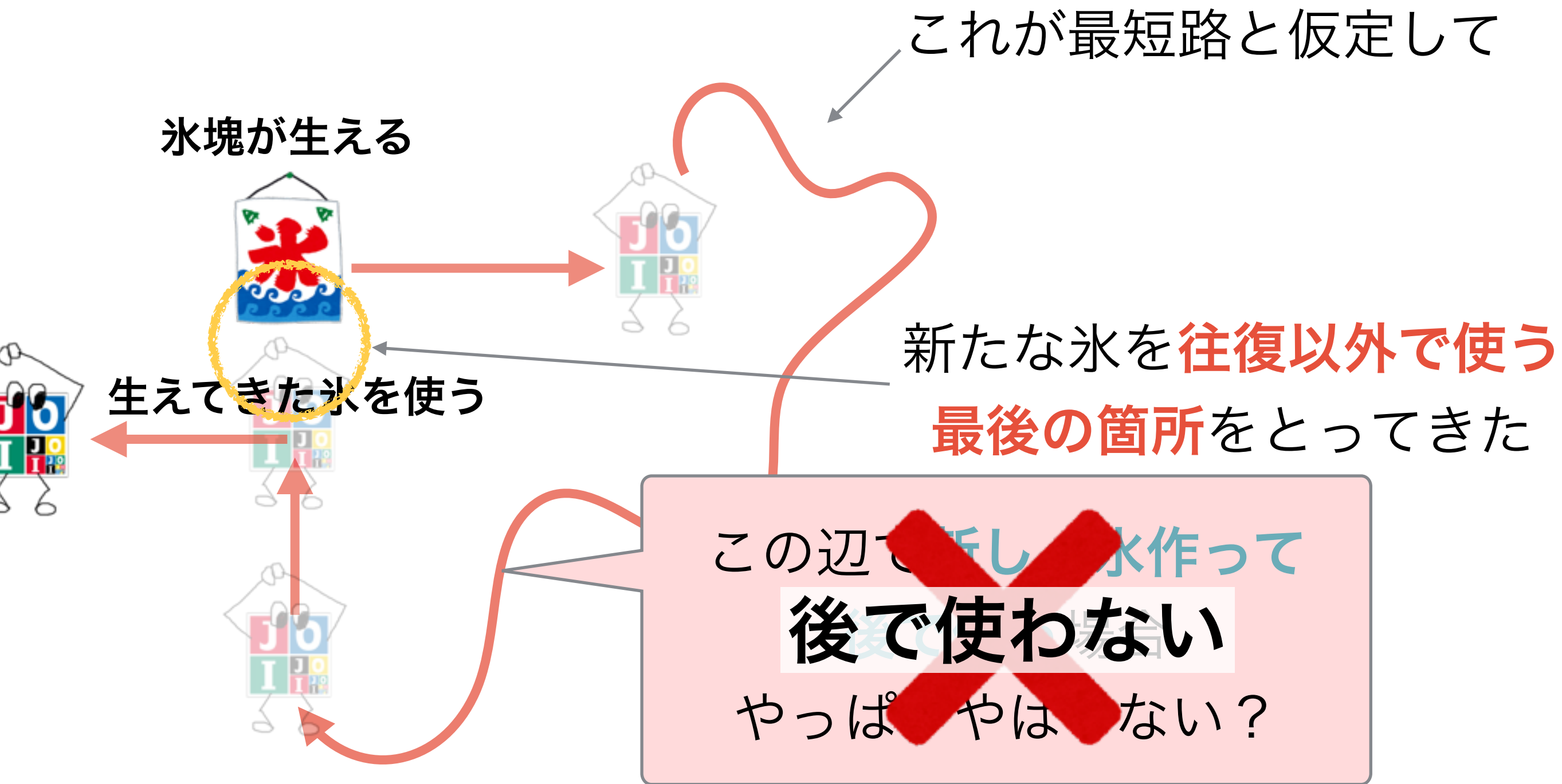
新たな氷を往復以外で使う
最後の箇所をとってきた

色々あって

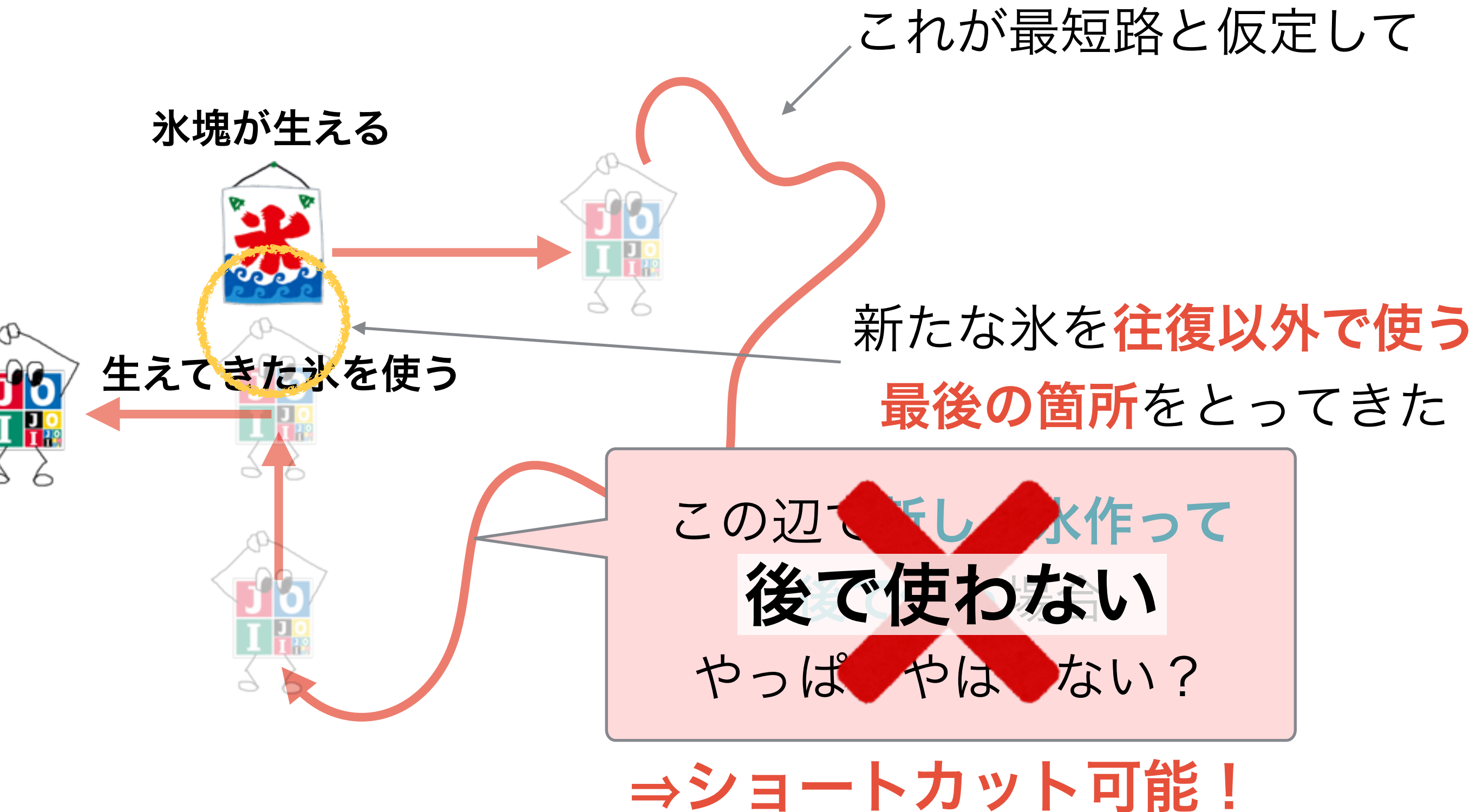
簡単な証明



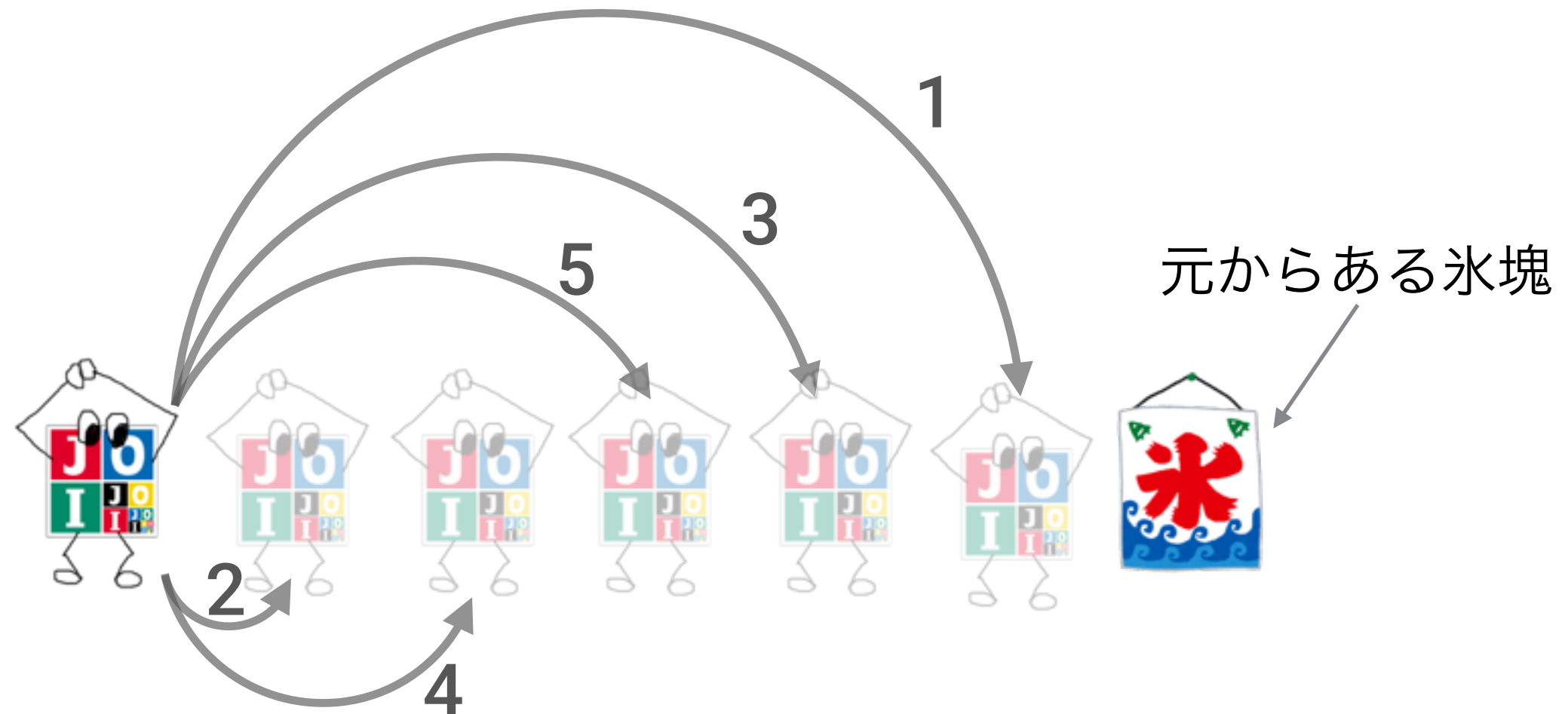
簡単な証明



簡単な証明

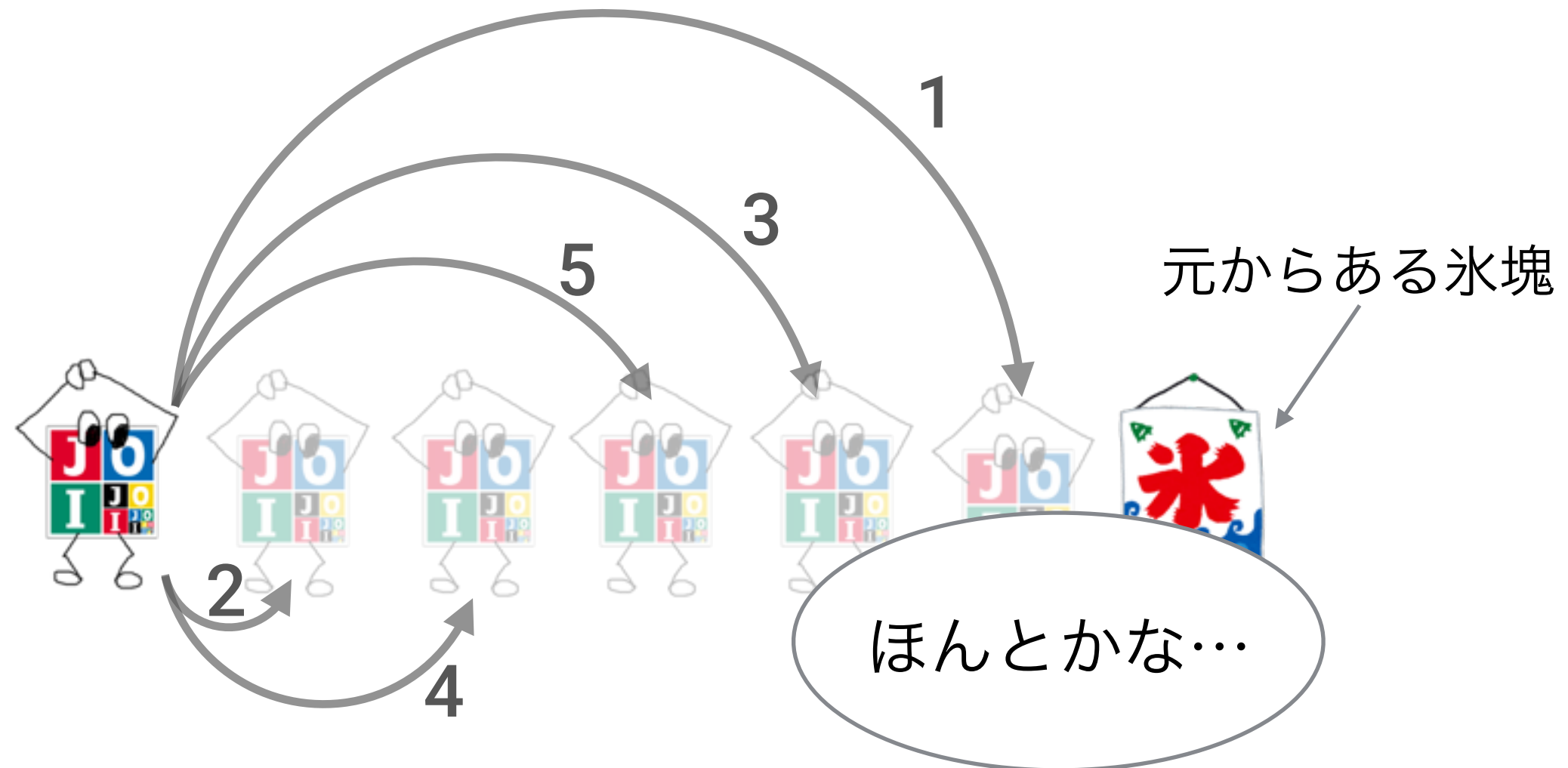


つまり



こういう辺を張って**最短路**を求めれば
良いのでは???????? (Dijkstra)

つまり

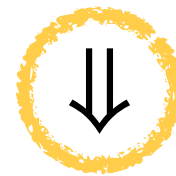


こういう辺を張って**最短路**を求めよう。
良いのでは???????? (Dijkstra)



懸念

往復以外で新しい氷塊を使わない

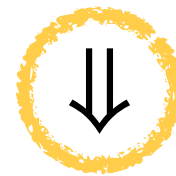


——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

懸念

往復以外で新しい氷塊を使わない



——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい



元からある氷塊

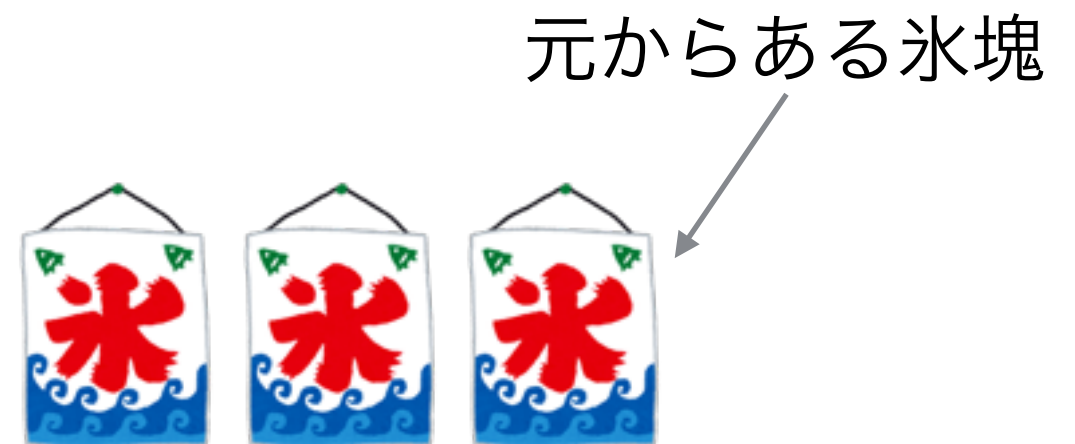


懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい



懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

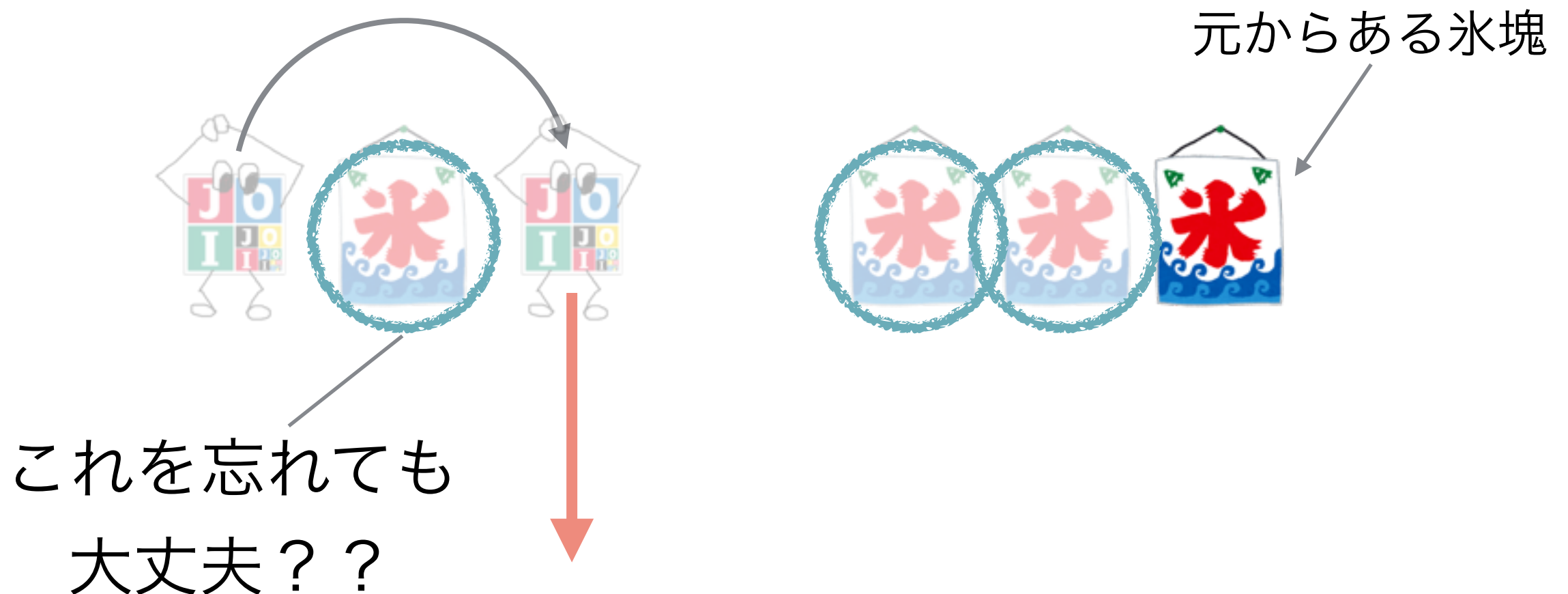


懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

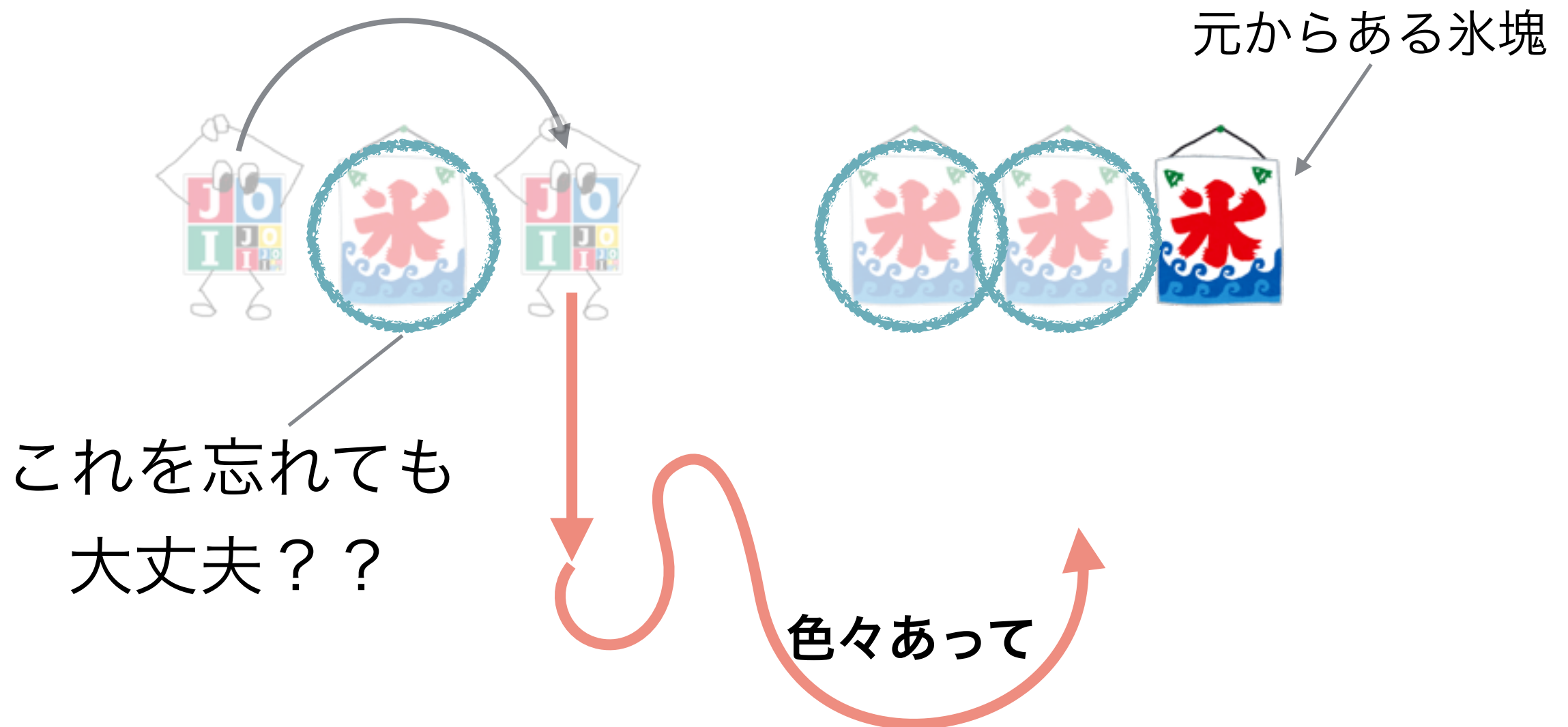


懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

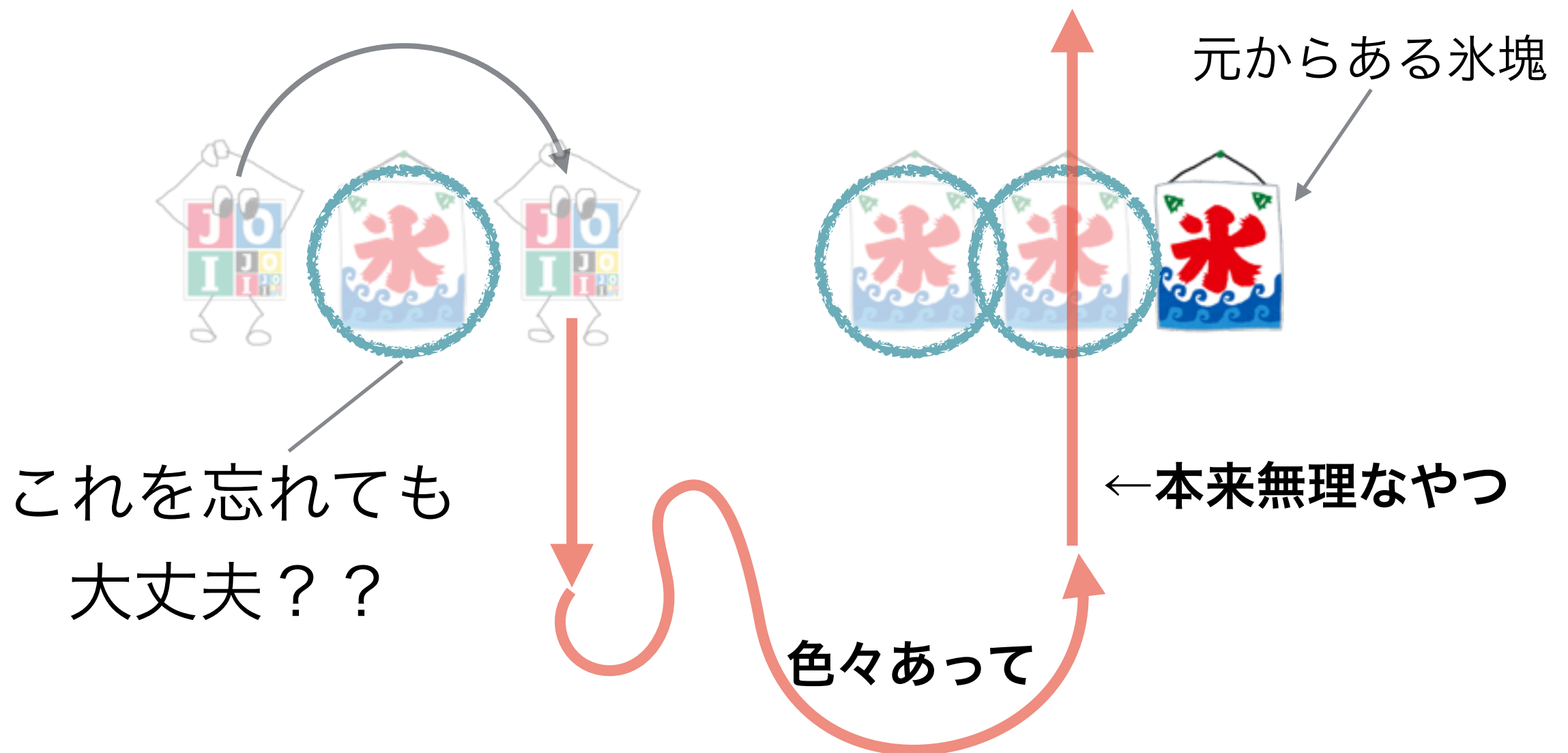


懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい



懸念

往復以外で新しい氷塊を使わない

これは非自明

往復だい

てよい

こういうのが最短路になると

やばくない？

(不当な解が出てしまう)

氷塊

これを忘れても
大丈夫??

←本来無理なやつ

色々あって

懸念

往復以外で新しい氷塊を使わない

これは非自明

往復だい

てよい

こういう場合になると

ならない

氷塊

(不当な点が出る)

これを忘れても
大丈夫??

←本来無理なやつ

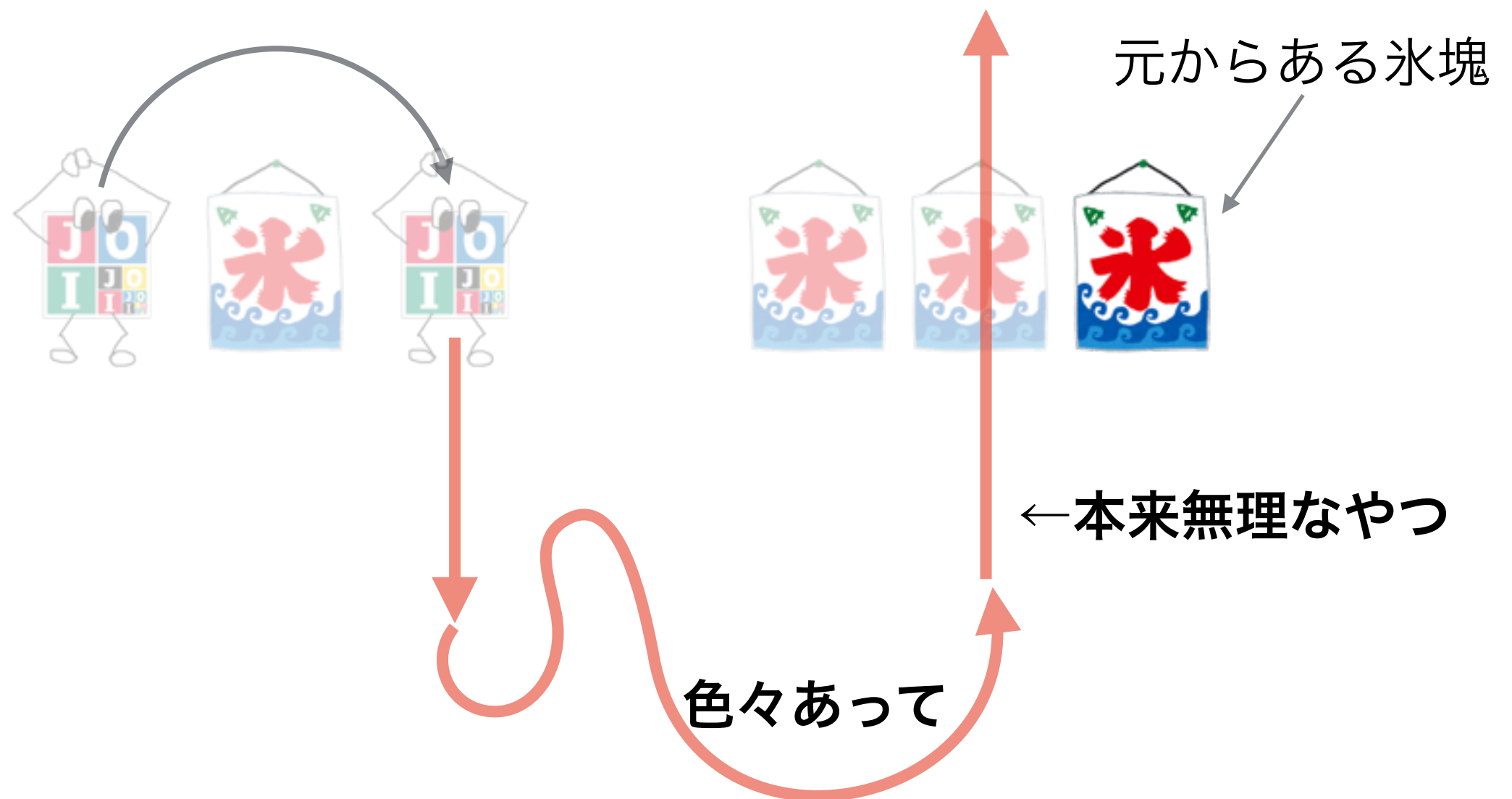
色々あって

懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

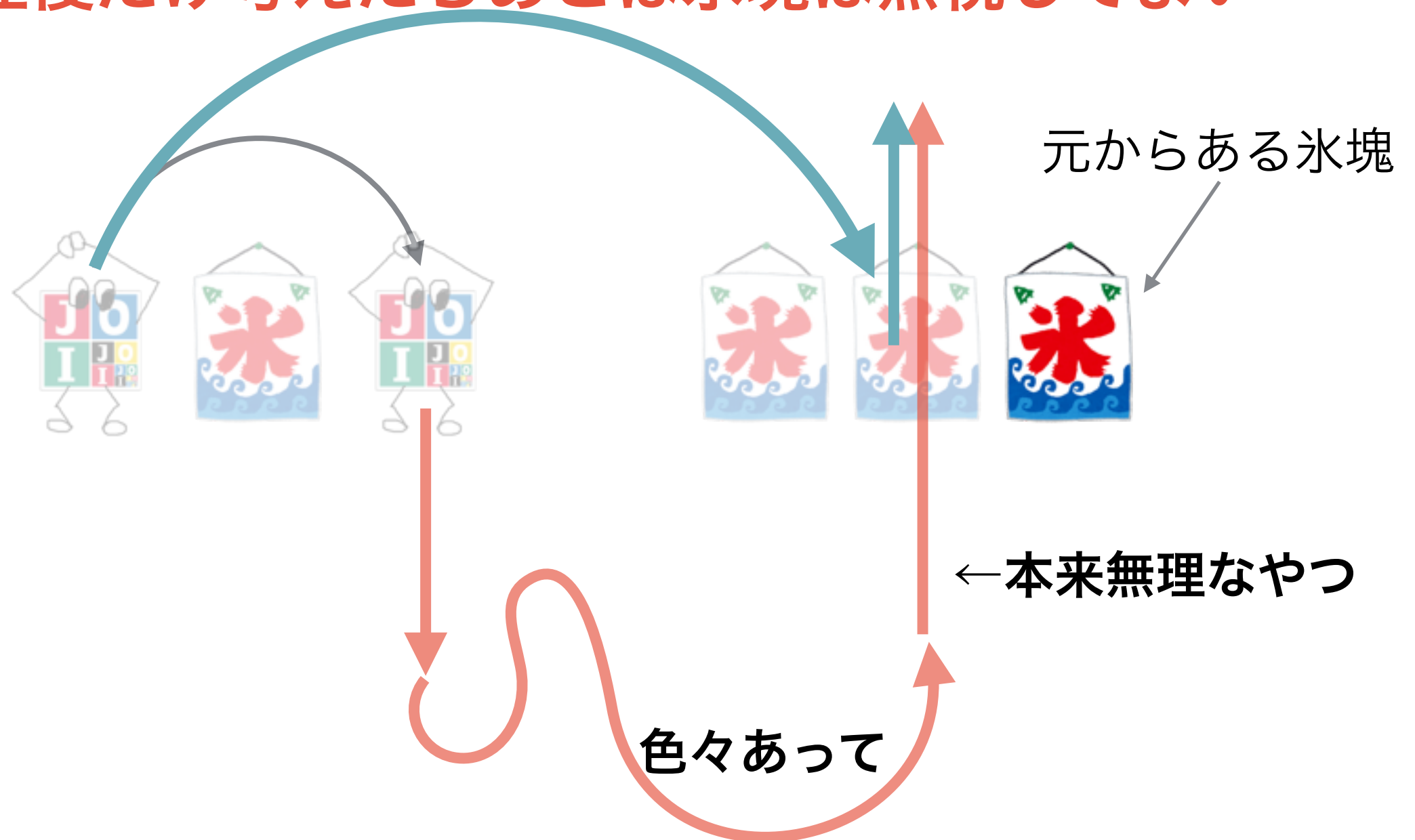


懸念

往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい

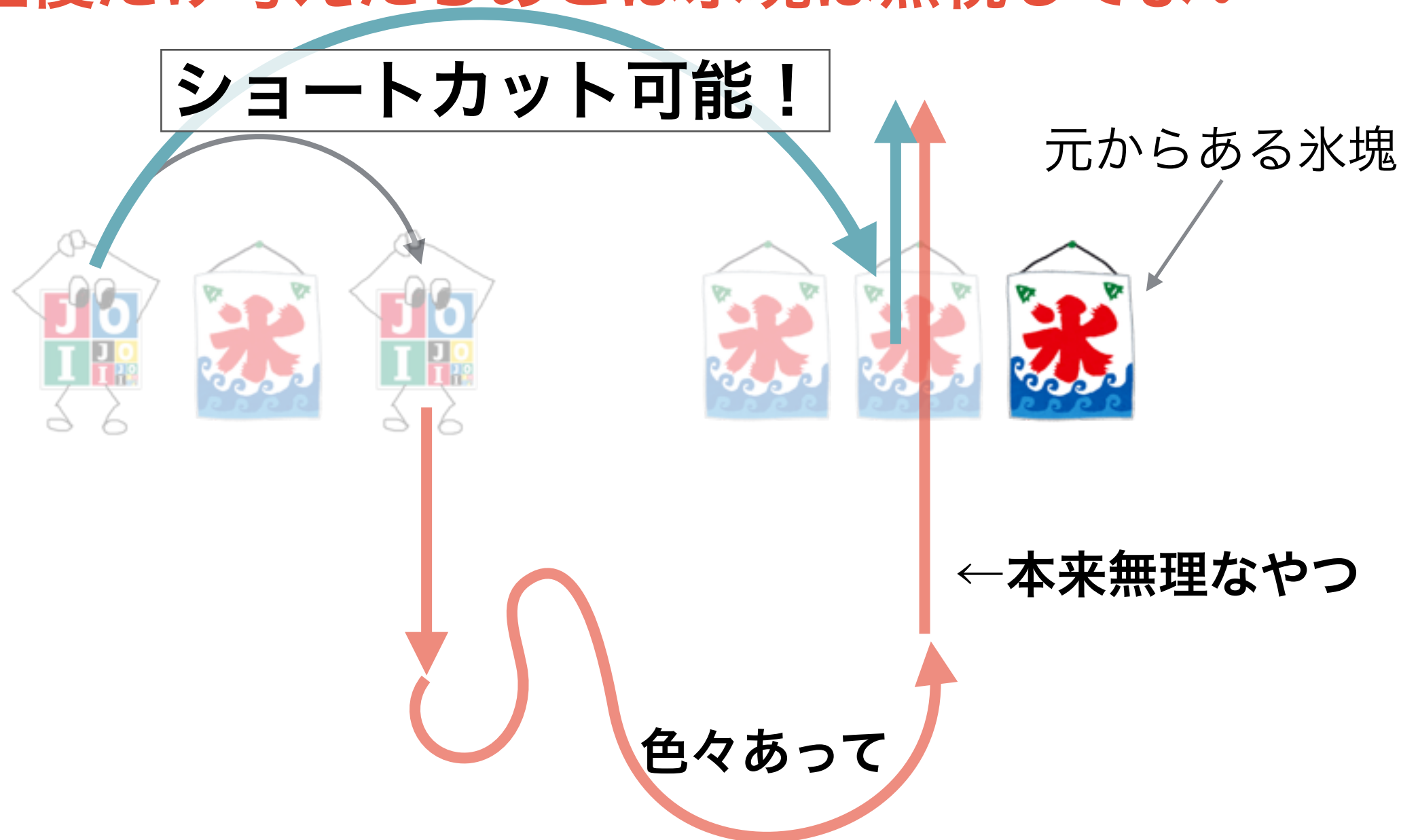


懸念

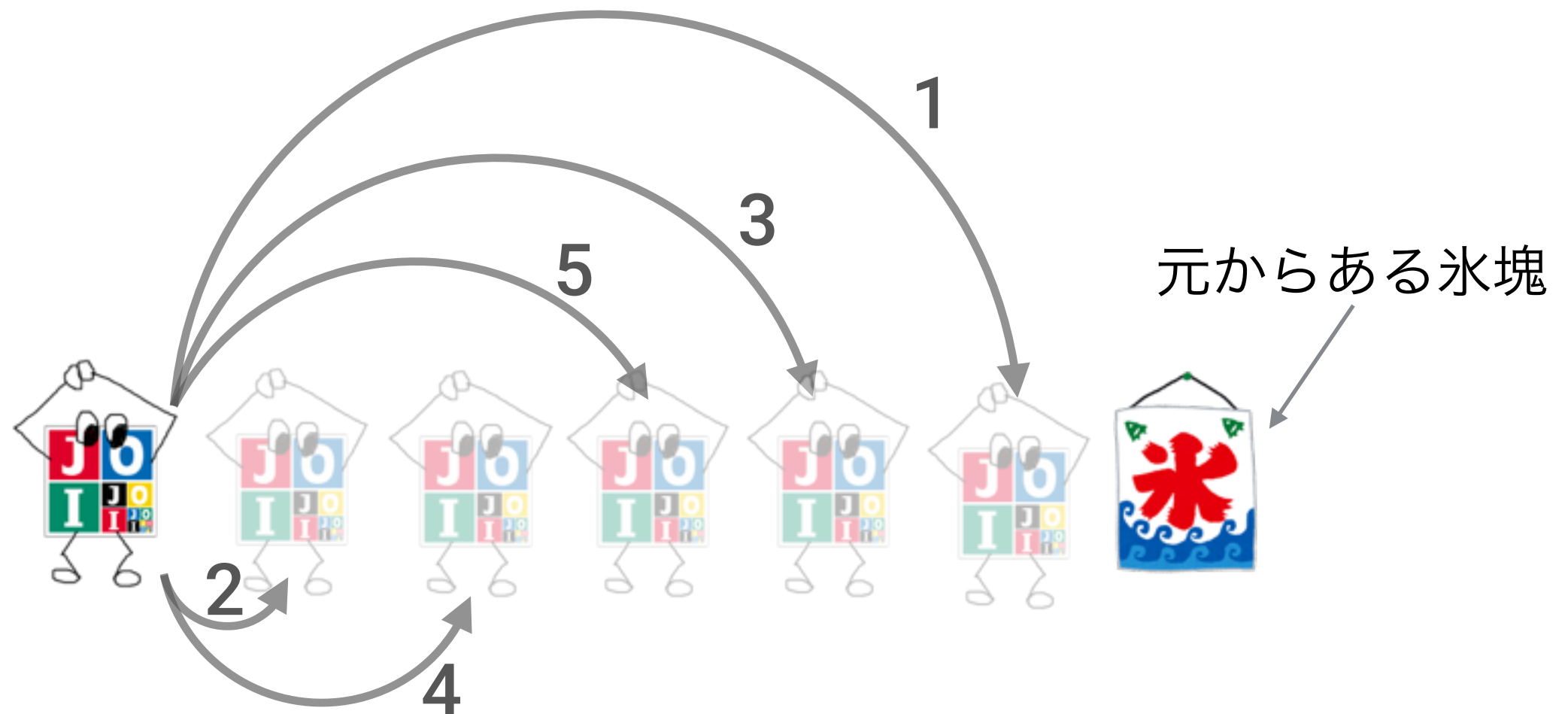
往復以外で新しい氷塊を使わない

↓——— これは非自明

往復だけ考えたらあとは氷塊は無視してよい



小課題 2 (65点)

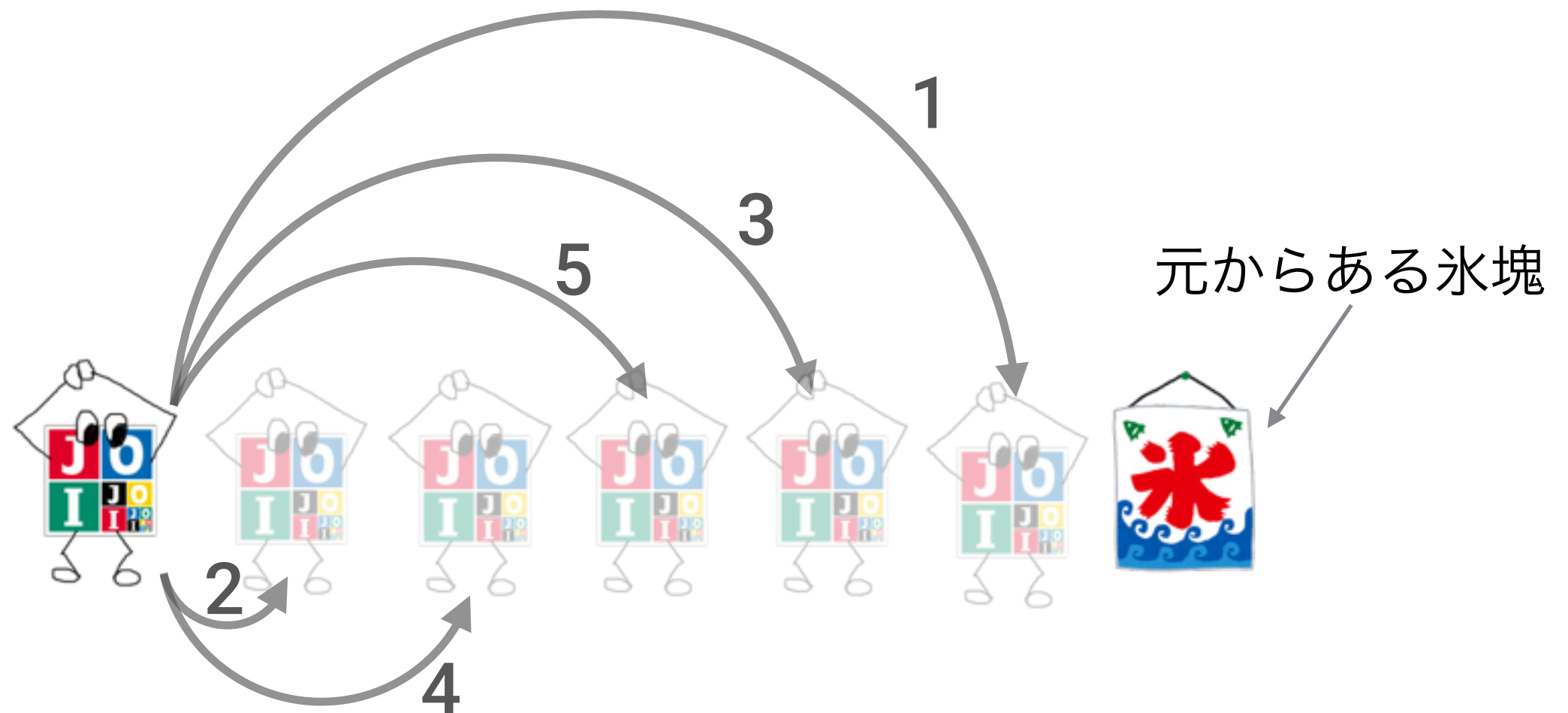


これで Dijkstra すれば, 頂点数 $\mathcal{O}(RC)$
各頂点の次数が $\mathcal{O}(R + C)$ なので計算量は

$$\mathcal{O}(RC(R + C) \log RC)$$

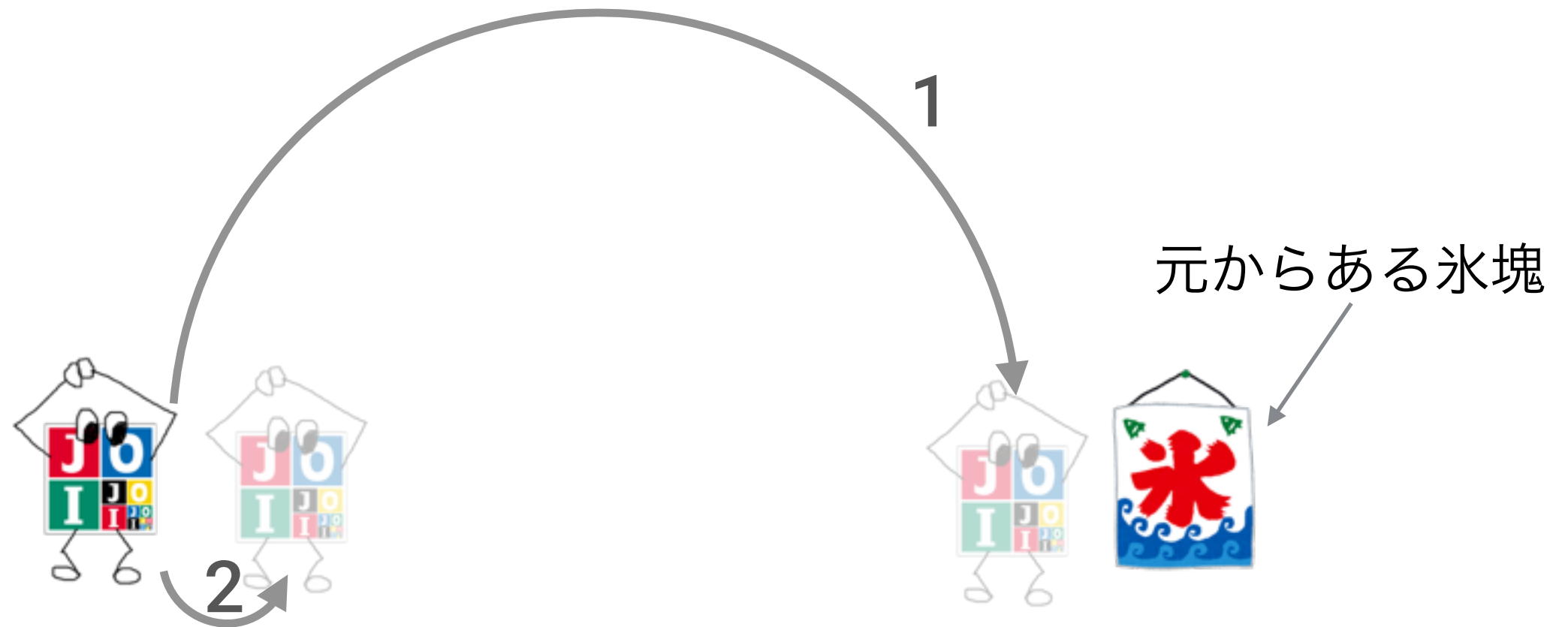
(queue を $\mathcal{O}(R+C)$ 個使えば $\log$ は消える)

辺の削減



実はこれは**無駄**が多い

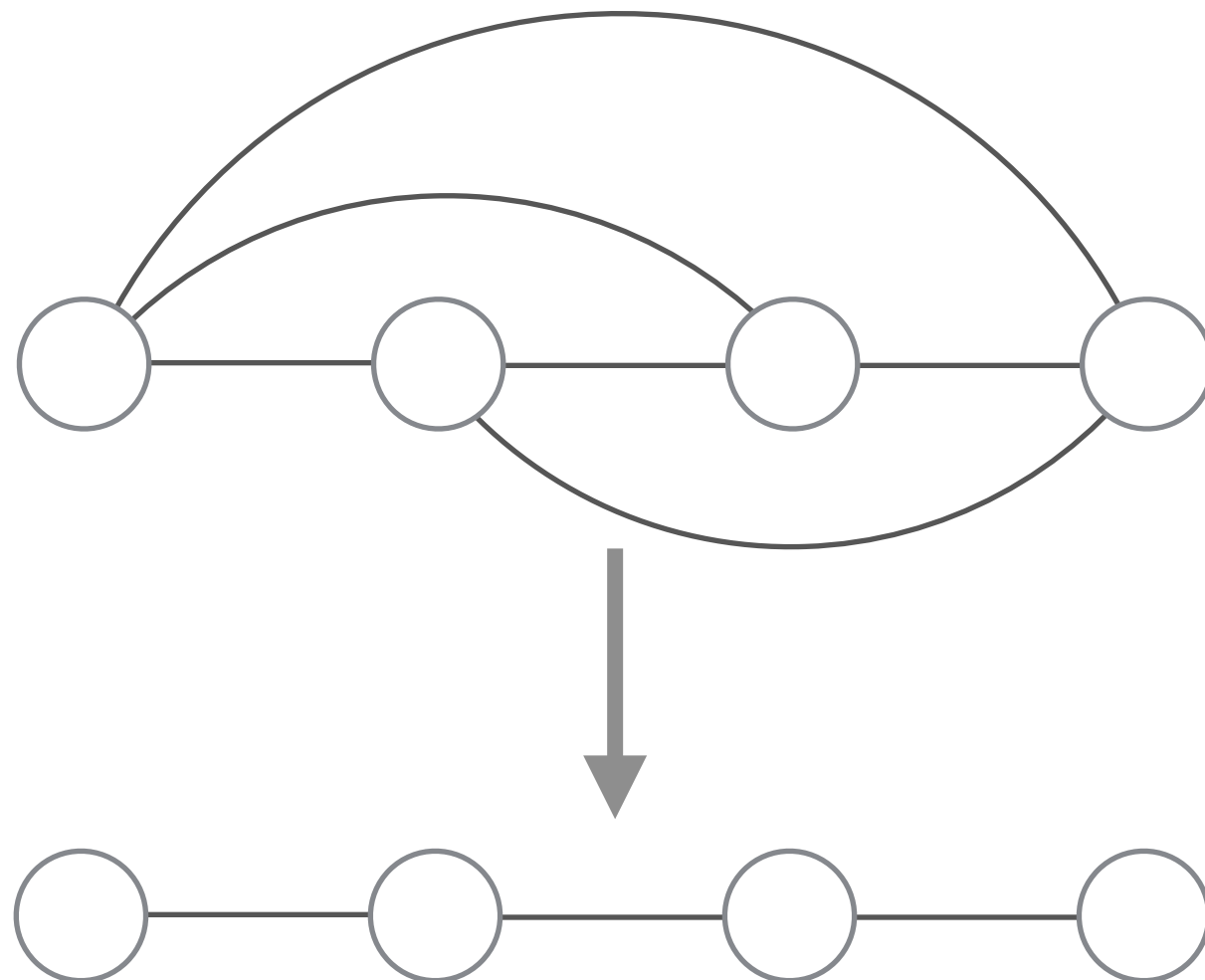
辺の削減



これだけで問題ない

辺の削減

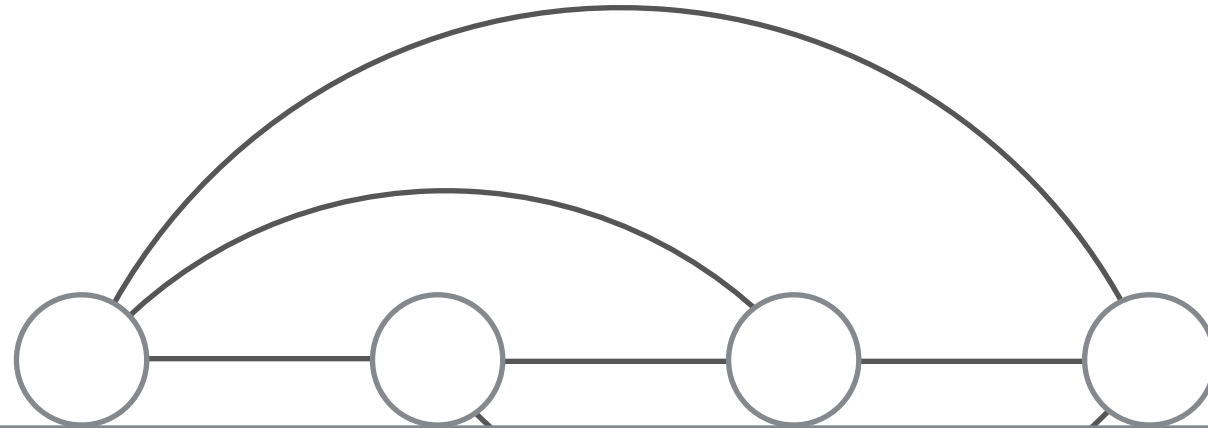
よくある, こういうのを



こうするやつ

辺の削減

よくある, こういうのを

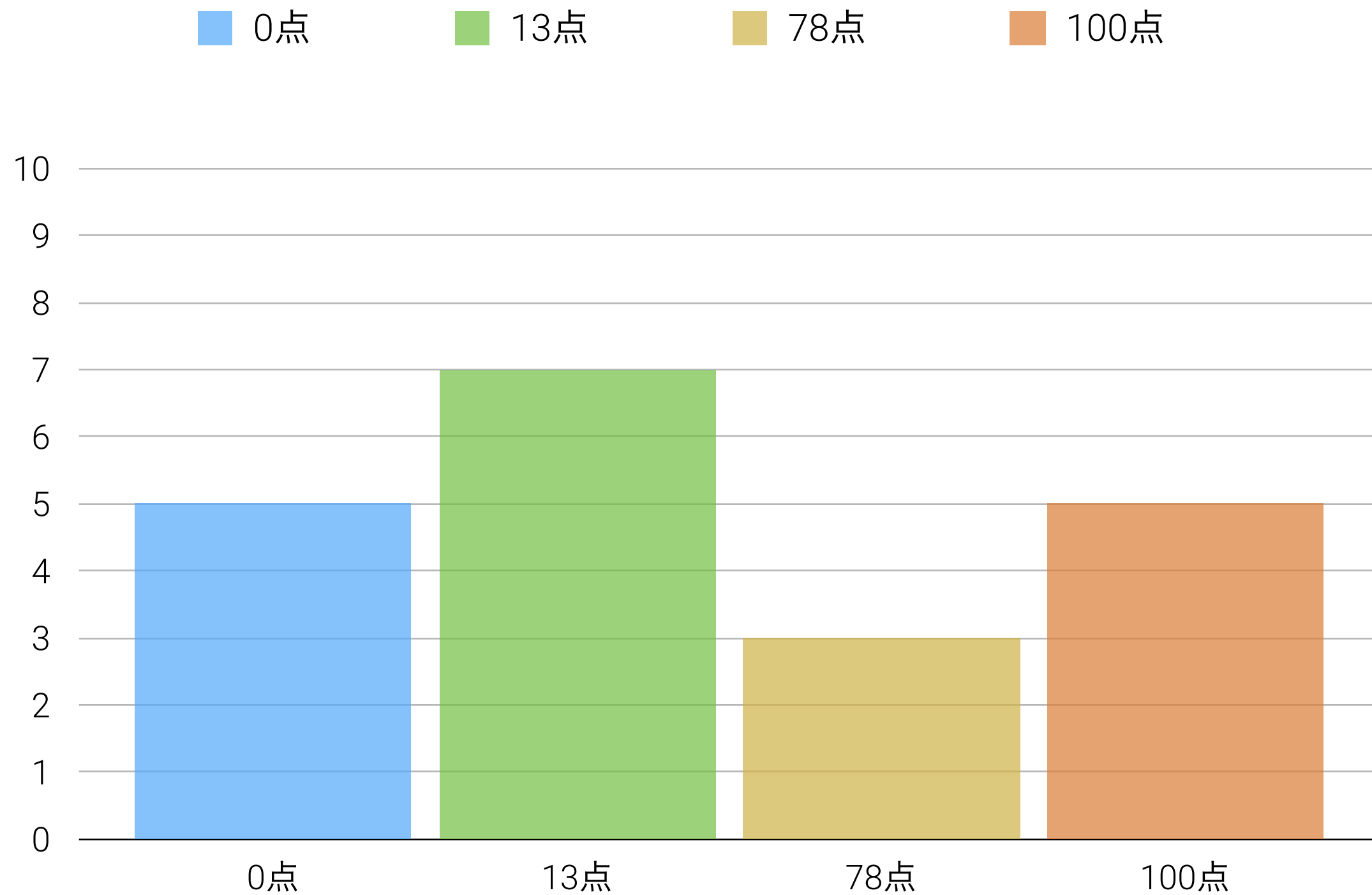


各頂点の次数が $\mathcal{O}(1)$ になって全体で
 $\mathcal{O}(RC \log RC)$ または $\mathcal{O}(RC)$



こうするやつ

得点分布



JOI 2015/2016 春合宿 Day4

雪降る道路(Snowy Roads) 解説

2016/3/23

解説担当 : 城下慎也 (IOI 2011 タイ大会 銀メダル)

最初に

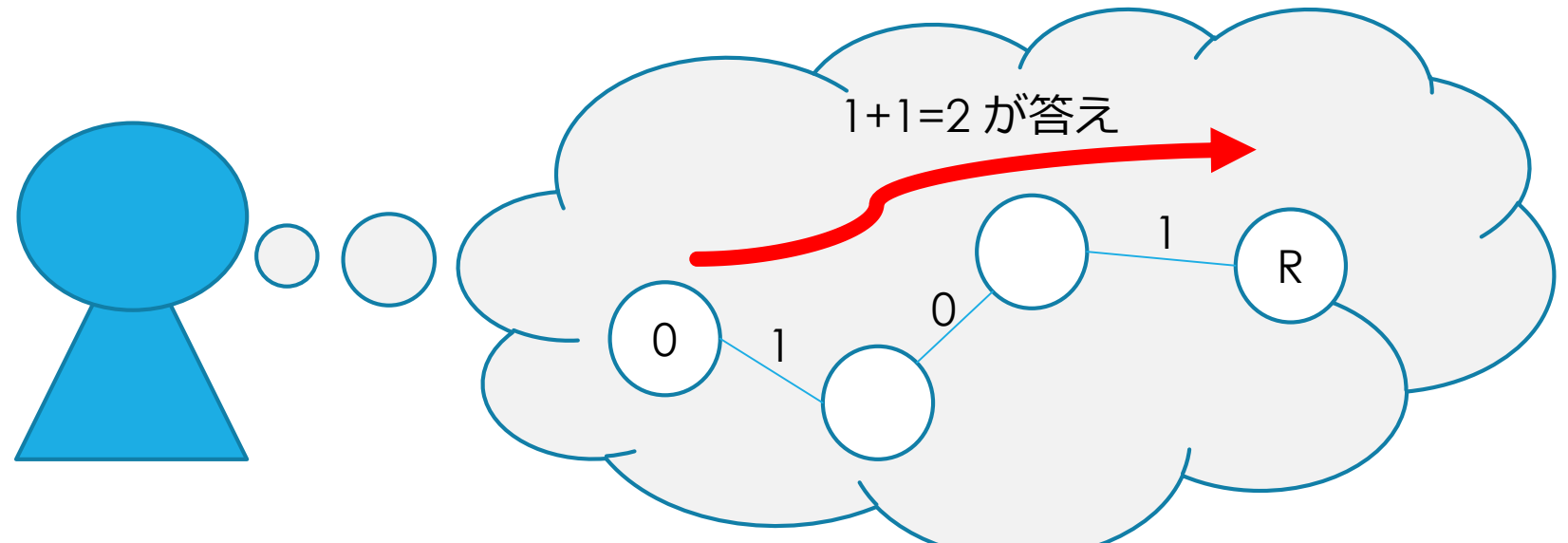
- ▶ **問題文はとても長いです。**
- ▶ 形式が特殊なので理解等が大変だったかもしれませんが、簡単な部分点等も含まれているので、今回のセットでは時間をかけてでもしっかり内容を理解をしましょう。

問題概要

- ▶ Anya と Boris がいて、Anya が中継サーバに保存し、Boris がそこから読みだして質問に回答していきます。
- ▶ 中継サーバには容量制限、質問回数制限があります。
- ▶ 質問は、都市 0 と別のある都市まで移動するのに経由する、雪降る道路の本数の最小値がいくつか、というものです。
- ▶ 同じ日に質問が複数あることもあります。
- ▶ ちなみに、同じ日の 2 回目以降の質問は以前のサーバから情報を活用できそうですが、直前に変更があるかもしれないので、実質的には活用できません。

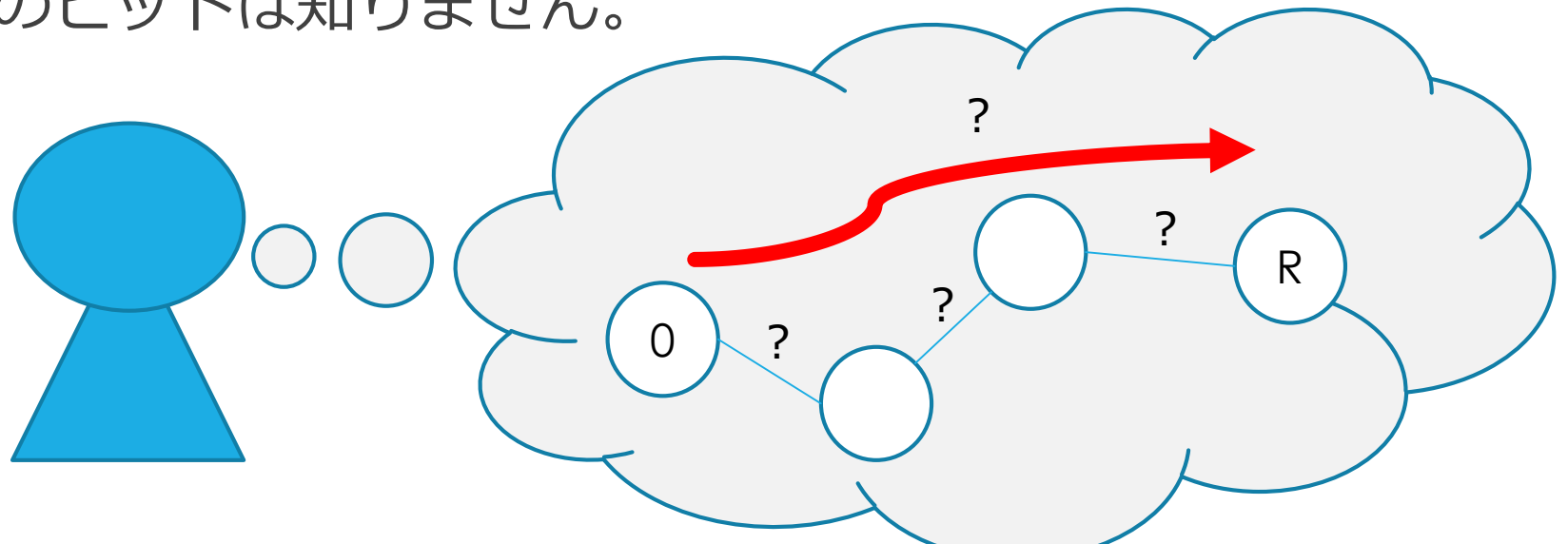
ということ？

- ▶ Boris は、都市 0 からある都市 R まで到達するために経路する必要のある雪が降っている道路の最小値を答えます。
- ▶ 木なのでパスは一意に定まります。



どうということ？

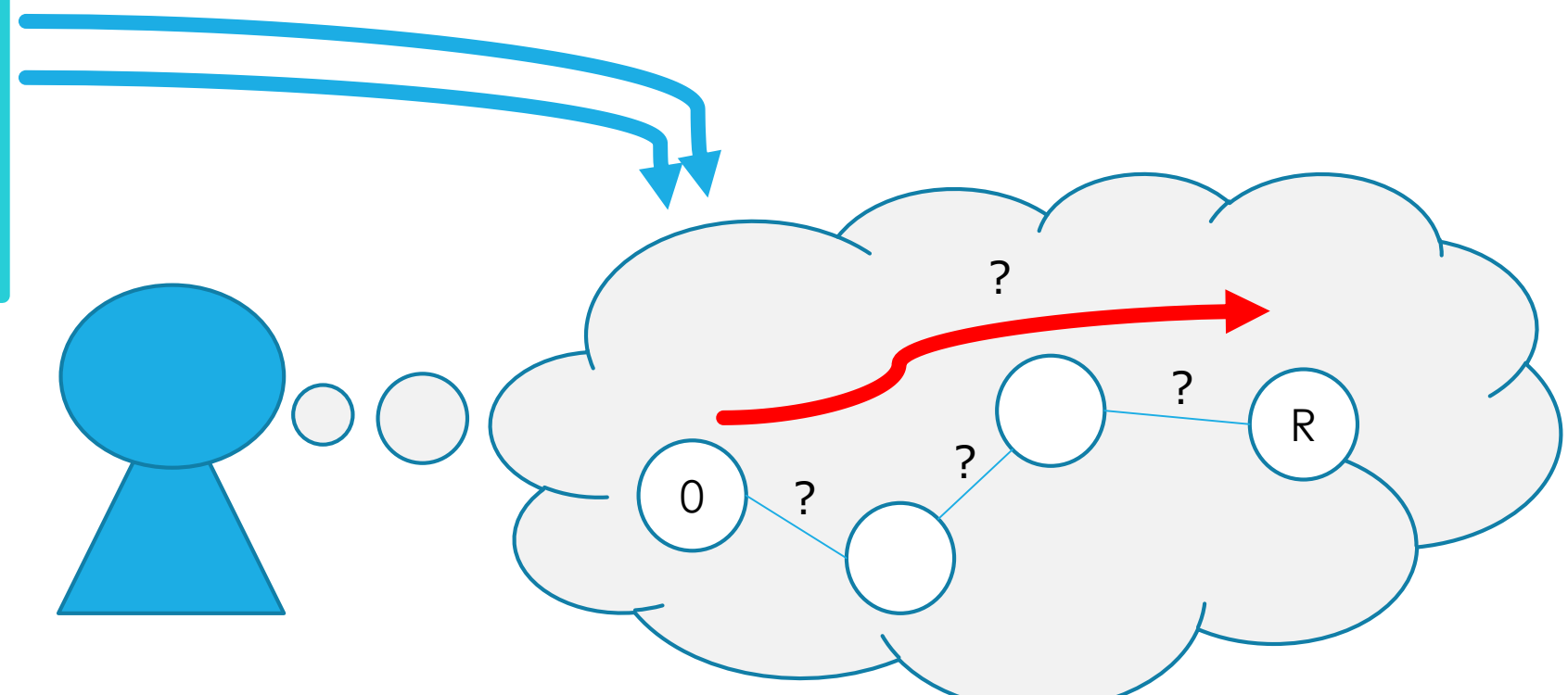
- ▶ Boris は、都市 0 からある都市 R まで到達するために経路する必要のある雪が降っている道路の最小値を答えます。
- ▶ 木なのでパスは一意に定まります。
- ▶ 困ったことに各辺のビットは知りません。



どうということ？



- ▶ そこで、答えを知っている Anya から、中継サーバを介して情報を受け取ります。



どうということ？

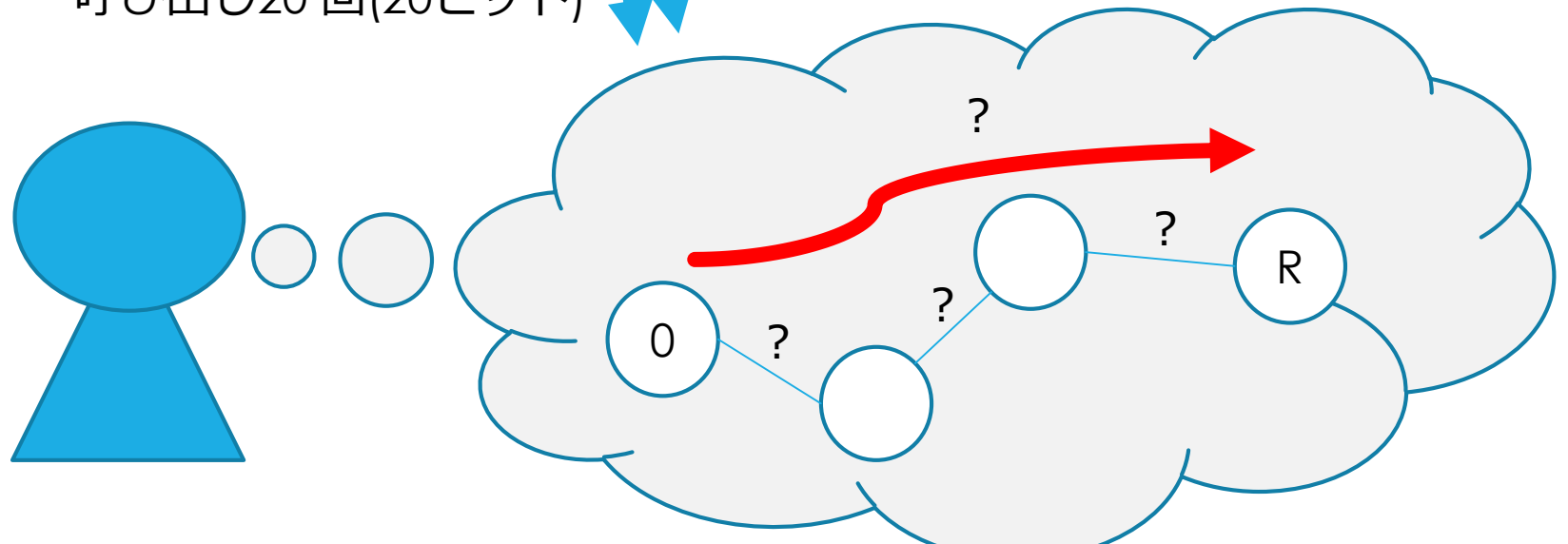


容量1,000 ビット

▶ そこで、答えを知っている Anya から、中継サーバを介して情報を受け取ります。

呼び出し20 回(20ビット)

▶ ただし、サーバや Ask に制限があります。



小課題 1 (15 点)

▶ $N \leq 20$

小課題 1 の特徴

- ▶ N がすごく小さい。
- ▶ TLE用？ でもパスをたどるのはあまり時間はかからないような...？
- ▶ 多方面から考察してみましょう。

小課題 1 以前に

- ▶ そもそも、サーバに直接答えを置いたらダメなのでしょうか？
- ▶ 降雪情報は全体でも 499 ビットなので、1,000 ビット制限に引っかかりません。
- ▶ Anya が降雪情報をそのまま保存し、Boris が普通にパスを辿って計算すれば良いのでは？

→小課題 1 のみ通ります。

どうして？

- ▶ パスが 20 より長いと、Ask 呼び出し制限に引っかかって不正解になります。
- ▶ 小課題 1 はパスが短いので、パスが 19 を超えることがなく、この手法でうまくいきます。

小課題 2 (5+15 点)

▶ $N \leq 100$

小課題 2 の特徴

- ▶ N がそこそこ。
- ▶ さすがに小課題 1 用の方針では厳しいです。

小課題 2 以前に

- ▶ そもそも、Boris が直接降雪情報を知る必要はあるのでしょうか？
- ▶ Boris が答える値は 0 以上 $N - 1$ 以下で、全体でも $500(< 2^9)$ 通りの答えしか取りません。
- ▶ すべて知っている Anya が予め答えを保存し、Boris が答えを読み込んでそのまま送れば良いのでは？

→小課題 1 と 2 が通ります。

どうして？

- ▶ サーバの容量制限に引っかかります。
- ▶ 各頂点につき 9 ビット消費するので、 $9N$ ビット全体でかかることになり、小課題 3 以降では 1,000 ビット制限に引っかかります。
- ▶ 小課題 2 なら、100 頂点以下なので、覚えるビット数は 900 ビットで済みます。

小課題 3 (35 点)

▶ 直線

2つの解法を比較してみる

- ▶ 先ほどの解法には、それぞれ長所短所があります。

- ▶ Anya サボる
- ▶ Boris 頑張る
- ▶ 記憶量 **少ない**
- ▶ アクセス **多すぎ**

解法 1

- ▶ Anya 頑張る
- ▶ Boris サボる
- ▶ 記憶量 **多すぎ**
- ▶ アクセス **少ない**

解法 2

2つの解法を比較してみる

- ▶ 先ほどの解法には、それぞれ長所短所があります。

- ▶ Anya サボる
- ▶ Boris 頑張る
- ▶ 記憶量 **少ない**
- ▶ アクセス **多すぎ**

解法 1

- ▶ Anya 頑張る
- ▶ Boris 頑張る
- ▶ 記憶量 **そこそこ**
- ▶ アクセス **そこそこ**

理想

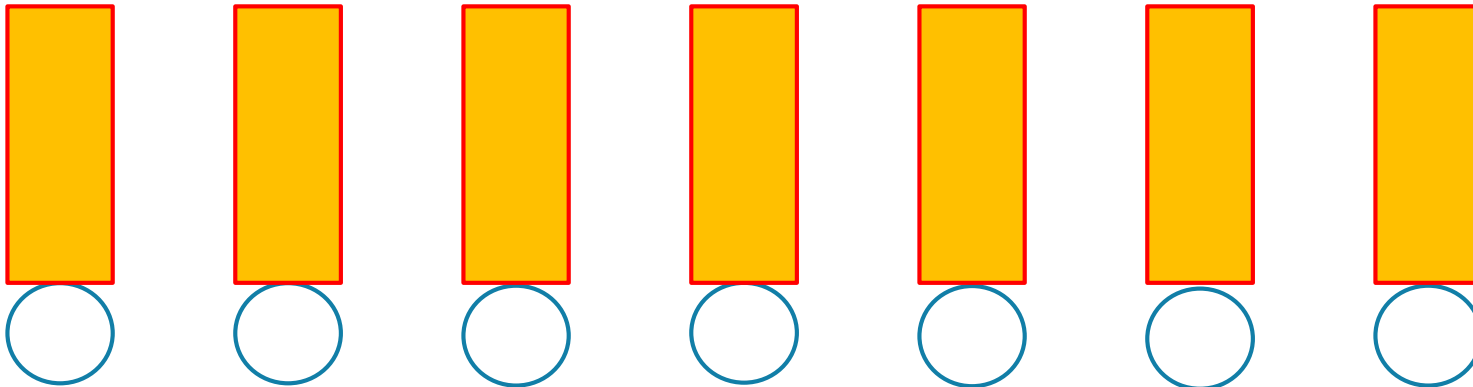
- ▶ Anya 頑張る
- ▶ Boris サボる
- ▶ 記憶量 **多すぎ**
- ▶ アクセス **少ない**

解法 2

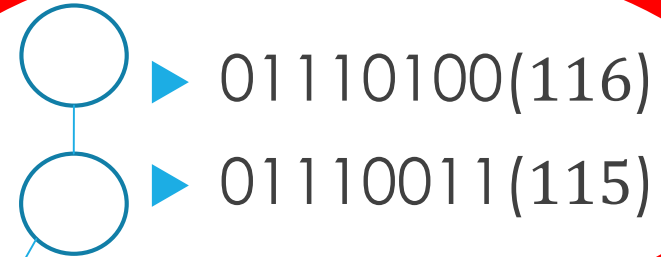
- ▶ 2つの解法の中間的な解法ならばうまくいきそう？

解法 2 の観察

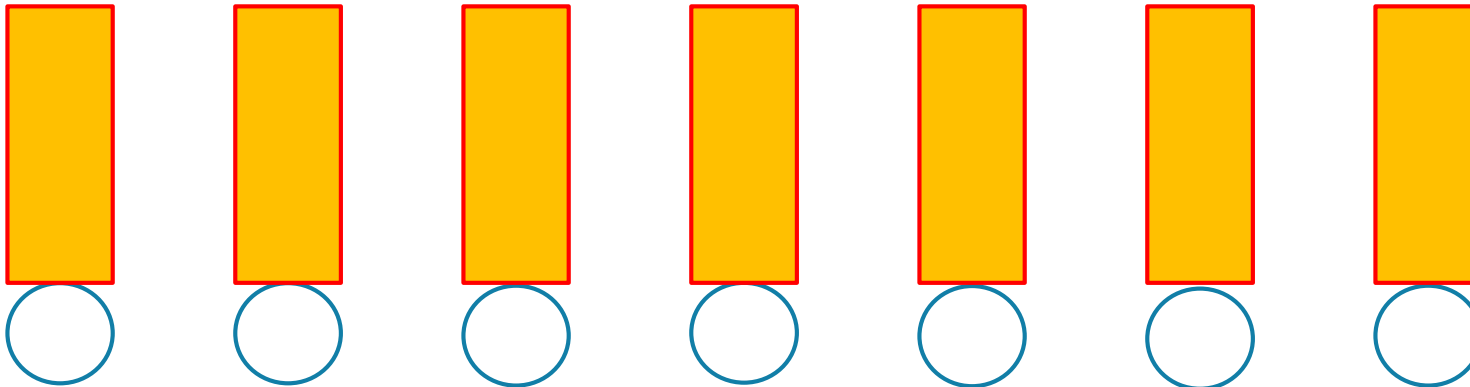
- ▶ 全頂点にビットを保存するのは、多くのメモリを消費します。



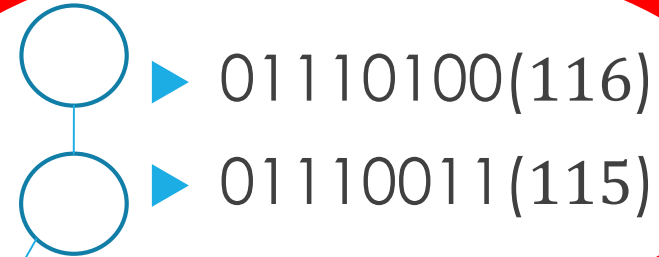
解法 2 の観察



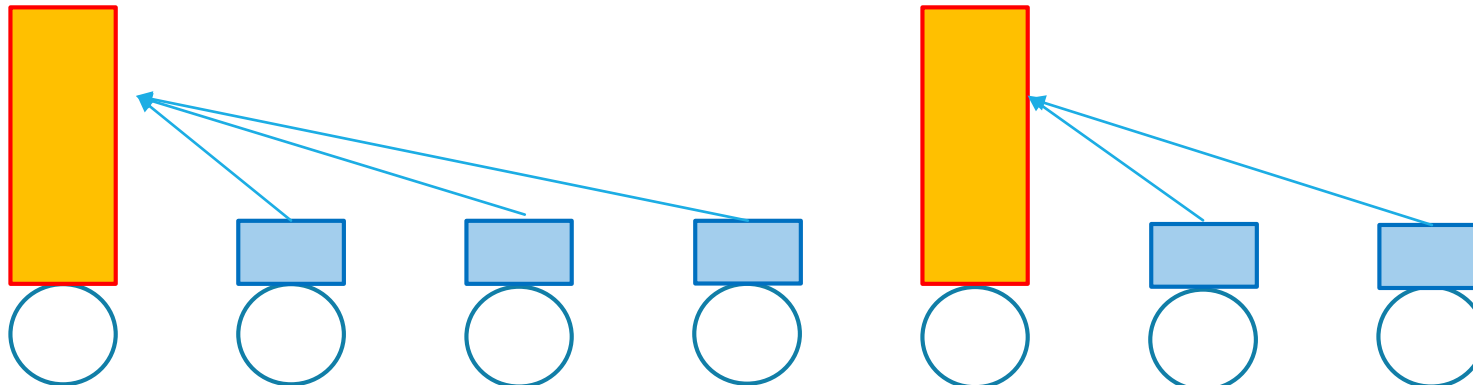
- ▶ 全頂点にビットを保存するのは、多くのメモリを消費します。
- ▶ そもそも、隣り合う頂点の解は高々 1 しか変わりません。



解法 2 の観察

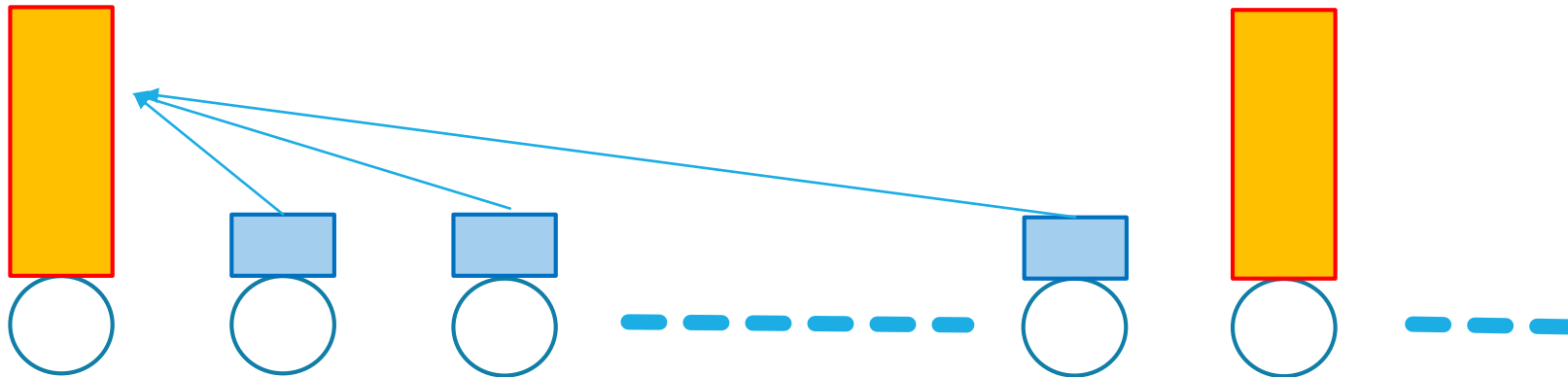


- ▶ 全頂点にビットを保存するのは、多くのメモリを消費します。
- ▶ そもそも、隣り合う頂点の解は高々 1 しか変わりません。
- ▶ そこで、近い位置関係にある頂点に、差分を保管することで節約することにします。



具体的な方針

- ▶ 例えば、12 個周期で覚える頂点を定めます。 ($(500/12) \times 9 \div 410$ ビット)
- ▶ 残りの頂点には、近くの覚える頂点との差分(実際には辺の情報を覚えておいて質問に対して実際にたどれば良いです。)を覚えます。 (約499ビット)
- ▶ 合計で 909 ビットとなり、小課題 3 を解くことができます。

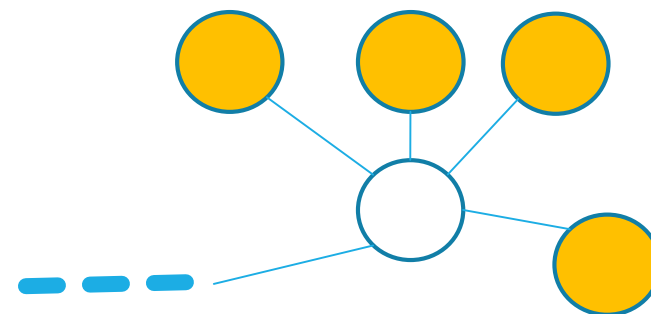


小課題 4 (15+5+35+45 点)

▶追加の制限なし

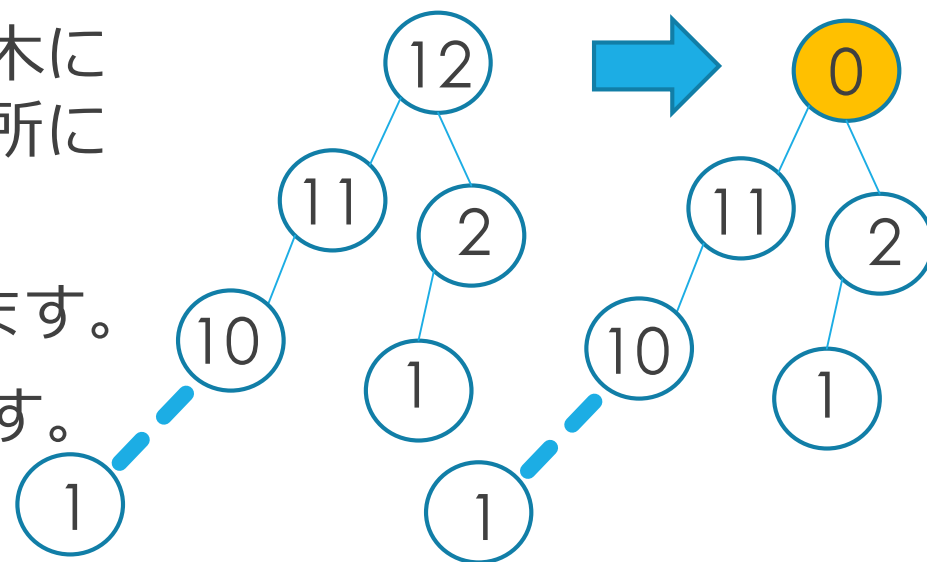
目的

- ▶ 先ほどの手法を木上で実装したい。
- ▶ ただ、適当に根からの距離で設定すると、同一距離が大量にあるケース (e.g. 星グラフ) で大量に保存してしまいます。
- ▶ うまく保存する場所を決めて保存する箇所を節約したいです。
- ▶ 以下、都市 0 を根ノードとして探索することになります。



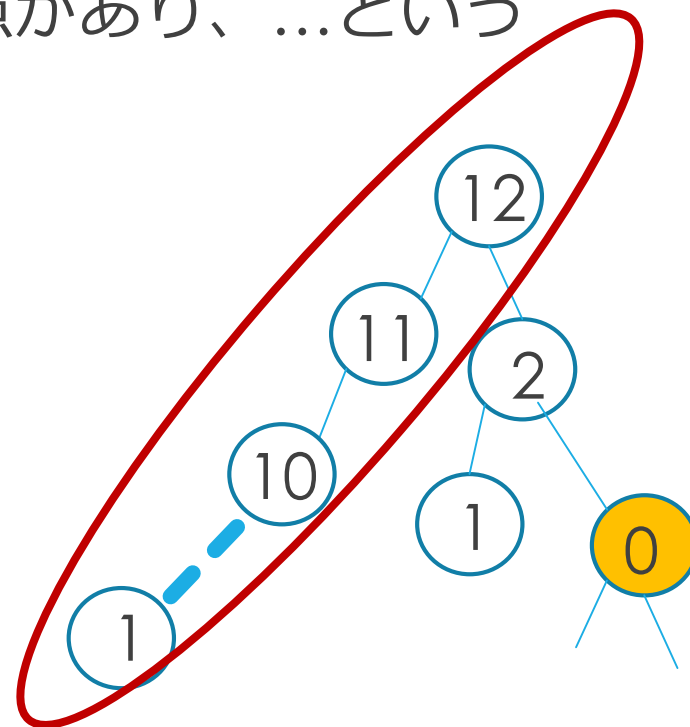
アルゴリズムの例

- ▶ 深さ優先探索の戻りがけのタイミングで、含めないと答えられなくなる頂点が子孫に登場するなら、その頂点の解を覚えるようにします。
- ▶ 例えば、各ノードに、ここより下の部分木にあるノードのうち、未解決で最も遠い場所にあるノードとの距離を計算します。
- ▶ その距離が 12 になれば覚える側に含めます。
- ▶ こうすることで満点を得ることができます。



証明

- ▶ 12 になって覚える必要のある頂点が登場したとき、その子供には 11 の頂点があり、さらにその子供には 10 の頂点があり、... というようにして、長さ 12 のパスが取れます。
- ▶ そのため、覚える頂点に数えるようにした際、必ず 11 個以上の頂点に関して覚えなくて済むことが保証できます。
- ▶ 唯一の例外は都市 0 ですが、都市 0 の値は常に 0 なので気にしなくて大丈夫です。
- ▶ $42 (\div 500/12)$ 個程の頂点記憶で足ります。



別解

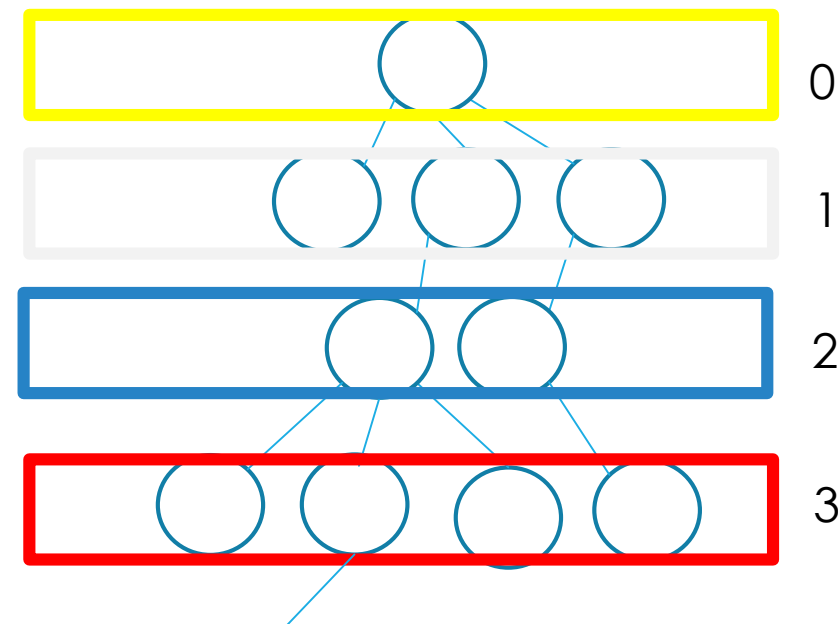
- ▶ 深さが mod 12 で一定の値となった場所を覚えることにします。
- ▶ もちろん、先ほどの星グラフに衝突した場合失敗します。

- ▶ ところで、深さの mod が k となる頂点の個数を $s(k)$ としたとき、

$$s(0)+s(1)+\dots+s(11)\leq 500$$

がいえます。

- ▶ よって、それらの min に合わせれば 42 個ほどの頂点記憶にできます。





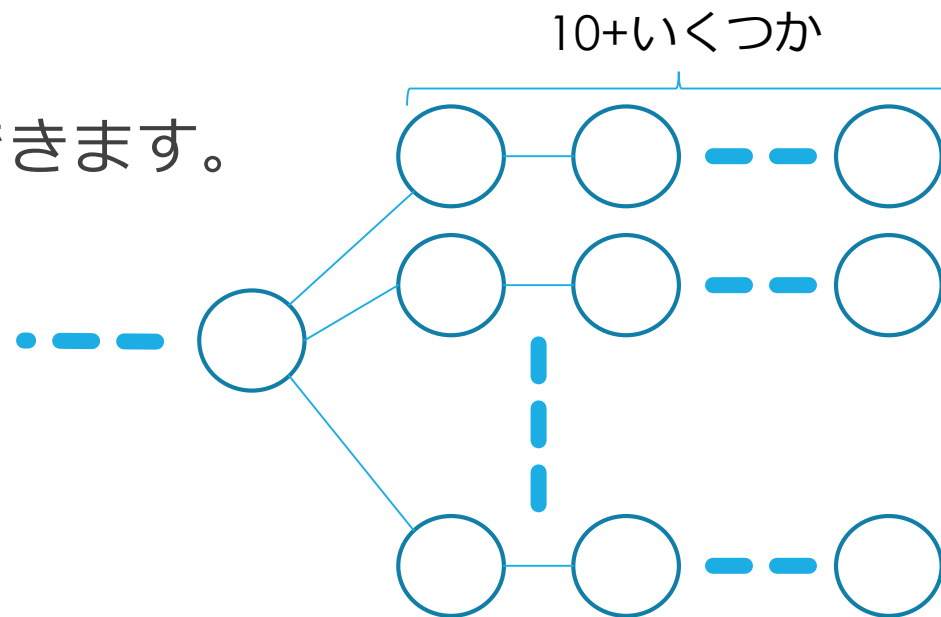
補足

どのように覚える頂点を決定するか

- ▶ うまく頂点を設定したいけれども、解析が大変/思いつかない...
- ▶ よく考えてみると、子孫側に移動することでも解決することができる！
- ▶ そう考えると、42 個くらいでのカバー自体はいっぱいありそう？

乱択アルゴリズム

- ▶ 覚える頂点集合を無作為に決定すればうまくいくこともあります。
- ▶ しかしながら、適当に決定すると下図のような部分を含んだ際に、失敗率が高くなります。
- ▶ 各種最適化で成功率を上げることができます。
- ▶ 重要なこととして、ここで紹介したアルゴリズムは**正しいことを保証していません**。
- ▶ この辺りを頑張れば、通るかもしれないという程度に思ってください。



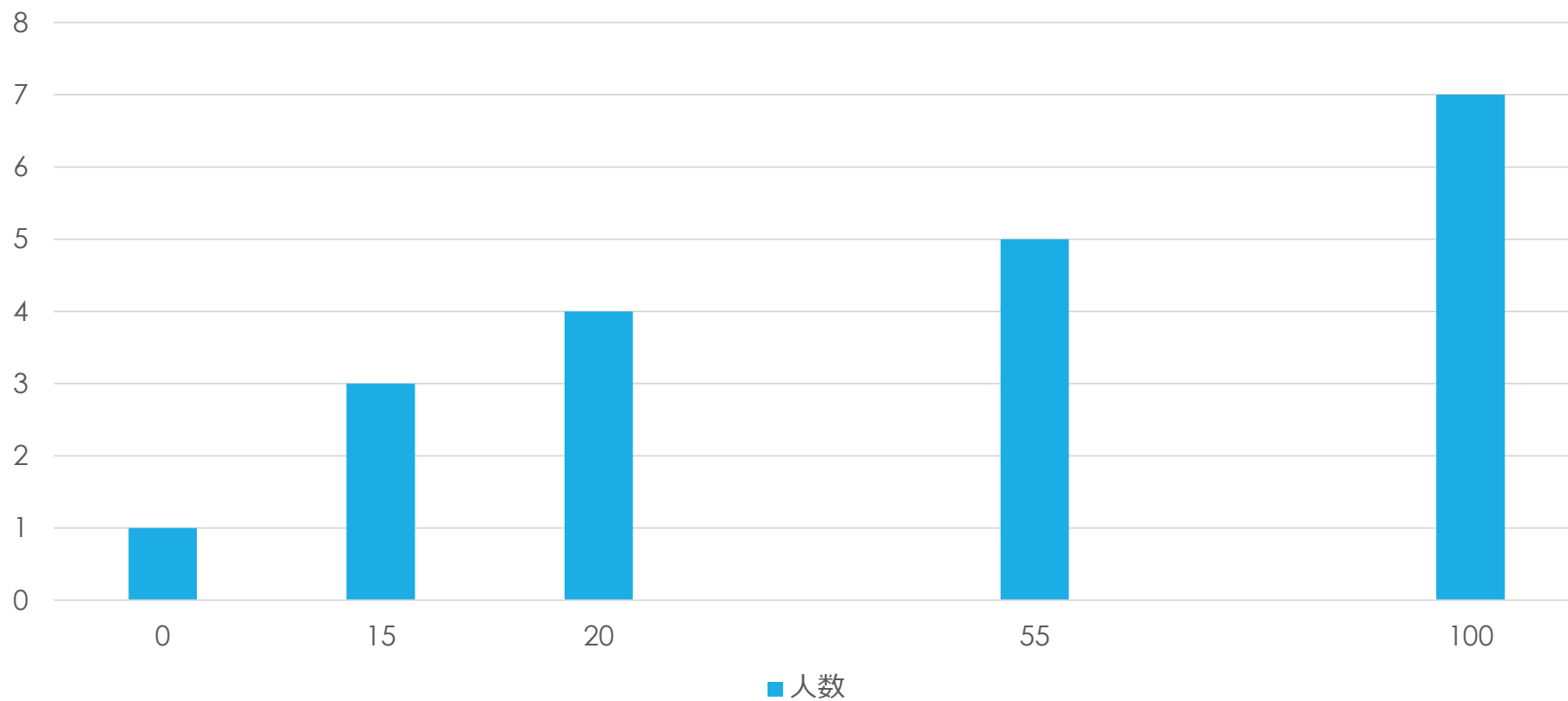
乱択アルゴリズム(各種最適化の一例)

- ▶ 覚えるビットは、0 以上(都市 0 からの距離) 以下なので、ある程度節約することができます。
- ▶ すでにいずれかの到達できる頂点を、次の抽選から除外すると、うまくいきやすくなります。
- ▶ 頂点集合を決めれば、今の頂点集合でいけるかは分かるので、もしあるシード値で失敗したときにうまくいくまで別のシード値で試し続ける解法も考えられます。

まとめ

- ▶ 長くても問題文をしっかりと読み、「何が本質的に問われているのか」を意識しましょう。
- ▶ 解法が分からなくても、乱択などを実装してみるとどうにかなることがあるかもしれません。
- ▶ もしも他に手を付けられそうになかったならば、試してみる価値はあるかもしれません。

得点分布



JOI春合宿Day4

最悪の記者2

Worst Reporter 2

問題概要

問題概要

- 競技開始 2h 後と 5h 後のN選手のランキングが与えられる

2時間	5時間
1	1
2	2
3	3
⋮	⋮
N	N

問題概要

- 競技開始 2h 後と 5h 後のN選手のランキングが与えられる
- ランキングといっても与えられるのは以下の情報のみ
 - i 位の選手の国籍と得点

2時間		5時間	
1	 600 _{pt}	1	 600 _{pt}
2	 334 _{pt}	2	 551 _{pt}
3	 100 _{pt}	3	 114 _{pt}
⋮		⋮	
N		N	

問題概要

- 競技開始 2h 後と 5h 後のN選手のランキングが与えられる
- ランキングといっても与えられるのは以下の情報のみ
 - i 位の選手の国籍と得点
- 同じ選手は時間経過とともに国籍が変化したり得点が減少することはない

問題概要

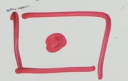
- どうやらランキングが矛盾しているので、国籍のみを書き換えて矛盾がないランキングにしたい

2時間			5日間		
1		600 _{pt}	1		600 _{pt}
2		<u>334</u> _{pt}	2		551 _{pt}
3		100 _{pt}	3		<u>114</u> _{pt}
⋮			⋮		
N			N		

A red arrow points from the 334 pt value in the 2-hour column to the 114 pt value in the 5-day column, with the text "NG" written in red next to the arrow.

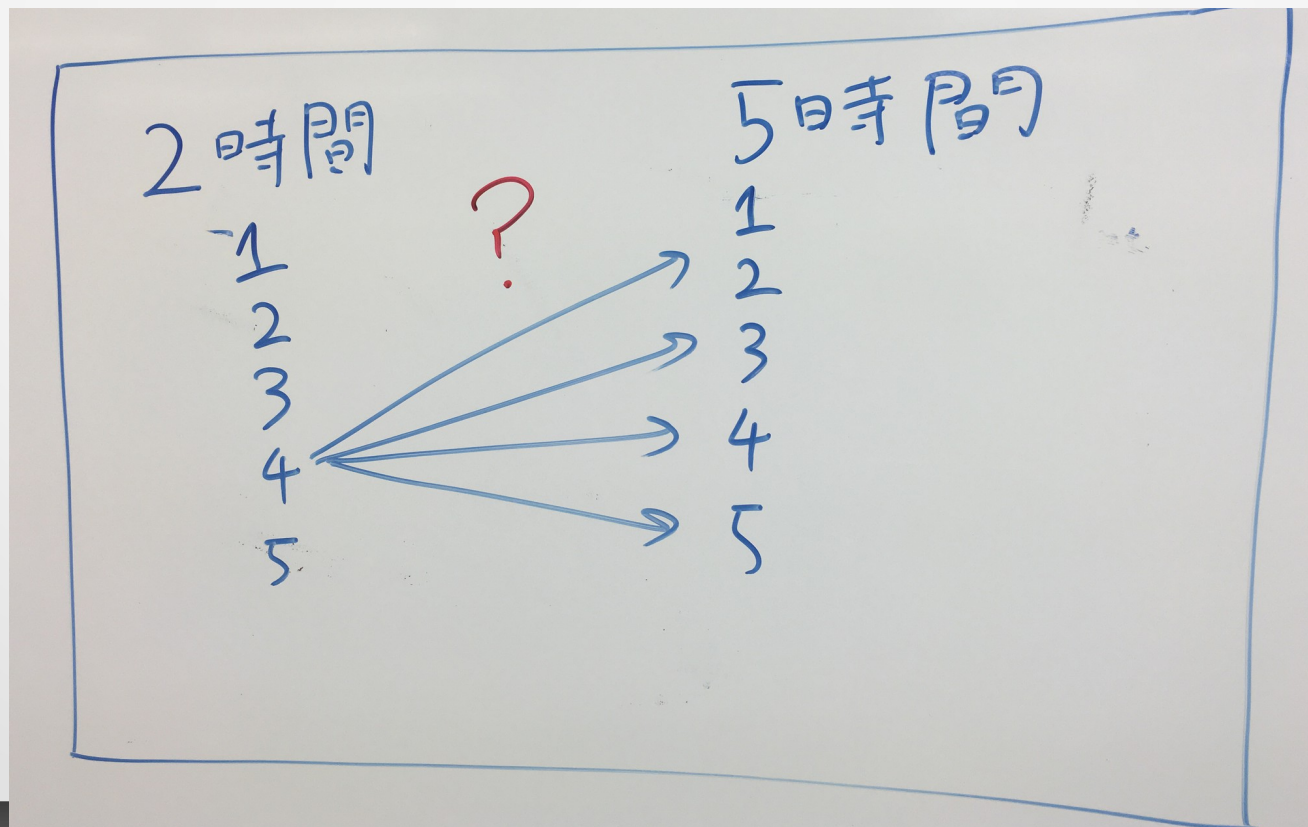
問題概要

- どうやらランキングが矛盾しているので、国籍のみを書き換えて矛盾がないランキングにしたい
- 書き換えの個数を最小化せよ

2時間		5時間	
1	 600 pt	1	 600 pt
2	 334 pt	2	 551 pt
3	 100 pt	3	 <u>114</u> pt
⋮		⋮	
N		N	

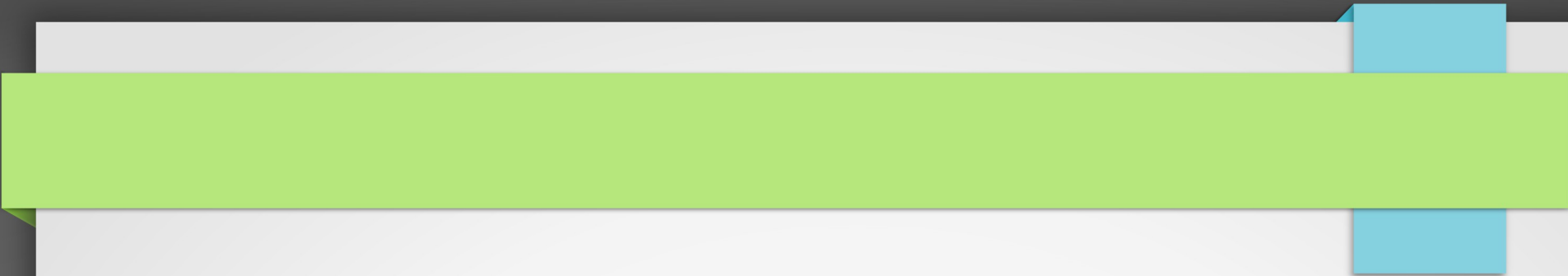
つまりどんな問題？

- 2h後のi位の選手が5h後の何位の選手に対応するかあてる問題



つまりどんな問題？

- 2h後の*i*位の選手が5h後の何位の選手に対応するかあてる問題
- できるだけ少ない書き換えで矛盾のない対応づけをめざす



小課題1

小課題1 ($N \leq 16$)

- $N \leq 16$

小課題1 ($N \leq 16$)

- $N \leq 16$
- 「何位の人は何位に対応するか」を全部ためせる？

小課題1 ($N \leq 16$)

- $N \leq 16$
- 「何位の人は何位に対応するか」を全部ためせる？
- $16! \doteq 2 * 10^{13}$

小課題1 ($N \leq 16$)

- $N \leq 16$
- 「何位の人が何位に対応するか」を全部ためせる？
- $16! \doteq 2 * 10^{13}$
- おわらない

小課題1 ($N \leq 16$)

- 16は階乗というよりは指数向けの数
 - $2^{16} = 65536$ は小さめでいい数字

小課題1 ($N \leq 16$)

- 16は階乗というよりは指数向けの数
 - $2^{16} = 65536$ は小さめでいい数字
- 階乗を指数にする一般的なテクニックがある

小課題1 ($N \leq 16$)

- 16は階乗というよりは指数向けの数
 - $2^{16} = 65536$ は小さめでいい数字
- 階乗を指数にする一般的なテクニックがある
- BitDP

小課題1 ($N \leq 16$)

- この問題におけるBit DPのやりかた

小課題1 ($N \leq 16$)

- この問題におけるBit DPのやりかた

- $dp[i][N$ 桁のbit列] =

—

—

—

2時間

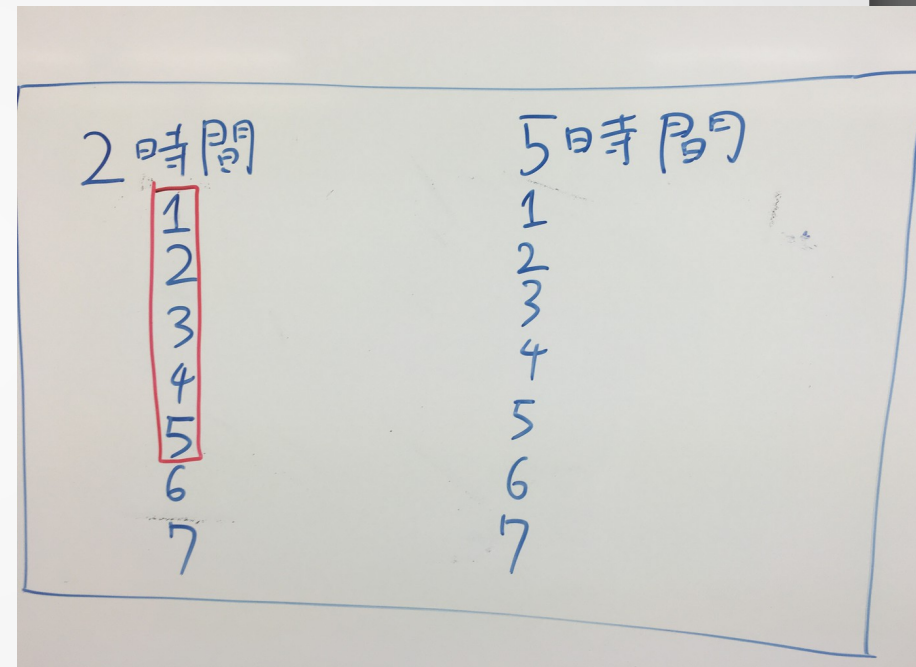
1
2
3
4
5
6
7

5時間

1
2
3
4
5
6
7

小課題1 ($N \leq 16$)

- この問題におけるBit DPのやりかた
- $dp[i][N\text{桁のbit列}] =$
 - 2h後の1位～i位までの選手と
 -
 -



小課題1 ($N \leq 16$)

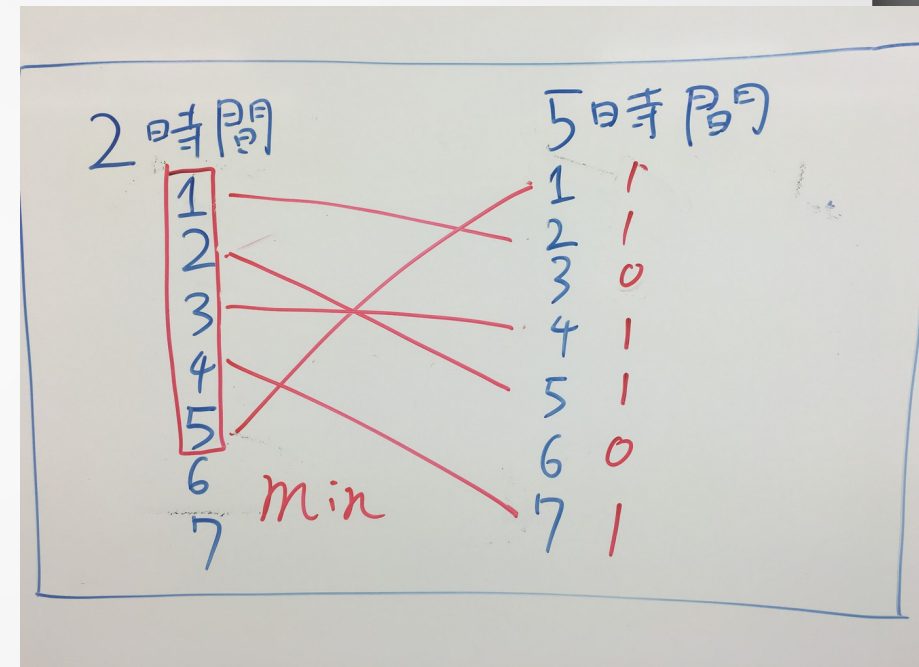
- この問題におけるBit DPのやりかた
- $dp[i][N\text{桁のbit列}] =$
 - 2h後の1位～i位までの選手と
 - 5h後の順位でbitが立っているところにいる選手で対応付したときの
 -

Handwritten table showing player rankings at 2h and 5h intervals. The table is enclosed in a blue border. The left column is labeled '2時間' and the right column is labeled '5時間'. The rows are numbered 1 through 7. The '2時間' column has a red box around the numbers 1 through 5. The '5時間' column has red marks next to the numbers 1 through 7.

2時間	5時間
1	1
2	1
3	0
4	1
5	1
6	0
7	1

小課題1 ($N \leq 16$)

- この問題におけるBit DPのやりかた
- $dp[i][N\text{桁のbit列}] =$
 - 2h後の1位～i位までの選手と
 - 5h後の順位でbitが立っているところにいる選手で対応付したときの
 - 必要な書き換え個数の最小値



小課題1 ($N \leq 16$)

- この問題におけるBit DPの計算量

小課題1 ($N \leq 16$)

- この問題におけるBit DPの計算量
- メモリは $O(N * 2^N)$

小課題1 ($N \leq 16$)

- この問題におけるBit DPの計算量
- メモリは $O(N \cdot 2^N)$
- dp配列の各要素につき $O(N)$ のループをまわす

小課題1 ($N \leq 16$)

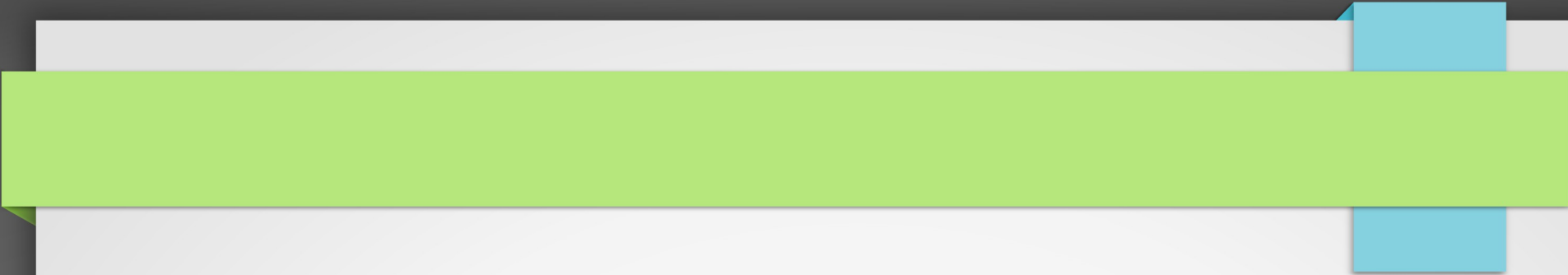
- この問題におけるBit DPの計算量
- メモリは $O(N * 2^N)$
- dp配列の各要素につき $O(N)$ のループをまわす
- $O(2^N * N^2)$

小課題1 ($N \leq 16$)

- この問題におけるBit DPの計算量
- メモリは $O(N * 2^N)$
- dp配列の各要素につき $O(N)$ のループをまわす
- $O(2^N * N^2)$
 - 15点獲得！

小課題1 ($N \leq 16$)

- この問題におけるBit DPの計算量
- メモリは $O(N * 2^N)$
- dp配列の各要素につき $O(N)$ のループをまわす
- $O(2^N * N^2)$
 - 15点獲得！
- ちなみに $O(N * N^2)$ でもできます



小課題2

小課題2 ($N \leq 50$)

- $N \leq 50$

小課題2 ($N \leq 50$)

- $N \leq 50$
 - $O(N^3)$ とか $O(N^4)$ くらいのアλゴリズムっぽい

小課題2 ($N \leq 50$)

- $N \leq 50$
 - $O(N^3)$ とか $O(N^4)$ くらいのアλゴリズムっぽい
- うーん DPかな？

小課題2 ($N \leq 50$)

- $N \leq 50$
 - $O(N^3)$ とか $O(N^4)$ くらいのアλゴリズムっぽい
- うーん DPかな？ 実はlogとかsqrtとかが入るのかな？

小課題2 ($N \leq 50$)

- $N \leq 50$
 - $O(N^3)$ とか $O(N^4)$ くらいのアアルゴリズムっぽい
- うーん DPかな？ 実はlogとかsqrtとかが入るのかな？
- うーん わからない

小課題2 ($N \leq 50$)

- この問題はどんな問題？

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題
- 2つの → Bipartite

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題
- 2つの → Bipartite
- できるだけ少ない → Minimum

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題
- 2つの → Bipartite
- できるだけ少ない → Minimum
- 全て → Perfect

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題
- 2つの → Bipartite
- できるだけ少ない → Minimum
- 全て → Perfect
- 対応付け → Matching
- おや これは...

小課題2 ($N \leq 50$)

- この問題はどんな問題？
- 2hの順位と5hの順位という「2つの」順位を「できるだけ少ない」書き換えで矛盾なく「全て」「対応付けさせる」問題
- 2つの → Bipartite
- できるだけ少ない → Minimum
- 全て → Perfect
- 対応付け → Matching

小課題2 ($N \leq 50$)

Min Cost Perfect Matching
in
Bipartite Graph

小課題2 ($N \leq 50$)

最小コスト完全2部マッチング

小課題2 ($N \leq 50$)

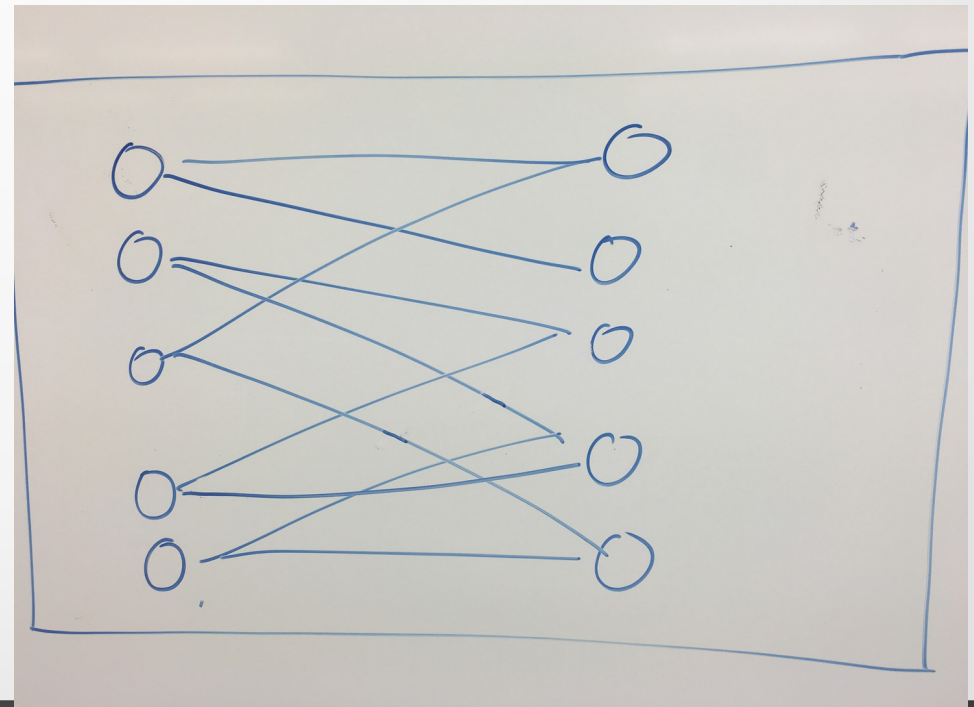
フロー
最小コスト完全2部マッチング

小課題2 ($N \leq 50$)

- 最小コスト完全二部マッチング問題とは？

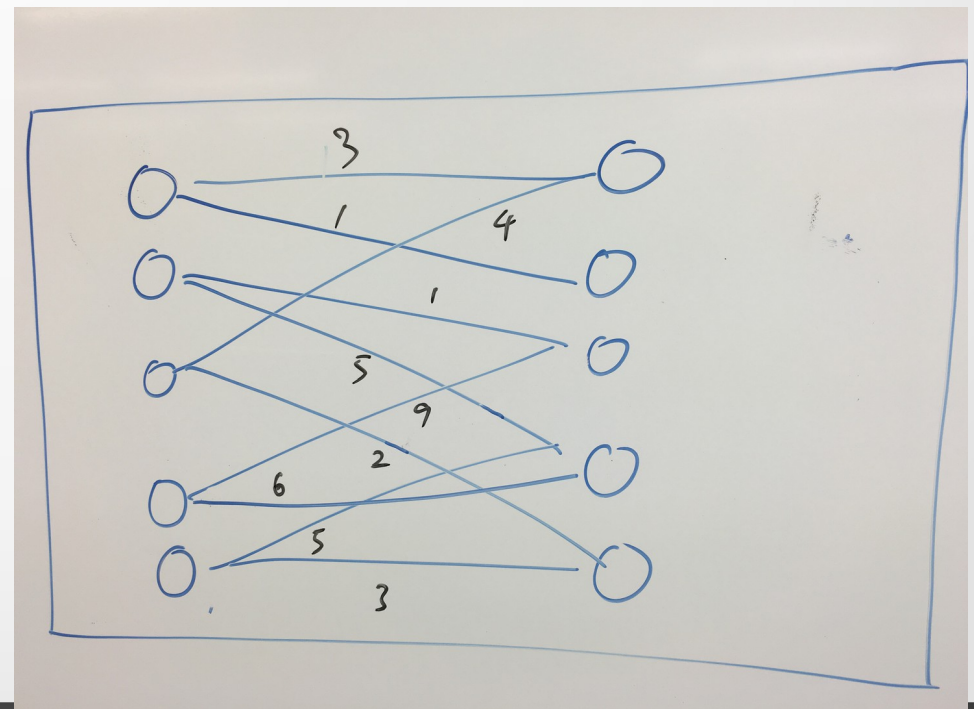
小課題2 ($N \leq 50$)

- 最小コスト完全二部マッチング問題とは？
 - 二部グラフがあります



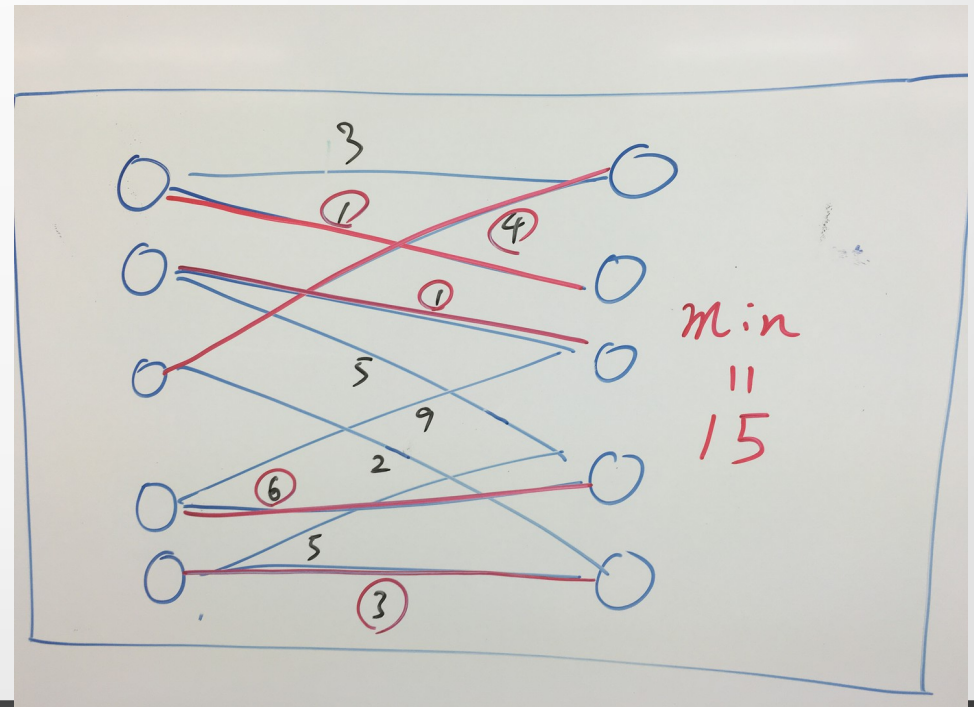
小課題2 ($N \leq 50$)

- 最小コスト完全二部マッチング問題とは？
 - 二部グラフがあります
 - 辺にはコストがあります



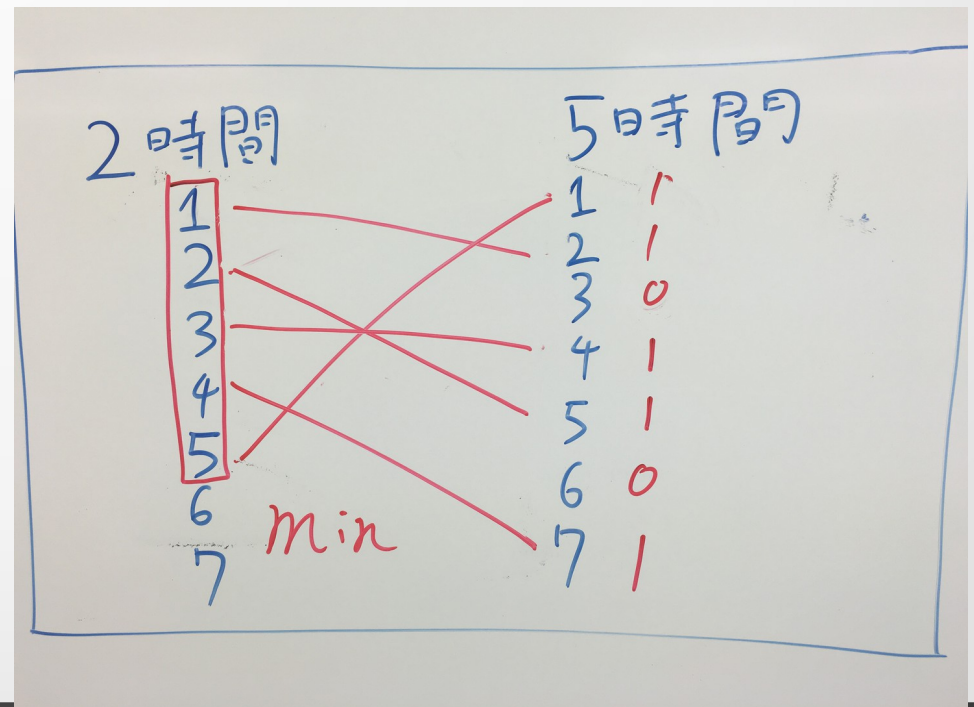
小課題2 ($N \leq 50$)

- 最小コスト完全二部マッチング問題とは？
 - 二部グラフがあります
 - 辺にはコストがあります
 - 完全マッチング(全てを一対一対応させる辺の組の選び方)のうち最小コストのものを求めてください



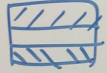
小課題2 ($N \leq 50$)

- 最小コスト完全二部マッチング問題とは？
 - **二部**グラフがあります
 - 辺には**コスト**があります
 - **完全マッチング**(全てを一対一対応させる辺の組の選び方)のうち**最小コスト**のものを求めてください
- 今回の問題にすごく似てる



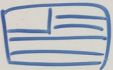
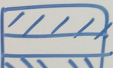
小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか

2時間	5時間
1  300	1  300
2  200	2  200
3  100	3  100

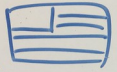
小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか
 - 2h後の*i*位の選手と5h後の*j*位の選手を対応付けても良い時

2時間		5時間	
1	 300	1	 300
<i>i</i> 2	 200	2	 200 <i>j</i>
3	 100	3	 100

小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか
 - 2h後の*i*位の選手と5h後の*j*位の選手を対応付けても良い時
 - 国籍が違うならコスト1の辺を張る

2時間		5時間	
1	 300	1	 300
<i>i</i> 2	 200	2	 200 <i>j</i>
3	 100	3	 100

小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか
 - 2h後の*i*位の選手と5h後の*j*位の選手を対応付けても良い時
 - 国籍が違うならコスト1の辺を張る
 - 国籍が同じならコスト0の辺を張る

小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか
 - 2h後の*i*位の選手と5h後の*j*位の選手を対応付けても良い時
 - 国籍が違うならコスト1の辺を張る
 - 国籍が同じならコスト0の辺を張る
 - 対応付けられない時

小課題2 ($N \leq 50$)

- 今回の問題をどう帰着させるか
 - 2h後の*i*位の選手と5h後の*j*位の選手を対応付けても良い時
 - 国籍が違うならコスト1の辺を張る
 - 国籍が同じならコスト0の辺を張る
 - 対応付けられない時
 - 辺を張らない OR コスト無限の辺を張る

小課題2 ($N \leq 50$)

- できた2部グラフの完全マッチングは、元の問題における対応付と互いに変換可能
 - しかもマッチングのコストは、元の問題における対応付けに必要な書き換えの量と一致

小課題2 ($N \leq 50$)

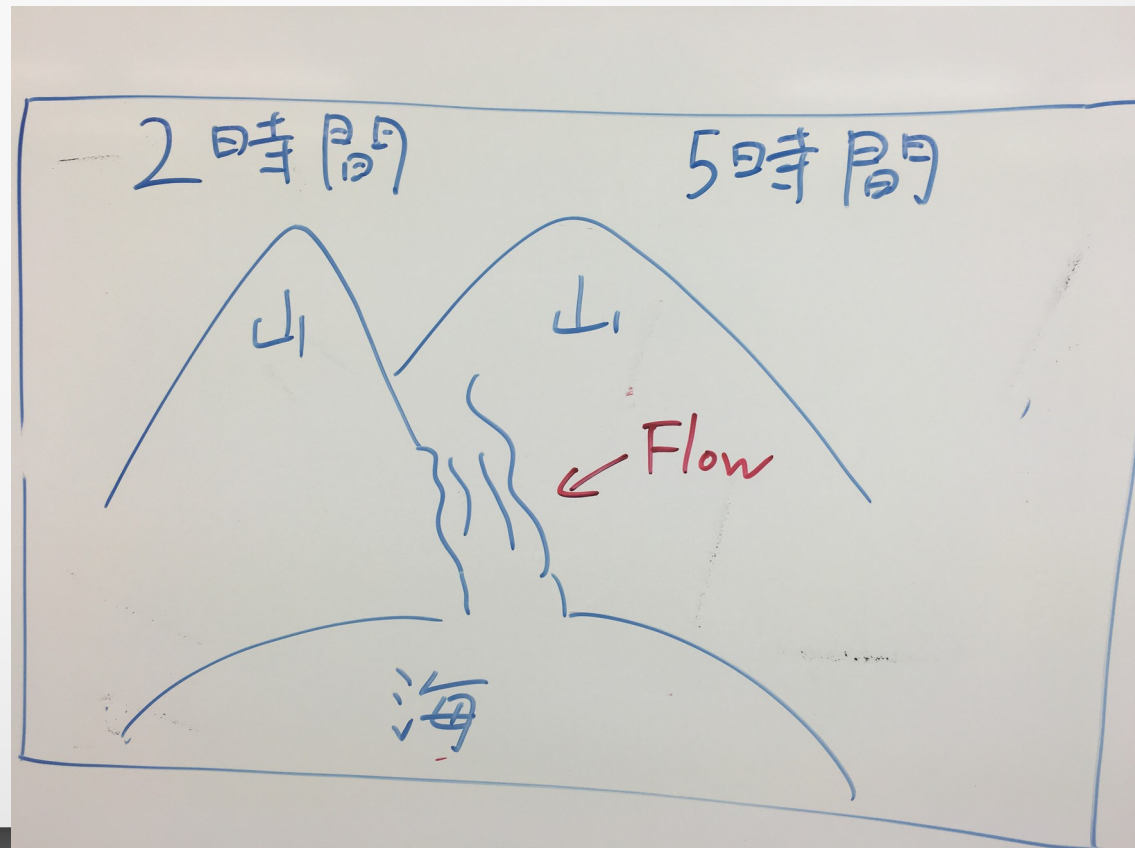
- できた2部グラフの完全マッチングは、元の問題における対応付と互いに変換可能
 - しかもマッチングのコストは、元の問題における対応付けに必要な書き換えの量と一致
- 帰着に成功したみたいなのであとは最小コスト完全マッチング問題を解くだけ！

小課題2 ($N \leq 50$)

- Q.最小コスト完全マッチングの求め方？

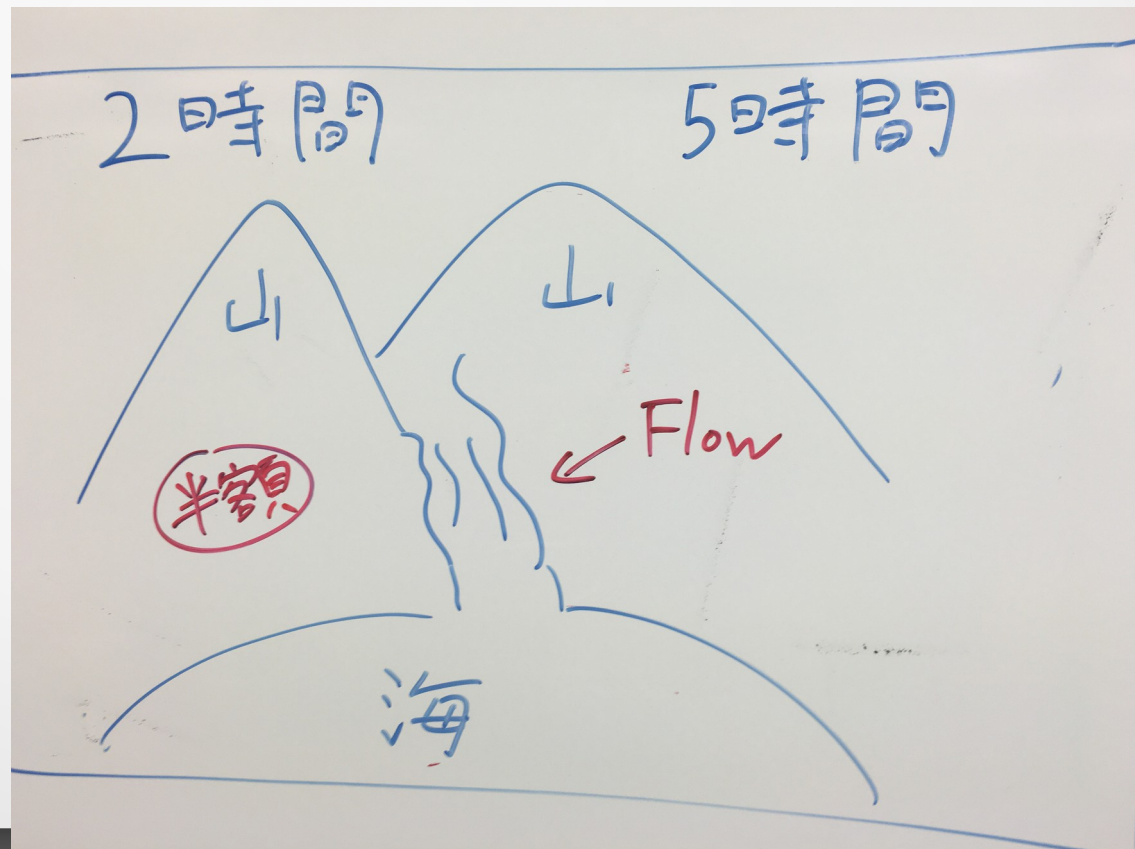
小課題2 ($N \leq 50$)

- Q.最小コスト完全マッチングの求め方？
- A.フローです。



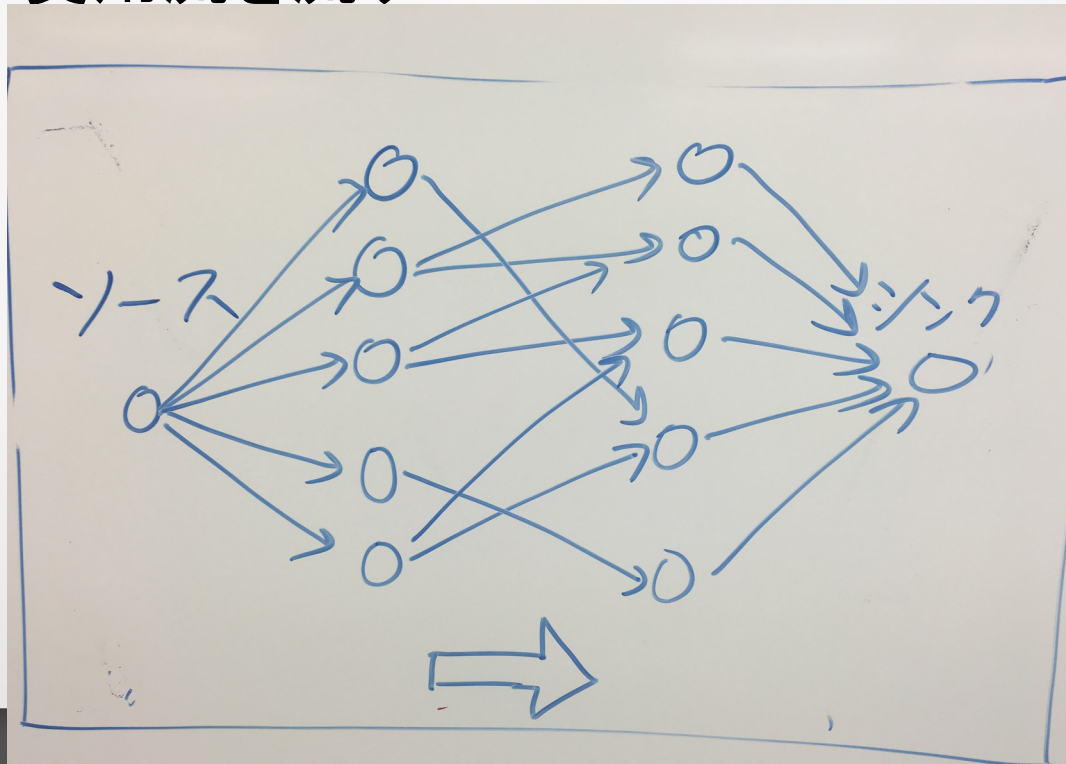
小課題2 ($N \leq 50$)

- Q.最小コスト完全マッチングの求め方？
- A.Min Cost フローです。



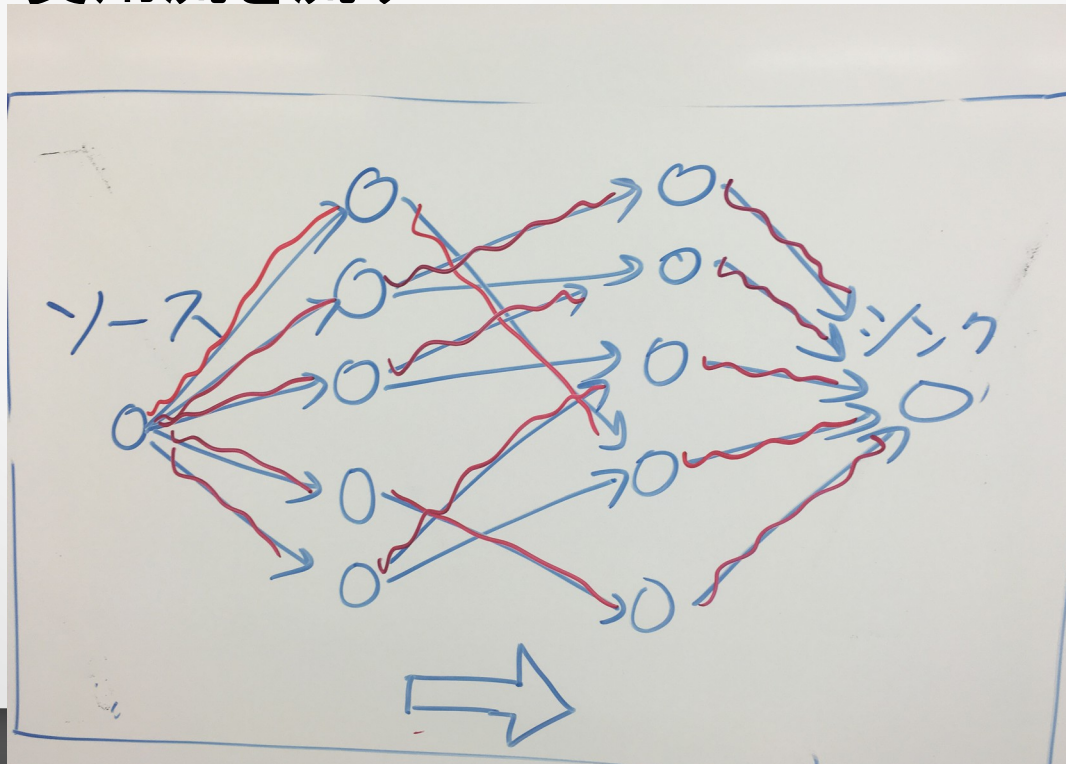
小課題2 ($N \leq 50$)

- シンクとソースを一個ずつつくって、2部グラフの各パートと辺を結ぶ
 - コストは0 流量制限は1
- シンク側からソース側に向かって、各辺に向きを付ける
- 流量Nの最小費用流を流す



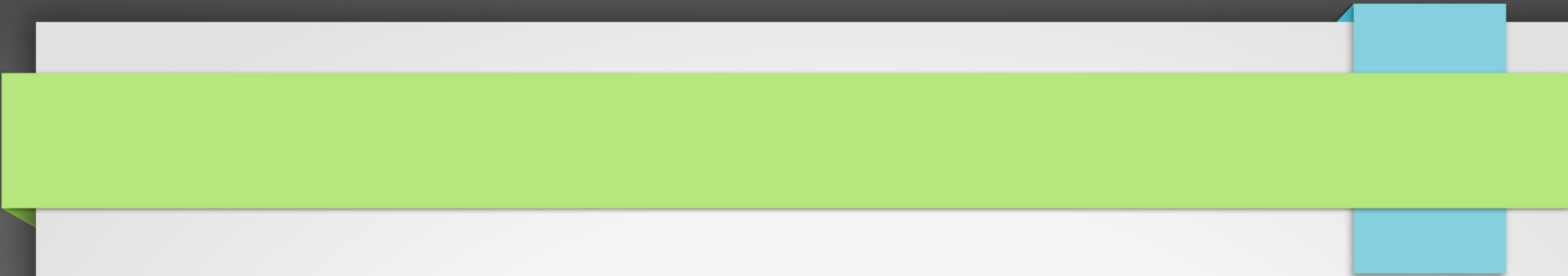
小課題2 ($N \leq 50$)

- シンクとソースを一個ずつつくって、2部グラフの各パートと辺を結ぶ
 - コストは0 流量制限は1
- シンク側からソース側に向かって、各辺に向きを付ける
- 流量Nの最小費用流を流す



小課題2 ($N \leq 50$)

- シンクとソースを一個ずつつくって、2部グラフの各パートと辺を結ぶ
 - コストは0 流量制限は1
- シンク側からソース側に向かって、各辺に向きを付ける
- 流量 N の最小費用流を流す
 - 詳しいアルゴリズムは2015年のチューター講義を参照
- もちろん最小費用流パートがボトルネック
 - 計算量は $O(N^3)$ 累計30点ゲット！



小課題3

小課題3 ($N \leq 5000$)

- $N \leq 5000$

小課題3 ($N \leq 5000$)

- $N \leq 5000$
 - $O(N^2)$ かな？

小課題3 ($N \leq 5000$)

- $N \leq 5000$
 - $O(N^2)$ かな？
 - フローからは離れたほうが良さそう

小課題3 ($N \leq 5000$) 考察

- 書き換えるのは片側のランキングだけで良い

小課題3 ($N \leq 5000$) 考察

- 書き換えるのは片側のランキングだけで良い
 - 対応付けした時のことを考えて、片側をもう片側に合わせるように書き換えを行えば良い

小課題3 ($N \leq 5000$) 考察

- 書き換えるのは片側のランキングだけで良い
 - 対応付けした時のことを考えて、片側をもう片側に合わせるように書き換えを行えば良い
 - 今回は5h後のランキングのみ書き換えることにする

小課題3 ($N \leq 5000$) 考察

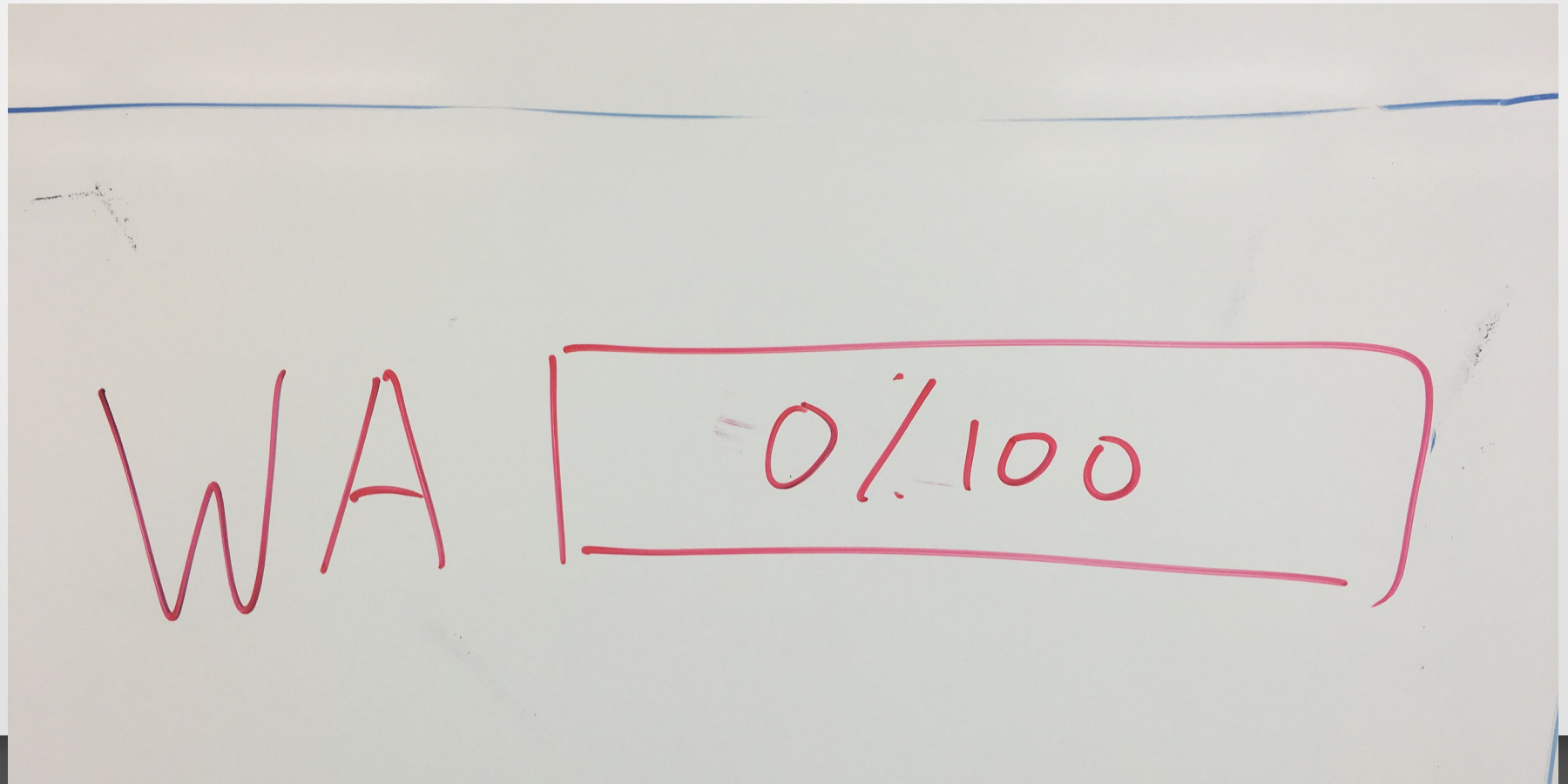
- 書き換えるのは片側のランキングだけで良い
 - 対応付けした時のことを考えて、片側をもう片側に合わせるように書き換えを行えば良い
 - 今回は5h後のランキングのみ書き換えることにする
- 5h後のランキングの中で書き換えない選手の数最大化すれば良いことになる

小課題3 ($N \leq 5000$) 考察

- 書き換えずにすむ対応付けをただひたすら多くすればよい？

小課題3 ($N \leq 5000$) 考察

- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - →Wrong Answer



小課題3 ($N \leq 5000$) 考察

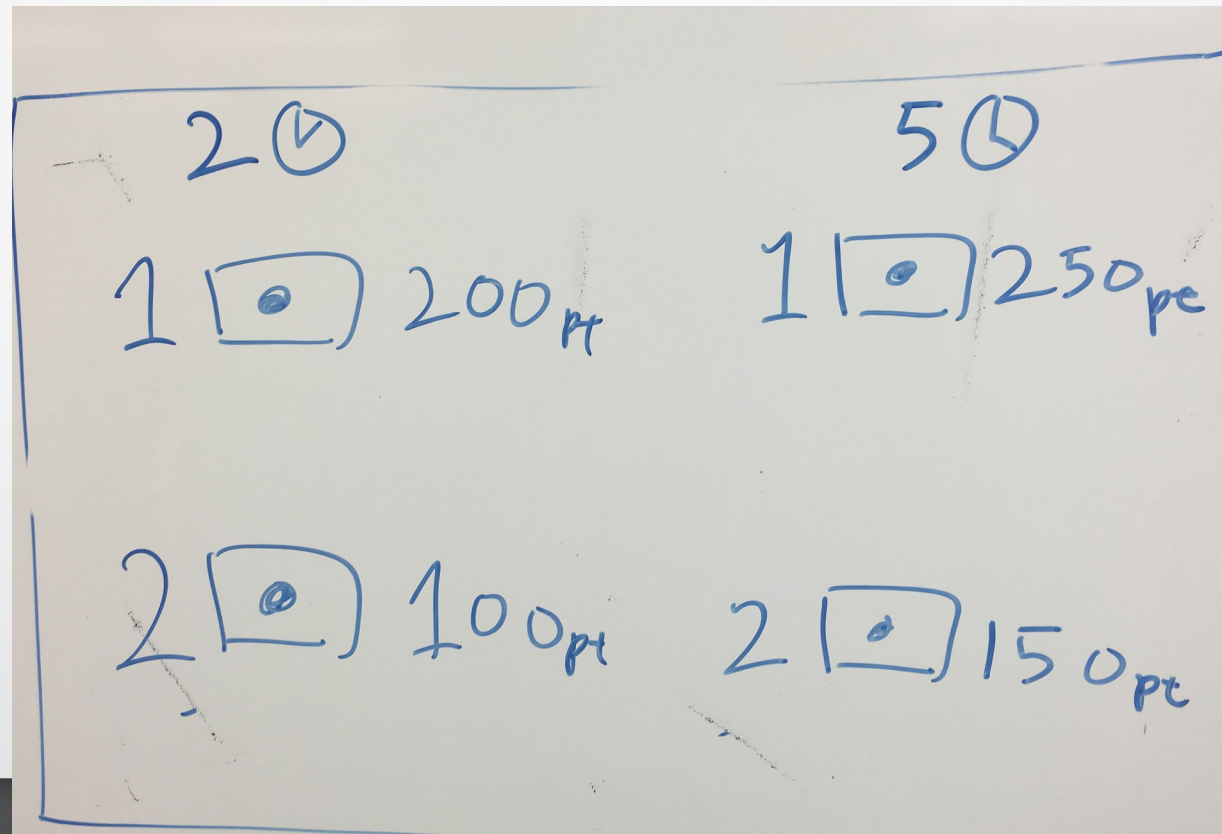
- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - →Wrong Answer
 - やたらめったら対応付けると完全マッチングが作れなくなることがある

小課題3 ($N \leq 5000$) 考察

- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - $\rightarrow$ Wrong Answer
 - やたらめったら対応付けると完全マッチングが作れなくなることがある
- 例

小課題3 ($N \leq 5000$) 考察

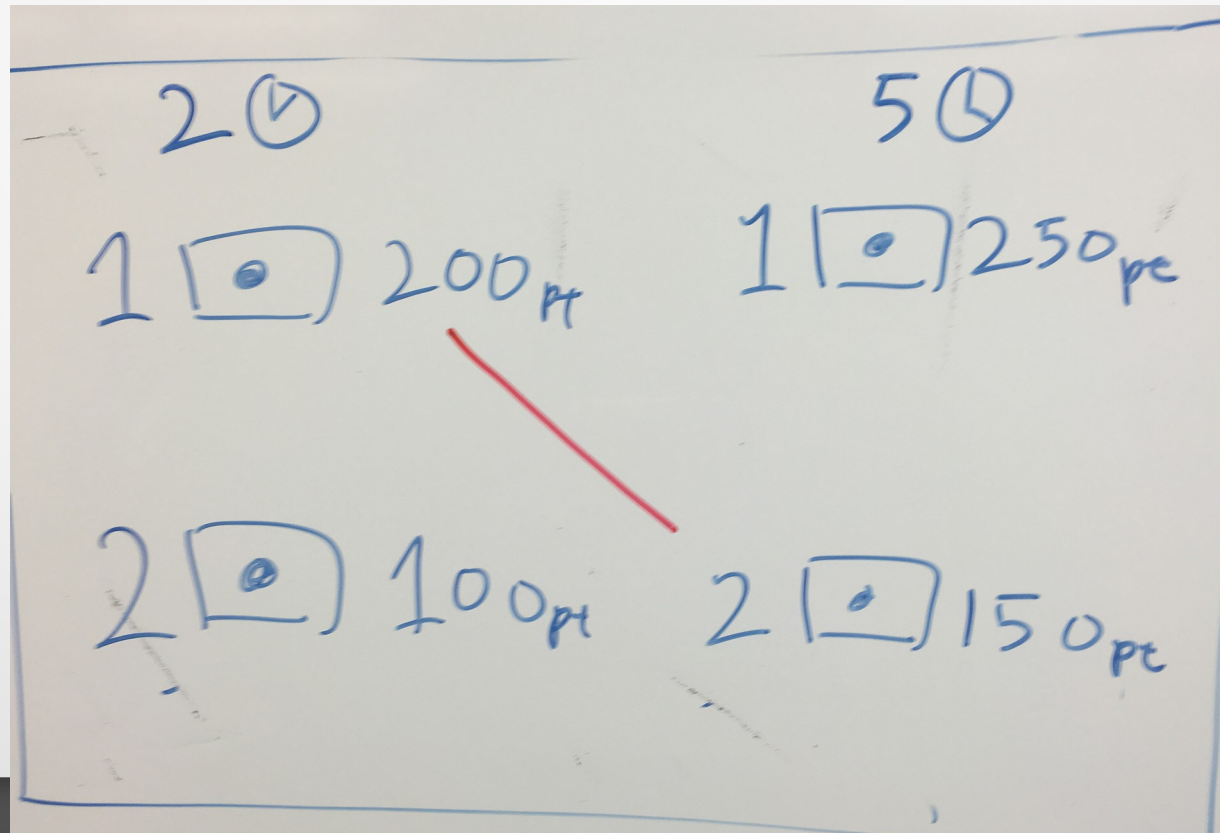
- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - $\rightarrow$ Wrong Answer
 - やたらめったら対応付けると完全マッチングが作れなくなることがある
- 例



小課題3 ($N \leq 5000$) 考察

- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - $\rightarrow$ Wrong Answer
 - やたらめったら対応付けると完全マッチングが作れなくなることがある

• 例



小課題3 ($N \leq 5000$) 考察

- 書き換えずにすむ対応付けをただひたすら多くすればよい？
 - →Wrong Answer
 - やたらめったら対応付けると完全マッチングが作れなくなることがある
- 例

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができなくなる条件とは？

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができなくなる条件とは？
- あきらかに駄目なパターン
 - 両ランキングのX点以下をとりだしたとき、2h後のランキングの選手の方が少ない時

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができなくなる条件とは？
- あきらかに駄目なパターン
 - 両ランキングのX点以下をとりだしたとき、2h後のランキングの選手の方が少ない時
- 例

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができなくなる条件とは？
- あきらかに駄目なパターン
 - 両ランキングのX点以下をとりだしたとき、2h後のランキングの選手の方が少ない時
- 実は全てのXでこれが成り立っていなければ完全マッチングを作ることができる。
 - 2h後から5h後にかけて誰も順位が変わらないように対応付ければ良い

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができるように気をつけながら、書き換えずすむ対応付けをひたすら多く結んでいけば良い？

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができるように気をつけながら、書き換えずすむ対応付けをひたすら多く結んでいけば良い？
 - WrongAnswer

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができるように気をつけながら、書き換えずすむ対応付けをひたすら多く結んでいけば良い？
 - WrongAnswer
- 反例

小課題3 ($N \leq 5000$) 考察

- 完全マッチングができるように気をつけながら、書き換えずすむ対応付けをひたすら多く結んでいけば良い？
 - WrongAnswer
- 反例

小課題3 ($N \leq 5000$) 考察

- 順番を気にすればOKかもしれない

小課題3 ($N \leq 5000$) 考察

- 順番を気にすればOKかもしれない
- 順番として考えられるものが得点ぐらいしか思いつかない

小課題3 ($N \leq 5000$) 考察

- 順番を気にすればOKかもしれない
- 順番として考えられるものが得点ぐらいしか思いつかない

ここでいきなり解法を思いつきます

小課題3 ($N \leq 5000$) 解法

- 5h後のランキングについて点数が低い選手から
 - 書き換えずに済むなら書き換えない
 - このとき、できるだけ点が高い相手と対応付ける
 - 書き換えなければならないなら書き換える
 - 何に書き換えるかは考えなくて良い
- というふうに貪欲に決めていくと答えが求まる。

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) 図

- 実際に以下の例でやってみる

小課題3 ($N \leq 5000$) アルゴリズム

- もう一度アルゴリズムをまとめると
- 点数が低い選手から順番に
 - 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける
 - 書き換えなければならないなら放置

小課題3 ($N \leq 5000$) アルゴリズム

- もう一度アルゴリズムをまとめると
- 点数が低い選手から順番に
 - 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける
 - 書き換えなければならないなら放置

小課題3 ($N \leq 5000$) アルゴリズム

- もう一度アルゴリズムをまとめると
- 点数が低い選手から順番に
 - 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける
 - 書き換えなければならないなら放置
- 貪欲法だが、この赤字の部分がすこし怪しい(非自明)

小課題3 ($N \leq 5000$) 貪欲法のミソ

- 貪欲法は証明が難しいことが多い

小課題3 ($N \leq 5000$) 貪欲法のミソ

- 貪欲法は証明が難しいことが多い
- 貪欲法を証明するときによく使えるキーワードがある

小課題3 ($N \leq 5000$) 貪欲法のミソ

- 貪欲法は証明が難しいことが多い
- 貪欲法を証明するときによく使えるキーワードがある

「交換しても悪化しない」

小課題3 ($N \leq 5000$) 証明

- 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける

小課題3 ($N \leq 5000$) 証明

- 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける

小課題3 ($N \leq 5000$) 証明

- 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける
 - 点数が低い人と対応付けした最適解があるとする
 - より点数が高い人と交換しても悪化しない

小課題3 ($N \leq 5000$) 証明

- 書き換えずに対応付けできるなら、できるだけ点数が高い選手と対応付ける
 - あえて書き換えて対応付けるような最適解があるとする
 - 1人でも書き換えずに対応付け可能な選手がいるなら、その選手と交換しても悪化しない

小課題3 ($N \leq 5000$) 実装

- 最終的に答えが格納される変数をAnsとする
- $Ans = N$
- 点数が低い順に対応付けを決めていく
 - できるだけ点数が高い同じ国籍の選手と対応付けできるか確認する
 - できたらその選手と対応付けて Ans を1減らす
 - できなかったら放置

小課題3 ($N \leq 5000$) 実装

- 最終的に答えが格納される変数をAnsとする
- $Ans = N$
- 点数が低い順に対応付けを決めていく
 - できるだけ点数が高い同じ国籍の選手と対応付けできるか
確認する
 - できたらその選手と対応付けて Ans を1減らす
 - できなかったら放置

小課題3 ($N \leq 5000$) 実装

- 対応付け可能かどうかの確認方法
 - まだ対応付けが完了していない選手だけを見て、先ほど示した「完全マッチングがでる条件」を満たしているか確認

全ての X について以下が成り立つ

両ランキングの X 点以下をとりだしたとき、2h後のランキングの選手の方が多い

- 各 X について X 点以下の選手の人数を両ランキングで求めれば良い
- これは $O(N)$ で簡単に実装できる

小課題3 ($N \leq 5000$) 実装

- 確認部分がボトルネックとなり全体の計算量は
 - $O(N^2)$ 累計60点獲得！

小課題4 ($N \leq 20000$)

- 先ほど $O(N)$ で実装していた部分
 - 同じ国籍、適切な点数の中で最も点が高い選手を見つける
 - 「完全マッチングが出来る条件」の判定
- これらを $O(\log N)$ とか $O(1)$ にできれば解決

小課題4 ($N \leq 20000$)

- 同じ国籍、適切な点数の中で最も点が高い選手を見つける
 - 国籍ごとに、まだだれともマッチングされていない選手の2h後の得点を適切なデータ構造で管理すれば良い
 - やりたい操作は、値の追加、最大値の検索、最大値の削除
 - setやpriority_queueを使えば $O(\log N)$ で実装できる
 - 点数が小さいほうから見ている関係で、実は追加された値はその時点で最大値になる
 - Stackで $O(1)$ で実装できる

小課題4 ($N \leq 20000$)

- 「完全マッチングができる条件」の判定
 - 小課題3での実装は
 - 各 X について X 点以下の人数を両ランキングで求める

全ての X について以下が成り立つ

両ランキングの X 点以下をとりだしたとき、2h後のランキングの選手の方が多い

小課題4 ($N \leq 20000$)

- 「完全マッチングができる条件」の判定
 - 小課題3での実装は「各XについてX点以下の人数を両ランキングで求める」というもの
 - 実際には大小関係しか気にしないので、2h後の人数－5h後の人数だけを管理すればよい
 - 符号を見れば大小関係がわかる
 - 負の値があるかどうかは最小値の符号を見れば確認できる
 - この数列はどのように更新されていくのだろうか？

小課題4 ($N \leq 20000$)

- 「完全マッチングができる条件」の判定
 - 小課題3での実装は「各XについてX点以下の人数を両ランキングで求める」というもの
 - 実際には大小関係しか気にしないので、2h後の人数－5h後の人数だけを管理すればよい
 - 符号を見れば大小関係がわかる
 - 負の値があるかどうかは最小値の符号を見れば確認できる
 - この数列はどのように更新されていくのだろうか？

小課題4 ($N \leq 20000$)

- X点以下の2hでの人数-5hでの人数
 - 2hで30点、5hで51点の選手が対応付けられたとする

小課題4 ($N \leq 20000$)

- X点以下の2hでの人数-5hでの人数
 - 2hで30点、5hで51点の選手が対応付けられたとする

小課題4 ($N \leq 20000$)

- X点以下の2hでの人数-5hでの人数
 - 2hで30点、5hで51点の選手が対応付けられたとする
 - 実際に計算すると以下のように値が変わる

小課題4 ($N \leq 20000$)

- X点以下の2hでの人数-5hでの人数
 - 2hで30点、5hで51点の選手が対応付けられたとする
 - 実際に計算すると以下のように値が変わる
 - おっこれは (Day1の記憶が想起される

小課題4 ($N \leq 20000$)

Segment Tree

小課題4 ($N \leq 20000$)

- 2hでA点の選手、5hでB点の選手を対応付けられるか確認したい
- 先ほどの差の配列を観てA以上B未満の範囲の最小値が0より大きいか確認する
 - 大きいならその区間の値を1減らして、対応付けて良い
 - 最小値が0なら対応付けしてはならない
 - 完全マッチングが崩れる

小課題4 ($N \leq 20000$)

- 2hでA点の選手、5hでB点の選手を対応付けられるか確認したい
- 先ほどの差の配列を観てA以上B未満の範囲の最小値が0より大きいか確認する
 - 大きいならその区間の値を1減らして、対応付けて良い
 - 最小値が0なら対応付けしてはならない
 - 完全マッチングが崩れる

小課題4 ($N \leq 20000$) 解法まとめ

- $Ans = N$
- $5h$ での点数が低い順に選手を見ていく
 - 国籍を書き変えずに対応付けができるかを確認する
 - このときに先ほどのSegment Treeを使う
 - もし対応付けできるなら、なるべく高い人と対応付けする
 - ここはSet等を使う
 - $Ans -= 1$
 - そうでないなら対応付けしない
- Ans が答えになっている

小課題4 ($N \leq 20000$) 解法まとめ

- $\text{Ans} = N$
- $5h$ での点数が低い順に選手を見ていく
 - 国籍を書き変えずに対応付けができるかを確認する
 - このときに先ほどのSegment Treeを使う
 - もし対応付けできるなら、なるべく高い人と対応付けする
 - ここはSet等を使う
 - $\text{Ans} -= 1$
 - そうでないなら対応付けしない
- Ans が答えになっている
- 総計算量は $O(N \log N)$
 - 満点獲得！

得点分布

